Apr 2026

Contents

1 Changes

1.1 .github/ISSUE_TEMPLATE/API.yml

```
        If it exists, the part of the Physlib which corresponds to this API.
```

1.2 .github/ISSUE_TEMPLATE/Bump.yml

```
      This issue is a tracker issue for bumping Physlib to new versions of Lean.
      - label: Put a bump notice on this [thread](https://leanprover.zulipchat.com/#
Physlib/topic/Bumps.20of.20PhysLean.2E/with/572707099).
      [v4.20.0](https://github.com/leanprover-community/physlib/pull/591),
      [v4.20.0-rc5](https://github.com/leanprover-community/physlib/pull/566)
      [v4.27.0](https://github.com/leanprover-community/physlib/pull/929)
```

1.3 .github/workflows/PhyslibTreemap.yml

```
# This is a workflow that is triggered manually to create a treemap (in docs folder)
name: Physlib treemap
```

1.4 .github/workflows/build.yml

```
      - name: build Physlib
      - name: runLinter on Physlib
          run: env LEAN_ABORT_ON_PANIC=1 lake exe runPhyslibLinters
          exclude_file: scripts/MetaPrograms/spellingWords.txt,scripts/physlibNoShak
```

1.5 CONTRIBUTING.md

```
# Contributing
```

```
See:
- https://physlib.io/GettingStarted to learn how to get started
- https://physlib.io/GetInvolved.html to see how to get involved.
```

1.6 Physlib.lean

```
module

public import Physlib.ClassicalFieldTheory.Local.Variation
public import Physlib.ClassicalMechanics.Basic
public import Physlib.ClassicalMechanics.DampedHarmonicOscillator.Basic
public import Physlib.ClassicalMechanics.EulerLagrange
public import Physlib.ClassicalMechanics.HamiltonsEquations
public import Physlib.ClassicalMechanics.HarmonicOscillator.Basic
public import Physlib.ClassicalMechanics.HarmonicOscillator.ConfigurationSp
public import Physlib.ClassicalMechanics.HarmonicOscillator.Solution
public import Physlib.ClassicalMechanics.Lagrangian.TotalDerivativeEquivale
public import Physlib.ClassicalMechanics.Mass.MassUnit
public import Physlib.ClassicalMechanics.Pendulum.CoplanarDoublePendulum
public import Physlib.ClassicalMechanics.Pendulum.MiscellaneousPendulumPivo
public import Physlib.ClassicalMechanics.Pendulum.SlidingPendulum
public import Physlib.ClassicalMechanics.RigidBody.Basic
public import Physlib.ClassicalMechanics.RigidBody.SolidSphere
public import Physlib.ClassicalMechanics.Scattering.RigidSphere
public import Physlib.ClassicalMechanics.Vibrations.LinearTriatomic
public import Physlib.ClassicalMechanics.WaveEquation.Basic
public import Physlib.ClassicalMechanics.WaveEquation.HarmonicWave
public import Physlib.CondensedMatter.Basic
public import Physlib.CondensedMatter.TightBindingChain.Basic
public import Physlib.Cosmology.Basic
public import Physlib.Cosmology.FLRW.Basic
public import Physlib.Electromagnetism.Basic
public import Physlib.Electromagnetism.Charge.ChargeUnit
public import Physlib.Electromagnetism.Current.CircularCoil
public import Physlib.Electromagnetism.Current.InfiniteWire
public import Physlib.Electromagnetism.Dynamics.Basic
public import Physlib.Electromagnetism.Dynamics.CurrentDensity
public import Physlib.Electromagnetism.Dynamics.Hamiltonian
public import Physlib.Electromagnetism.Dynamics.IsExtrema
public import Physlib.Electromagnetism.Dynamics.KineticTerm
public import Physlib.Electromagnetism.Dynamics.Lagrangian
public import Physlib.Electromagnetism.Kinematics.Boosts
public import Physlib.Electromagnetism.Kinematics.EMPotential
public import Physlib.Electromagnetism.Kinematics.ElectricField
```

```
public import Physlib.Electromagnetism.Kinematics.FieldStrength
public import Physlib.Electromagnetism.Kinematics.MagneticField
public import Physlib.Electromagnetism.Kinematics.ScalarPotential
public import Physlib.Electromagnetism.Kinematics.VectorPotential
public import Physlib.Electromagnetism.PointParticle.OneDimension
public import Physlib.Electromagnetism.PointParticle.ThreeDimension
public import Physlib.Electromagnetism.ThreeDimension.Basic
public import Physlib.Electromagnetism.ThreeDimension.MaxwellEquations
public import Physlib.Electromagnetism.Vacuum.Constant
public import Physlib.Electromagnetism.Vacuum.HarmonicWave
public import Physlib.Electromagnetism.Vacuum.IsPlaneWave
public import Physlib.Mathematics.Calculus.AdjFDeriv
public import Physlib.Mathematics.Calculus.Divergence
public import Physlib.Mathematics.DataStructures.FourTree.Basic
public import Physlib.Mathematics.DataStructures.FourTree.UniqueMap
public import Physlib.Mathematics.DataStructures.Matrix.LieTrace
public import Physlib.Mathematics.Distribution.Basic
public import Physlib.Mathematics.Distribution.PowMul
public import Physlib.Mathematics.FDerivCurry
public import Physlib.Mathematics.Fin
public import Physlib.Mathematics.Fin.Involutions
public import Physlib.Mathematics.Geometry.Metric.PseudoRiemannian.Defs
public import Physlib.Mathematics.Geometry.Metric.Riemannian.Defs
public import Physlib.Mathematics.InnerProductSpace.Adjoint
public import Physlib.Mathematics.InnerProductSpace.Basic
public import Physlib.Mathematics.InnerProductSpace.Calculus
public import Physlib.Mathematics.InnerProductSpace.Submodule
public import Physlib.Mathematics.KroneckerDelta
public import Physlib.Mathematics.LinearMaps
public import Physlib.Mathematics.List
public import Physlib.Mathematics.List.InsertIdx
public import Physlib.Mathematics.List.InsertionSort
public import Physlib.Mathematics.PiTensorProduct
public import Physlib.Mathematics.RatComplexNum
public import Physlib.Mathematics.SO3.Basic
public import Physlib.Mathematics.SchurTriangulation
public import Physlib.Mathematics.SpecialFunctions.PhysHermite
public import Physlib.Mathematics.Trigonometry.Tanh
public import Physlib.Mathematics.VariationalCalculus.Basic
public import Physlib.Mathematics.VariationalCalculus.HasVarAdjDeriv
public import Physlib.Mathematics.VariationalCalculus.HasVarAdjoint
public import Physlib.Mathematics.VariationalCalculus.HasVarGradient
public import Physlib.Mathematics.VariationalCalculus.IsLocalizedfunctionTransform
public import Physlib.Mathematics.VariationalCalculus.IsTestFunction
public import Physlib.Meta.AllFilePaths
public import Physlib.Meta.Basic
```

```
public import Physlib.Meta.Informal.Basic
public import Physlib.Meta.Informal.Post
public import Physlib.Meta.Informal.SemiFormal
public import Physlib.Meta.Linters.Sorry
public import Physlib.Meta.Notes.Basic
public import Physlib.Meta.Notes.HTMLNote
public import Physlib.Meta.Notes.NoteFile
public import Physlib.Meta.Notes.ToHTML
public import Physlib.Meta.Remark.Basic
public import Physlib.Meta.Remark.Properties
public import Physlib.Meta.Sorry
public import Physlib.Meta.TODO.Basic
public import Physlib.Meta.TODO.Global
public import Physlib.Meta.TransverseTactics
public import Physlib.Optics.Basic
public import Physlib.Optics.Polarization.Basic
public import Physlib.Particles.BeyondTheStandardModel.GeorgiGlashow.Basic
public import Physlib.Particles.BeyondTheStandardModel.PatiSalam.Basic
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
public import Physlib.Particles.BeyondTheStandardModel.Spin10.Basic
public import Physlib.Particles.BeyondTheStandardModel.TwoHDM.Basic
public import Physlib.Particles.BeyondTheStandardModel.TwoHDM.GramMatrix
public import Physlib.Particles.BeyondTheStandardModel.TwoHDM.Potential
public import Physlib.Particles.FlavorPhysics.CKMMatrix.Basic
public import Physlib.Particles.FlavorPhysics.CKMMatrix.Invariants
public import Physlib.Particles.FlavorPhysics.CKMMatrix.PhaseFreedom
public import Physlib.Particles.FlavorPhysics.CKMMatrix.Relations
public import Physlib.Particles.FlavorPhysics.CKMMatrix.Rows
public import Physlib.Particles.FlavorPhysics.CKMMatrix.StandardParameteriz
public import Physlib.Particles.FlavorPhysics.CKMMatrix.StandardParameteriz
public import Physlib.Particles.NeutrinoPhysics.Basic
public import Physlib.Particles.StandardModel.AnomalyCancellation.Basic
```

```
public import Physlib.Particles.StandardModel.AnomalyCancellation.FamilyMaps
public import Physlib.Particles.StandardModel.AnomalyCancellation.NoGrav.Basic
public import Physlib.Particles.StandardModel.AnomalyCancellation.NoGrav.One.Lemmas
public import Physlib.Particles.StandardModel.AnomalyCancellation.NoGrav.One.LinearP
public import Physlib.Particles.StandardModel.AnomalyCancellation.Permutations
public import Physlib.Particles.StandardModel.Basic
public import Physlib.Particles.StandardModel.HiggsBoson.Basic
public import Physlib.Particles.StandardModel.HiggsBoson.Potential
public import Physlib.Particles.StandardModel.Representations
public import Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.B3
public import Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.Basic
public import Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.HyperCharge
public import Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.LineY3B3
public import Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.OrthogY3B3.
public import Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.OrthogY3B3.
public import Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.OrthogY3B3.
public import Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.Permutation
public import Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.Y3
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.AllowsTerm
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.Basic
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.Completions
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.Map
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.MinimalSuperSet
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.MinimallyAllowsTerm
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.MinimallyAllowsTerm
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.MinimallyAllowsTerm
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.OfFieldLabel
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.OfPotentialTerm
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.PhenoClosed
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.PhenoConstrained
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.Yukawa
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.ZMod
public import Physlib.Particles.SuperSymmetry.SU5.FieldLabels
public import Physlib.Particles.SuperSymmetry.SU5.Potential
public import Physlib.QFT.AnomalyCancellation.Basic
public import Physlib.QFT.AnomalyCancellation.GroupActions
public import Physlib.QFT.PerturbationTheory.CreateAnnihilate
public import Physlib.QFT.PerturbationTheory.FeynmanDiagrams.Basic
public import Physlib.QFT.PerturbationTheory.FieldOpFreeAlgebra.Basic
public import Physlib.QFT.PerturbationTheory.FieldOpFreeAlgebra.Grading
public import Physlib.QFT.PerturbationTheory.FieldOpFreeAlgebra.NormTimeOrder
public import Physlib.QFT.PerturbationTheory.FieldOpFreeAlgebra.NormalOrder
public import Physlib.QFT.PerturbationTheory.FieldOpFreeAlgebra.SuperCommute
public import Physlib.QFT.PerturbationTheory.FieldOpFreeAlgebra.TimeOrder
public import Physlib.QFT.PerturbationTheory.FieldSpecification.Basic
public import Physlib.QFT.PerturbationTheory.FieldSpecification.CrAnFieldOp
```

```
public import Physlib.QFT.PerturbationTheory.FieldSpecification.CrAnSection
public import Physlib.QFT.PerturbationTheory.FieldSpecification.Filters
public import Physlib.QFT.PerturbationTheory.FieldSpecification.NormalOrder
public import Physlib.QFT.PerturbationTheory.FieldSpecification.TimeOrder
public import Physlib.QFT.PerturbationTheory.FieldStatistics.Basic
public import Physlib.QFT.PerturbationTheory.FieldStatistics.ExchangeSign
public import Physlib.QFT.PerturbationTheory.FieldStatistics.OfFinset
public import Physlib.QFT.PerturbationTheory.Koszul.KoszulSign
public import Physlib.QFT.PerturbationTheory.Koszul.KoszulSignInsert
public import Physlib.QFT.PerturbationTheory.WickAlgebra.Basic
public import Physlib.QFT.PerturbationTheory.WickAlgebra.Grading
public import Physlib.QFT.PerturbationTheory.WickAlgebra.NormalOrder.Basic
public import Physlib.QFT.PerturbationTheory.WickAlgebra.NormalOrder.Lemmas
public import Physlib.QFT.PerturbationTheory.WickAlgebra.NormalOrder.WickCo
public import Physlib.QFT.PerturbationTheory.WickAlgebra.StaticWickTerm
public import Physlib.QFT.PerturbationTheory.WickAlgebra.StaticWickTheorem
public import Physlib.QFT.PerturbationTheory.WickAlgebra.SuperCommute
public import Physlib.QFT.PerturbationTheory.WickAlgebra.TimeContraction
public import Physlib.QFT.PerturbationTheory.WickAlgebra.TimeOrder
public import Physlib.QFT.PerturbationTheory.WickAlgebra.Universality
public import Physlib.QFT.PerturbationTheory.WickAlgebra.WickTerm
public import Physlib.QFT.PerturbationTheory.WickAlgebra.WicksTheorem
public import Physlib.QFT.PerturbationTheory.WickAlgebra.WicksTheoremNormal
public import Physlib.QFT.PerturbationTheory.WickContraction.Basic
public import Physlib.QFT.PerturbationTheory.WickContraction.Card
public import Physlib.QFT.PerturbationTheory.WickContraction.Erase
public import Physlib.QFT.PerturbationTheory.WickContraction.ExtractEquiv
public import Physlib.QFT.PerturbationTheory.WickContraction.InsertAndContr
public import Physlib.QFT.PerturbationTheory.WickContraction.InsertAndContr
public import Physlib.QFT.PerturbationTheory.WickContraction.Involutions
public import Physlib.QFT.PerturbationTheory.WickContraction.IsFull
public import Physlib.QFT.PerturbationTheory.WickContraction.Join
public import Physlib.QFT.PerturbationTheory.WickContraction.Perm
public import Physlib.QFT.PerturbationTheory.WickContraction.Sign.Basic
public import Physlib.QFT.PerturbationTheory.WickContraction.Sign.InsertNon
public import Physlib.QFT.PerturbationTheory.WickContraction.Sign.InsertSom
public import Physlib.QFT.PerturbationTheory.WickContraction.Sign.Join
public import Physlib.QFT.PerturbationTheory.WickContraction.Singleton
public import Physlib.QFT.PerturbationTheory.WickContraction.StaticContract
public import Physlib.QFT.PerturbationTheory.WickContraction.SubContraction
public import Physlib.QFT.PerturbationTheory.WickContraction.TimeCond
public import Physlib.QFT.PerturbationTheory.WickContraction.TimeContract
public import Physlib.QFT.PerturbationTheory.WickContraction.Uncontracted
public import Physlib.QFT.PerturbationTheory.WickContraction.UncontractedLi
public import Physlib.QFT.QED.AnomalyCancellation.Basic
public import Physlib.QFT.QED.AnomalyCancellation.BasisLinear
```

```
public import Physlib.QFT.QED.AnomalyCancellation.ConstAbs
public import Physlib.QFT.QED.AnomalyCancellation.Even.BasisLinear
public import Physlib.QFT.QED.AnomalyCancellation.Even.LineInCubic
public import Physlib.QFT.QED.AnomalyCancellation.Even.Parameterization
public import Physlib.QFT.QED.AnomalyCancellation.LineInPlaneCond
public import Physlib.QFT.QED.AnomalyCancellation.LowDim.One
public import Physlib.QFT.QED.AnomalyCancellation.LowDim.Three
public import Physlib.QFT.QED.AnomalyCancellation.LowDim.Two
public import Physlib.QFT.QED.AnomalyCancellation.Odd.BasisLinear
public import Physlib.QFT.QED.AnomalyCancellation.Odd.LineInCubic
public import Physlib.QFT.QED.AnomalyCancellation.Odd.Parameterization
public import Physlib.QFT.QED.AnomalyCancellation.Permutations
public import Physlib.QFT.QED.AnomalyCancellation.Sorts
public import Physlib.QFT.QED.AnomalyCancellation.VectorLike
public import Physlib.QuantumMechanics.DDimensions.Hydrogen.Basic
public import Physlib.QuantumMechanics.DDimensions.Hydrogen.LaplaceRungeLenzVector
public import Physlib.QuantumMechanics.DDimensions.Operators.AngularMomentum
public import Physlib.QuantumMechanics.DDimensions.Operators.Commutation
public import Physlib.QuantumMechanics.DDimensions.Operators.Momentum
public import Physlib.QuantumMechanics.DDimensions.Operators.Position
public import Physlib.QuantumMechanics.DDimensions.Operators.Unbounded
public import Physlib.QuantumMechanics.DDimensions.SpaceDHilbertSpace.Basic
public import Physlib.QuantumMechanics.DDimensions.SpaceDHilbertSpace.PolyBddSchwart
public import Physlib.QuantumMechanics.DDimensions.SpaceDHilbertSpace.SchwartzSubmod
public import Physlib.QuantumMechanics.FiniteTarget.Basic
public import Physlib.QuantumMechanics.FiniteTarget.HilbertSpace
public import Physlib.QuantumMechanics.OneDimension.GeneralPotential.Basic
public import Physlib.QuantumMechanics.OneDimension.HarmonicOscillator.Basic
public import Physlib.QuantumMechanics.OneDimension.HarmonicOscillator.Completeness
public import Physlib.QuantumMechanics.OneDimension.HarmonicOscillator.Eigenfunction
public import Physlib.QuantumMechanics.OneDimension.HarmonicOscillator.Examples
public import Physlib.QuantumMechanics.OneDimension.HarmonicOscillator.TISE
public import Physlib.QuantumMechanics.OneDimension.HilbertSpace.Basic
public import Physlib.QuantumMechanics.OneDimension.HilbertSpace.Gaussians
public import Physlib.QuantumMechanics.OneDimension.HilbertSpace.PlaneWaves
public import Physlib.QuantumMechanics.OneDimension.HilbertSpace.PositionStates
public import Physlib.QuantumMechanics.OneDimension.HilbertSpace.SchwartzSubmodule
public import Physlib.QuantumMechanics.OneDimension.Operators.Commutation
public import Physlib.QuantumMechanics.OneDimension.Operators.Momentum
public import Physlib.QuantumMechanics.OneDimension.Operators.Parity
public import Physlib.QuantumMechanics.OneDimension.Operators.Position
public import Physlib.QuantumMechanics.OneDimension.Operators.Unbounded
public import Physlib.QuantumMechanics.OneDimension.ReflectionlessPotential.Basic
public import Physlib.QuantumMechanics.PlanckConstant
public import Physlib.Relativity.Bispinors.Basic
public import Physlib.Relativity.CliffordAlgebra
```

```
public import Physlib.Relativity.LorentzAlgebra.Basic
public import Physlib.Relativity.LorentzAlgebra.Basis
public import Physlib.Relativity.LorentzAlgebra.ExponentialMap
public import Physlib.Relativity.LorentzGroup.Basic
public import Physlib.Relativity.LorentzGroup.Boosts.Apply
public import Physlib.Relativity.LorentzGroup.Boosts.Basic
public import Physlib.Relativity.LorentzGroup.Boosts.Generalized
public import Physlib.Relativity.LorentzGroup.Orthochronous.Basic
public import Physlib.Relativity.LorentzGroup.Proper
public import Physlib.Relativity.LorentzGroup.Restricted.Basic
public import Physlib.Relativity.LorentzGroup.Restricted.FromBoostRotation
public import Physlib.Relativity.LorentzGroup.Rotations
public import Physlib.Relativity.LorentzGroup.ToVector
public import Physlib.Relativity.MinkowskiMatrix
public import Physlib.Relativity.PauliMatrices.AsTensor
public import Physlib.Relativity.PauliMatrices.Basic
public import Physlib.Relativity.PauliMatrices.CliffordAlgebra
public import Physlib.Relativity.PauliMatrices.Relations
public import Physlib.Relativity.PauliMatrices.SelfAdjoint
public import Physlib.Relativity.PauliMatrices.ToTensor
public import Physlib.Relativity.SL2C.Basic
public import Physlib.Relativity.SL2C.SelfAdjoint
public import Physlib.Relativity.Special.ProperTime
public import Physlib.Relativity.Special.TwinParadox.Basic
public import Physlib.Relativity.SpeedOfLight
public import Physlib.Relativity.Tensors.Basic
public import Physlib.Relativity.Tensors.Color.Basic
public import Physlib.Relativity.Tensors.Color.Discrete
public import Physlib.Relativity.Tensors.Color.Lift
public import Physlib.Relativity.Tensors.ComplexTensor.Basic
public import Physlib.Relativity.Tensors.ComplexTensor.Lemmas
public import Physlib.Relativity.Tensors.ComplexTensor.Matrix.Pre
public import Physlib.Relativity.Tensors.ComplexTensor.Metrics.Basic
public import Physlib.Relativity.Tensors.ComplexTensor.Metrics.Lemmas
public import Physlib.Relativity.Tensors.ComplexTensor.Metrics.Pre
public import Physlib.Relativity.Tensors.ComplexTensor.OfRat
public import Physlib.Relativity.Tensors.ComplexTensor.Units.Basic
public import Physlib.Relativity.Tensors.ComplexTensor.Units.Pre
public import Physlib.Relativity.Tensors.ComplexTensor.Units.Symm
public import Physlib.Relativity.Tensors.ComplexTensor.Vector.Pre.Basic
public import Physlib.Relativity.Tensors.ComplexTensor.Vector.Pre.Contracti
public import Physlib.Relativity.Tensors.ComplexTensor.Vector.Pre.Modules
public import Physlib.Relativity.Tensors.ComplexTensor.Weyl.Basic
public import Physlib.Relativity.Tensors.ComplexTensor.Weyl.Contraction
public import Physlib.Relativity.Tensors.ComplexTensor.Weyl.Metric
public import Physlib.Relativity.Tensors.ComplexTensor.Weyl.Modules
```

```
public import Physlib.Relativity.Tensors.ComplexTensor.Weyl.Two
public import Physlib.Relativity.Tensors.ComplexTensor.Weyl.Unit
public import Physlib.Relativity.Tensors.Constructors
public import Physlib.Relativity.Tensors.Contraction.Basic
public import Physlib.Relativity.Tensors.Contraction.Basis
public import Physlib.Relativity.Tensors.Contraction.Products
public import Physlib.Relativity.Tensors.Contraction.Pure
public import Physlib.Relativity.Tensors.Dual
public import Physlib.Relativity.Tensors.Elab
public import Physlib.Relativity.Tensors.Evaluation
public import Physlib.Relativity.Tensors.MetricTensor
public import Physlib.Relativity.Tensors.OfInt
public import Physlib.Relativity.Tensors.Product
public import Physlib.Relativity.Tensors.RealTensor.Basic
public import Physlib.Relativity.Tensors.RealTensor.CoVector.Basic
public import Physlib.Relativity.Tensors.RealTensor.Matrix.Pre
public import Physlib.Relativity.Tensors.RealTensor.Metrics.Basic
public import Physlib.Relativity.Tensors.RealTensor.Metrics.Pre
public import Physlib.Relativity.Tensors.RealTensor.ToComplex
public import Physlib.Relativity.Tensors.RealTensor.Units.Pre
public import Physlib.Relativity.Tensors.RealTensor.Vector.Basic
public import Physlib.Relativity.Tensors.RealTensor.Vector.Causality.Basic
public import Physlib.Relativity.Tensors.RealTensor.Vector.Causality.LightLike
public import Physlib.Relativity.Tensors.RealTensor.Vector.Causality.TimeLike
public import Physlib.Relativity.Tensors.RealTensor.Vector.MinkowskiProduct
public import Physlib.Relativity.Tensors.RealTensor.Vector.Pre.Basic
public import Physlib.Relativity.Tensors.RealTensor.Vector.Pre.Contraction
public import Physlib.Relativity.Tensors.RealTensor.Vector.Pre.Modules
public import Physlib.Relativity.Tensors.RealTensor.Velocity.Basic
public import Physlib.Relativity.Tensors.TensorSpecies.Basic
public import Physlib.Relativity.Tensors.Tensorial
public import Physlib.Relativity.Tensors.UnitTensor
public import Physlib.SpaceAndTime.Space.Basic
public import Physlib.SpaceAndTime.Space.ConstantSliceDist
public import Physlib.SpaceAndTime.Space.CrossProduct
public import Physlib.SpaceAndTime.Space.Derivatives.Basic
public import Physlib.SpaceAndTime.Space.Derivatives.Curl
public import Physlib.SpaceAndTime.Space.Derivatives.Div
public import Physlib.SpaceAndTime.Space.Derivatives.Grad
public import Physlib.SpaceAndTime.Space.Derivatives.Iterated
public import Physlib.SpaceAndTime.Space.Derivatives.Laplacian
public import Physlib.SpaceAndTime.Space.Derivatives.MultiIndex
public import Physlib.SpaceAndTime.Space.DistConst
public import Physlib.SpaceAndTime.Space.DistOfFunction
public import Physlib.SpaceAndTime.Space.Integrals.Basic
public import Physlib.SpaceAndTime.Space.Integrals.NormPow
```

```
public import Physlib.SpaceAndTime.Space.Integrals.RadialAngularMeasure
public import Physlib.SpaceAndTime.Space.IsDistBounded
public import Physlib.SpaceAndTime.Space.LengthUnit
public import Physlib.SpaceAndTime.Space.Module
public import Physlib.SpaceAndTime.Space.Norm
public import Physlib.SpaceAndTime.Space.Slice
public import Physlib.SpaceAndTime.Space.Translations
public import Physlib.SpaceAndTime.SpaceTime.Basic
public import Physlib.SpaceAndTime.SpaceTime.Boosts
public import Physlib.SpaceAndTime.SpaceTime.Derivatives
public import Physlib.SpaceAndTime.SpaceTime.LorentzAction
public import Physlib.SpaceAndTime.SpaceTime.TimeSlice
public import Physlib.SpaceAndTime.Time.Basic
public import Physlib.SpaceAndTime.Time.Derivatives
public import Physlib.SpaceAndTime.Time.TimeMan
public import Physlib.SpaceAndTime.Time.TimeTransMan
public import Physlib.SpaceAndTime.Time.TimeUnit
public import Physlib.SpaceAndTime.TimeAndSpace.Basic
public import Physlib.SpaceAndTime.TimeAndSpace.ConstantTimeDist
public import Physlib.StatisticalMechanics.BoltzmannConstant
public import Physlib.StatisticalMechanics.CanonicalEnsemble.Basic
public import Physlib.StatisticalMechanics.CanonicalEnsemble.Finite
public import Physlib.StatisticalMechanics.CanonicalEnsemble.Lemmas
public import Physlib.StatisticalMechanics.CanonicalEnsemble.TwoState
public import Physlib.StringTheory.Basic
public import Physlib.StringTheory.FTheory.SU5.Basic
public import Physlib.StringTheory.FTheory.SU5.Charges.AnomalyFree
public import Physlib.StringTheory.FTheory.SU5.Charges.OfRationalSection
public import Physlib.StringTheory.FTheory.SU5.Charges.Viable
public import Physlib.StringTheory.FTheory.SU5.Fluxes.Basic
public import Physlib.StringTheory.FTheory.SU5.Fluxes.NoExotics.ChiralIndic
public import Physlib.StringTheory.FTheory.SU5.Fluxes.NoExotics.Completenes
public import Physlib.StringTheory.FTheory.SU5.Fluxes.NoExotics.Elems
public import Physlib.StringTheory.FTheory.SU5.Quanta.Basic
public import Physlib.StringTheory.FTheory.SU5.Quanta.FiveQuanta
public import Physlib.StringTheory.FTheory.SU5.Quanta.IsViable
public import Physlib.StringTheory.FTheory.SU5.Quanta.TenQuanta
public import Physlib.Thermodynamics.Basic
public import Physlib.Thermodynamics.IdealGas.Basic
public import Physlib.Thermodynamics.Temperature.Basic
public import Physlib.Thermodynamics.Temperature.TemperatureUnits
public import Physlib.Units.Basic
public import Physlib.Units.Dimension
public import Physlib.Units.Examples
public import Physlib.Units.FDeriv
public import Physlib.Units.Integral
```

```
public import Physlib.Units.UnitDependent
public import Physlib.Units.WithDim.Area
public import Physlib.Units.WithDim.Basic
public import Physlib.Units.WithDim.Energy
public import Physlib.Units.WithDim.Mass
public import Physlib.Units.WithDim.Momentum
public import Physlib.Units.WithDim.Pressure
public import Physlib.Units.WithDim.Speed
public import Physlib.Units.WithDim.Velocity
```

1.7 Physlib/ClassicalFieldTheory/Local/Variation.lean

```
/-
Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Juan Jose Fernandez Morales
-/
module

public import Physlib.Mathematics.VariationalCalculus.IsTestFunction
/-!
# Admissible local variations

## i. Overview

This module packages the admissible variations used in the local first-
variation problem.

For the first local stage, a variation is admissible if it is a smooth compactly sup
`Space d → U` for a real normed vector space `U`. This reuses the existing `IsTestFu
predicate rather than introducing a second support calculus.

## ii. Key results

- `ClassicalFieldTheory.Local.AdmissibleVariation` : compactly supported smooth vari

## iii. Table of contents

- A. Admissible variations
- B. Basic operations on admissible variations

## iv. References

-/
```

```
@[expose] public section

open MeasureTheory
open Physlib

namespace ClassicalFieldTheory
namespace Local

/-!
## A. Admissible variations

-/

/-- An admissible local variation is a smooth compactly supported map `Spac
/
structure AdmissibleVariation (d : ℕ) (U : Type*) [NormedAddCommGroup U] [N
  /-- The underlying variation function. -/
  toFun : Space d → U
  /-- Smoothness and compact support of the variation. -/
  isTestFunction : IsTestFunction toFun

namespace AdmissibleVariation

variable {d : ℕ} {U : Type*} [NormedAddCommGroup U] [NormedSpace ℝ U]

/-!
## B. Basic operations on admissible variations

-/

instance : CoeFun (AdmissibleVariation d U) (fun _ => Space d → U) where
  coe η := η.toFun

attribute [fun_prop] AdmissibleVariation.isTestFunction

/-- An admissible variation has compact support. -/
lemma hasCompactSupport (η : AdmissibleVariation d U) : HasCompactSupport (
  η.isTestFunction.supp

/-- View an admissible variation as a compactly supported continuous map. -
/
noncomputable def toCompactlySupportedContinuousMap (η : AdmissibleVariatio
    CompactlySupportedContinuousMap (Space d) U :=
  η.isTestFunction.toCompactlySupportedContinuousMap
```

```
noncomputable instance : Zero (AdmissibleVariation d U) where
  zero :=
    { toFun := fun _ => 0
      isTestFunction := IsTestFunction.zero }

@[simp]
lemma zero_apply (x : Space d) : (0 : AdmissibleVariation d U) x = 0 := rfl

noncomputable instance : Neg (AdmissibleVariation d U) where
  neg η :=
    { toFun := fun x => -η x
      isTestFunction := η.isTestFunction.neg }

@[simp]
lemma neg_apply (η : AdmissibleVariation d U) (x : Space d) : (-
η) x = -η x := rfl

noncomputable instance : Add (AdmissibleVariation d U) where
  add η ξ :=
    { toFun := fun x => η x + ξ x
      isTestFunction := η.isTestFunction.add ξ.isTestFunction }

@[simp]
lemma add_apply (η ξ : AdmissibleVariation d U) (x : Space d) : (η + ξ) x = η x + ξ

noncomputable instance : Sub (AdmissibleVariation d U) where
  sub η ξ :=
    { toFun := fun x => η x - ξ x
      isTestFunction := η.isTestFunction.sub ξ.isTestFunction }

@[simp]
lemma sub_apply (η ξ : AdmissibleVariation d U) (x : Space d) : (η - ξ) x = η x - ξ

noncomputable instance : SMul ℝ (AdmissibleVariation d U) where
  smul c η :=
    { toFun := fun x => c • η x
      isTestFunction := IsTestFunction.smul_left (f := fun _ : Space d => c) (g := η
        (by fun_prop) η.isTestFunction }

@[simp]
lemma smul_apply (c : ℝ) (η : AdmissibleVariation d U) (x : Space d) :
    (c • η) x = c • η x := rfl

@[fun_prop]
lemma coord {m : ℕ} (η : AdmissibleVariation d (Space m)) (a : Fin m) :
    IsTestFunction (fun x => (η.toFun x).coord a) := by
```

```
  simpa [Space.coord] using IsTestFunction.coord η.isTestFunction a

end AdmissibleVariation

end Local
end ClassicalFieldTheory
```

1.8 Physlib/ClassicalMechanics/DampedHarmonicOscillator/Basic.l

```
public import Physlib.SpaceAndTime.Time.Derivatives
TODO "Derive solutions for the underdamped case (oscillatory with exponenti
TODO "Derive solutions for the critically damped case (fastest non-
oscillatory return)."
TODO "Derive solutions for the overdamped case (slow non-
oscillatory return)."
TODO "Define and prove properties of the quality factor Q."
TODO "Define and prove properties of the relaxation time τ."
TODO "Prove that the damped harmonic oscillator reduces to the undamped cas
```

1.9 Physlib/ClassicalMechanics/EulerLagrange.lean

```
public import Physlib.Mathematics.VariationalCalculus.HasVarGradient
public import Physlib.SpaceAndTime.Time.Derivatives
set_option backward.isDefEq.respectTransparency false in
```

1.10 Physlib/ClassicalMechanics/HamiltonsEquations.lean

```
public import Physlib.Mathematics.VariationalCalculus.HasVarGradient
public import Physlib.SpaceAndTime.Time.Derivatives
set_option backward.isDefEq.respectTransparency false in
```

1.11 Physlib/ClassicalMechanics/HarmonicOscillator/Basic.lean

```
public import Physlib.ClassicalMechanics.EulerLagrange
public import Physlib.ClassicalMechanics.HamiltonsEquations
public import Mathlib.Data.Real.Hom
TODO "Create a new folder for the damped harmonic oscillator, initially as
holder."
TODO "Create a new file for the geometric model which properly models the p
    configuration space and velocity as its tangent space, then show explic
```

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.12 Physlib/ClassicalMechanics/HarmonicOscillator/ConfigurationSpace.

```
public import Physlib.SpaceAndTime.Space.Basic
public import Mathlib.Analysis.InnerProductSpace.Calculus
TODO "Configuration Space should be refactored to take `EuclideanSpace ℝ (Fin 1)`
TODO "The API around the configuration space
    should be improved to allow further development of a proper
    geometric model of the Harmonic Oscillator."
  dist_eq x y := by
    simp [dist_eq_val, norm]
    refine abs_eq_abs.mpr ?_
    ring_nf
    simp
  dist_eq x y := by
    simp [dist_eq_val, norm]
    refine abs_eq_abs.mpr ?_
    ring_nf
    simp
```

1.13 Physlib/ClassicalMechanics/HarmonicOscillator/Solution.lean

```
public import Physlib.ClassicalMechanics.HarmonicOscillator.Basic
TODO "Implement other initial conditions for the classical harmonic oscillator. For
  simp [trajectory_eq, smul_add, smul_smul, mul_comm]
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
          rw [← sub_sub_sub_eq (S.m * ‖IC.v₀‖ ^ 2) (S.k * ‖IC.x₀‖ ^ 2)
TODO "For the classical harmonic oscillator find the time for which it returns to
TODO "For the classical harmonic oscillator find the times for
```

1.14 Physlib/ClassicalMechanics/Lagrangian/TotalDerivativeEquiv

```
public import Physlib.Mathematics.InnerProductSpace.Basic
public import Mathlib.Analysis.InnerProductSpace.Dual
set_option backward.isDefEq.respectTransparency false in
```

1.15 Physlib/ClassicalMechanics/Mass/MassUnit.lean

```
public import Mathlib.Analysis.RCLike.Basic
set_option backward.isDefEq.respectTransparency false in
```

1.16 Physlib/ClassicalMechanics/Pendulum/CoplanarDoublePendulum

```
public import Physlib.Meta.Linters.Sorry
```

1.17 Physlib/ClassicalMechanics/Pendulum/SlidingPendulum.lean

```
public import Physlib.Meta.Linters.Sorry
```

1.18 Physlib/ClassicalMechanics/RigidBody/Basic.lean

```
public import Physlib.SpaceAndTime.Space.Module
public import Physlib.Meta.Informal.Basic
TODO "The definition of a rigid body is currently defined via linear maps
noncomputable def mass {d : ℕ} (R : RigidBody d) : ℝ := R.ρ (fun _ => 1, co
```

1.19 Physlib/ClassicalMechanics/RigidBody/SolidSphere.lean

```
public import Physlib.ClassicalMechanics.RigidBody.Basic
public import Physlib.SpaceAndTime.Space.Integrals.Basic
public import Physlib.Meta.Linters.Sorry

set_option backward.isDefEq.respectTransparency false in
  simp only [← integral_indicator measurableSet_closedBall, Set.indicator,
```

1.20 Physlib/ClassicalMechanics/Scattering/RigidSphere.lean

```
public import Physlib.Meta.TODO.Basic
```

```
TODO "Derive the scattering cross section of a perfectly rigid sphere."
```

1.21 Physlib/ClassicalMechanics/Vibrations/LinearTriatomic.lean

```
public import Physlib.Meta.TODO.Basic
TODO "Derive the frequencies of vibaration of a symmetric linear triatomic molecule.
```

1.22 Physlib/ClassicalMechanics/WaveEquation/Basic.lean

```
public import Physlib.SpaceAndTime.Space.CrossProduct
public import Physlib.SpaceAndTime.Space.Derivatives.Laplacian
  simp only [PiLp.inner_apply, fderiv_fun_const, Pi.zero_apply,
    PiLp.single_apply, zero_add]
set_option backward.isDefEq.respectTransparency false in
```

1.23 Physlib/ClassicalMechanics/WaveEquation/HarmonicWave.lean

```
public import Physlib.ClassicalMechanics.WaveEquation.Basic
TODO "Show that the wave equation is invariant under rotations and any direction `s`
set_option backward.isDefEq.respectTransparency false in
TODO "Show that any disturbance (subject to certain conditions) can be expressed
```

1.24 Physlib/CondensedMatter/TightBindingChain/Basic.lean

```
public import Physlib.QuantumMechanics.FiniteTarget.HilbertSpace
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.25 Physlib/Cosmology/FLRW/Basic.lean

```
public import Mathlib.Analysis.SpecialFunctions.Trigonometric.Deriv
public import Physlib.Meta.Linters.Sorry
public import Physlib.Meta.Informal.Basic
public import Physlib.SpaceAndTime.Time.Derivatives
open Time

- `a : Time → ℝ` is the FLRW scale factor as a function of cosmic time `t`.
```

```
- `ρ : Time → ℝ` is the total energy density as a function of cosmic time `
def FirstOrderFriedmann (a ρ : Time → ℝ) (k Λ G c : ℝ) (t : Time) : Prop :=
    ((∂ₜ a t / a t)^2
- `a : Time → ℝ` is the FLRW scale factor as a function of cosmic time `t`.
- `ρ : Time → ℝ` is the total energy density as a function of cosmic time `
- `p : Time → ℝ` is the pressure. It is related to `ρ` via `p = w * ρ `
def SecondOrderFriedmann (a ρ p : Time → ℝ) (Λ G c : ℝ) (t : Time) : Prop :
    ∂ₜ (∂ₜ a) t / a t = - (4 * Real.pi * G / 3) * (ρ t + 3 * p t / c^2) + Λ
noncomputable def hubbleConstant (a : Time → ℝ) (t : Time) : ℝ :=
    ∂ₜ a t / a t

/-- The Hubble constant is nonzero whenever `∂ₜ a t` and `a t` are both non
/
lemma hubbleConstant_ne_zero {a : Time → ℝ} {t : Time}
    (hd_az : ∂ₜ a t ≠ 0) (haz : a t ≠ 0) :
    hubbleConstant a t ≠ 0 :=
  div_ne_zero hd_az haz
noncomputable def decelerationParameter (a : Time → ℝ) (t : Time) : ℝ :=
    - (∂ₜ (∂ₜ a) t * a t) / (∂ₜ a t)^2

/-- Quotient-rule expression for the time derivative of the Hubble constant
  `dₜ H = (a'' a - (a')^2) / a^2`. -/
lemma deriv_hubbleConstant {a : Time → ℝ} {t : Time}
    (ha : DifferentiableAt ℝ a t) (hd_a : DifferentiableAt ℝ (∂ₜ a) t)
    (haz : a t ≠ 0) :
    ∂ₜ (hubbleConstant a) t =
      (∂ₜ (∂ₜ a) t * a t - (∂ₜ a t) ^ 2) / (a t) ^ 2 := by
  show ∂ₜ (fun s => ∂ₜ a s / a s) t = _
  rw [Time.deriv_div hd_a ha haz]
  ring
lemma decelerationParameter_eq_one_plus_hubbleConstant
    {a : Time → ℝ} {t : Time}
    (ha : DifferentiableAt ℝ a t) (hd_a : DifferentiableAt ℝ (∂ₜ a) t)
    (haz : a t ≠ 0) (hd_az : ∂ₜ a t ≠ 0) :
    decelerationParameter a t =
      -(1 + ∂ₜ (hubbleConstant a) t / (hubbleConstant a t) ^ 2) := by
  rw [deriv_hubbleConstant ha hd_a haz]
  simp only [decelerationParameter, hubbleConstant]
  field_simp
  ring

/-- The time derivative of the Hubble constant equals `-H² (1 + q)`. -
/
lemma deriv_hubbleConstant_eq_neg_sq_mul
    {a : Time → ℝ} {t : Time}
    (ha : DifferentiableAt ℝ a t) (hd_a : DifferentiableAt ℝ (∂ₜ a) t)
```

```
    (haz : a t ≠ 0) (hd_az : ∂ₜ a t ≠ 0) :
    ∂ₜ (hubbleConstant a) t =
      -(hubbleConstant a t) ^ 2 * (1 + decelerationParameter a t) := by
  rw [deriv_hubbleConstant ha hd_a haz]
  simp only [hubbleConstant, decelerationParameter]
  field_simp
  ring

/-- Pointwise: `∂ₜ H t < 0` iff `-1 < q t` (assuming `a` is twice differentiable at
  and both `a t` and `∂ₜ a t` are nonzero). -/
lemma deriv_hubbleConstant_neg_iff
    {a : Time → ℝ} {t : Time}
    (ha : DifferentiableAt ℝ a t) (hd_a : DifferentiableAt ℝ (∂ₜ a) t)
    (haz : a t ≠ 0) (hd_az : ∂ₜ a t ≠ 0) :
    ∂ₜ (hubbleConstant a) t < 0 ↔ -1 < decelerationParameter a t := by
  have hH : hubbleConstant a t ≠ 0 := hubbleConstant_ne_zero hd_az haz
  have hHsq : 0 < (hubbleConstant a t) ^ 2 :=
    (sq_nonneg _).lt_of_ne (Ne.symm (pow_ne_zero _ hH))
  rw [deriv_hubbleConstant_eq_neg_sq_mul ha hd_a haz hd_az]
  constructor <;> intro h <;> nlinarith

/-- There exists a time at which `∂ₜ H < 0` iff there exists a time with `q > -
1`.

  (The corresponding informal statement was written with `q < -
1`. Since
  `dₜ H = -H² (1 + q)` and `H ≠ 0`, one has `dₜ H < 0 ↔ q > -
1`, so the formal
  statement uses the corrected inequality `-1 < q`.) -/
lemma exists_deriv_hubbleConstant_neg_iff
    {a : Time → ℝ}
    (ha : ∀ t, DifferentiableAt ℝ a t) (hd_a : ∀ t, DifferentiableAt ℝ (∂ₜ a) t)
    (haz : ∀ t, a t ≠ 0) (hd_az : ∀ t, ∂ₜ a t ≠ 0) :
    (∃ t, ∂ₜ (hubbleConstant a) t < 0) ↔ (∃ t, -1 < decelerationParameter a t) :=
  exists_congr fun t => deriv_hubbleConstant_neg_iff (ha t) (hd_a t) (haz t) (hd_az
```

1.26 Physlib/Electromagnetism/Basic.lean

```
public import Physlib.SpaceAndTime.SpaceTime.Basic
TODO "Charge density and current density should be generalized to signed measures,
  https://leanprover.zulipchat.com/#narrow/channel/479953-
Physlib/topic/Maxwell's.20Equations"
```

1.27 Physlib/Electromagnetism/Charge/ChargeUnit.lean

```
set_option backward.isDefEq.respectTransparency false in
```

1.28 Physlib/Electromagnetism/Current/CircularCoil.lean

```
public import Physlib.Meta.TODO.Basic
TODO "Prove that the magnetic field around a circular current loop is as gi
  in the reference https://ntrs.nasa.gov/api/citations/20140002333/download
```

1.29 Physlib/Electromagnetism/Current/InfiniteWire.lean

```
public import Physlib.Electromagnetism.Dynamics.IsExtrema
public import Physlib.SpaceAndTime.Space.Norm
public import Physlib.SpaceAndTime.Space.ConstantSliceDist
public import Physlib.SpaceAndTime.TimeAndSpace.ConstantTimeDist
open Physlib

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  simp only [ContinuousLinearMap.zero_apply, PiLp.zero_apply]
  rw [infiniteWire_vectorPotential _ I, constantTime_distTimeDeriv]
  simp
```

1.30 Physlib/Electromagnetism/Dynamics/Basic.lean

```
public import Physlib.Relativity.SpeedOfLight
public import Mathlib.Data.Real.Sqrt
```

1.31 Physlib/Electromagnetism/Dynamics/CurrentDensity.lean

```
public import Physlib.SpaceAndTime.SpaceTime.TimeSlice
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.32 Physlib/Electromagnetism/Dynamics/Hamiltonian.lean

```
public import Physlib.Electromagnetism.Dynamics.Lagrangian
    lagrangian □ ⟨fun x => A x + x (Sum.inl 0) • v⟩ J x) 0
    lagrangian □ ⟨fun x => A x + v⟩ J x) 0
    kineticTerm □ ⟨fun x => A x + x (Sum.inl 0) • v⟩ x) 0:= by
  have hx : DifferentiableAt ℝ (fun v => kineticTerm □ ⟨fun x => A x + x (Sum.inl 0)
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.33 Physlib/Electromagnetism/Dynamics/IsExtrema.lean

```
public import Physlib.Electromagnetism.Dynamics.Lagrangian
attribute [-simp] Nat.reduceAdd Nat.reduceSucc Fin.isValue

set_option backward.isDefEq.respectTransparency false in
      simp only [one_div, map_smul, map_neg, map_add,
      simp_all only [false_or, ne_eq, one_div, map_smul,
    simp only [EmbeddingLike.map_eq_zero_iff] at h1
set_option maxHeartbeats 600000 in
set_option backward.isDefEq.respectTransparency false in
    IsExtrema □ (Λ • A) (fun x => Λ • J (Λ⁻¹ • x)) ↔
    change tensorDeriv (fun x => toFieldStrength (Λ • A) x) x
    simp only [one_div, map_smul, actionT_smul,
    simp only [one_div, map_smul, map_neg,
set_option backward.isDefEq.respectTransparency false in
set_option maxHeartbeats 600000 in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    simp only [one_div, map_smul, actionT_smul,
    simp only [one_div, map_smul, map_neg,
    simpa only [Fin.isValue, schwartzAction_mul_apply, inv_mul_cancel, map_one,
      ContinuousLinearMap.one_apply, smul_add, actionT_smul, smul_neg] using h (schw
    simp only [Nat.reduceAdd, Nat.succ_eq_add_one, Fin.isValue, one_div, map_smul, m
```

1.34 Physlib/Electromagnetism/Dynamics/KineticTerm.lean

```
public import Physlib.Electromagnetism.Kinematics.MagneticField
public import Physlib.Electromagnetism.Dynamics.Basic
public import Physlib.Mathematics.VariationalCalculus.HasVarGradient
open Tensor ContDiff Physlib
set_option backward.isDefEq.respectTransparency false in
    kineticTerm □ (Λ • A) x = kineticTerm □ A (Λ⁻¹ • x) := by
```

```
    change η μ' μ
    change η (ν') (ν)
      (μ', ν')
      (μ, ν)
    kineticTerm □ ⟨fun _ : SpaceTime d => A₀⟩ = 0 := by
    kineticTerm □ ⟨fun x => A x + A₀⟩ = kineticTerm □ A := by
    kineticTerm □ ⟨fun x => A x + x (Sum.inl 0) • c⟩ x = A.kineticTerm □ x
  (δ (q':=A), ∫ x, kineticTerm □ ⟨q'⟩ x)
  change HasVarGradientAt (fun A' x => ElectromagneticPotential.kineticTerm
set_option backward.isDefEq.respectTransparency false in
    (hA : ContDiff ℝ ∞ A) :
    HasVarGradientAt (fun A => kineticTerm □ ⟨A⟩) (A.gradKineticTerm □) A :
  change HasVarGradientAt (fun A' x => ElectromagneticPotential.kineticTerm
set_option backward.isDefEq.respectTransparency false in
attribute [-simp] Nat.reduceAdd Nat.reduceSucc Fin.isValue in
  rfl
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
attribute [-simp] Nat.reduceAdd Nat.reduceSucc Fin.isValue in
      (fun | 0 => μ | 1 => ν)
    simp only [Basis.repr_reindex, Finsupp.mapDomain_equiv_apply,
    rfl
    rfl
  · simp only [Function.comp_apply]
    simp [ComponentIdx.prod]
  · simp only [Function.comp_apply]
    simp [ComponentIdx.prod]
```

1.35 Physlib/Electromagnetism/Dynamics/Lagrangian.lean

```
public import Physlib.Electromagnetism.Dynamics.CurrentDensity
public import Physlib.Electromagnetism.Dynamics.KineticTerm
public import Physlib.Relativity.Tensors.RealTensor.Vector.MinkowskiProduct
    freeCurrentPotential ⟨fun x => A x + c⟩ J x = freeCurrentPotential A J
    HasVarGradientAt (fun A => freeCurrentPotential ⟨A⟩ J)
  (δ (q':=A), ∫ x, freeCurrentPotential ⟨q'⟩ J x)
    inl_0_inl_0, one_mul, inr_i_inr_i, neg_mul, _root_.neg_smul, Pi.add_app
    lagrangian □ ⟨fun x => A x + c⟩ J x = lagrangian □ A J x - ⟪c, J x⟫ₘ :=
    HasVarGradientAt (fun A => lagrangian □ ⟨A⟩ J)
  (δ (q':=A), ∫ x, lagrangian □ ⟨q'⟩ J x)
    HasVarGradientAt (fun A => lagrangian □ ⟨A⟩ J) (A.gradLagrangian □ J) A
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
attribute [-simp] Nat.reduceAdd Nat.reduceSucc Fin.isValue in
set_option backward.isDefEq.respectTransparency false in
  simp only [Pi.sub_apply, apply_sub, one_div, map_smul,
  simp only [one_div, map_smul, apply_smul,
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
attribute [-simp] Nat.reduceAdd Nat.reduceSucc Fin.isValue in
set_option backward.isDefEq.respectTransparency false in
    (permT id (PermCond.auto) {((1/ □.μ₀ : ℝ) • (distTensorDeriv A.fieldStrength ε)
  simp only [ContinuousLinearMap.coe_sub', Pi.sub_apply, apply_sub, one_div,
    map_smul, map_neg, map_add, permT_permT, CompTriple.comp_eq, apply_add,
  simp only [one_div, map_smul, apply_smul,
```

1.36 Physlib/Electromagnetism/Kinematics/Boosts.lean

```
public import Physlib.Electromagnetism.Kinematics.MagneticField
public import Physlib.SpaceAndTime.SpaceTime.Boosts
    electricField c (Λ • A) t x 0 =
  · fun_prop
set_option backward.isDefEq.respectTransparency false in
    electricField c (Λ • A) t x i.succ =
  · fun_prop
    magneticFieldMatrix c (Λ • A) t x (0, i.succ) =
    magneticFieldMatrix c (Λ • A) t x (i.succ, j.succ) =
```

1.37 Physlib/Electromagnetism/Kinematics/EMPotential.lean

```
public import Physlib.SpaceAndTime.SpaceTime.Derivatives
public import Physlib.Mathematics.VariationalCalculus.HasVarAdjDeriv
public import Physlib.Relativity.Tensors.Elab
  - A.1. Basic instances on the type of electromagnetic potentials
  - A.2. The group action on the ElectromagneticPotential
  - A.3. Differentiability
  - A.4. The action on the space-time derivatives
  - A.5. Variational adjoint derivative of component
  - A.6. Variational adjoint derivative of derivatives of the potential
structure ElectromagneticPotential (d : ℕ := 3) where
  /-- The underlying map from `SpaceTime d` to `Lorentz.Vector d` associated
    with an electromagnetic potential. -/
  val : SpaceTime d → Lorentz.Vector d
```

```
@[ext]
lemma eq_of_val_eq (A B : ElectromagneticPotential d) (h : A.val = B.val) :
  cases A; cases B
  simp at h
  rw [h]

/-!

## A.1. Basic instances on the type of electromagnetic potentials

-/

instance {d} : CoeFun (ElectromagneticPotential d)
    (fun _ => SpaceTime d → Lorentz.Vector d) where
  coe A := A.val

instance {d} : Add (ElectromagneticPotential d) where
  add A B := ⟨fun x => A x + B x⟩

@[simp]
lemma add_val {d} (A B : ElectromagneticPotential d) :
    (A + B).val = A.val + B.val := rfl

lemma add_apply {d} (A B : ElectromagneticPotential d) (x : SpaceTime d) :
    (A + B) x = A x + B x := by simp

noncomputable instance {d} : SMul ℝ (ElectromagneticPotential d) where
  smul r A := ⟨fun x => r • A x⟩

@[simp]
lemma smul_val {d} (r : ℝ) (A : ElectromagneticPotential d) :
    (r • A).val = r • A.val := rfl

lemma smul_apply {d} (r : ℝ) (A : ElectromagneticPotential d) (x : SpaceTim
    (r • A) x = r • A x := by simp

/-!

## A.2. The group action on the ElectromagneticPotential

-/

noncomputable instance {d} : SMul (LorentzGroup d) (ElectromagneticPotentia
  smul Λ A := ⟨fun x => Λ • A (Λ⁻¹ • x)⟩

lemma action_val {d} (Λ : LorentzGroup d) (A : ElectromagneticPotential d)
```

```
    (Λ • A).val = fun x => Λ • A (Λ⁻¹ • x) := rfl

noncomputable instance {d} : MulAction (LorentzGroup d) (ElectromagneticPotential d)
  mul_smul Λ₁ Λ₂ A := by
    ext i
    simp [action_val, mul_smul]
  one_smul A := by
    ext i
    simp [action_val, one_smul]

### A.3. Differentiability

We show that the components of field strength tensor are differentiable if the poten
-/

@[fun_prop]
lemma differentiable_component {d : ℕ}
    (A : ElectromagneticPotential d) (hA : Differentiable ℝ A) (μ : Fin 1 ⊕ Fin d) :
    Differentiable ℝ (fun x => A x μ) := by
  revert μ
  rw [SpaceTime.differentiable_vector]
  exact hA

@[fun_prop]
lemma differentiable_action {d} (Λ : LorentzGroup d) (A : ElectromagneticPotential d
    (hA : Differentiable ℝ A) : Differentiable ℝ (fun x => Λ • A (Λ⁻¹ • x)) := by
  apply Differentiable.comp
  · exact ContinuousLinearMap.differentiable (Lorentz.Vector.actionCLM Λ)
  · apply Differentiable.comp
    · exact hA
    · exact ContinuousLinearMap.differentiable (Lorentz.Vector.actionCLM Λ⁻¹)

/-!

### A.4. The action on the space-time derivatives
`∂_ μ (Λ • A)`, we write an expression for this in terms of the tensor.
set_option backward.isDefEq.respectTransparency false in
    ∂_ μ (Λ • A) x ν = ∑ κ, ∑ ρ, (Λ.1 ν κ * Λ⁻¹.1 ρ μ) * ∂_ ρ A (Λ⁻¹ • x) κ := by
        simp only [action_val]
        fun_prop
      simp only [ContinuousLinearMap.coe_sum', ContinuousLinearMap.coe_smul',
  simp only [smul_eq_mul]
### A.5. Variational adjoint derivative of component
set_option backward.isDefEq.respectTransparency false in
### A.6. Variational adjoint derivative of derivatives of the potential
lemma deriv_eq_tensorDeriv {d} (A : ElectromagneticPotential d)
```

```
    (hA : Differentiable ℝ A) (x : SpaceTime d) :
    A.deriv x = tensorDeriv A.val x := by
  rw [deriv, tensorDeriv_eq_sum_tensor_basis (by fun_prop)]
  /- Match the basis sum. -/
  let e : ComponentIdx (Fin.append ![Color.down] ![Color.up])
      ≃ (Fin 1 ⊕ Fin d) × (Fin 1 ⊕ Fin d) := ComponentIdx.prod.trans <|
    Lorentz.CoVector.indexEquiv.prodCongr Lorentz.Vector.indexEquiv
  rw [← e.symm.sum_comp, Fintype.sum_prod_type]
  /- Getting rid of the sums -/
  refine Finset.sum_congr rfl (fun μ _ => Finset.sum_congr rfl (fun ν _ =>
  congr
  /- The coefficients. -/
  · simp [e]
    rw [deriv_apply_eq _ _ _ (by fun_prop)]
    congr
    simp [Lorentz.Vector.tensor_basis_repr_toTensor_apply]
  /- The basis elements. -/
  · change _ = ((Tensor.basis (S := realLorentzTensor d) (Fin.append ![Colo
      (Tensorial.toTensor (M := (Lorentz.CoVector d) ⊗[ℝ] (Lorentz.Vector d
    rw [Tensorial.basis_map_prod, ← Lorentz.Vector.toTensor_symm_basis,
      ← Lorentz.CoVector.toTensor_symm_basis]
    simp [e]

    (hf : Differentiable ℝ A) : deriv (Λ • A) x = Λ • (deriv A (Λ⁻¹ • x)) :
  rw [deriv_eq_tensorDeriv, deriv_eq_tensorDeriv]
  rw [action_val, tensorDeriv_equivariant]
  all_goals fun_prop
open Tensorial
/-- Evaluation of the tensor components of `∂_ μ A x ν`. -
/
lemma tensorDeriv_eval_eq {d} {A : ElectromagneticPotential d} (hA : Differ
    (x : SpaceTime d) (μ ν : Fin 1 ⊕ Fin d) :
    toField {tensorDeriv A.val x | [μ] [ν]}ᵀ = ∂_ μ A x ν := by
  trans  (Lorentz.CoVector.basis.tensorProduct Lorentz.Vector.basis).repr (
  · simp [deriv, Basis.tensorProduct_repr_tmul_apply, Finsupp.single_apply]
  rw [deriv_eq_tensorDeriv _ hA]
  generalize (tensorDeriv A.val x) = t
  obtain ⟨t, rfl⟩ := toTensor.symm.surjective t
  induction' t using Tensor.induction_on_basis with b a t h t1 t2 h1 h2
  · simp only [LinearEquiv.apply_symm_apply, basis_apply, evalT_pure, Pure.
      toField_pure, smul_eq_mul, mul_one, Pure.evalPCoeff]
    change _ * ((realLorentzTensor d).basis (Color.up)).repr
      ((realLorentzTensor d).basis (Color.up) (b 1)) ν = _
    /- Transforming the basis -/
    let e : ComponentIdx (Fin.append ![Color.down] ![Color.up])
      ≃ (Fin 1 ⊕ Fin d) × (Fin 1 ⊕ Fin d) := ComponentIdx.prod.trans <|
```

```
      Lorentz.CoVector.indexEquiv.prodCongr Lorentz.Vector.indexEquiv
    have h1 :  Lorentz.CoVector.basis.tensorProduct Lorentz.Vector.basis =
        (((Tensor.basis (Fin.append ![Color.down] ![Color.up]))).map toTensor.symm).
      ext ⟨i, j⟩
      simp_rw [Tensorial.basis_map_prod, Basis.tensorProduct_apply,
        ← Lorentz.Vector.toTensor_symm_basis, ← Lorentz.CoVector.toTensor_symm_basis
      simp
    simp [Pure.basisVector, h1, Finsupp.single_apply]
    grind [show e b = (b 0,  b 1) from rfl]
  · simp only [map_zero, Finsupp.coe_zero, Pi.zero_apply]
  · simp only [map_smul, h, smul_eq_mul, Finsupp.coe_smul, Pi.smul_apply]
  · simp only [map_add, h1, h2, Finsupp.coe_add, Pi.add_apply]

  rcases μν  with ⟨μ, ν⟩
  simp [deriv, Basis.tensorProduct_repr_tmul_apply, Finsupp.single_apply]
    ∂_ (b 0) A x (b 1) := by
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    distDeriv (b 0) A ε (b 1) := by
set_option backward.isDefEq.respectTransparency false in
```

1.38 Physlib/Electromagnetism/Kinematics/ElectricField.lean

```
public import Physlib.Electromagnetism.Kinematics.VectorPotential
public import Physlib.Electromagnetism.Kinematics.ScalarPotential
public import Physlib.Electromagnetism.Kinematics.FieldStrength
public import Physlib.Electromagnetism.Basic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.39 Physlib/Electromagnetism/Kinematics/FieldStrength.lean

```
public import Physlib.Electromagnetism.Kinematics.EMPotential
public import Physlib.Relativity.Tensors.RealTensor.Metrics.Basic
public import Mathlib.Data.Real.Hom
lemma toFieldStrength_eq_deriv {d} (A : ElectromagneticPotential d) (x : SpaceTime d
    toFieldStrength A x =
    Tensorial.toTensor.symm (permT id PermCond.auto {(η d | μ μ' ⊗ A.deriv x | μ' ν)
    + - (η d | ν ν' ⊗ A.deriv x | ν' μ)}ᵀ) := by
  rw [toFieldStrength]

lemma toFieldStrength_eq_tensorDeriv {d} {A : ElectromagneticPotential d}
    (hA : Differentiable ℝ A) (x : SpaceTime d) :
```

```
    toFieldStrength A x =
    Tensorial.toTensor.symm (permT id PermCond.auto {(η d | μ μ' ⊗ tensorDe
    + - (η d | ν ν' ⊗ tensorDeriv A x | ν' μ)}ᵀ) := by
  rw [toFieldStrength_eq_deriv, deriv_eq_tensorDeriv _ hA]

set_option backward.isDefEq.respectTransparency false in
    ∑ κ, (η (b 0) κ * ∂_ κ A x (b 1) - η (b 1) κ * ∂_ κ A x (b 0)) := by
    change η (b 0) (n)
    change ∂_ (n) A x (b 1)
    change η (b 1) (n)
    change ∂_ (n) A x (b 0)
      (b 0, b 1) := by
    (fun | 0 => μ | 1 => ν); swap
  rfl
    (fun | 0 => μ | 1 => ν) := by
set_option backward.isDefEq.respectTransparency false in
  simp only [Fin.isValue, neg_sub]
set_option backward.isDefEq.respectTransparency false in
    toFieldStrength (Λ • A) x = Λ • toFieldStrength A (Λ⁻¹ • x) := by
set_option backward.isDefEq.respectTransparency false in
    fieldStrengthMatrix (Λ • A) x (μ, ν) =
set_option backward.isDefEq.respectTransparency false in
  simp only [add_val]
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  simp only [smul_val]
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    ∑ κ, (η (b 0) κ * SpaceTime.distDeriv κ A ε (b 1) -
      η (b 1) κ * SpaceTime.distDeriv κ A ε (b 0)) := by
    change η (b 0) n
    change distDeriv n A ε (b 1)
    change η (b 1) n
    change distDeriv n A ε (b 0)
      (b 0, b 1) := by
    (fun | 0 => μ | 1 => ν); swap
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.40 Physlib/Electromagnetism/Kinematics/MagneticField.lean

```
public import Physlib.Electromagnetism.Kinematics.ElectricField
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    Basis.tensorProduct_repr_tmul_apply, PiLp.basisFun_repr, PiLp.single_apply,
```

1.41 Physlib/Electromagnetism/Kinematics/ScalarPotential.lean

```
public import Physlib.Electromagnetism.Kinematics.EMPotential
public import Physlib.SpaceAndTime.SpaceTime.TimeSlice
public import Mathlib.Data.Real.Hom
@[fun_prop]
open ContDiff

@[fun_prop]
lemma scalarPotential_contDiff_space_of_smooth {n : ℕ} {d} (c : SpeedOfLight)
    (A : ElectromagneticPotential d)
    (hA : ContDiff ℝ ∞ A) (t : Time) : ContDiff ℝ n (A.scalarPotential c t) := by
  apply scalarPotential_contDiff_space
  exact hA.of_le (ENat.LEInfty.out)

set_option backward.isDefEq.respectTransparency false in
```

1.42 Physlib/Electromagnetism/Kinematics/VectorPotential.lean

```
public import Physlib.Electromagnetism.Kinematics.EMPotential
public import Physlib.SpaceAndTime.SpaceTime.TimeSlice
public import Mathlib.Data.Real.Hom
@[fun_prop]
open ContDiff
@[fun_prop]
lemma vectorPotential_contDiff_of_smooth {n : ℕ} {d} {c : SpeedOfLight}
    (A : ElectromagneticPotential d) (hA : ContDiff ℝ ∞ A) :
    ContDiff ℝ n ₁(A.vectorPotential c) := by
  apply vectorPotential_contDiff
  exact hA.of_le (ENat.LEInfty.out)

set_option backward.isDefEq.respectTransparency false in
```

1.43 Physlib/Electromagnetism/PointParticle/OneDimension.lean

```
public import Physlib.Electromagnetism.Dynamics.IsExtrema
public import Physlib.SpaceAndTime.Space.Norm
public import Physlib.SpaceAndTime.Space.Translations
```

```
public import Physlib.SpaceAndTime.TimeAndSpace.ConstantTimeDist
open Physlib

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  rw [oneDimPointParticle_electricField, map_smul, constantTime_distTimeDer
  module
set_option backward.isDefEq.respectTransparency false in
```

1.44 Physlib/Electromagnetism/PointParticle/ThreeDimension.lean

```
public import Physlib.Electromagnetism.Dynamics.IsExtrema
public import Physlib.SpaceAndTime.Space.Norm
public import Physlib.SpaceAndTime.Space.Translations
public import Physlib.SpaceAndTime.TimeAndSpace.ConstantTimeDist
open Physlib

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  rw [threeDimPointParticle_electricField, map_smul, constantTime_distTimeD
  module
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.45 Physlib/Electromagnetism/ThreeDimension/Basic.lean

```
public import Physlib.Electromagnetism.Kinematics.MagneticField
```

1.46 Physlib/Electromagnetism/ThreeDimension/MaxwellEquations.l

```
/-
Copyright (c) 2026 Zhi Kai Pong. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Zhi Kai Pong
-/
module

public import Physlib.Electromagnetism.ThreeDimension.Basic
```

```
public import Physlib.Electromagnetism.Dynamics.IsExtrema
/-!

# Maxwell equations in three dimensions

This file states Maxwell's equations in the familiar vector-
calculus language
and connects them to the tensorial backend formulation.

-/
namespace Electromagnetism
namespace ThreeDimension

open Time
open Space
open ElectromagneticPotential
open ContDiff

variable {𝓒 : FreeSpace} (V : ElectromagneticPotential 3) (J₄ : LorentzCurrentDensit

local notation "φ" => V.scalarPotential 𝓒.c
local notation "A" => V.vectorPotential 𝓒.c
local notation "E" => V.electricField 𝓒.c
local notation "B" => V.magneticField 𝓒.c
local notation "ρ" => J₄.chargeDensity 𝓒.c
local notation "J" => J₄.currentDensity 𝓒.c
local notation "ε₀" => 𝓒.ε₀
local notation "μ₀" => 𝓒.μ₀

/-- Gauss's law for the electric field. -/
theorem gaussLawElectric (t : Time) (x : Space)
    (h : IsExtrema 𝓒 V J₄) (hV : ContDiff ℝ ∞ V) (hJ : ContDiff ℝ ∞ J₄) :
    (∇ ⬝ E t) x = ρ t x / ε₀ := by
  rw [((isExtrema_iff_gauss_ampere_magneticFieldMatrix hV J₄ hJ (𝓒 := 𝓒)).mp h t x).

/-- Gauss's law for the magnetic field. -/
theorem gaussLawMagnetic (t : Time) (x : Space) (hV : ContDiff ℝ ∞ V) :
    (∇ ⬝ B t) x = 0 := by
  rw [magneticField_eq_3D, div_of_curl_eq_zero _ (by fun_prop)]
  simp only [Pi.zero_apply]

/-- Ampère's law. -/
theorem ampereLaw (t : Time) (x : Space)
    (h : IsExtrema 𝓒 V J₄) (hV : ContDiff ℝ ∞ V) (hJ : ContDiff ℝ ∞ J₄) :
      (∇ × B t) x = μ₀ • J t x + μ₀ • ε₀ • ∂ₜ (fun t => E t x) t := by
  ext i
```

```
  have hdE := ((isExtrema_iff_gauss_ampere_magneticFieldMatrix hV J₄ hJ (□
  rw [← magneticField_curl_eq_magneticFieldMatrix] at hdE
  simp only [PiLp.add_apply, PiLp.smul_apply, smul_eq_mul, ← mul_assoc, hdE
  exact hV.of_le ENat.LEInfty.out

/-- Faraday's law. -/
theorem faradayLaw (t : Time) (x : Space) (hV : ContDiff ℝ ∞ V) :
    (∇ × E t) x = - ∂ₜ (fun t => B t x) t := by
  rw [electricField_eq_3D, magneticField_eq_3D]
  change curl ((-(fun t x => ∇ (scalarPotential □.c V t) x) t -
      (fun t x => ∂ₜ (fun t => vectorPotential □.c V t x) t) t)) x = _
  rw [curl_sub, curl_neg, curl_of_grad_eq_zero, time_deriv_curl_commute]
  simp only [neg_zero, Pi.sub_apply, Pi.zero_apply, zero_sub]
  all_goals fun_prop

end ThreeDimension
end Electromagnetism
```

1.47 Physlib/Electromagnetism/Vacuum/Constant.lean

```
public import Physlib.Electromagnetism.Dynamics.IsExtrema
    (B₀_antisymm : ∀ i j, B₀ (i, j) = - B₀ (j, i)) : ElectromagneticPotenti
  val := fun x μ =>
set_option backward.isDefEq.respectTransparency false in
```

1.48 Physlib/Electromagnetism/Vacuum/HarmonicWave.lean

```
public import Physlib.Electromagnetism.Vacuum.IsPlaneWave

  (φ : Fin d → ℝ) : ElectromagneticPotential d.succ where
  val := fun x μ =>
@[simp]
lemma harmonicWaveX_inl_zero {d} (□ : FreeSpace) (k : ℝ) (E₀ : Fin d → ℝ) (
    (x : SpaceTime d.succ) :
    harmonicWaveX □ k E₀ φ x (Sum.inl 0) = 0 := by
  simp [harmonicWaveX]

@[simp]
lemma harmonicWaveX_inr_zero {d} (□ : FreeSpace) (k : ℝ) (E₀ : Fin d → ℝ) (
    (x : SpaceTime d.succ) :
    harmonicWaveX □ k E₀ φ x (Sum.inr 0) = 0 := by
  simp [harmonicWaveX]
  rfl
```

```
  | Sum.inl 0 => simp
  | Sum.inr ⟨0, h⟩ => simp
```

1.49 Physlib/Electromagnetism/Vacuum/IsPlaneWave.lean

```
public import Physlib.ClassicalMechanics.WaveEquation.Basic
public import Physlib.Electromagnetism.Dynamics.IsExtrema
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.50 Physlib/Mathematics/Calculus/AdjFDeriv.lean

```
public import Physlib.Mathematics.FDerivCurry
public import Physlib.Mathematics.InnerProductSpace.Adjoint
public import Physlib.Mathematics.InnerProductSpace.Calculus
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
open InnerProductSpace in
open InnerProductSpace in
```

1.51 Physlib/Mathematics/Calculus/Divergence.lean

```
public import Mathlib.LinearAlgebra.Trace
public import Physlib.Mathematics.Calculus.AdjFDeriv
public import Physlib.SpaceAndTime.Space.Derivatives.Div
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.52 Physlib/Mathematics/DataStructures/FourTree/Basic.lean

```
namespace Physlib
end Physlib
```

1.53 Physlib/Mathematics/DataStructures/FourTree/UniqueMap.lean

```
public import Physlib.Mathematics.DataStructures.FourTree.Basic
```

```
namespace Physlib
end Physlib
```

1.54 Physlib/Mathematics/DataStructures/Matrix/LieTrace.lean

```
public import Physlib.Mathematics.SchurTriangulation
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.55 Physlib/Mathematics/Distribution/Basic.lean

```
public import Physlib.Meta.TODO.Basic
public import Mathlib.Analysis.Distribution.SchwartzSpace.Fourier
public import Mathlib.Topology.Algebra.Module.PointwiseConvergence
- In this file we will define distributions generally, in `Physlib.SpaceAnd
/- Need the `Physlib` namespace to prevent conflict with Mathlib's distribu
/
namespace Physlib

set_option backward.isDefEq.respectTransparency false in
TODO "For distributions, prove that the derivative fderivD commutes with
set_option backward.isDefEq.respectTransparency false in
  · fun_prop
open ContinuousLinearMap
set_option backward.isDefEq.respectTransparency false in
end Physlib
```

1.56 Physlib/Mathematics/Distribution/PowMul.lean

```
public import Mathlib.Analysis.Distribution.SchwartzSpace.Basic
namespace Physlib
        rw [iteratedFDeriv_fun_zero]
set_option backward.isDefEq.respectTransparency false in
end Physlib
```

1.57 Physlib/Mathematics/Fin.lean

```
public import Mathlib.Algebra.Order.Group.Nat
public import Mathlib.Algebra.Order.Monoid.NatCast
namespace Physlib.Fin
end Physlib.Fin
```

1.58 Physlib/Mathematics/Fin/Involutions.lean

```
public import Mathlib.Data.Nat.Factorial.DoubleFactorial
namespace Physlib.Fin
set_option backward.isDefEq.respectTransparency false in
    ring
end Physlib.Fin
```

1.59 Physlib/Mathematics/Geometry/Metric/PseudoRiemannian/Defs.lean

```
set_option backward.isDefEq.respectTransparency false in
```

1.60 Physlib/Mathematics/Geometry/Metric/Riemannian/Defs.lean

```
public import Physlib.Mathematics.Geometry.Metric.PseudoRiemannian.Defs
@[reducible]
@[reducible]
@[reducible]
@[reducible]
```

1.61 Physlib/Mathematics/InnerProductSpace/Adjoint.lean

```
public import Physlib.Mathematics.InnerProductSpace.Basic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.62 Physlib/Mathematics/InnerProductSpace/Basic.lean

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

```
      one_div, prod_inner_apply', map_add]
set_option backward.isDefEq.respectTransparency false in
    simp only [WithLp.equiv_apply, PiLp.inner_apply, inner_self_eq_norm_sq_
      re_ofReal_pow]
          conv_rhs => rw [inner]
          simp [PiLp.inner_apply]
          conv_lhs => rw [inner]
          simp only [PiLp.inner_apply, inner_self_eq_norm_sq_to_K, map_sum,
          exact InnerProductSpace.Core.inner_self_nonneg
      conv_rhs => rw [inner]
      simp [PiLp.inner_apply]
```

1.63 Physlib/Mathematics/InnerProductSpace/Calculus.lean

```
public import Physlib.Mathematics.InnerProductSpace.Basic
```

1.64 Physlib/Mathematics/KroneckerDelta.lean

```
set_option backward.isDefEq.respectTransparency false in
```

1.65 Physlib/Mathematics/LinearMaps.lean

```
public import Physlib.Meta.TODO.Basic
TODO "Replace the definitions of bi-linear maps in `./Mathematics/LinaerMap
 with definitions from Mathlib."
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.66 Physlib/Mathematics/List.lean

```
public import Physlib.Mathematics.Fin
namespace Physlib.List
open Physlib
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    (List.insertionSort le1 r).eraseIdx ↑((Physlib.List.insertionSortEquiv le1 r) n)
end Physlib.List
```

1.67 Physlib/Mathematics/List/InsertIdx.lean

```
public import Mathlib.Algebra.GroupWithZero.Nat
public import Mathlib.Algebra.Order.Group.Nat
public import Mathlib.Algebra.Order.ZeroLEOne
public import Mathlib.Data.Fin.SuccPred
namespace Physlib.List
open Physlib
set_option backward.isDefEq.respectTransparency false in
end Physlib.List
```

1.68 Physlib/Mathematics/List/InsertionSort.lean

```
public import Physlib.Mathematics.List
import all Physlib.Mathematics.List
namespace Physlib.List
open Physlib
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
end Physlib.List
```

1.69 Physlib/Mathematics/PiTensorProduct.lean

```
namespace Physlib.PiTensorProduct
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
end Physlib.PiTensorProduct
```

1.70 Physlib/Mathematics/RatComplexNum.lean

```
namespace Physlib
set_option backward.isDefEq.respectTransparency false in
```

```
end Physlib
```

1.71 Physlib/Mathematics/SO3/Basic.lean

```
set_option backward.isDefEq.respectTransparency false in
```

1.72 Physlib/Mathematics/SchurTriangulation.lean

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.73 Physlib/Mathematics/SpecialFunctions/PhysHermite.lean

```
namespace Physlib
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
noncomputable instance : CoeFun (Polynomial ℤ) (fun _ ↦ ℝ → ℝ)where
set_option backward.isDefEq.respectTransparency false in
  simp [physHermite_succ', aevalEquiv, aeval, mul_assoc]
set_option backward.isDefEq.respectTransparency false in
  simp [physHermite_succ', aevalEquiv, aeval, mul_assoc]
set_option backward.isDefEq.respectTransparency false in
  simp [aeval, aevalEquiv, mul_assoc]
set_option backward.isDefEq.respectTransparency false in
  simp only [aeval, aevalEquiv, Equiv.coe_fn_mk, eval₂AlgHom_apply, Algebra
    algebraMap_int_eq, eval₂_ofNat]
    simp [physHermite_one, aeval, aevalEquiv]
set_option backward.isDefEq.respectTransparency false in
      simp only [aeval, aevalEquiv, Equiv.coe_fn_mk, eval₂AlgHom_apply, Alg
        algebraMap_int_eq, eval₂_ofNat]
set_option backward.isDefEq.respectTransparency false in
    simp only [neg_mul, mul_assoc, Real.rpow_natCast, Pi.smul_apply, smul_e
      · exact fun _ _ ↦ DifferentiableAt.hasDerivAt (deriv_physHermite_diff
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
      simp [aeval, aevalEquiv]
end Physlib
```

1.74 Physlib/Mathematics/Trigonometry/Tanh.lean

```
public import Mathlib.Analysis.Calculus.Deriv.Polynomial
public import Mathlib.Analysis.Distribution.TemperateGrowth
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.75 Physlib/Mathematics/VariationalCalculus/Basic.lean

```
public import Physlib.Mathematics.VariationalCalculus.IsTestFunction
public import Mathlib.Analysis.Calculus.BumpFunction.InnerProduct
The variational calculus API in Physlib is designed to match and formalize the physi
$$S[u] = \int φ(t) L(u, t) dt.$$
- `L` and `Gᵤ` by `HasVarAdjDeriv`.
- https://leanprover.zulipchat.com/#narrow/channel/479953-
Physlib/topic/Variational.20Calculus/with/529022834
set_option backward.isDefEq.respectTransparency false in
      have hclose : ‖f₂ x - x₂‖ < ‖x₂‖ / 2 := by
        convert hδ₂ hx using 1
        exact mem_sphere_iff_norm.mp rfl
```

1.76 Physlib/Mathematics/VariationalCalculus/HasVarAdjDeriv.lean

```
public import Physlib.Mathematics.VariationalCalculus.HasVarAdjoint
set_option backward.isDefEq.respectTransparency false in
```

1.77 Physlib/Mathematics/VariationalCalculus/HasVarAdjoint.lean

```
public import Physlib.Mathematics.VariationalCalculus.IsLocalizedfunctionTransform
public import Physlib.Mathematics.VariationalCalculus.Basic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
            fun_prop) (by fun_prop) (fun _ _ => by
            apply Differentiable.differentiableAt
            fun_prop)
            · intro _ _
              apply Differentiable.differentiableAt
              change Differentiable ℝ fun y => f' i (ψ y)
              fun_prop
            · intro _ _
              apply Differentiable.differentiableAt
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.78 Physlib/Mathematics/VariationalCalculus/HasVarGradient.lea

```
public import Physlib.Mathematics.VariationalCalculus.HasVarAdjDeriv
```

1.79 Physlib/Mathematics/VariationalCalculus/IsLocalizedfunctio

```
public import Physlib.SpaceAndTime.Space.Derivatives.Div
public import Physlib.Mathematics.Calculus.AdjFDeriv
```

1.80 Physlib/Mathematics/VariationalCalculus/IsTestFunction.lea

```
public import Physlib.Mathematics.Calculus.Divergence
public import Mathlib.Topology.ContinuousMap.CompactlySupported
```

1.81 Physlib/Meta/AllFilePaths.lean

```
public import Physlib.Meta.TODO.Basic
## Getting an array of all file paths in Physlib.
/-- Gets an array of all file paths in `Physlib`. -/
  allFilePaths.go (#[] : Array FilePath) "./Physlib" ("./Physlib" : FilePat
/-- Gets an array of all module names in `Physlib`. -/
def allPhyslibModules : IO (Array Lean.Name) := do
    let relativePath := path.toString.replace "./Physlib/" "Physlib."
```

1.82 Physlib/Meta/Basic.lean

```
/-- Gets all imports within Physlib. -/
def Physlib.allImports : IO (Array Import) := do
  let mods := `Physlib
  let (physlibMod, _) ← readModuleData mFile
  physlibMod.imports.filterM fun c => return c.module != `Init
/-- Number of files within Physlib. -/
def Physlib.noImports : IO Nat := do
def Physlib.Imports.getConsts (imp : Import) : IO (Array ConstantInfo) := d
```

```
def Physlib.Imports.getUserConsts (imp : Import) : CoreM (Array ConstantInfo) := do
def Physlib.Imports.getLines (imp : Import) : IO (Array String) := do
  s!"https://github.com/leanprover-community/physlib/blob/master/{c.toFilePath}#L{li
  | some (.inr _) => return "" -- Verso docstring, ignore for now
namespace Physlib
  let imports ← Physlib.allImports
  let x ← imports.mapM Physlib.Imports.getUserConsts
  let imports ← Physlib.allImports
  let x ← imports.mapM Physlib.Imports.getUserConsts
/-- All docstrings present in Physlib. -/
  let imports ← Physlib.allImports
  let x ← imports.mapM Physlib.Imports.getUserConsts
  let imports ← Physlib.allImports
  let x ← imports.mapM Physlib.Imports.getUserConsts
  let imports ← Physlib.allImports
  let x ← imports.mapM Physlib.Imports.getUserConsts
/-- The number of lines in Physlib. -/
  let imports ← Physlib.allImports
  let x ← imports.mapM Physlib.Imports.getLines
  let imports ← Physlib.allImports
  let x ← imports.mapM Physlib.Imports.getLines
  let imports ← Physlib.allImports
  let x ← imports.mapM Physlib.Imports.getLines
  let imports ← Physlib.allImports
  return (← imports.flatMapM Physlib.Imports.getUserConsts)
end Physlib
```

1.83 Physlib/Meta/Informal/Basic.lean

```
`note_attr_informal` from `Physlib.Meta.Notes.Basic` to mark the informal definition
`note_attr_informal` from `Physlib.Meta.Notes.Basic` to mark the informal definition
```

1.84 Physlib/Meta/Informal/Post.lean

```
public import Physlib.Meta.Basic
public import Physlib.Meta.Informal.Basic
public import Physlib.Meta.TODO.Basic
namespace Physlib
/-- The number of informal lemmas in Physlib. -/
/-- The number of informal definitions in Physlib. -/
end Physlib
```

1.85 Physlib/Meta/Informal/SemiFormal.lean

```
they appear in Physlib's TODO list and must be tagged accordingly.
```

1.86 Physlib/Meta/Linters/Sorry.lean

```
public meta import Physlib.Meta.Basic
```

1.87 Physlib/Meta/Notes/Basic.lean

```
namespace Physlib
end Physlib
```

1.88 Physlib/Meta/Notes/HTMLNote.lean

```
public import Physlib.Meta.Basic
public import Physlib.Meta.Notes.Basic
public import Physlib.Meta.Informal.Basic
namespace Physlib
end Physlib
```

1.89 Physlib/Meta/Notes/NoteFile.lean

```
public import Lean.Setup
namespace Physlib
end Physlib
```

1.90 Physlib/Meta/Notes/ToHTML.lean

```
public meta import Physlib.Meta.Notes.HTMLNote
public import Physlib.Meta.Notes.NoteFile
namespace Physlib
  <a href=\"https://github.com/leanprover-community/physlib\">Physlib</a>,
  <a href=\"https://github.com/leanprover-community/physlib/issues\">here</
end Physlib
```

1.91 Physlib/Meta/Remark/Basic.lean

```
namespace Physlib
end Physlib
```

1.92 Physlib/Meta/Remark/Properties.lean

```
public import Physlib.Meta.Remark.Basic
namespace Physlib
end Physlib
```

1.93 Physlib/Meta/Sorry.lean

```
public import Physlib.Meta.Linters.Sorry

namespace Physlib
        if v.name == ``Lean.ofReduceBool || (toString v.name).contains "native_decid
  let (allWithSorry, allWithPseudo) ← Physlib.allWithSorryPseudo
  let allConst ← Physlib.allUserConsts
  The following names depend on `Lean.ofReduceBool` or `native_decide`
  but are not attributed `pseudo`:
  The following names are attributed `pseudo` but do not depend on `Lean.ofReduceBoo
  or `native_decide`:
end Physlib
```

1.94 Physlib/Meta/TODO/Basic.lean

```
namespace Physlib
syntax (name := todo_comment) "TODO " str : command
  | `(TODO $s) => do
    let tag : String := toString (String.hash str)
      Elab.Command.liftTermElabM <| Lean.Elab.Term.addTermInfo' s
        (Lean.mkStrLit s!"TODO tag: {tag}") (expectedType? := none)
end Physlib
```

1.95 Physlib/Meta/TODO/Global.lean

```
/-
Copyright (c) 2026 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license.
Authors: Joseph Tooby-Smith
```

```
-/
module

public import Physlib.Meta.TODO.Basic
/-!

# Global TODO items

This module contains global TODO items, which are not specific to any parti
but are still important for the development of Physlib.

-/

@[expose] public section


/-!

## Style

-/

TODO "Ensure that all the lines in QuantumInfo are below 100 characters lon

TODO "Remove all instances of `erw` within Physlib. This usually
  indicates the need for better API."
```

1.96 Physlib/Meta/TransverseTactics.lean

```
public import Physlib.Meta.TODO.Basic
TODO "Find a way to free the environment `env` in `transverseTactics`.
```

1.97 Physlib/Optics/Polarization/Basic.lean

```
`Physlib.Electromagnetism.Vacuum.HarmonicWave` for better organization.
```

1.98 Physlib/Particles/BeyondTheStandardModel/GeorgiGlashow/Bas

```
public import Physlib.Particles.StandardModel.Basic
```

1.99 Physlib/Particles/BeyondTheStandardModel/PatiSalam/Basic.lean

```
public import Physlib.Particles.StandardModel.Basic
```

1.100 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/

```
public import Physlib.QFT.AnomalyCancellation.Basic
```

1.101 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/

```
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Basic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  simp only [SMνSpecies_numberCharges, Function.comp_apply, toSpecies_apply, toSpeci
    Fin.zero_eta, Fin.isValue, Nat.reduceMul]
set_option backward.isDefEq.respectTransparency false in
  simp only [SMνSpecies_numberCharges, toSpecies_apply, toSpeciesEquiv_apply, Fin.ze
    Fin.isValue, Nat.reduceMul]
set_option backward.isDefEq.respectTransparency false in
  simp only [SMνSpecies_numberCharges, toSpecies_apply, toSpeciesEquiv_apply, Fin.ze
    Fin.isValue, Nat.reduceMul]
    simp only [SMνSpecies_numberCharges, Fin.zero_eta, Fin.isValue, toSpecies_apply,
      toSpeciesEquiv_apply, Nat.reduceMul]
```

1.102 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/

```
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Permu
public import Physlib.QFT.AnomalyCancellation.GroupActions
```

1.103 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/

```
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Permu
public import Physlib.QFT.AnomalyCancellation.GroupActions
```

1.104 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/

```
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Ordin
set_option backward.isDefEq.respectTransparency false in
  simp only [B₀, Equiv.invFun_as_coe, cubeTriLin_toFun_apply_apply, Fin.isValue,
```

```
    toSpeciesEquiv_apply, Nat.reduceMul, finProdFinEquiv, Fin.divNat, Fin.m
    Fin.coe_ofNat_eq_mod, Nat.zero_mod, mul_zero, add_zero, toSpeciesEquiv_
    mul_one, Nat.ofNat_pos, Nat.add_div_right, Nat.add_mod_right, zero_mul,
    Nat.mod_succ, Fin.sum_univ_three, Fin.zero_eta, one_mul, Fin.mk_one, ne
    Fin.reduceFinMk]
set_option backward.isDefEq.respectTransparency false in
  simp only [B₁, Equiv.invFun_as_coe, cubeTriLin_toFun_apply_apply, Fin.isV
    toSpeciesEquiv_apply, Nat.reduceMul, finProdFinEquiv, Fin.divNat, Fin.m
    Fin.coe_ofNat_eq_mod, Nat.zero_mod, mul_zero, add_zero, toSpeciesEquiv_
    mul_one, Nat.ofNat_pos, Nat.add_div_right, Nat.add_mod_right, Nat.reduc
    Fin.sum_univ_three, Fin.zero_eta, zero_mul, zero_add, Fin.reduceFinMk,
    Nat.reduceAdd, neg_mul, mul_neg]
set_option backward.isDefEq.respectTransparency false in
  simp only [B₂, Equiv.invFun_as_coe, cubeTriLin_toFun_apply_apply, Fin.isV
    toSpeciesEquiv_apply, Nat.reduceMul, finProdFinEquiv, Fin.divNat, Fin.m
    Fin.coe_ofNat_eq_mod, Nat.zero_mod, mul_zero, add_zero, toSpeciesEquiv_
    mul_one, Nat.ofNat_pos, Nat.add_div_right, Nat.add_mod_right, Nat.reduc
    Fin.sum_univ_three, Fin.zero_eta, zero_mul, zero_add, Fin.reduceFinMk,
    Nat.reduceAdd, neg_mul, mul_neg]
set_option backward.isDefEq.respectTransparency false in
  simp only [B₃, Equiv.invFun_as_coe, cubeTriLin_toFun_apply_apply, Fin.isV
    toSpeciesEquiv_apply, Nat.reduceMul, finProdFinEquiv, Fin.divNat, Fin.m
    Fin.coe_ofNat_eq_mod, Nat.zero_mod, mul_zero, add_zero, toSpeciesEquiv_
    mul_one, Nat.ofNat_pos, Nat.add_div_right, Nat.add_mod_right, Nat.reduc
    Fin.sum_univ_three, Fin.zero_eta, zero_mul, zero_add, Fin.reduceFinMk,
    Nat.reduceAdd, neg_mul, mul_neg]
set_option backward.isDefEq.respectTransparency false in
  simp only [B₄, Equiv.invFun_as_coe, cubeTriLin_toFun_apply_apply, Fin.isV
    toSpeciesEquiv_apply, Nat.reduceMul, finProdFinEquiv, Fin.divNat, Fin.m
    Fin.coe_ofNat_eq_mod, Nat.zero_mod, mul_zero, add_zero, toSpeciesEquiv_
    mul_one, Nat.ofNat_pos, Nat.add_div_right, Nat.add_mod_right, Nat.reduc
    Fin.sum_univ_three, Fin.zero_eta, zero_mul, zero_add, Fin.reduceFinMk,
    Nat.reduceAdd, neg_mul]
set_option backward.isDefEq.respectTransparency false in
  simp only [B₅, Equiv.invFun_as_coe, cubeTriLin_toFun_apply_apply, Fin.isV
    toSpeciesEquiv_apply, Nat.reduceMul, finProdFinEquiv, Fin.divNat, Fin.m
    Fin.coe_ofNat_eq_mod, Nat.zero_mod, mul_zero, add_zero, toSpeciesEquiv_
    mul_one, Nat.ofNat_pos, Nat.add_div_right, Nat.add_mod_right, Nat.reduc
    Fin.sum_univ_three, Fin.zero_eta, zero_mul, zero_add, Fin.reduceFinMk,
    Nat.reduceAdd, neg_mul]
set_option backward.isDefEq.respectTransparency false in
  simp only [B₆, Equiv.invFun_as_coe, cubeTriLin_toFun_apply_apply, Fin.isV
    toSpeciesEquiv_apply, Nat.reduceMul, finProdFinEquiv, Fin.divNat, Fin.m
    Fin.coe_ofNat_eq_mod, Nat.zero_mod, mul_zero, add_zero, toSpeciesEquiv_
    mul_one, Nat.ofNat_pos, Nat.add_div_right, Nat.add_mod_right, Nat.reduc
    Fin.sum_univ_three, Fin.zero_eta, zero_mul, zero_add, Fin.reduceFinMk,
```

```
    Nat.reduceAdd, one_mul, neg_mul, mul_neg]
TODO "Remove the definitions of elements `(SM 3).Charges` B₀, B₁ etc, here are
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.105 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/

```
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Ordin
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Famil
```

1.106 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/

```
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Basic
    simp only [PermGroup]
    simp only [chargeMap_apply, Pi.inv_apply, Module.End.mul_apply]
```

1.107 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/

```
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.PlusU
    mul_one, neg_mul, sub_neg_eq_add]
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.108 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/

```
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Permu
public import Physlib.QFT.AnomalyCancellation.GroupActions
```

1.109 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/

```
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.PlusU
set_option backward.isDefEq.respectTransparency false in
```

1.110 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/

```
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.PlusU
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Famil
```

1.111 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCance

```
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.112 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCance

```
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.113 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCance

```
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
set_option backward.isDefEq.respectTransparency false in
    OfNat.ofNat_ne_zero, false_or, α₂, HomogeneousQuadratic.eq_1, accQuad]
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.114 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCance

```
public import Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
set_option backward.isDefEq.respectTransparency false in
```

1.115 Physlib/Particles/BeyondTheStandardModel/Spin10/Basic.lea

```
public import Physlib.Particles.BeyondTheStandardModel.PatiSalam.Basic
public import Physlib.Particles.BeyondTheStandardModel.GeorgiGlashow.Basic
```

1.116 Physlib/Particles/BeyondTheStandardModel/TwoHDM/Basic.lea

```
public import Physlib.Particles.StandardModel.HiggsBoson.Basic
```

1.117 Physlib/Particles/BeyondTheStandardModel/TwoHDM/GramMatrix.lean

```
public import Physlib.Particles.BeyondTheStandardModel.TwoHDM.Basic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.118 Physlib/Particles/BeyondTheStandardModel/TwoHDM/Potential.lean

```
public import Physlib.Particles.BeyondTheStandardModel.TwoHDM.GramMatrix
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  suffices hs : ∀ x, x ^ 2 ≤ c/b from not_lt_of_ge (hs √(|c/b| + 1)) <| by
    rw [sq_sqrt (by positivity)]
    grind
      (by rw [sqrt_sq_eq_abs]; grind)) (hs (|x| + |d| + 1) (by positivity))
set_option backward.isDefEq.respectTransparency false in
```

1.119 Physlib/Particles/FlavorPhysics/CKMMatrix/Invariants.lean

```
public import Physlib.Particles.FlavorPhysics.CKMMatrix.Basic
```

1.120 Physlib/Particles/FlavorPhysics/CKMMatrix/PhaseFreedom.lean

```
public import Physlib.Particles.FlavorPhysics.CKMMatrix.Relations
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.121 Physlib/Particles/FlavorPhysics/CKMMatrix/Relations.lean

```
public import Physlib.Particles.FlavorPhysics.CKMMatrix.Rows
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.122 Physlib/Particles/FlavorPhysics/CKMMatrix/Rows.lean

```
public import Physlib.Particles.FlavorPhysics.CKMMatrix.Basic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.123 Physlib/Particles/FlavorPhysics/CKMMatrix/StandardParamet

```
public import Physlib.Particles.FlavorPhysics.CKMMatrix.Rows
public import Physlib.Particles.FlavorPhysics.CKMMatrix.Invariants

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.124 Physlib/Particles/FlavorPhysics/CKMMatrix/StandardParamet

```
public import Physlib.Particles.FlavorPhysics.CKMMatrix.PhaseFreedom
public import Physlib.Particles.FlavorPhysics.CKMMatrix.StandardParameteriz

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.125 Physlib/Particles/NeutrinoPhysics/Basic.lean

```
public import Mathlib.Analysis.Complex.Exponential
```

1.126 Physlib/Particles/StandardModel/AnomalyCancellation/Basic

```
public import Physlib.QFT.AnomalyCancellation.Basic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.127 Physlib/Particles/StandardModel/AnomalyCancellation/FamilyMaps.l

```
public import Physlib.Particles.StandardModel.AnomalyCancellation.Basic
set_option backward.isDefEq.respectTransparency false in
```

1.128 Physlib/Particles/StandardModel/AnomalyCancellation/NoGrav/Basic

```
public import Physlib.Particles.StandardModel.AnomalyCancellation.Basic
```

1.129 Physlib/Particles/StandardModel/AnomalyCancellation/NoGrav/One/L

```
public import Physlib.Particles.StandardModel.AnomalyCancellation.NoGrav.One.LinearP
```

1.130 Physlib/Particles/StandardModel/AnomalyCancellation/NoGrav/One/L

```
public import Physlib.Particles.StandardModel.AnomalyCancellation.NoGrav.Basic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
      Fin.isValue, toSpecies_apply, Finset.sum_singleton,
      Fin.isValue, toSpecies_apply, Finset.sum_singleton,
      simp only [asCharges, Fin.isValue, toSpecies_apply]
      simp only [asCharges, Fin.isValue, neg_add_rev, toSpecies_apply]
      simp only [asCharges, Fin.isValue, neg_mul, toSpecies_apply]
set_option backward.isDefEq.respectTransparency false in
  have h1 : -216 - S.v ^ 3 * 216 - S.w ^ 3 * 216 = - 216 *(S.v ^3 + S.w ^3 +1) := by
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.131 Physlib/Particles/StandardModel/AnomalyCancellation/Permutations

```
public import Physlib.Particles.StandardModel.AnomalyCancellation.Basic
    simp only [PermGroup]
    simp only [chargeMap_apply, Pi.inv_apply, Module.End.mul_apply]
```

1.132 Physlib/Particles/StandardModel/Basic.lean

```
public import Physlib.SpaceAndTime.SpaceTime.Basic
```

```
public import Physlib.Meta.Linters.Sorry
TODO "Redefine the gauge group as a quotient of SU(3) x SU(2) x U(1) by a s
noncomputable def ofU1Subgroup (u1 : unitary ℂ) : GaugeGroupI :=
```

1.133 Physlib/Particles/StandardModel/HiggsBoson/Basic.lean

```
public import Physlib.Particles.StandardModel.Basic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
TODO "Make `HiggsBundle` an associated bundle."
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
TODO "Define the global gauge action on HiggsField."
TODO "Prove `⟪φ1, φ2⟫_H` invariant under the global gauge action. (norm_map
TODO "Prove invariance of potential under global gauge action."
```

1.134 Physlib/Particles/StandardModel/HiggsBoson/Potential.lean

```
public import Physlib.Particles.StandardModel.HiggsBoson.Basic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.135 Physlib/Particles/StandardModel/Representations.lean

```
public import Mathlib.Analysis.Complex.Basic
```

```
public import Mathlib.LinearAlgebra.UnitaryGroup
```

1.136 Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/B3.le

```
public import Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.Basic
set_option backward.isDefEq.respectTransparency false in
  simp [Fin.isValue, Prod.mk_zero_zero, Prod.mk_one_one]
```

1.137 Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/Basic

```
public import Physlib.QFT.AnomalyCancellation.Basic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.138 Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/Hyper

```
public import Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.Basic
```

1.139 Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/LineY

```
public import Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.Y3
public import Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.B3
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  simp only [Nat.reduceMul, Fin.isValue, Prod.mk_zero_zero, Prod.mk_one_one, add_sub
    sub_self]
set_option backward.isDefEq.respectTransparency false in
```

1.140 Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/Ortho

```
public import Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.LineY3B3
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.141 Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellatio

```
public import Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.Or
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.142 Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellatio

```
public import Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.Or
set_option backward.isDefEq.respectTransparency false in
```

1.143 Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellatio

```
public import Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.Ba
    simp only [PermGroup]
    simp only [Pi.inv_apply]
```

1.144 Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellatio

```
public import Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.Ba
set_option backward.isDefEq.respectTransparency false in
  simp [Fin.isValue, Prod.mk_zero_zero, Prod.mk_one_one]
```

1.145 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/Allows

```
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.OfPotentia
```

1.146 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/Basic.

```
They were created specifically for the purpose of Physlib.
```

```
lemma empty_eq : (∅ : ChargeSpectrum ℤ) = ⟨none, none, {}, {}⟩ := rfl

  simp [Subset, empty_eq]
  simp [empty_eq]
  simp [empty_eq]
  simp [empty_eq]
  simp [empty_eq]
  simp [card, empty_eq]
  simp [subset_def] at h
```

1.147 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/Completions.le

```
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.MinimallyAllowsTerm
```

1.148 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/Map.lean

```
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.Yukawa
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.Completions
public import Mathlib.Tactic.FinCases
```

1.149 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/MinimalSuperS

```
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.Completions
    · simp_all [subset_def]
    · simp_all [subset_def]
```

1.150 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/MinimallyAllow

```
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.AllowsTerm
```

1.151 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/MinimallyAllow

```
public import Mathlib.Tactic.FinCases
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.MinimallyAllowsTerm
```

1.152 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/MinimallyAllow

```
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.MinimallyAllowsTerm
```

```
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.PhenoConst
```

1.153 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/OfFiel

```
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.Basic
public import Physlib.Particles.SuperSymmetry.SU5.FieldLabels
```

1.154 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/OfPote

```
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.OfFieldLab
public import Physlib.Particles.SuperSymmetry.SU5.Potential
```

1.155 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/PhenoC

```
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.Yukawa
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.MinimalSup
public import Physlib.Meta.TODO.Basic
TODO "Make the result `viableChargesMultiset` a safe definition, that is to
```

1.156 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/PhenoC

```
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.AllowsTerm
```

1.157 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/Yukawa

```
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.PhenoConst
```

1.158 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/ZMod.l

```
meta import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.Yukawa
meta import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.Completions
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.Yukawa
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.Completion
public import Physlib.Meta.Linters.Sorry
```

1.159 Physlib/Particles/SuperSymmetry/SU5/Potential.lean

```
public import Physlib.Particles.SuperSymmetry.SU5.FieldLabels
```

1.160 Physlib/QFT/AnomalyCancellation/Basic.lean

```
public import Physlib.Mathematics.LinearMaps
TODO "Replace `ACCSystemChargesMk` with `⟨n⟩` notation everywhere. "
TODO "Anomaly cancellation conditions can be derived formally from the gauge group
TODO "Anomaly cancellation conditions can be defined using algebraic varieties.
```

1.161 Physlib/QFT/AnomalyCancellation/GroupActions.lean

```
public import Physlib.QFT.AnomalyCancellation.Basic
set_option backward.isDefEq.respectTransparency false in
```

1.162 Physlib/QFT/PerturbationTheory/FieldOpFreeAlgebra/Basic.lean

```
public import Physlib.QFT.PerturbationTheory.FieldSpecification.CrAnSection


  | position φ => simp [fieldOpToCrAnType]; erw [CreateAnnihilate.sum_eq]
set_option backward.isDefEq.respectTransparency false in
```

1.163 Physlib/QFT/PerturbationTheory/FieldOpFreeAlgebra/Grading.lean

```
public import Physlib.QFT.PerturbationTheory.FieldOpFreeAlgebra.Basic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
        simp only [add_eq_mul, mul_self]
      simp only [add_eq_mul, mul_self]
        simp only [add_eq_mul, mul_self]
        simp only [add_eq_mul, mul_self]
```

1.164 Physlib/QFT/PerturbationTheory/FieldOpFreeAlgebra/NormTimeOrder.

```
public import Physlib.QFT.PerturbationTheory.FieldSpecification.TimeOrder
public import Physlib.QFT.PerturbationTheory.FieldOpFreeAlgebra.Basic
```

```
open Module Physlib.List
```

1.165 Physlib/QFT/PerturbationTheory/FieldOpFreeAlgebra/NormalO

```
public import Physlib.QFT.PerturbationTheory.FieldSpecification.NormalOrder
public import Physlib.QFT.PerturbationTheory.FieldOpFreeAlgebra.SuperCommut
`Physlib.QFT.PerturbationTheory.FieldSpecification.NormalOrder`
          rw [Physlib.List.insertionSort_append_insertionSort_append]
    LinearMap.coe_comp, Function.comp_apply]
    LinearMap.flip_apply, LinearMap.coe_mk, AddHom.coe_mk, ← mul_assoc,
  simp only [superCommuteF_ofCrAnOpF_ofCrAnOpF, Algebra.smul_mul_assoc]
  simp only [neg_smul, mul_neg, Algebra.mul_smul_comm, neg_mul,
    simp only [crPartF_position, anPartF_position]
    simp only [crAnStatistics, Function.comp_apply, crAnFieldOpToFieldOp_pr
    simp only [crPartF_negAsymp, anPartF_posAsymp]
    simp only [crAnStatistics, Function.comp_apply, crAnFieldOpToFieldOp_pr
    simp only [crPartF_negAsymp, anPartF_position]
    simp only [crAnStatistics, Function.comp_apply, crAnFieldOpToFieldOp_pr
    simp only [crPartF_position, anPartF_posAsymp]
    simp only [crAnStatistics, Function.comp_apply, crAnFieldOpToFieldOp_pr
    normalOrderF_crPartF_mul_anPartF, normalOrderF_anPartF_mul_anPartF]
TODO "Split the following two lemmas up into smaller parts."
set_option backward.isDefEq.respectTransparency false in
      Algebra.smul_mul_assoc, Algebra.mul_smul_comm, map_smul]
    simp only [List.singleton_append, List.append_assoc, List.cons_append,
set_option backward.isDefEq.respectTransparency false in
      Algebra.smul_mul_assoc, Algebra.mul_smul_comm, map_smul]
    simp only [List.singleton_append, List.append_assoc, List.cons_append,
    ofCrAnListF_append, normalOrderList_statistics, smul_sub, smul_smul,
```

1.166 Physlib/QFT/PerturbationTheory/FieldOpFreeAlgebra/SuperCo

```
public import Physlib.QFT.PerturbationTheory.FieldOpFreeAlgebra.Grading
public import Physlib.QFT.PerturbationTheory.FieldStatistics.ExchangeSign
  simp only [map_sum, LinearMap.coe_sum, Finset.sum_apply, Algebra.smul_mul
  simp only [CrAnSection.statistics_eq_state_statistics,
    simp only [crPartF_posAsymp, map_zero, mul_zero, smul_zero, zero_mul,
    simp only [anPartF_position, crPartF_position, Algebra.smul_mul_assoc]
    simp only [anPartF_posAsymp, crPartF_position, Algebra.smul_mul_assoc]
    simp only [anPartF_position, crPartF_negAsymp, Algebra.smul_mul_assoc]
    simp only [List.singleton_append, crAnStatistics,
    simp only [anPartF_posAsymp, crPartF_negAsymp, Algebra.smul_mul_assoc]
    simp only [crPartF_posAsymp, map_zero, LinearMap.zero_apply, zero_mul,
```

```
    simp only [anPartF_negAsymp, map_zero, mul_zero, smul_zero, zero_mul,
    simp only [crPartF_position, anPartF_position, Algebra.smul_mul_assoc]
    simp only [crPartF_position, anPartF_posAsymp, Algebra.smul_mul_assoc]
    simp only [crPartF_negAsymp, anPartF_position, Algebra.smul_mul_assoc]
    simp only [crPartF_negAsymp, anPartF_posAsymp, Algebra.smul_mul_assoc]
  simp only [crPartF_posAsymp, map_zero, LinearMap.zero_apply, zero_mul, mul_zero, s
  simp only [crPartF_posAsymp, map_zero, mul_zero, smul_zero, zero_mul,
  simp only [crPartF_position, Algebra.smul_mul_assoc]
  simp only [crPartF_position, crPartF_negAsymp, Algebra.smul_mul_assoc]
  simp only [crPartF_negAsymp, crPartF_position, Algebra.smul_mul_assoc]
  simp only [crPartF_negAsymp, Algebra.smul_mul_assoc]
    simp only [anPartF_position, Algebra.smul_mul_assoc]
    simp only [anPartF_position, anPartF_posAsymp, Algebra.smul_mul_assoc]
    simp only [anPartF_posAsymp, anPartF_position, Algebra.smul_mul_assoc]
    simp only [anPartF_posAsymp, Algebra.smul_mul_assoc]
    simp only [crPartF_negAsymp, Algebra.smul_mul_assoc]
    simp only [crPartF_position, Algebra.smul_mul_assoc]
    simp only [anPartF_position, Algebra.smul_mul_assoc]
    simp only [anPartF_posAsymp, Algebra.smul_mul_assoc]
  simp only [neg_smul, sub_neg_eq_add]
  simp only [Algebra.smul_mul_assoc, neg_smul, sub_neg_eq_add]
  simp only [List.cons_append, List.append_assoc, List.nil_append,
  simp only [map_mul, mul_comm]
  simp only [Algebra.smul_mul_assoc, List.singleton_append, Algebra.mul_smul_comm,
    simp only [ofCrAnListF_nil, List.length_nil, Finset.univ_eq_empty,
    simp only [superCommuteF_ofCrAnListF_ofFieldOpFsList, ofList_empty,
  simp only [map_sub, map_smul, neg_smul]
  simp only [ofList_append_eq_mul, List.append_assoc]
    simp only [add_eq_mul, p]
      simp only [add_eq_mul, p]
      simp only [add_eq_mul, map_add, LinearMap.add_apply, p]
      simp only [add_eq_mul, map_smul, LinearMap.smul_apply, p]
    simp only [add_eq_mul, map_add, p]
    simp only [add_eq_mul, map_smul, p]
      simp only [add_eq_mul, mul_self]
      simp only [add_eq_mul]
      simp only [add_eq_mul]
      simp only [add_eq_mul, mul_self]
      simp only [add_eq_mul, mul_self]
      simp only [add_eq_mul]
      simp only [add_eq_mul]
      simp only [add_eq_mul, mul_self]
    simp only [List.get_eq_getElem, Algebra.smul_mul_assoc, p]
    simp_all only [p, List.get_eq_getElem, Algebra.smul_mul_assoc, map_add,
    simp_all only [p, List.get_eq_getElem, Algebra.smul_mul_assoc, map_smul,
      simp only [add_eq_mul, mul_self]
```

```
      simp only [add_eq_mul, mul_self]
```

1.167 Physlib/QFT/PerturbationTheory/FieldOpFreeAlgebra/TimeOrd

```
public import Physlib.QFT.PerturbationTheory.FieldSpecification.TimeOrder
public import Physlib.QFT.PerturbationTheory.FieldOpFreeAlgebra.SuperCommut
open Physlib.List
  simp only [List.singleton_append, Algebra.smul_mul_assoc]
  simp only [Algebra.smul_mul_assoc]
  simp only [Algebra.smul_mul_assoc, map_sub, map_smul]
    simp only [mul_one]
  simp only [ofList_singleton, ofCrAnListF_singleton, neg_smul, map_smul,
  simp only [smul_zero, neg_zero, neg_smul, map_neg, map_smul, smul_neg,
  simp only [ofList_singleton, ofCrAnListF_singleton, neg_smul, map_smul,
  simp only [neg_smul, map_neg, map_smul, smul_neg, neg_neg]
    simp only [neg_smul, map_neg, map_smul, neg_eq_zero, smul_eq_zero]
  simp only [Algebra.smul_mul_assoc, map_sub, map_smul]
  simp only [Algebra.mul_smul_comm, Algebra.smul_mul_assoc, smul_smul]
set_option backward.isDefEq.respectTransparency false in
```

1.168 Physlib/QFT/PerturbationTheory/FieldSpecification/Basic.l

```
public import Physlib.SpaceAndTime.SpaceTime.Basic
public import Physlib.Units.WithDim.Momentum
public import Physlib.QFT.PerturbationTheory.FieldStatistics.OfFinset
```

1.169 Physlib/QFT/PerturbationTheory/FieldSpecification/CrAnFie

```
public import Physlib.QFT.PerturbationTheory.FieldSpecification.Basic
public import Physlib.QFT.PerturbationTheory.CreateAnnihilate
(see: `Physlib.QFT.PerturbationTheory.FieldSpecification.Basic`),
```

1.170 Physlib/QFT/PerturbationTheory/FieldSpecification/CrAnSec

```
public import Physlib.QFT.PerturbationTheory.FieldSpecification.CrAnFieldOp
public import Physlib.Mathematics.List
`Physlib.QFT.PerturbationTheory.FieldSpecification.Basic`
`Physlib.QFT.PerturbationTheory.FieldStatistics.CrAnFieldOp`
  simp only [List.map_take]
      simp only [fieldOpToCrAnType]
```

```
      erw [CreateAnnihilate.CreateAnnihilate_card_eq_two]
set_option backward.isDefEq.respectTransparency false in
    rw [Physlib.List.take_eraseIdx_same, Physlib.List.drop_eraseIdx_succ]
    rw [← Physlib.List.eraseIdx_length _ ⟨n, hn⟩] at hn
```

1.171 Physlib/QFT/PerturbationTheory/FieldSpecification/Filters.lean

```
public import Physlib.QFT.PerturbationTheory.FieldSpecification.CrAnFieldOp
```

1.172 Physlib/QFT/PerturbationTheory/FieldSpecification/NormalOrder.lea

```
public import Physlib.QFT.PerturbationTheory.FieldSpecification.Filters
public import Physlib.QFT.PerturbationTheory.Koszul.KoszulSign
  simp only [mul_eq_mul_right_iff]
    simp only [mul_eq_mul_left_iff]
  Physlib.List.insertionSortEquiv □.normalOrderRel φs
set_option backward.isDefEq.respectTransparency false in
  erw [← Physlib.List.insertionSortEquiv_get]
set_option backward.isDefEq.respectTransparency false in
  rw [Physlib.List.eraseIdx_insertionSort_fin]
```

1.173 Physlib/QFT/PerturbationTheory/FieldSpecification/TimeOrder.lean

```
public import Physlib.QFT.PerturbationTheory.FieldSpecification.CrAnSection
public import Physlib.QFT.PerturbationTheory.FieldSpecification.NormalOrder
open Physlib.List
  simp only [timeOrderSign, Wick.koszulSign, Wick.koszulSignInsert, mul_one]
  simp only [crAnTimeOrderSign, Wick.koszulSign, Wick.koszulSignInsert, mul_one]
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.174 Physlib/QFT/PerturbationTheory/FieldStatistics/Basic.lean

```
public import Physlib.Mathematics.List.InsertIdx
public import Mathlib.Data.List.Sort
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    simp only [List.length_cons, List.map_cons, List.get_eq_getElem, List.mem_cons,
    simp only [List.length_cons, mul_ite, ite_mul, one_mul, mul_one]
```

```
open Physlib.List
  simp only [List.take_succ_cons]
    cases a <;> rfl
    cases a <;> rfl
    cases a <;> cases b <;> cases c <;> rfl
    simp only [Finset.univ_eq_empty, Finset.prod_const, Finset.card_empty,
    simp only [Finset.prod_const, Finset.card_univ, Fintype.card_fin]
```

1.175 Physlib/QFT/PerturbationTheory/FieldStatistics/ExchangeSi

```
public import Physlib.QFT.PerturbationTheory.FieldStatistics.Basic
```

1.176 Physlib/QFT/PerturbationTheory/FieldStatistics/OfFinset.l

```
public import Physlib.QFT.PerturbationTheory.FieldStatistics.Basic
  simp only [ofFinset, Fin.getElem_fin]
  simp only [Fin.getElem_fin, mul_self, one_mul]
  simp only [Fin.getElem_fin, mul_ite, ite_mul, mul_self, one_mul, mul_one,
```

1.177 Physlib/QFT/PerturbationTheory/Koszul/KoszulSign.lean

```
public import Physlib.QFT.PerturbationTheory.Koszul.KoszulSignInsert
public import Physlib.Mathematics.List.InsertionSort
import all Physlib.Mathematics.List
open Physlib.List
set_option backward.isDefEq.respectTransparency false in
      Physlib.Fin.equivCons_trans, Equiv.trans_apply, Physlib.Fin.equivCons
        Equiv.trans_apply, Physlib.Fin.equivCons_succ]
set_option backward.isDefEq.respectTransparency false in
  simp only [List.get_eq_getElem]
    simp only [Fin.castOrderIso,
  simp only [Fin.getElem_fin, mul_one]
  simp only [List.get_eq_getElem, List.length_cons, List.insertionSort,
    simp only [List.take_left', List.length_insertionSort]
    simp only [mul_assoc, List.length_insertionSort, List.take_left',
```

1.178 Physlib/QFT/PerturbationTheory/Koszul/KoszulSignInsert.le

```
public import Physlib.QFT.PerturbationTheory.FieldStatistics.ExchangeSign
public import Physlib.Mathematics.List
```

```
open Physlib.List
```

1.179 Physlib/QFT/PerturbationTheory/WickAlgebra/Basic.lean

```
public import Physlib.QFT.PerturbationTheory.FieldOpFreeAlgebra.SuperCommute
open Physlib.List
def WickAlgebra : Type := (TwoSidedIdeal.span 𝓕.fieldOpIdealSet).ringCon.Quotient
instance : Semiring (𝓕.WickAlgebra) := inferInstanceAs <| Semiring <|
  (TwoSidedIdeal.span 𝓕.fieldOpIdealSet).ringCon.Quotient

instance : Algebra ℂ (𝓕.WickAlgebra) := inferInstanceAs <| Algebra ℂ <|
  (TwoSidedIdeal.span 𝓕.fieldOpIdealSet).ringCon.Quotient

instance: Ring (𝓕.WickAlgebra) := inferInstanceAs <| Ring <|
  (TwoSidedIdeal.span 𝓕.fieldOpIdealSet).ringCon.Quotient

instance : Coe (𝓕.FieldOpFreeAlgebra) (𝓕.WickAlgebra) :=
  ⟨(TwoSidedIdeal.span 𝓕.fieldOpIdealSet).ringCon.toQuotient⟩
    simp only [ofCrAnListF_singleton, List.get_eq_getElem, Algebra.smul_mul_assoc, m
      map_smul, map_mul, ι_superCommuteF_superCommuteF_ofCrAnOpF_ofCrAnOpF_ofCrAnOpF
      zero_mul, MulActionWithZero.smul_zero, Finset.sum_const_zero]
TODO "The lemma `bosonicProjF_mem_ideal` has a proof which is really long.
set_option backward.isDefEq.respectTransparency false in
lemma ofCrAnList_nil : ofCrAnList (𝓕 := 𝓕) [] = 1 := by
  simp only [ofCrAnList, ofCrAnListF_nil]
  simp
```

1.180 Physlib/QFT/PerturbationTheory/WickAlgebra/Grading.lean

```
public import Physlib.QFT.PerturbationTheory.WickAlgebra.Basic
open Physlib.List
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.181 Physlib/QFT/PerturbationTheory/WickAlgebra/NormalOrder/Basic.lea

```
public import Physlib.QFT.PerturbationTheory.FieldOpFreeAlgebra.NormalOrder
public import Physlib.QFT.PerturbationTheory.WickAlgebra.Basic
```

```
open Physlib.List
  simp only [ofList_singleton, Algebra.mul_smul_comm, Algebra.smul_mul_asso
```

1.182 Physlib/QFT/PerturbationTheory/WickAlgebra/NormalOrder/Le

```
public import Physlib.QFT.PerturbationTheory.WickAlgebra.NormalOrder.Basic
public import Physlib.QFT.PerturbationTheory.WickAlgebra.SuperCommute
open Physlib.List
  simp only [normalOrderSign_nil, normalOrderList_nil, ofCrAnList_nil]
  module
  erw [normalOrder_ofCrAnList, smul_smul, normalOrderSign, Wick.koszulSign_
    simp only [anPart_inAsymp, zero_mul, map_zero, mul_zero]
    module
    simp only [anPart_position]
    simp only [anPart_outAsymp]
  erw [smul_smul]
  simp [FieldStatistic.exchangeSign_mul_self]
  simp only [map_add, map_smul]
  erw [superCommute_ofCrAnOp_ofCrAnList_eq_sum, Finset.smul_sum,
  simp only [List.get_eq_getElem, normalOrderList_get_normalOrderEquiv,
  erw [ofCrAnList_eq_normalOrder, mul_smul_comm, smul_smul, smul_smul]
    simp only [zero_mul, smul_zero]
  simp only [Fin.val_cast, Fin.getElem_fin,
    simp only [anPart_inAsymp, map_zero, LinearMap.zero_apply, Fin.getElem_
      Algebra.smul_mul_assoc, zero_mul]
    conv_rhs =>
      enter [2, s]
      rw [smul_zero]
    simp only [anPart_position, Fin.getElem_fin, Algebra.smul_mul_assoc]
    simp only [crAnStatistics, Function.comp_apply, crAnFieldOpToFieldOp_pr
    simp only [anPart_outAsymp, Fin.getElem_fin, Algebra.smul_mul_assoc]
    simp only [crAnStatistics, Function.comp_apply, crAnFieldOpToFieldOp_pr
  simp only [add_left_inj]
  simp only [Fin.getElem_fin, Algebra.smul_mul_assoc, contractStateAtIndex,
    rw [Physlib.List.insertIdx_eq_take_drop]
  simp only [Nat.succ_eq_add_one, ofList_singleton, Algebra.smul_mul_assoc,
```

1.183 Physlib/QFT/PerturbationTheory/WickAlgebra/NormalOrder/Wi

```
public import Physlib.QFT.PerturbationTheory.WickAlgebra.NormalOrder.Lemmas
public import Physlib.QFT.PerturbationTheory.WickContraction.InsertAndContr
open Physlib.List
```

```
  simp only [Nat.succ_eq_add_one]
  simp only [uncontractedListGet]
    uncontractedListGet]
```

1.184 Physlib/QFT/PerturbationTheory/WickAlgebra/StaticWickTerm.lean

```
public import Physlib.QFT.PerturbationTheory.WickAlgebra.NormalOrder.WickContraction
public import Physlib.QFT.PerturbationTheory.WickContraction.Sign.InsertNone
public import Physlib.QFT.PerturbationTheory.WickContraction.Sign.InsertSome
public import Physlib.QFT.PerturbationTheory.WickContraction.StaticContract

open Physlib.List
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
      simp only [ZeroMemClass.coe_zero, zero_mul, smul_zero, mul_zero]
    · simp_all only [not_not, forall_const]
          Fin.val_cast, Fin.getElem_fin, smul_eq_zero]
set_option backward.isDefEq.respectTransparency false in
```

1.185 Physlib/QFT/PerturbationTheory/WickAlgebra/StaticWickTheorem.lean

```
public import Physlib.QFT.PerturbationTheory.WickAlgebra.StaticWickTerm
open Physlib.List

set_option backward.isDefEq.respectTransparency false in
```

1.186 Physlib/QFT/PerturbationTheory/WickAlgebra/SuperCommute.lean

```
public import Physlib.QFT.PerturbationTheory.WickAlgebra.Basic
open Physlib.List
  simp only [ofList_singleton, List.get_eq_getElem, Algebra.smul_mul_assoc]
  simp only [ofList_singleton, List.get_eq_getElem, Algebra.smul_mul_assoc]
```

1.187 Physlib/QFT/PerturbationTheory/WickAlgebra/TimeContraction.lean

```
public import Physlib.QFT.PerturbationTheory.WickAlgebra.NormalOrder.Lemmas
public import Physlib.QFT.PerturbationTheory.WickAlgebra.TimeOrder
  simp only [timeContract, Algebra.smul_mul_assoc, add_zero]
  simp only [smul_add]
    simp only [smul_zero]
```

```
    simp only [map_smul, smul_eq_zero]
    simp only [map_smul, smul_eq_zero]
```

1.188 Physlib/QFT/PerturbationTheory/WickAlgebra/TimeOrder.lean

```
public import Physlib.QFT.PerturbationTheory.WickAlgebra.SuperCommute
public import Physlib.QFT.PerturbationTheory.FieldOpFreeAlgebra.TimeOrder
open Physlib.List
  calc _
      _ = ι (𝓕.ofCrAnListF (φs1 ++ φ1 :: φ2 :: φ3 :: φs2)) -
          𝓢(𝓕.crAnStatistics φ1, (ofList 𝓕.crAnStatistics [φ2, φ3])) •
          ι (𝓕.ofCrAnListF (φs1 ++ φ2 :: φ3 :: φ1 :: φs2)) -
          𝓢(𝓕.crAnStatistics φ2, 𝓕.crAnStatistics φ3) •
          (ι (𝓕.ofCrAnListF (φs1 ++ φ1 :: φ3 :: φ2 :: φs2))) -
          𝓢(𝓕.crAnStatistics φ1, ofList 𝓕.crAnStatistics [φ3, φ2]) •
          ι 𝓕.ofCrAnListF (φs1 ++ φ3 :: φ2 :: φ1 :: φs2))) := by
        rw [← ofCrAnListF_singleton, ← ofCrAnListF_singleton, ← ofCrAnListF
        rw [superCommuteF_ofCrAnListF_ofCrAnListF]
        simp only [List.singleton_append, ofList_singleton, map_sub, map_sm
        rw [superCommuteF_ofCrAnListF_ofCrAnListF, superCommuteF_ofCrAnList
        simp only [List.cons_append, List.nil_append, ofList_singleton, mul
          ofCrAnListF_append, Algebra.mul_smul_comm, sub_mul, List.append_a
          Algebra.smul_mul_assoc, map_sub, map_smul]
  trans crAnTimeOrderSign (φs1 ++ φ1 :: φ2 :: φ3 :: φs2) • (ι (ofCrAnListF
    ι [ofCrAnOpF φ1, [ofCrAnOpF φ2, ofCrAnOpF φ3]ₛF]ₛF *
    ι (ofCrAnListF l2)); swap
  · simp
  simp only [List.singleton_append, ofList_singleton, map_sub, map_smul]
  simp [List.cons_append, List.nil_append, ofList_singleton, map_sub,
    map_smul, mul_sub, sub_mul, Semigroup.mul_assoc]
  module
example (c1 c2 : ℂ) (a : 𝓕.WickAlgebra) : c1 • c2 • a =
  c2 • c1 • a := by exact smul_comm c1 c2 a
      calc _
        /- Split the commutator. -/
        _ = crAnTimeOrderSign (φs' ++ φ :: ψ :: φs) •
          ι (ofCrAnListF (crAnTimeOrderList (φs' ++ φ :: ψ :: φs))) -
            crAnTimeOrderSign (φs' ++ ψ :: φ :: φs) •
          𝓢(𝓕.crAnStatistics φ, 𝓕.crAnStatistics ψ) •
            ι (ofCrAnListF (crAnTimeOrderList (φs' ++ ψ :: φ :: φs))) := by
          conv_lhs =>
            rw [← ofCrAnListF_singleton, ← ofCrAnListF_singleton,
              superCommuteF_ofCrAnListF_ofCrAnListF]
            simp [mul_sub, sub_mul, ← ofCrAnListF_append]
```

```
      rw [timeOrderF_ofCrAnListF, timeOrderF_ofCrAnListF]
    simp only [map_smul, sub_right_inj]
    module
  /- Simplify the signs. -/
  _ = crAnTimeOrderSign (φs' ++ φ :: ψ :: φs) •
    (ι (ofCrAnListF (crAnTimeOrderList (φs' ++ φ :: ψ :: φs))) -
    𝓢(𝓕.crAnStatistics φ, 𝓕.crAnStatistics ψ) •
      ι (ofCrAnListF (crAnTimeOrderList (φs' ++ ψ :: φ :: φs)))) := by
    have h1 : crAnTimeOrderSign (φs' ++ φ :: ψ :: φs) =
        crAnTimeOrderSign (φs' ++ ψ :: φ :: φs) := by
      trans crAnTimeOrderSign (φs' ++ [φ, ψ] ++ φs)
      simp only [List.append_assoc, List.cons_append, List.nil_append]
      rw [crAnTimeOrderSign]
      have hp : List.Perm [φ,ψ] [ψ,φ] := by exact List.Perm.swap ψ φ []
      rw [Wick.koszulSign_perm_eq _ _ φ _ _ _ _ _ hp]
      simp only [List.append_assoc, List.cons_append]
      rfl
      simp_all
    rw [h1]
    module
  /- Splitting the time-ordered lists. -/
  _ = crAnTimeOrderSign (φs' ++ [φ, ψ] ++ φs) •
    (ι (ofCrAnListF (List.takeWhile (fun c => ¬crAnTimeOrderRel φ c)
          (List.insertionSort crAnTimeOrderRel (φs' ++ φs)))) *
        ι (ofCrAnListF (List.filter (fun c =>
          (crAnTimeOrderRel φ c ∧ crAnTimeOrderRel c φ)) φs')) *
        ι (ofCrAnListF [φ, ψ]) *
        ι (ofCrAnListF (List.filter (fun c =>
          (crAnTimeOrderRel φ c ∧ crAnTimeOrderRel c φ)) φs)) *
        ι (ofCrAnListF (List.filter (fun c => (crAnTimeOrderRel φ c ∧ ¬crAnTim
          (List.insertionSort crAnTimeOrderRel (φs' ++ φs)))) -
    𝓢(𝓕.crAnStatistics φ, 𝓕.crAnStatistics ψ) •
    ι (ofCrAnListF (List.takeWhile (fun c => ¬crAnTimeOrderRel φ c)
                (List.insertionSort crAnTimeOrderRel (φs' ++ φs)))) *
        ι (ofCrAnListF (List.filter (fun c =>
          (crAnTimeOrderRel φ c ∧ crAnTimeOrderRel c φ)) φs')) *
        ι (ofCrAnListF [ψ, φ]) *
        ι (ofCrAnListF (List.filter (fun c =>
          (crAnTimeOrderRel φ c ∧ crAnTimeOrderRel c φ)) φs)) *
        ι (ofCrAnListF
        (List.filter (fun c => decide (crAnTimeOrderRel φ c ∧ ¬crAnTimeOrderRe
          (List.insertionSort crAnTimeOrderRel (φs' ++ φs))))) := by
    have h1 := insertionSort_of_eq_list 𝓕.crAnTimeOrderRel φ φs' [φ, ψ] φs
      (by simp_all)
    rw [crAnTimeOrderList, show φs' ++ φ :: ψ :: φs = φs' ++ [φ, ψ] ++ φs by s
    have h2 := insertionSort_of_eq_list 𝓕.crAnTimeOrderRel φ φs' [ψ, φ] φs
```

```
            (by simp_all)
          rw [crAnTimeOrderList, show φs' ++ ψ :: φ :: φs = φs' ++ [ψ, φ] +
          repeat rw [ofCrAnListF_append]
          simp
        _ = crAnTimeOrderSign (φs' ++ [φ, ψ] ++ φs) •
          (ι (ofCrAnListF (List.takeWhile (fun c => ¬crAnTimeOrderRel φ c
              (List.insertionSort crAnTimeOrderRel (φs' ++ φs)))) *
            ι (ofCrAnListF (List.filter (fun c => (crAnTimeOrderRel φ c ∧
              crAnTimeOrderRel c φ)) φs')) *
            (ι (ofCrAnListF [φ, ψ]) - □(□.crAnStatistics φ, □.crAnStatist
              ι (ofCrAnListF [ψ, φ])) *
            ι (ofCrAnListF (List.filter (fun c => (crAnTimeOrderRel φ c ∧
              crAnTimeOrderRel c φ)) φs)) *
            ι (ofCrAnListF (List.filter (fun c => (crAnTimeOrderRel φ c ∧
              (List.insertionSort crAnTimeOrderRel (φs' ++ φs))))) := by
          simp [mul_sub, sub_mul]
        _ = crAnTimeOrderSign (φs' ++ [φ, ψ] ++ φs) •
          (ι (ofCrAnListF (List.takeWhile (fun c => ¬crAnTimeOrderRel φ c
              (List.insertionSort crAnTimeOrderRel (φs' ++ φs)))) *
            ι (ofCrAnListF (List.filter (fun c => (crAnTimeOrderRel φ c ∧
              crAnTimeOrderRel c φ)) φs')) *
            ι [ofCrAnOpF φ, ofCrAnOpF ψ]ₛF *
            ι (ofCrAnListF (List.filter (fun c => (crAnTimeOrderRel φ c ∧
              crAnTimeOrderRel c φ)) φs)) *
            ι (ofCrAnListF (List.filter (fun c => (crAnTimeOrderRel φ c ∧
              (List.insertionSort crAnTimeOrderRel (φs' ++ φs))))) := by
          congr
          rw [superCommuteF_ofCrAnOpF_ofCrAnOpF]
          rw [← ofCrAnListF_singleton, ← ofCrAnListF_singleton, ← ofCrAnL
          simp only [List.singleton_append, Algebra.smul_mul_assoc, map_s
            map_smul]
          rw [← ofCrAnListF_append]
          simp
      rw [← map_mul, ← map_mul, ← map_mul]
      rw [← ofCrAnListF_append, ← ofCrAnListF_append, ← ofCrAnListF_append]
      simp [decide_not, Bool.decide_and, List.append_assoc, List.cons_appen
        rw [← ofCrAnListF_append]
        exact hψφ
        simp_all
    simp only [neg_smul, map_neg, map_smul, mul_neg, Algebra.mul_smul_comm,
  simp only [map_smul]
```

1.189 Physlib/QFT/PerturbationTheory/WickAlgebra/Universality.lean

```
public import Physlib.QFT.PerturbationTheory.WickAlgebra.Basic
open Physlib.List
```

1.190 Physlib/QFT/PerturbationTheory/WickAlgebra/WickTerm.lean

```
public import Physlib.QFT.PerturbationTheory.WickContraction.Sign.InsertNone
public import Physlib.QFT.PerturbationTheory.WickContraction.Sign.InsertSome
public import Physlib.QFT.PerturbationTheory.WickContraction.TimeContract
public import Physlib.QFT.PerturbationTheory.WickAlgebra.NormalOrder.WickContraction
open Physlib.List
set_option backward.isDefEq.respectTransparency false in
    simp only [Nat.succ_eq_add_one, timeContract_insert_none,
  · simp only [Nat.succ_eq_add_one, timeContract_insert_none, Algebra.smul_mul_assoc
set_option backward.isDefEq.respectTransparency false in
        simp only [smul_smul, Algebra.smul_mul_assoc]
        simp only [smul_smul, Algebra.smul_mul_assoc]
        contractStateAtIndex, uncontractedFieldOpEquiv, Equiv.optionCongr_apply,
        zero_mul]
  simp only [Nat.succ_eq_add_one]
      Equiv.coe_trans, Option.map_none, one_mul, Algebra.smul_mul_assoc, smul_smul]
      Function.comp_apply, finCongr_apply, Algebra.smul_mul_assoc, smul_smul]
```

1.191 Physlib/QFT/PerturbationTheory/WickAlgebra/WicksTheorem.lean

```
public import Physlib.QFT.PerturbationTheory.WickAlgebra.WickTerm
open Physlib.List
set_option backward.isDefEq.respectTransparency false in
    simp only [exchangeSign_mul_self, Nat.succ_eq_add_one, Fintype.sum_option,
```

1.192 Physlib/QFT/PerturbationTheory/WickAlgebra/WicksTheoremNormal.le

```
public import Physlib.QFT.PerturbationTheory.WickAlgebra.StaticWickTheorem
public import Physlib.QFT.PerturbationTheory.WickAlgebra.WicksTheorem
public import Physlib.QFT.PerturbationTheory.WickContraction.Sign.Join
public import Physlib.QFT.PerturbationTheory.WickContraction.TimeCond

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.193 Physlib/QFT/PerturbationTheory/WickContraction/Basic.lean

```
public import Physlib.QFT.PerturbationTheory.FieldSpecification.Basic

open Physlib.List
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.194 Physlib/QFT/PerturbationTheory/WickContraction/Card.lean

```
public import Physlib.QFT.PerturbationTheory.WickContraction.ExtractEquiv
open Physlib.List
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.195 Physlib/QFT/PerturbationTheory/WickContraction/Erase.lean

```
public import Physlib.QFT.PerturbationTheory.WickContraction.Uncontracted
public import Physlib.Mathematics.Fin
open Physlib.List
open Physlib.Fin
set_option backward.isDefEq.respectTransparency false in
```

1.196 Physlib/QFT/PerturbationTheory/WickContraction/ExtractEqu

```
meta import Physlib.QFT.PerturbationTheory.WickContraction.InsertAndContrac
public import Physlib.QFT.PerturbationTheory.WickContraction.InsertAndContr
open Physlib.List
open Physlib.Fin
```

1.197 Physlib/QFT/PerturbationTheory/WickContraction/InsertAndC

```
public import Physlib.QFT.PerturbationTheory.WickContraction.UncontractedLi
open Physlib.List
```

```
open Physlib.Fin
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.198 Physlib/QFT/PerturbationTheory/WickContraction/InsertAndContract

```
public import Physlib.QFT.PerturbationTheory.WickContraction.Erase
open Physlib.List
open Physlib.Fin
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.199 Physlib/QFT/PerturbationTheory/WickContraction/Involutions.lean

```
public import Physlib.QFT.PerturbationTheory.WickContraction.Basic
open Physlib.List
set_option backward.isDefEq.respectTransparency false in
```

1.200 Physlib/QFT/PerturbationTheory/WickContraction/IsFull.lean

```
public import Physlib.Mathematics.Fin.Involutions
public import Physlib.QFT.PerturbationTheory.WickContraction.ExtractEquiv
public import Physlib.QFT.PerturbationTheory.WickContraction.Involutions
open Physlib.List
  exact Physlib.Fin.involutionNoFixed_card_even n he
  exact Physlib.Fin.involutionNoFixed_card_odd n ho
```

1.201 Physlib/QFT/PerturbationTheory/WickContraction/Join.lean

```
public import Physlib.QFT.PerturbationTheory.WickContraction.Singleton
open Physlib.List
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    Equiv.trans_toEmbedding]
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.202 Physlib/QFT/PerturbationTheory/WickContraction/Perm.lean

```
public import Physlib.QFT.PerturbationTheory.WickAlgebra.WickTerm
public import Physlib.QFT.PerturbationTheory.WickContraction.IsFull
public import Physlib.Meta.Linters.Sorry
open Physlib.List
```

1.203 Physlib/QFT/PerturbationTheory/WickContraction/Sign/Basic

```
public import Physlib.QFT.PerturbationTheory.WickContraction.Basic
public import Physlib.QFT.PerturbationTheory.FieldStatistics.ExchangeSign
open Physlib.List
```

1.204 Physlib/QFT/PerturbationTheory/WickContraction/Sign/Inser

```
public import Physlib.QFT.PerturbationTheory.WickContraction.Sign.Basic
public import Physlib.QFT.PerturbationTheory.WickContraction.InsertAndContr
open Physlib.List
  simp only [Nat.succ_eq_add_one, insertAndContract_sndFieldOfContract,
    simp only [Nat.succ_eq_add_one, finCongr_apply, Fin.getElem_fin, Fin.va
```

```
    simp only [exchangeSign_symm]
    simp only [Fin.getElem_fin, h.1, ↓reduceIte, mul_ite, exchangeSign_mul_self,
    · simp_all only [forall_const, Fin.getElem_fin, mul_ite,
  simp only [Fin.getElem_fin, Fintype.prod_sum_type]
  simp only [Bool.decide_and, Bool.decide_eq_true, List.mem_filter,
  simp only [Finset.mem_filter, Finset.mem_univ, true_and, Fin.getElem_fin]
```

1.205 Physlib/QFT/PerturbationTheory/WickContraction/Sign/InsertSome.l

```
public import Physlib.QFT.PerturbationTheory.WickContraction.Sign.Basic
public import Physlib.QFT.PerturbationTheory.WickContraction.InsertAndContract
open Physlib.List
  simp only [Fin.getElem_fin, Fintype.prod_sum_type]
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  simp only [Nat.succ_eq_add_one, insertAndContract_sndFieldOfContract,
    simp only [Fin.getElem_fin, Fin.val_cast, List.getElem_insertIdx_self, map_mul]
    simp only [exchangeSign_symm]
      simp only [Fin.getElem_fin, Fin.val_cast, insertIdx_getElem_fin, map_mul]
      simp only [exchangeSign_symm]
    simp only [Fin.getElem_fin, h, Nat.succ_eq_add_one, and_self,
      simp only [Fin.getElem_fin, h1, Nat.succ_eq_add_one, false_and,
      simp only [Nat.succ_eq_add_one, not_lt, Fin.getElem_fin]
  · simp only [Nat.succ_eq_add_one, not_lt, Fin.getElem_fin,
  simp only [Fin.getElem_fin, Fintype.prod_sum_type]
    simp only [Nat.succ_eq_add_one, not_lt, Finset.mem_filter, Finset.mem_univ,
  simp only [signInsertSomeCoef, Nat.succ_eq_add_one,
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.206 Physlib/QFT/PerturbationTheory/WickContraction/Sign/Join.lean

```
public import Physlib.QFT.PerturbationTheory.WickContraction.Join
open Physlib.List
  simp only [Fin.getElem_fin, Fintype.prod_sum_type]
    simp only [hj1, and_true, Fin.getElem_fin, true_and]
          simp only [ofFinset_singleton, List.get_eq_getElem]
  simp only [Fin.getElem_fin, h1, mul_one]
set_option backward.isDefEq.respectTransparency false in
```

1.207 Physlib/QFT/PerturbationTheory/WickContraction/Singleton.

```
public import Physlib.QFT.PerturbationTheory.WickContraction.SubContraction
public import Physlib.QFT.PerturbationTheory.WickContraction.StaticContract
public import Physlib.QFT.PerturbationTheory.WickContraction.TimeContract
public import Physlib.QFT.PerturbationTheory.WickContraction.Sign.Basic
open Physlib.List
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.208 Physlib/QFT/PerturbationTheory/WickContraction/StaticCont

```
public import Physlib.QFT.PerturbationTheory.WickContraction.InsertAndContr
public import Physlib.QFT.PerturbationTheory.WickAlgebra.NormalOrder.Lemmas
open Physlib.List
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  simp only [Nat.succ_eq_add_one, Fin.getElem_fin, ite_mul,
set_option backward.isDefEq.respectTransparency false in
```

1.209 Physlib/QFT/PerturbationTheory/WickContraction/SubContrac

```
public import Physlib.QFT.PerturbationTheory.WickContraction.UncontractedLi
open Physlib.List
```

1.210 Physlib/QFT/PerturbationTheory/WickContraction/TimeCond.l

```
public import Physlib.QFT.PerturbationTheory.WickContraction.Join
open Physlib.List
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.211 Physlib/QFT/PerturbationTheory/WickContraction/TimeContract.lean

```
public import Physlib.QFT.PerturbationTheory.WickAlgebra.TimeContraction
public import Physlib.QFT.PerturbationTheory.WickContraction.InsertAndContract
open Physlib.List
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  simp only [Nat.succ_eq_add_one, Fin.getElem_fin, ite_mul,
set_option backward.isDefEq.respectTransparency false in
  simp only [Nat.succ_eq_add_one, Fin.getElem_fin, ite_mul,
  simp only [Algebra.smul_mul_assoc, smul_smul]
set_option backward.isDefEq.respectTransparency false in
```

1.212 Physlib/QFT/PerturbationTheory/WickContraction/Uncontracted.lean

```
public import Physlib.QFT.PerturbationTheory.WickContraction.Basic
open Physlib.List
```

1.213 Physlib/QFT/PerturbationTheory/WickContraction/UncontractedList.

```
public import Physlib.QFT.PerturbationTheory.WickContraction.ExtractEquiv
open Physlib.List
open Physlib.Fin
    (f : Fin n → Fin m) (hf : StrictMono f) : (List.map f l).Pairwise (· ≤ ·) := by
  · erw [fin_list_sorted_indexOf_filter_le_mem l hl i hi]
    simp
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  apply Physlib.List.eraseIdx_sorted
```

1.214 Physlib/QFT/QED/AnomalyCancellation/Basic.lean

```
public import Physlib.QFT.AnomalyCancellation.Basic
TODO "The implementation of pure U(1) anomaly cancellation conditions is done
```

1.215 Physlib/QFT/QED/AnomalyCancellation/BasisLinear.lean

```
public import Physlib.QFT.QED.AnomalyCancellation.Basic
```

```
set_option backward.isDefEq.respectTransparency false in
TODO "The definition of `coordinateMap` here may be improved by replacing
```

1.216 Physlib/QFT/QED/AnomalyCancellation/ConstAbs.lean

```
public import Physlib.QFT.QED.AnomalyCancellation.VectorLike
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.217 Physlib/QFT/QED/AnomalyCancellation/Even/BasisLinear.lean

```
public import Physlib.QFT.QED.AnomalyCancellation.BasisLinear
public import Physlib.QFT.QED.AnomalyCancellation.VectorLike
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.218 Physlib/QFT/QED/AnomalyCancellation/Even/LineInCubic.lean

```
public import Physlib.QFT.QED.AnomalyCancellation.Even.BasisLinear
public import Physlib.QFT.QED.AnomalyCancellation.LineInPlaneCond
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.219 Physlib/QFT/QED/AnomalyCancellation/Even/Parameterization

```
public import Physlib.QFT.QED.AnomalyCancellation.Even.LineInCubic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.220 Physlib/QFT/QED/AnomalyCancellation/LineInPlaneCond.lean

```
public import Physlib.QFT.QED.AnomalyCancellation.ConstAbs
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.221 Physlib/QFT/QED/AnomalyCancellation/LowDim/One.lean

```
public import Physlib.QFT.QED.AnomalyCancellation.Basic
```

1.222 Physlib/QFT/QED/AnomalyCancellation/LowDim/Three.lean

```
public import Physlib.QFT.QED.AnomalyCancellation.Basic
```

1.223 Physlib/QFT/QED/AnomalyCancellation/LowDim/Two.lean

```
public import Physlib.QFT.QED.AnomalyCancellation.Basic
```

1.224 Physlib/QFT/QED/AnomalyCancellation/Odd/BasisLinear.lean

```
public import Physlib.QFT.QED.AnomalyCancellation.BasisLinear
public import Physlib.QFT.QED.AnomalyCancellation.VectorLike
### A.3. The shifted shifted split: Splting the charges up via `((1+n)+1) + n.succ`
lemma oddShiftZero_eq_oddFst : oddShiftZero = oddFst (0 : Fin n.succ) := by
  ext
  simp [oddShiftZero, oddFst]

lemma oddShiftFst_castSucc_eq_oddFst_succ (j : Fin n) :
    oddShiftFst j.castSucc = oddFst j.succ := by
  rw [Fin.ext_iff]
  simp only [oddShiftFst, Fin.val_cast, Fin.val_castAdd, Fin.val_natAdd, oddFst, Fin
  exact Nat.add_comm 1 ↑j

lemma oddShiftFst_last_eq_oddMid : oddShiftFst (Fin.last n) = oddMid := by
  rw [Fin.ext_iff]
  simp only [oddShiftFst, Fin.val_cast, Fin.val_castAdd, Fin.val_natAdd, oddMid, Fin
  exact Nat.add_comm 1 n

lemma oddShiftSnd_eq_oddSnd (j : Fin n) : oddShiftSnd j = oddSnd j := by
  rw [Fin.ext_iff]
  simp only [oddShiftSnd, Fin.val_cast, Fin.val_natAdd, oddSnd, add_left_inj]
```

```
  ring

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  rw [P!_oddShiftZero, oddShiftZero_eq_oddFst, P_oddFst]
  rw [P!_oddShiftFst, oddShiftFst_castSucc_eq_oddFst_succ, P_oddFst]
  rw [P!_oddShiftFst, oddShiftFst_last_eq_oddMid, P_oddMid]
  rw [P!_oddShiftSnd, oddShiftSnd_eq_oddSnd, P_oddSnd]
set_option backward.isDefEq.respectTransparency false in
```

1.225 Physlib/QFT/QED/AnomalyCancellation/Odd/LineInCubic.lean

```
public import Physlib.QFT.QED.AnomalyCancellation.LineInPlaneCond
public import Physlib.QFT.QED.AnomalyCancellation.Odd.BasisLinear
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.226 Physlib/QFT/QED/AnomalyCancellation/Odd/Parameterization.

```
public import Physlib.QFT.QED.AnomalyCancellation.Odd.LineInCubic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.227 Physlib/QFT/QED/AnomalyCancellation/Permutations.lean

```
public import Physlib.QFT.QED.AnomalyCancellation.Basic
public import Physlib.QFT.AnomalyCancellation.GroupActions
  simp only [accGrav, PermGroup, permCharges, LinearMap.coe_mk, AddHom.coe_
```

1.228 Physlib/QFT/QED/AnomalyCancellation/Sorts.lean

```
public import Physlib.QFT.QED.AnomalyCancellation.Permutations
  simp only [Sorted, PureU1_numberCharges, sort, FamilyPermutations, PermGr
set_option backward.isDefEq.respectTransparency false in
```

1.229 Physlib/QFT/QED/AnomalyCancellation/VectorLike.lean

```
public import Physlib.QFT.QED.AnomalyCancellation.Sorts
```

1.230 Physlib/QuantumMechanics/DDimensions/Hydrogen/Basic.lean

```
public import Physlib.QuantumMechanics.DDimensions.Operators.Momentum
public import Physlib.QuantumMechanics.DDimensions.Operators.Position
Hamiltonian in which the potential is replaced by `-k·r(ε)⁻¹`, where `r(ε)² = ‖x‖² +
This goes by several names including "soft-core" and "truncated" Coulomb potential.
e.g. see https://doi.org/10.1103/PhysRevA.80.032507 and https://doi.org/10.1063/1.32
set_option backward.isDefEq.respectTransparency false in
  `□(ε) ≔ (2m)⁻¹□² - k·□(ε)⁻¹`. -/
  (2 * H.m)⁻¹ • (□ □ᵥ □) - H.k • □₀ ε (-1)
```

1.231 Physlib/QuantumMechanics/DDimensions/Hydrogen/LaplaceRungeLenzVe

```
/-
Copyright (c) 2026 Gregory J. Loges. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Gregory J. Loges
-/
module

public import Physlib.QuantumMechanics.DDimensions.Hydrogen.Basic
public import Physlib.QuantumMechanics.DDimensions.Operators.Commutation
public import Physlib.Meta.Linters.Sorry
/-!

# Laplace-Runge-Lenz vector

In this file we define
- The (regularized) LRL vector operator for the quantum mechanical hydrogen atom,
  `□(ε)ᵢ ≔ ½(□ⱼ□ᵢⱼ + □ᵢⱼ□ⱼ) - mk·□(ε)⁻¹□ᵢ`.

The main results are
- The commutators `⁅□ᵢⱼ, □(ε)ₖ⁆ = iℏ(δᵢₖ□(ε)ⱼ - δⱼₖ□(ε)ᵢ)` in `angularMomentum_commu
- The commutators `⁅□(ε)ᵢ, □(ε)ⱼ⁆ = (-2iℏm·□(ε) + iℏmkε²·□(ε)⁻³))□ᵢⱼ` in `lrl_commut
- The commutators `⁅□(ε), □(ε)ᵢ⁆ = iℏε²(⋯)` in `hamiltonianReg_commutation_lrl`
- The relation `□(ε)² = 2m □(ε)(□² + ¼ℏ²(d-1)²) + m²k² + ε²(⋯)` in `lrlOperatorSqr_e

-/

@[expose] public section

namespace QuantumMechanics
```

```
namespace HydrogenAtom
noncomputable section
open Complex Constants
open KroneckerDelta
open ContinuousLinearMap SchwartzMap

variable (H : HydrogenAtom)

/-- The (regularized) Laplace-Runge-Lenz vector operator for the `d`-
dimensional hydrogen atom,
  `𝐀(ε)ᵢ ≔ ½(∇ⱼ𝐋ᵢⱼ + 𝐋ᵢⱼ∇ⱼ) - mk·𝐫(ε)⁻¹𝐱ᵢ`. -/
def lrlOperator (ε : ℝˣ) (i : Fin H.d) : 𝓢(Space H.d, ℂ) →L[ℂ] 𝓢(Space H.d,
  (2 : ℝ)⁻¹ • (∇ ⬝ᵥ ∇ i + ∇ i ⬝ᵥ ∇) - (H.m * H.k) • 𝐫₀ ε (-
1) ∘L ∇ i

/-- `𝐀(ε)ᵢ = 𝐱ᵢ𝐇² - (∇ⱼ𝐋ⱼ)𝐱ᵢ + ½iℏ(d-1)∇ᵢ - mk·𝐫(ε)⁻¹𝐱ᵢ` -
/
lemma lrlOperator_eq (ε : ℝˣ) (i : Fin H.d) : H.lrlOperator ε i = 𝐱 i ∘L (𝐇
    - (∇ ⬝ᵥ ∇) ∘L ∇ i + (2⁻¹ * I * ℏ * (H.d - 1)) • ∇ i - (H.m * H.k) • 𝐫₀
1) ∘L ∇ i := by
  rw [lrlOperator, sub_left_inj] -- mk·r⁻¹x terms match exactly
  calc
    _ = (2 : ℝ)⁻¹ • ∑ j, ((∇ j ∘L 𝐱 i) ∘L ∇ j + 𝐱 i ∘L ∇ j ∘L ∇ j
        - ((∇ j ∘L 𝐱 j) ∘L ∇ i + 𝐱 j ∘L ∇ j ∘L ∇ i)) := by
      simp_rw [dotProduct, mul_def, ← Finset.sum_add_distrib, angularMoment
        sub_comp, comp_assoc, momentum_comp_commute, ← sub_sub, add_sub, su
    _ = (2 : ℂ)⁻¹ • ∑ j, ((2 : ℂ) • 𝐱 i ∘L ∇ j ∘L ∇ j - (I * ℏ) • δ[i,j] •
        - ((2 : ℂ) • (∇ j ∘L 𝐱 j) ∘L ∇ i - (I * ℏ) • ∇ i)) := by
      simp only [momentum_comp_position_eq, sub_comp, comp_assoc, smul_comp
        sub_add_eq_add_sub, eq_one_of_same, one_smul, ← Complex.coe_smul, o
    _ = (2 : ℂ)⁻¹ • ∑ j, ((2 : ℂ) • 𝐱 i ∘L ∇ j ∘L ∇ j - (2 : ℂ) • (∇ j ∘L 𝐱
        + (I * ℏ) • ∇ i - (I * ℏ) • δ[i,j] • ∇ j) := by
      simp_rw [sub_sub_sub_comm, sub_sub_eq_add_sub]
    _ = 𝐱 i ∘L (∇ ⬝ᵥ ∇) - (∇ ⬝ᵥ ∇) ∘L ∇ i
        + ((2⁻¹ * I * ℏ) • H.d • ∇ i - (2⁻¹ * I * ℏ) • ∇ i) := by
      simp only [add_sub_assoc, Finset.sum_add_distrib, Finset.sum_sub_dist
        ← comp_finset_sum, ← finset_sum_comp, sum_smul, smul_add, smul_sub,
      norm_num
      rfl
    _ = 𝐱 i ∘L (∇ ⬝ᵥ ∇) - (∇ ⬝ᵥ ∇) ∘L ∇ i + (2⁻¹ * I * ℏ * (H.d - 1)) • ∇ i
      simp only [← Nat.cast_smul_eq_nsmul ℂ, smul_smul, ← sub_smul, mul_sub

/-- `𝐀(ε)ᵢ = 𝐋ᵢⱼ∇ⱼ + ½iℏ(d-1)∇ᵢ - mk·𝐫(ε)⁻¹𝐱ᵢ` -/
lemma lrlOperator_eq' (ε : ℝˣ) (i : Fin H.d) : H.lrlOperator ε i =
    ∇ i ⬝ᵥ ∇ + (2⁻¹ * I * ℏ * (H.d - 1)) • ∇ i - (H.m * H.k) • 𝐫₀ ε (-
1) ∘L ∇ i := by
```

```
  rw [lrlOperator_eq, sub_left_inj, add_left_inj]
  symm
  trans ∑ j, ⁅ i ∘L ⁅ j ∘L ⁅ j - ∑ j, (⁅ j ∘L ⁅ j) ∘L ⁅ i
  · simp [dotProduct, mul_def, angularMomentumOperator, comp_assoc, momentum_comp_co
  simp [← comp_finset_sum, ← finset_sum_comp, dotProduct, mul_def]

set_option backward.isDefEq.respectTransparency false in
/-- `⁅(ε)ᵢ = ⁅ⱼ⁅ᵢⱼ - ½iℏ(d-1)⁅ᵢ - mk·⁅(ε)⁻¹⁅ᵢ` -/
lemma lrlOperator_eq'' (ε : ℝˣ) (i : Fin H.d) : H.lrlOperator ε i =
    ⁅ ⁅ᵥ ⁅ i - (2⁻¹ * I * ℏ * (H.d - 1)) • ⁅ i - (H.m * H.k) • ⁅₀ ε (-
1) ∘L ⁅ i := by
  trans (2 : ℝ) • H.lrlOperator ε i - H.lrlOperator ε i
  · simp [two_smul]
  nth_rw 2 [lrlOperator_eq']
  simp only [lrlOperator, smul_add, smul_sub, smul_smul]
  ring_nf
  ext
  simp only [one_smul, coe_sub', coe_smul', Pi.sub_apply, ContinuousLinearMap.add_ap
    Pi.smul_apply, SchwartzMap.sub_apply, SchwartzMap.add_apply, SchwartzMap.smul_ap
    ofReal_mul, ofReal_ofNat]
  ring

/-
## Angular momentum / LRL vector commutators
-/

/-- `⁅⁅ᵢⱼ, ⁅(ε)ₖ⁆ = iℏ(δᵢₖ⁅(ε)ⱼ - δⱼₖ⁅(ε)ᵢ)` -/
@[sorryful]
lemma angularMomentum_commutation_lrl (ε : ℝˣ) (i j k : Fin H.d) : ⁅⁅ i j, H.lrlOper
    (I * ℏ) • (δ[i,k] • H.lrlOperator ε j - δ[j,k] • H.lrlOperator ε i) := by
  sorry

/-- `⁅⁅ᵢⱼ, ⁅(ε)²⁆ = 0` -/
@[sorryful, simp]
lemma angularMomentum_commutation_lrlSqr (ε : ℝˣ) (i j : Fin H.d) :
    ⁅⁅ i j, H.lrlOperator ε ⁅ᵥ H.lrlOperator ε⁆ = 0 := by
  simp only [dotProduct, mul_def, lie_sum, lie_leibniz, H.angularMomentum_commutatio
    comp_smul, comp_sub, smul_comp, sub_comp, ← smul_add, ← Finset.smul_sum, Finset.
    Finset.sum_sub_distrib, sum_smul, sub_add_sub_cancel, sub_self, smul_zero]

/-- `⁅⁅², ⁅(ε)²⁆ = 0` -/
@[sorryful, simp]
lemma angularMomentumSqr_commutation_lrlSqr (ε : ℝˣ) :
    ⁅⁅²[H.d], H.lrlOperator ε ⁅ᵥ H.lrlOperator ε⁆ = 0 := by
  simp [angularMomentumOperatorSqr, sum_lie, leibniz_lie]
```

```
/-

## LRL / LRL commutators

To compute the commutator `⁅□ᵢ(ε), □ⱼ(ε)⁆` we take the following approach:
- Write `□(ε)ᵢ = □ᵢ□² - (□ⱼ□ⱼ)□ᵢ + ½iℏ(d-1)□ᵢ - mk·□(ε)⁻¹□ᵢ =: f1ᵢ - f2ᵢ + f
- Organize the sixteen terms which result from expanding `⁅f1ᵢ-
f2ᵢ+f3ᵢ-f4ᵢ, f1ⱼ-f2ⱼ+f3ⱼ-f4ⱼ⁆`
  into four diagonal terms such as `⁅f1ᵢ, f1ⱼ⁆` and six off-
diagonal pairs such as
  `⁅f1ᵢ, f3ⱼ⁆ + ⁅f3ᵢ, f1ⱼ⁆ = ⁅f1ᵢ, f3ⱼ⁆ - ⁅f1ⱼ, f3ᵢ⁆`.
- Compute the diagonal commutators and off-diagonal pairs individually. Man
  that don't are all of the form `iℏ (…) □ᵢⱼ` (as they must to be antisymme
- Collect terms.

-/

private lemma positionDotMomentum_commutation_position {d : ℕ} (i : Fin d)
    ⁅□[d] □ᵥ □, □ i⁆ = (-I * ℏ) • □ i := by
  trans ∑ j, □ j ∘L ⁅□ j, □ i⁆
  · simp [dotProduct, mul_def, sum_lie, leibniz_lie]
  simp_rw [← lie_skew (□ _), position_commutation_momentum, ← neg_smul, ← n
    comp_id, ← Finset.smul_sum, sum_smul]

private lemma positionDotMomentum_commutation_momentum {d : ℕ} (i : Fin d)
    ⁅□[d] □ᵥ □, □ i⁆ = (I * ℏ) • □ i := by
  trans ∑ j, ⁅□ j, □ i⁆ ∘L □ j
  · simp [dotProduct, mul_def, sum_lie, leibniz_lie]
  simp_rw [position_commutation_momentum, smul_comp, id_comp, ← Finset.smul

private lemma positionDotMomentum_commutation_momentumSqr (d : ℕ) :
    ⁅□[d] □ᵥ □, □[d] □ᵥ □⁆ = (2 * I * ℏ) • (□ □ᵥ □) := by
  trans ∑ i, ⁅□ i, □ □ᵥ □⁆ ∘L □ i
  · rw [dotProduct]
    simp [mul_def, sum_lie, leibniz_lie, ← lie_skew (□ _) (□ □ᵥ □)]
  simp_rw [position_commutation_momentumSqr, smul_comp, ← Finset.smul_sum,

private lemma positionDotMomentum_commutation_radiusRegPow (d : ℕ) (ε : ℝˣ)
    ⁅□[d] □ᵥ □, □₀[d] ε s⁆ = (-s * I * ℏ) • (□₀ ε s - ε.1 ^ 2 • □₀ ε (s-
2)) := by
  calc
    _ = ∑ i, □ i ∘L ⁅□ i, □₀ ε s⁆ := by simp [dotProduct, mul_def, sum_lie,
    _ = (-s * I * ℏ) • (∑ i, □ i ∘L □ i) ∘L □₀ ε (s-2) := by
      simp [← lie_skew (□ _), radiusRegPow_commutation_momentum, Finset.smu
        position_comp_radiusRegPow_commute, finset_sum_comp, comp_assoc]
    _ = (-s * I * ℏ) • (□₀ ε s - ε.1 ^ 2 • □₀ ε (s-2)) := by simp [position
```

```
private lemma positionCompMomentumSqr_comm {d : ℕ} (i j : Fin d) :
    ⁅□ i ∘L (□ ⬝ᵥ □), □ j ∘L (□ ⬝ᵥ □)⁆ = (-2 * I * ℏ) • (□ ⬝ᵥ □) ∘L □ i j := by
  calc
    _ = (□ j ∘L ⁅□ i, □[d] ⬝ᵥ □⁆ + □ i ∘L ⁅□[d] ⬝ᵥ □, □ j⁆) ∘L (□ ⬝ᵥ □) := by
      simp [lie_leibniz, leibniz_lie, comp_assoc]
    _ = (2 * I * ℏ) • □ j i ∘L (□ ⬝ᵥ □) := by
      simp_rw [← lie_skew (□ ⬝ᵥ □) _, position_commutation_momentumSqr, comp_neg, co
        ← sub_eq_add_neg, ← smul_sub, smul_comp, angularMomentumOperator]
    _ = (-2 * I * ℏ) • (□ ⬝ᵥ □) ∘L □ i j := by
      rw [angularMomentumOperator_antisymm j i, neg_comp, smul_neg, ← neg_smul, ← ne
        ← neg_mul, momentumSqr_comp_angularMomentum_commute]

private lemma positionCompMomentumSqr_comm_positionDotMomentumCompMomentum_add
    {d : ℕ} (i j : Fin d) : ⁅□ i ∘L (□ ⬝ᵥ □), (□ ⬝ᵥ □) ∘L □ j⁆ +
    ⁅(□ ⬝ᵥ □) ∘L □ i, □ j ∘L (□ ⬝ᵥ □)⁆ = (-I * ℏ) • (□ ⬝ᵥ □) ∘L □ i j := by
  suffices ∀ k l : Fin d, ⁅□ k ∘L (□ ⬝ᵥ □), (□ ⬝ᵥ □) ∘L □ l⁆
      = (-I * ℏ) • (□ k ∘L □ l - δ[k,l] • (□ ⬝ᵥ □)) ∘L (□ ⬝ᵥ □) by
    nth_rw 2 [← lie_skew]
    simp_rw [this, ← sub_eq_add_neg, ← smul_sub, ← sub_comp, symm j i, sub_sub_sub_c
      momentumSqr_comp_angularMomentum_commute, angularMomentumOperator]
  intro k l
  calc
    _ = (□ ⬝ᵥ □) ∘L ⁅□ k, □ l⁆ ∘L (□ ⬝ᵥ □) + □ k ∘L ⁅□[d] ⬝ᵥ □, □[d] ⬝ᵥ □⁆ ∘L □ l
        + ⁅□ k, □[d] ⬝ᵥ □⁆ ∘L (□ ⬝ᵥ □) ∘L □ l := by
      simp [lie_leibniz, leibniz_lie, add_assoc, comp_assoc]
    _ = (□ ⬝ᵥ □) ∘L ⁅□ k, □ l⁆ ∘L (□ ⬝ᵥ □) + (-I * ℏ) • □ k ∘L □ l ∘L (□ ⬝ᵥ □) := by
      simp only [← lie_skew _ (□ ⬝ᵥ □), positionDotMomentum_commutation_position,
        positionDotMomentum_commutation_momentumSqr, momentumSqr_comp_momentum_commu
        ← neg_smul, ← neg_mul, smul_comp, comp_smul, add_assoc, ← add_smul]
      ring_nf
    _ = (-I * ℏ) • (□ k ∘L □ l - δ[k,l] • (□ ⬝ᵥ □)) ∘L (□ ⬝ᵥ □) := by
      simp_rw [position_commutation_momentum, sub_comp, smul_sub, smul_comp, comp_sm
        id_comp, sub_eq_add_neg, ← neg_smul, neg_mul, neg_neg, comp_assoc, add_comm]

private lemma positionDotMomentumCompMomentum_comm {d : ℕ} (i j : Fin d) :
    ⁅(□ ⬝ᵥ □) ∘L □ i, (□ ⬝ᵥ □) ∘L □ j⁆ = 0 := by
  simp [lie_leibniz, leibniz_lie, momentum_comp_commute,
    ← lie_skew (□ _) (□ ⬝ᵥ □), positionDotMomentum_commutation_momentum, comp_assoc]

private lemma positionCompMomentumSqr_comm_momentum_add {d : ℕ} (i j : Fin d) :
    ⁅□ i ∘L (□ ⬝ᵥ □), □ j⁆ + ⁅□ i, □ j ∘L (□ ⬝ᵥ □)⁆ = 0 := by
  nth_rw 2 [← lie_skew]
  simp [leibniz_lie, position_commutation_momentum, symm j]

private lemma positionDotMomentumCompMomentum_comm_momentum_add {d : ℕ} (i j : Fin d
```

```
    ⁅(□ □ᵥ □) ∘L □ i, □ j⁆ + ⁅□ i, (□ □ᵥ □) ∘L □ j⁆ = 0 := by
  nth_rw 2 [← lie_skew]
  simp [leibniz_lie, positionDotMomentum_commutation_momentum, momentum_com

set_option backward.isDefEq.respectTransparency false in
private lemma positionCompMomentumSqr_comm_radiusRegInvCompPosition_add
    {d : ℕ} (ε : ℝˣ) (i j : Fin d) : ⁅□ i ∘L (□ □ᵥ □), □₀ ε (-
1) ∘L □ j⁆
    + ⁅□₀ ε (-1) ∘L □ i, □ j ∘L (□ □ᵥ □)⁆ = (-2 * I * ℏ) • □₀ ε (-
1) ∘L □ i j := by
  let A := ⁅□[d] □ᵥ □, □₀[d] ε (-1)⁆
  have hA : □ i ∘L A ∘L □ j = □ j ∘L A ∘L □ i := by
    suffices A = (2 * I * ℏ) • □₀[d] ε (-3) ∘L (□ □ᵥ □)
        + ((d - 3) * ℏ ^ 2 : ℝ) • □₀ ε (-3) + (3 * ε.1 ^ 2 * ℏ ^ 2) • □₀ ε
5) by
      simp_rw [this, add_comp, comp_add, smul_comp, comp_smul, ← comp_assoc
        position_comp_radiusRegPow_commute, comp_assoc,
        comp_eq_comp_add_commute (□ □ᵥ □), positionDotMomentum_commutation_
        comp_add, comp_smul, ← comp_assoc (□ _) (□ _) _, position_comp_comm
    simp_rw [A, ← lie_skew (□ □ᵥ □) _, radiusRegPow_commutation_momentumSqr
      sub_eq_add_neg, ← neg_smul, ← neg_mul, neg_neg, ofReal_neg, ofReal_on
    ring_nf
    rw [add_rotate]
  calc
    _ = □₀ ε (-1) ∘L □ i ∘L ⁅□[d] □ᵥ □, □ j⁆ + □ i ∘L A ∘L □ j
        - (□₀ ε (-1) ∘L □ j ∘L ⁅□[d] □ᵥ □, □ i⁆ + □ j ∘L A ∘L □ i) := by
      nth_rw 2 [← lie_skew]
      simp_rw [← sub_eq_add_neg, lie_leibniz, leibniz_lie, ← sub_sub]
      simp [A, comp_assoc]
    _ = □₀ ε (-1) ∘L (□ i ∘L ⁅□[d] □ᵥ □, □ j⁆ - □ j ∘L ⁅□[d] □ᵥ □, □ i⁆) :=
    _ = (-2 * I * ℏ) • □₀ ε (-1) ∘L □ i j := by
      simp_rw [← lie_skew _ (□ _), position_commutation_momentumSqr, comp_n
        ← neg_smul, ← neg_mul, ← smul_sub, comp_smul, angularMomentumOperat

private lemma momentum_comm_radiusRegPow_position_symm {d : ℕ} (ε : ℝˣ) (s
    ⁅□ i, □₀ ε s ∘L □ j⁆ = ⁅□ j, □₀ ε s ∘L □ i⁆ := by
  simp [← lie_skew (□ _), leibniz_lie, position_commutation_momentum, symm
    radiusRegPow_commutation_momentum, position_comp_commute j i, comp_asso

set_option backward.isDefEq.respectTransparency false in
private lemma positionDotMomentumCompMomentum_comm_radiusRegInvCompPosition
    {d : ℕ} (ε : ℝˣ) (i j : Fin d) : ⁅(□ □ᵥ □) ∘L □ i, □₀ ε (-
1) ∘L □ j⁆ +
    ⁅□₀ ε (-1) ∘L □ i, (□ □ᵥ □) ∘L □ j⁆ = (I * ℏ * ε.1 ^ 2) • □₀ ε (-
3) ∘L □ i j := by
  suffices ∀ k, ⁅□[d] □ᵥ □, □₀[d] ε (-1) ∘L □ k⁆ = (-I * ℏ * ε.1 ^ 2) • □₀
```

```
3) ∘L ∇ k by
    nth_rw 2 [← lie_skew]
    simp_rw [leibniz_lie, this,
      momentum_comm_radiusRegPow_position_symm, ← sub_eq_add_neg, add_sub_add_left_e
      smul_comp, ← smul_sub, comp_assoc, ← comp_sub, angularMomentumOperator_antisym
      smul_neg, neg_mul, neg_smul, angularMomentumOperator]
  intro k
  calc
    _ = -(I * ℏ) • (ε.1 ^ 2) • ∇₀ ε (-1-2) ∘L ∇ k := by
      simp [lie_leibniz, positionDotMomentum_commutation_position,
        positionDotMomentum_commutation_radiusRegPow, sub_comp, smul_sub, ← add_sub_
    _ = (-I * ℏ * ε.1 ^ 2) • ∇₀ ε (-3) ∘L ∇ k := by
      simp only [← Complex.coe_smul, ofReal_pow, smul_smul]
      ring_nf

set_option backward.isDefEq.respectTransparency false in
private lemma momentum_comm_radiusRegInvCompPosition_add {d : ℕ} (ε : ℝˣ) (i j : Fin
    ⁅∇ i, ∇₀ ε (-1) ∘L ∇ j⁆ + ⁅∇₀ ε (-1) ∘L ∇ i, ∇ j⁆ = 0 := by
  rw [← lie_skew _ (∇ _), momentum_comm_radiusRegPow_position_symm, add_neg_cancel]

set_option backward.isDefEq.respectTransparency false in
private lemma radiusRegInvCompPosition_comm {d : ℕ} (ε : ℝˣ) (i j : Fin d) :
    ⁅∇₀ ε (-1) ∘L ∇ i, ∇₀ ε (-1) ∘L ∇ j⁆ = 0 := by
  simp [lie_leibniz, leibniz_lie, ← lie_skew (∇₀ _ _) (∇ _)]

/-- `⁅∇(ε)ᵢ, ∇(ε)ⱼ⁆ = (-2iℏm·𝓡(ε) + iℏmkε²·𝓡(ε)⁻³)∇ᵢⱼ` -/
lemma lrl_commutation_lrl (ε : ℝˣ) (i j : Fin H.d) :
    ⁅H.lrlOperator ε i, H.lrlOperator ε j⁆ = ((-2 * I * ℏ * H.m) • H.hamiltonianReg
    + (I * ℏ * H.m * H.k * ε.1 ^ 2) • ∇₀ ε (-3)) ∘L ∇ i j := by
  repeat rw [lrlOperator_eq]
  let c₁ : ℂ := 2⁻¹ * I * ℏ * (H.d - 1)
  let c₂ : ℂ := H.m * H.k
  trans ⁅∇ i ∘L (∇ ⬝ᵥ ∇), ∇ j ∘L (∇ ⬝ᵥ ∇)⁆ + ⁅(∇ ⬝ᵥ ∇) ∘L ∇ i, (∇ ⬝ᵥ ∇) ∘L ∇ j⁆
      + (c₂ ^ 2) • ⁅∇₀ ε (-1) ∘L ∇ i, ∇₀ ε (-1) ∘L ∇ j⁆
      - (⁅∇ i ∘L (∇ ⬝ᵥ ∇), (∇ ⬝ᵥ ∇) ∘L ∇ j⁆ + ⁅(∇ ⬝ᵥ ∇) ∘L ∇ i, ∇ j ∘L (∇ ⬝ᵥ ∇)⁆)
      + c₁ • (⁅∇ i ∘L (∇ ⬝ᵥ ∇), ∇ j⁆ + ⁅∇ i, ∇ j ∘L (∇ ⬝ᵥ ∇)⁆)
      - c₂ • (⁅∇ i ∘L (∇ ⬝ᵥ ∇), ∇₀ ε (-1) ∘L ∇ j⁆ + ⁅∇₀ ε (-
1) ∘L ∇ i, ∇ j ∘L (∇ ⬝ᵥ ∇)⁆)
      - c₁ • (⁅(∇ ⬝ᵥ ∇) ∘L ∇ i, ∇ j⁆ + ⁅∇ i, (∇ ⬝ᵥ ∇) ∘L ∇ j⁆)
      + c₂ • (⁅(∇ ⬝ᵥ ∇) ∘L ∇ i, ∇₀ ε (-1) ∘L ∇ j⁆ + ⁅∇₀ ε (-
1) ∘L ∇ i, (∇ ⬝ᵥ ∇) ∘L ∇ j⁆)
      - (c₁ * c₂) • (⁅∇ i, ∇₀ ε (-1) ∘L ∇ j⁆ + ⁅∇₀ ε (-1) ∘L ∇ i, ∇ j⁆)
  · simp only [lie_add, add_lie, lie_sub, sub_lie, lie_smul, smul_lie,
      momentum_commutation_momentum, smul_zero, add_zero, ← Complex.coe_smul, ofReal
    subst c₁ c₂
    ext
```

```
    simp only [coe_sub', coe_smul', Pi.sub_apply, ContinuousLinearMap.add_a
      SchwartzMap.sub_apply, SchwartzMap.add_apply, SchwartzMap.smul_apply,
    ring
  rw [positionCompMomentumSqr_comm]
  rw [positionDotMomentumCompMomentum_comm]
  rw [positionCompMomentumSqr_comm_positionDotMomentumCompMomentum_add]
  rw [positionCompMomentumSqr_comm_momentum_add]
  rw [positionDotMomentumCompMomentum_comm_momentum_add]
  rw [positionCompMomentumSqr_comm_radiusRegInvCompPosition_add]
  rw [positionDotMomentumCompMomentum_comm_radiusRegInvCompPosition_add]
  rw [momentum_comm_radiusRegInvCompPosition_add]
  rw [radiusRegInvCompPosition_comm]
  subst c₁ c₂
  simp_rw [hamiltonianReg, smul_zero, add_zero, sub_zero, ← sub_smul, ← Com
    ofReal_inv, ofReal_mul, ofReal_ofNat, smul_sub, smul_smul, add_comp, su
  ring_nf
  simp

/-
## Hamiltonian / LRL vector commutators
-/

private lemma pSqr_comm_pL_Lp {d : ℕ} (i : Fin d) :
    ⁅□[d] □ᵥ □, □ □ᵥ □ i + □ i □ᵥ □⁆ = 0 := by
  show ⁅□ □ᵥ □, ∑ j, □ j ∘L □ i j + ∑ j, □ i j ∘L □ j⁆ = 0
  simp [lie_sum, lie_leibniz, ← lie_skew (□ □ᵥ □) (□ _ _)]

private lemma r_comm_rx {d : ℕ} (ε : ℝˣ) (i : Fin d) : ⁅□₀[d] ε (-
1), □₀ ε (-1) ∘L □ i⁆ = 0 := by
  simp [lie_leibniz, ← lie_skew (□₀ _ _) (□ _)]

private lemma xL_Lx_eq {d : ℕ} (ε : ℝˣ) (i : Fin d) :
    □ □ᵥ □ i + □ i □ᵥ □ = (2 : ℝ) • (□ □ᵥ □) ∘L □ i + (-I * ℏ * (d - 3)) •
    + ((-2 : ℝ) • □₀ ε 2 ∘L □ i + (2 * ε.1 ^ 2 : ℝ) • □ i) := by
  -- Change summand
  simp_rw [dotProduct, mul_def, ← Finset.sum_add_distrib, angularMomentumOp
    sub_comp, comp_assoc, sub_add_sub_comm, momentum_comp_position_eq, comp
    comp_id, ← comp_assoc, position_comp_commute i _, ← add_sub_assoc, ← tw
    sub_sub_eq_add_sub, sub_add_eq_add_sub, comp_assoc, comp_eq_comp_add_co
    position_commutation_momentum, symm _ i, comp_add, comp_smul, smul_add,
    ← Complex.coe_smul, smul_smul, ← add_smul, ← comp_assoc, eq_one_of_same
  -- Split/do sums
  simp_rw [Finset.sum_sub_distrib, Finset.sum_add_distrib, ← Finset.smul_su
    sum_smul, Finset.sum_const, Finset.card_univ, Fintype.card_fin, ← Nat.c
    positionOperatorSqr_eq ε, sub_comp, smul_comp, id_comp, smul_sub]
  -- Clean up coefficients
```

```
  simp_rw [add_sub_assoc, add_right_inj, smul_smul, ← sub_smul, sub_eq_add_neg, neg_
    ← Complex.coe_smul, smul_smul, ofReal_neg, ofReal_mul, ofReal_pow, ofReal_ofNat,
  ring_nf

set_option backward.isDefEq.respectTransparency false in
private lemma pSqr_comm_rx {d : ℕ} (ε : ℝˣ) (i : Fin d) :
    ⁅□[d] □ᵥ □, □₀ ε (-1) ∘L □ i⁆ = (I * ℏ) • □₀ ε (-3) ∘L (□ □ᵥ □ i + □ i □ᵥ □)
    + ((-2 * I * ℏ * ε.1 ^ 2) • □₀ ε (-3) ∘L □ i
      + (3 * ℏ ^ 2 * ε.1 ^ 2) • □₀ ε (-5) ∘L □ i) := by
  trans (-2 * I * ℏ) • □₀ ε (-1) ∘L □ i + (2 * I * ℏ) • □₀ ε (-
3) ∘L (□ □ᵥ □) ∘L □ i
      + (ℏ ^ 2 * (d - 3) : ℝ) • □₀ ε (-3) ∘L □ i + (3 * ℏ ^ 2 * ε.1 ^ 2) • □₀ ε (-
5) ∘L □ i
  · rw [← lie_skew]
    simp_rw [leibniz_lie, position_commutation_momentumSqr, radiusRegPow_commutation
      sub_comp, add_comp, smul_comp, comp_smul, comp_assoc, neg_add, neg_sub', neg_a
      sub_neg_eq_add, ← neg_smul, add_assoc, ofReal_neg, ofReal_one, dotProduct, mul
    ring_nf
  rw [← add_assoc, add_left_inj, add_rotate]
  simp_rw [xL_Lx_eq ε i, comp_add, comp_smul, smul_add, ← Complex.coe_smul, smul_smu
    radiusRegPowOperator_comp_eq, comp_assoc, add_assoc, ← add_smul, ofReal_mul, ofR
    ofReal_neg, ofReal_pow, ofReal_ofNat]
  ring_nf
  simp [I_sq]

private lemma r_comm_pL_Lp {d : ℕ} (ε : ℝˣ) (i : Fin d) :
    ⁅□₀[d] ε (-1), □ □ᵥ □ i + □ i □ᵥ □⁆ =
    -((I * ℏ) • □₀ ε (-3) ∘L (□ □ᵥ □ i + □ i □ᵥ □)) := by
  calc
    _ = ∑ j, (⁅□₀[d] ε (-1), □ j⁆ ∘L □ i j + □ i j ∘L ⁅□₀[d] ε (-
1), □ j⁆) := by
      simp [dotProduct, mul_def, ← Finset.sum_add_distrib, lie_sum, lie_leibniz,
        ← lie_skew _ (□ _ _)]
    _ = -((I * ℏ) • ∑ j, (□₀ ε (-3) ∘L □ j ∘L □ i j + (□ i j ∘L □₀ ε (-
3)) ∘L □ j)) := by
      simp only [radiusRegPow_commutation_momentum, smul_comp, comp_smul, ← smul_add
        ← Finset.smul_sum, comp_assoc, ofReal_neg, ofReal_one, ← neg_smul]
      ring_nf
    _ = -((I * ℏ) • □₀ ε (-3) ∘L (□ □ᵥ □ i + □ i □ᵥ □)) := by
      simp_rw [angularMomentum_comp_radiusRegPow_commute, comp_assoc, ← comp_add, ←
        Finset.sum_add_distrib, dotProduct, mul_def]

set_option backward.isDefEq.respectTransparency false in
/-- `⁅□(ε), □(ε)ᵢ⁆ = iℏk·ε²□(ε)⁻³□ᵢ - 3ℏ²k/2·ε²□(ε)⁻⁵□ᵢ` -
/
lemma hamiltonianReg_commutation_lrl (ε : ℝˣ) (i : Fin H.d) :
```

```
    ⁅H.hamiltonianReg ε, H.lrlOperator ε i⁆ = (I * ℏ * H.k * ε.1 ^ 2) • ∇₀
3) ∘L ∇ i
    - (3 / 2 * ℏ ^ 2 * H.k * ε.1 ^ 2) • ∇₀ ε (-5) ∘L ∇ i := by
  trans (-2⁻¹ * H.k) • (⁅∑[H.d] ∇ᵥ ∇, ∇₀ ε (-1) ∘L ∇ i⁆
      + ⁅∇₀[H.d] ε (-1), ∑ ∇ᵥ ∇ i + ∑ i ∇ᵥ ∇⁆)
  · have h : H.m * H.k * (H.m⁻¹ * 2⁻¹) = 2⁻¹ * H.k := by grind [m_ne_zero]
    simp only [hamiltonianReg, lrlOperator, lie_sub, sub_lie, smul_lie, lie
    simp [r_comm_rx, h, smul_smul, sub_eq_neg_add, add_comm]
  simp_rw [pSqr_comm_rx, r_comm_pL_Lp, add_neg_cancel_comm, smul_add, sub_e
    ← neg_mul, ← Complex.coe_smul, smul_smul, ofReal_mul, ofReal_neg, ofRea
    ofReal_pow, ofReal_ofNat]
  ring_nf

/-

## LRL vector squared

To compute `∑(ε)²` we take the following approach:
- Write `∑(ε)ᵢ = ∇ᵢⱼ∇ⱼ + ½iℏ(d-1)∇ᵢ - mk·∑(ε)⁻¹∇ᵢ` for the first term and
  `∑(ε)ᵢ = ∇ⱼ∇ᵢⱼ - ½iℏ(d-1)∇ᵢ - mk·∑(ε)⁻¹∇ᵢ` for the second.
- Expand out to nine terms: one is a triple sum, two are double sums and th
- Compute each term, symmetrizing the sums (see `sum_symmetrize` and `sum_s
- Collect terms.

-/

private lemma sum_symmetrize {d : ℕ} (f : Fin d → Fin d → ∑(Space d, ℂ) →L[
    ∑ i, ∑ j, f i j = (2 : ℂ)⁻¹ • ∑ i, ∑ j, (f i j + f j i) := by
  simp only [Finset.sum_add_distrib]
  nth_rw 3 [Finset.sum_comm]
  rw [← two_smul ℂ, smul_smul, inv_mul_cancel_of_invertible, one_smul]

private lemma sum_symmetrize' {d : ℕ}
    (f : Fin d → Fin d → Fin d → ∑(Space d, ℂ) →L[ℂ] ∑(Space d, ℂ)) :
    ∑ i, ∑ j, ∑ k, f i j k = (2 : ℂ)⁻¹ • ∑ i, ∑ k, ∑ j, (f i j k + f k j i)
  simp only [Finset.sum_add_distrib]
  nth_rw 3 [Finset.sum_comm]
  rw [← two_smul ℂ, smul_smul, inv_mul_cancel_of_invertible, one_smul]
  congr with
  rw [Finset.sum_comm]

private lemma sum_Lpp (d : ℕ) : ∑ i : Fin d, ∑ j, ∇ i j ∘L ∇ j ∘L ∇ i = 0 :
  rw [sum_symmetrize]
  conv_lhs =>
    enter [2, 2, i, 2, j]
    rw [angularMomentumOperator_antisymm j i, momentum_comp_commute j i]
```

```
  simp

private lemma sum_ppL (d : ℕ) : ∑ i : Fin d, ∑ j, ∇ i ◇L ∇ j ◇L 𝐫 i j = 0 := by
  rw [sum_symmetrize]
  conv_lhs =>
    enter [2, 2, i, 2, j]
    rw [← comp_assoc, ← comp_assoc, angularMomentumOperator_antisymm j i, momentum_c
  simp

private lemma sum_LppL (d : ℕ) :
    ∑ i : Fin d, ∑ j, ∑ k, ∇ i j ◇L ∇ j ◇L ∇ k ◇L 𝐫 i k = (∇ ⬝ᵥ ∇) ◇L 𝐫² := by
  rw [sum_symmetrize']
  conv_lhs =>
    enter [2, 2, i, 2, j, 2, k]
    calc
      _ = (∇ i k ◇L ∇ k ◇L ∇ j - ∇ j k ◇L ∇ k ◇L ∇ i) ◇L 𝐫 i j := by
        simp [angularMomentumOperator_antisymm j i, comp_neg, ← comp_assoc, sub_eq_a
      _ = (∇ i ◇L ∇ k ◇L ∇ k ◇L ∇ j - ∇ k ◇L ∇ i ◇L ∇ k ◇L ∇ j
          - (∇ j ◇L ∇ k ◇L ∇ k ◇L ∇ i - ∇ k ◇L ∇ j ◇L ∇ k ◇L ∇ i)) ◇L 𝐫 i j := by
        simp_rw [angularMomentumOperator, sub_comp, comp_assoc]
      _ = (∇ i ◇L ∇ j ◇L ∇ k ◇L ∇ k - ∇ j ◇L ∇ i ◇L ∇ k ◇L ∇ k) ◇L 𝐫 i j := by
        simp_rw [momentum_comp_commute k, ← comp_assoc (∇ _), momentum_comp_commute
          momentum_comp_commute i j, sub_sub_sub_cancel_right]
      _ = ∇ i j ◇L (∇ k ◇L ∇ k) ◇L 𝐫 i j := by
        simp_rw [← comp_assoc, ← sub_comp, angularMomentumOperator]
  trans (2 : ℂ)⁻¹ • ∑ i, ∑ j, ∇ i j ◇L (∇ ⬝ᵥ ∇) ◇L 𝐫 i j
  · simp_rw [← comp_finset_sum, ← finset_sum_comp, ← comp_assoc, dotProduct, mul_def
  simp_rw [← comp_assoc, ← momentumSqr_comp_angularMomentum_commute, comp_assoc, ← c
    ← comp_smul, angularMomentumOperatorSqr]

private lemma sum_Lprx (d : ℕ) (ε : ℝˣ) :
    ∑ i, ∑ j, ∇ i j ◇L 𝐫 j ◇L 𝐫₀[d] ε (-1) ◇L 𝐫 i = 𝐫₀ ε (-
1) ◇L 𝐫² := by
  simp_rw [← position_comp_radiusRegPow_commute, ← comp_assoc, ← finset_sum_comp _ (
  rw [sum_symmetrize]
  conv_lhs =>
    enter [1, 2, 2, i, 2, j]
    calc
      _ = ∇ i j ◇L (∇ j ◇L 𝐫 i - ∇ i ◇L 𝐫 j) := by
        simp [angularMomentumOperator_antisymm j i, comp_assoc, sub_eq_add_neg]
      _ = ∇ i j ◇L 𝐫 i j := by
        simp [momentum_comp_position_eq, symm j i, angularMomentumOperator]
  rw [← angularMomentumSqr_comp_radiusRegPow_commute, angularMomentumOperatorSqr]

private lemma sum_rxpL (d : ℕ) (ε : ℝˣ) :
    ∑ i, ∑ j, 𝐫₀[d] ε (-1) ◇L 𝐫 i ◇L ∇ j ◇L 𝐫 i j = 𝐫₀ ε (-
```

```
1) ∘L □² := by
  simp_rw [← comp_finset_sum (□₀ _ _)]
  rw [sum_symmetrize]
  conv_lhs =>
    enter [2, 2, 2, i, 2, j]
    rw [angularMomentumOperator_antisymm j i, comp_neg, comp_neg, ← sub_eq_
      ← comp_assoc, ← sub_comp, ← angularMomentumOperator]
  rw [angularMomentumOperatorSqr]

private lemma sum_prx (d : ℕ) (ε : ℝˣ) :
    ∑ i, □ i ∘L □₀[d] ε (-1) ∘L □ i = □₀ ε (-1) ∘L (□ ⬝ᵥ □)
    - (I * ℏ * (d - 1)) • □₀ ε (-1) - (I * ℏ * ε.1 ^ 2) • □₀ ε (-
3) := by
  calc
    _ = ∑ i, (□₀ ε (-1) ∘L □ i ∘L □ i + (I * ℏ) • □₀ ε (-
3) ∘L □ i ∘L □ i) := by
      simp_rw [← comp_assoc, momentum_comp_radiusRegPow_eq]
      ring_nf
      simp
    _ = ∑ i, (□₀ ε (-1) ∘L □ i ∘L □ i - (I * ℏ) • □₀ ε (-
1)
        + (I * ℏ) • □₀ ε (-3) ∘L □ i ∘L □ i) := by simp [momentum_comp_posi
    _ = □₀ ε (-1) ∘L ∑ i, □ i ∘L □ i + (-d * I * ℏ) • □₀ ε (-
1)
        + (I * ℏ) • □₀ ε (-3) ∘L ∑ i, □ i ∘L □ i := by
      simp [Finset.sum_add_distrib, ← comp_finset_sum, ← Finset.smul_sum,
        ← Nat.cast_smul_eq_nsmul ℂ, smul_smul, mul_assoc, sub_eq_add_neg]
    _ = □₀ ε (-1) ∘L (□ ⬝ᵥ □) - (I * ℏ * (d - 1)) • □₀ ε (-
1)
        - (I * ℏ * ε.1 ^ 2) • □₀ ε (-3) := by
      simp only [dotProduct, mul_def, positionOperatorSqr_eq ε, comp_sub, c
        radiusRegPowOperator_comp_eq, smul_sub, ← Complex.coe_smul, ofReal_
      ring_nf
      simp_rw [← add_sub_assoc, add_assoc, ← add_smul, sub_eq_add_neg, ← ne
      ring_nf

private lemma sum_rxp (d : ℕ) (ε : ℝˣ) :
    ∑ i, □₀[d] ε (-1) ∘L □ i ∘L □ i = □₀ ε (-1) ∘L (□ ⬝ᵥ □) := by rw [← com

set_option backward.isDefEq.respectTransparency false in
private lemma sum_rxrx (d : ℕ) (ε : ℝˣ) : ∑ i, □₀[d] ε (-
1) ∘L □ i ∘L □₀ ε (-1) ∘L □ i =
    ContinuousLinearMap.id ℂ □(Space d, ℂ) - (ε.1 ^ 2) • □₀ ε (-
2) := by
  simp_rw [← comp_finset_sum, ← comp_assoc, position_comp_radiusRegPow_comm
    ← comp_finset_sum, ← comp_assoc, radiusRegPowOperator_comp_eq, position
```

```
  ring_nf
  simp

set_option backward.isDefEq.respectTransparency false in
/-- The square of the (regularized) LRL vector operator is related to the (regulariz
  `□(ε)` of the hydrogen atom, square of the angular momentum `□²` and powers of `□(
  `□(ε)² = 2m·□(ε)(□² + ¼ħ²(d-1)²) + m²k²(□ - ε²·□(ε)⁻²) - ½(d-
1)mkħ²ε²□(ε)⁻³`. -/
lemma lrlOperatorSqr_eq (ε : ℝˣ) : H.lrlOperator ε □ᵥ H.lrlOperator ε =
    (2 * H.m) • (H.hamiltonianReg ε) ∘L
      (□² + (4⁻¹ * ℏ ^ 2 * (H.d - 1) ^ 2 : ℝ) • ContinuousLinearMap.id ℂ □(Space H.d
    + (H.m ^ 2 * H.k ^ 2) • (ContinuousLinearMap.id ℂ □(Space H.d, ℂ) - ε.1 ^ 2 • □₀
2))
    - (2⁻¹ * ℏ^2 * H.m * H.k * (H.d - 1) * ε.1 ^ 2) • □₀ ε (-
3) := by
  simp_rw [dotProduct, mul_def]
  conv_lhs => enter [2, i, 1]; rw [lrlOperator_eq']
  conv_lhs => enter [2, i, 2]; rw [lrlOperator_eq'']
  simp_rw [dotProduct, mul_def, sub_eq_add_neg, ← neg_smul, add_comp, comp_add, smul
    comp_smul, finset_sum_comp, comp_finset_sum, Finset.sum_add_distrib, ← Finset.sm
    comp_assoc, sum_LppL, sum_Lpp, sum_Lprx, sum_ppL, sum_prx, sum_rxpL, sum_rxp, su
  simp only [hamiltonianReg, sub_eq_add_neg, ← neg_smul, smul_zero, zero_add, add_ze
    smul_smul, dotProduct, mul_def, ← Complex.coe_smul, ofReal_mul, ofReal_neg, ofRe
    ofReal_pow, smul_add, ofReal_ofNat]
  ring_nf
  ext
  simp only [ContinuousLinearMap.add_apply, ContinuousLinearMap.smul_apply, Schwartz
    SchwartzMap.smul_apply, Function.comp_apply, coe_comp', coe_id', smul_eq_mul, of
    ofReal_neg, ofReal_one, ofReal_natCast]
  grind [I_sq, m_ne_zero, mul_inv_cancel₀, ofReal_eq_zero]

end
end HydrogenAtom
end QuantumMechanics
```

1.232 Physlib/QuantumMechanics/DDimensions/Operators/AngularMomentum.l

```
public import Physlib.QuantumMechanics.DDimensions.Operators.Position
public import Physlib.QuantumMechanics.DDimensions.Operators.Momentum
- `□` for `angularMomentumOperator`
set_option backward.isDefEq.respectTransparency false in
  □ i ∘L □ j - □ j ∘L □ i
notation "□" => angularMomentumOperator
```

```
@[inherit_doc angularMomentumOperator]
notation "□[" d' "]" => angularMomentumOperator (d := d')
    □ i j ψ = □ i (□ j ψ) - □ j (□ i ψ) := rfl
    □ i j ψ x = □ i (□ j ψ) x - □ j (□ i ψ) x := rfl
lemma angularMomentumOperator_antisymm {d : ℕ} (i j : Fin d) : □ i j = -
□ j i :=
lemma angularMomentumOperator_eq_zero {d : ℕ} (i : Fin d) : □ i i = 0 := su
set_option backward.isDefEq.respectTransparency false in
  (2 : ℂ)⁻¹ • ∑ i, ∑ j, □ i j ∘L □ i j
@[inherit_doc angularMomentumOperatorSqr]
notation "□²[" d' "]" => angularMomentumOperatorSqr (d := d')

set_option backward.isDefEq.respectTransparency false in
    □² ψ = (2 : ℂ)⁻¹ • ∑ i, ∑ j, □ i j (□ i j ψ) := by
    □² ψ x = (2 : ℂ)⁻¹ * ∑ i, ∑ j, □ i j (□ i j ψ) x := by
lemma angularMomentumOperator1D_trivial : □[1] = 0 := by
  ext i j
  simp [angularMomentumOperator_eq_zero]
set_option backward.isDefEq.respectTransparency false in
def angularMomentumOperator2D : □(Space 2, ℂ) →L[ℂ] □(Space 2, ℂ) := □ 0 1
set_option backward.isDefEq.respectTransparency false in
    | 0 => □ 1 2
    | 1 => □ 2 0
    | 2 => □ 0 1
```

1.233 Physlib/QuantumMechanics/DDimensions/Operators/Commutatio

```
public import Physlib.Mathematics.KroneckerDelta
public import Physlib.QuantumMechanics.DDimensions.Operators.AngularMomentu
lemma position_commutation_position : ⁅□ i, □ j⁆ = 0 := by
lemma position_comp_commute : □ i ∘L □ j = □ j ∘L □ i := by
lemma position_commutation_radiusRegPow : ⁅□ i, □₀[d] ε s⁆ = 0 := by
lemma position_comp_radiusRegPow_commute : □ i ∘L □₀ ε s = □₀ ε s ∘L □ i :=
lemma radiusRegPow_commutation_radiusRegPow : ⁅□₀[d] ε s, □₀[d] ε t⁆ = 0 :=
set_option backward.isDefEq.respectTransparency false in
lemma momentum_commutation_momentum : ⁅□ i, □ j⁆ = 0 := by
  show □ i (□ j ψ) x - □ j (□ i ψ) x = 0
lemma momentum_comp_commute : □ i ∘L □ j = □ j ∘L □ i := by
lemma momentumSqr_commutation_momentum : ⁅□[d] □ᵥ □, □ i⁆ = 0 := by
  simp [dotProduct, mul_def, sum_lie, leibniz_lie]
lemma momentumSqr_comp_momentum_commute : (□ □ᵥ □) ∘L □ i = □ i ∘L (□ □ᵥ □)
set_option backward.isDefEq.respectTransparency false in
lemma position_commutation_momentum : ⁅□ i, □ j⁆ =
  show □ i (□ j ψ) x - □ j (□ i ψ) x = _
```

```
lemma momentum_comp_position_eq : 𝐏 j ∘L 𝐗 i =
    𝐗 i ∘L 𝐏 j - (I * ℏ) • δ[i,j] • ContinuousLinearMap.id ℂ 𝓢(Space d, ℂ) := by
lemma position_position_commutation_momentum : ⁅𝐗 i ∘L 𝐏 j, 𝐗 k⁆ =
    (I * ℏ) • (δ[i,k] • 𝐗 j + δ[j,k] • 𝐗 i) := by
lemma position_commutation_momentum_momentum : ⁅𝐗 i, 𝐏 j ∘L 𝐏 k⁆ =
    (I * ℏ) • (δ[i,k] • 𝐏 j + δ[i,j] • 𝐏 k) := by
lemma position_commutation_momentumSqr : ⁅𝐗 i, 𝐏 ⬝ᵥ 𝐏⁆ = (2 * I * ℏ) • 𝐏 i := by
  simp only [dotProduct, mul_def, lie_sum, lie_leibniz, position_commutation_momentu
set_option backward.isDefEq.respectTransparency false in
    ⁅𝐗₀[d] ε s, 𝐏 i⁆ = (s * I * ℏ) • 𝐗₀ ε (s-2) ∘L 𝐏 i := by
  show 𝐗₀ ε s (𝐏 i ψ) x - 𝐏 i (𝐗₀ ε s ψ) x = (s * I * ℏ) * 𝐗₀ ε (s-
2) (𝐏 i ψ) x
    𝐏 i ∘L 𝐗₀ ε s = 𝐗₀ ε s ∘L 𝐏 i - (s * I * ℏ) • 𝐗₀ ε (s-
2) ∘L 𝐏 i := by
set_option backward.isDefEq.respectTransparency false in
    ⁅𝐗₀[d] ε s, 𝐗[d] ⬝ᵥ 𝐏⁆ = (2 * s * I * ℏ) • 𝐗₀ ε (s-2) ∘L (𝐗 ⬝ᵥ 𝐏)
    + (s * (d + s - 2) * ℏ ^ 2) • 𝐗₀ ε (s-2) - (ε ^ 2 * s * (s - 2) * ℏ ^ 2) • 𝐗₀ ε
4) := by
    _ = (s * I * ℏ) • ∑ i, ((𝐗 i ∘L 𝐗₀ ε (s-2)) ∘L 𝐏 i + 𝐗₀ ε (s-
2) ∘L 𝐏 i ∘L 𝐏 i) := by
      simp [dotProduct, mul_def, lie_sum, lie_leibniz, radiusRegPow_commutation_mome
    _ = (s * I * ℏ) • ∑ i, (𝐗₀ ε (s-2) ∘L 𝐏 i ∘L 𝐏 i + 𝐗₀ ε (s-
2) ∘L 𝐏 i ∘L 𝐏 i
      - (↑(s - 2) * I * ℏ) • 𝐗₀ ε (s-4) ∘L 𝐏 i ∘L 𝐏 i) := by
    _ = (s * I * ℏ) • ∑ i, ((2 : ℂ) • 𝐗₀ ε (s-2) ∘L 𝐏 i ∘L 𝐏 i - (I * ℏ) • 𝐗₀ ε (s-
2)
      - ((s - 2) * I * ℏ) • 𝐗₀ ε (s-4) ∘L 𝐏 i ∘L 𝐏 i) := by
    _ = (s * I * ℏ) • ((2 : ℂ) • 𝐗₀ ε (s-2) ∘L (𝐗 ⬝ᵥ 𝐏) - (d * I * ℏ) • 𝐗₀ ε (s-
2)
      - ((s - 2) * I * ℏ) • 𝐗₀ ε (s-4) ∘L ∑ i, 𝐏 i ∘L 𝐏 i) := by
      simp [Finset.sum_sub_distrib, ← Finset.smul_sum, ← comp_finset_sum,
        ← Nat.cast_smul_eq_nsmul ℂ, smul_smul, dotProduct, mul_def, mul_assoc]
    _ = (2 * s * I * ℏ) • 𝐗₀ ε (s-2) ∘L (𝐗 ⬝ᵥ 𝐏) + (s * (d + s - 2) * ℏ ^ 2) • 𝐗₀ ε
2)
      - (ε ^ 2 * s * (s - 2) * ℏ ^ 2) • 𝐗₀ ε (s-4) := by
      simp_rw [positionOperatorSqr_eq ε, comp_sub, comp_smul, comp_id, radiusRegPowO
      simp only [smul_sub, smul_smul, ← Complex.coe_smul, ofReal_mul, ofReal_add, of
        ofReal_pow, ofReal_ofNat, ofReal_natCast]
      ring_nf
      simp_rw [← sub_add, sub_sub, ← add_smul, I_sq, sub_eq_add_neg, ← neg_smul]
      ring_nf
    ⁅𝐗 i j, 𝐏 k⁆ = (I * ℏ) • (δ[i,k] • 𝐏 j - δ[j,k] • 𝐏 i) := by
  trans 𝐗 i ∘L ⁅𝐏 j, 𝐏 k⁆ - 𝐏 j ∘L ⁅𝐗 i, 𝐏 k⁆
  simp only [← lie_skew (𝐏 _), comp_neg, sub_neg_eq_add, add_comm, ← sub_eq_add_neg,
lemma angularMomentum_commutation_radiusRegPow : ⁅𝐗 i j, 𝐗₀[d] ε s⁆ = 0 := by
  trans 𝐗 i ∘L ⁅𝐏 j, 𝐗₀ ε s⁆ - 𝐏 j ∘L ⁅𝐗 i, 𝐗₀ ε s⁆
```

```
  simp [← lie_skew (𝐋 _), radiusRegPow_commutation_momentum, comp_neg,
lemma angularMomentum_comp_radiusRegPow_commute : 𝐋 i j ∘L 𝓡₀ ε s = 𝓡₀ ε s
  rw [comp_eq_comp_add_commute, angularMomentum_commutation_radiusRegPow, a

lemma angularMomentumSqr_commutation_radiusRegPow : ⁅𝐋²[d], 𝓡₀[d] ε s⁆ = 0
lemma angularMomentumSqr_comp_radiusRegPow_commute : 𝐋² ∘L 𝓡₀[d] ε s = 𝓡₀ ε
  rw [comp_eq_comp_add_commute, angularMomentumSqr_commutation_radiusRegPow

lemma angularMomentum_commutation_momentum :
    ⁅𝐋 i j, 𝐏 k⁆ = (I * ℏ) • (δ[i,k] • 𝐏 j - δ[j,k] • 𝐏 i) := by
  trans ⁅𝐋 i, 𝐏 k⁆ ∘L 𝐏 j - ⁅𝐋 j, 𝐏 k⁆ ∘L 𝐏 i
lemma momentum_comp_angularMomentum_eq :
    𝐏 k ∘L 𝐋 i j = 𝐋 i j ∘L 𝐏 k - (I * ℏ) • (δ[i,k] • 𝐏 j - δ[j,k] • 𝐏 i) :
lemma angularMomentum_commutation_momentumSqr : ⁅𝐋 i j, 𝐏[d] ⬝ᵥ 𝐏⁆ = 0 := b
  simp only [dotProduct, mul_def, lie_sum, lie_leibniz, angularMomentum_com
lemma momentumSqr_comp_angularMomentum_commute : (𝐏 ⬝ᵥ 𝐏) ∘L 𝐋 i j = 𝐏 i j
lemma angularMomentumSqr_commutation_momentumSqr : ⁅𝐋²[d], 𝐏[d] ⬝ᵥ 𝐏⁆ = 0 :
set_option backward.isDefEq.respectTransparency false in
lemma angularMomentum_commutation_angularMomentum : ⁅𝐋 i j, 𝐋 k l⁆ =
    (I * ℏ) • (δ[i,k] • 𝐋 j l - δ[i,l] • 𝐋 j k - δ[j,k] • 𝐋 i l + δ[j,l] •
    ContinuousLinearMap.add_apply, coe_mul', coe_comp', Function.comp_apply
    SchwartzMap.smul_apply, SchwartzMap.sub_apply, SchwartzMap.add_apply, s
    map_comp_sub]
@[simp]
lemma angularMomentumSqr_commutation_angularMomentum : ⁅𝐋²[d], 𝐋 i j⁆ = 0 :
  simp only [angularMomentumOperatorSqr, smul_lie, sum_lie, leibniz_lie, ←
    comp_add, comp_sub, smul_comp, add_comp, sub_comp, angularMomentum_comm
    angularMomentumOperator_antisymm _ i, angularMomentumOperator_antisymm
    sum_smul, ← Finset.smul_sum, Finset.sum_add_distrib, Finset.sum_sub_dis
  abel_nf
  simp [smul_zero]
```

1.234 Physlib/QuantumMechanics/DDimensions/Operators/Momentum.l

```
public import Physlib.QuantumMechanics.DDimensions.Operators.Unbounded
public import Physlib.QuantumMechanics.DDimensions.SpaceDHilbertSpace.Schwa
public import Physlib.QuantumMechanics.PlanckConstant
public import Physlib.SpaceAndTime.Space.Derivatives.Basic
- `𝐏` for `momentumOperator`
- B. Unbounded momentum vector operator
open Constants Complex
set_option backward.isDefEq.respectTransparency false in
notation "𝐏" => momentumOperator
@[inherit_doc momentumOperator]
```

```
notation "∇[" d' "]" => momentumOperator (d := d')
lemma momentumOperator_apply_fun (ψ : 𝓢(Space d, ℂ)) : ∇ i ψ = (-
I * ℏ) • ∂[i] ψ := rfl
@[simp]
lemma momentumOperator_apply (ψ : 𝓢(Space d, ℂ)) (x : Space d) : ∇ i ψ x = -
I * ℏ * ∂[i] ψ x :=
  rfl
## B. Unbounded momentum vector operator
set_option backward.isDefEq.respectTransparency false in
  schwartzEquiv.toLinearMap ∘ₗ (𝓕 i).toLinearMap ∘ₗ schwartzEquiv.symm.toLinearMap
set_option backward.isDefEq.respectTransparency false in
```

1.235 Physlib/QuantumMechanics/DDimensions/Operators/Position.lean

```
/-
Copyright (c) 2026 Gregory J. Loges. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Gregory J. Loges
-/
module

public import Physlib.QuantumMechanics.DDimensions.Operators.Unbounded
public import Physlib.QuantumMechanics.DDimensions.SpaceDHilbertSpace.SchwartzSubmod
public import Physlib.SpaceAndTime.Space.Integrals.NormPow
public import Physlib.SpaceAndTime.Space.Derivatives.Basic
/-!

# Position operators

## i. Overview

In this module we introduce several position operators for quantum mechanics on `Spa

## ii. Key results

Definitions:
- `positionOperator` : (components of) the position vector operator acting on Schwar
    `𝓢(Space d, ℂ)` by multiplication by `xᵢ`.
- `radiusRegPowOperator` : operator acting on Schwartz maps by multiplication by
    `(‖x‖² + ε²)^(s/2)`, a smooth regularization of `‖x‖ˢ`.
- `positionUnboundedOperator` : a symmetric unbounded operator acting on the Schwart
    of the Hilbert space `SpaceDHilbertSpace d`.
- `readiusRegPowUnboundedOperator` : a symmetric unbounded operator acting on the Sc
    submodule of the Hilbert space `SpaceDHilbertSpace d`. For `s ≤ 0` this operator
```

```
    bounded (by `|ε|ˢ`) and has natural domain the entire Hilbert space, bu
    use the same domain for all `s`.

Notation:
- `𝐱` for `positionOperator`
- `𝐱₀` for `radiusRegPowOperator`
- `𝐫` for `radiusPowOperator`

## iii. Table of contents

- A. Schwartz operators
  - A.1. Position vector
  - A.2. Radius powers (regularized)
  - A.3. Radius powers
    - A.3.1. As limit of regularized operators
- B. Unbounded operators
  - B.1. Position vector
  - B.2. Radius powers (regularized)

## iv. References

-/

@[expose] public section

open Physlib

namespace QuantumMechanics

variable {d : ℕ} (i : Fin d)

/-!
## A. Schwartz operators
-/

noncomputable section
open Space Function
open SchwartzMap

/-!
### A.1. Position vector
-/

set_option backward.isDefEq.respectTransparency false in
/-- Component `i` of the position operator is the continuous linear map
  from `𝓢(Space d, ℂ)` to itself which maps `ψ` to `xᵢψ`. -
```

```
/
def positionOperator : □(Space d, ℂ) →L[ℂ] □(Space d, ℂ) :=
  SchwartzMap.smulLeftCLM ℂ (Complex.ofRealCLM ∘L coordCLM i)

@[inherit_doc positionOperator]
notation "□" => positionOperator

@[inherit_doc positionOperator]
notation "□[" d' "]" => positionOperator (d := d')

lemma positionOperator_apply_fun (ψ : □(Space d, ℂ)) : □ i ψ = (fun x : Space d ↦ x
  ext
  simp [positionOperator, coordCLM_apply, coord_apply,
    smulLeftCLM_apply_apply (g := Complex.ofRealCLM ∘ (coordCLM i)) (by fun_prop)]

@[simp]
lemma positionOperator_apply (ψ : □(Space d, ℂ)) (x : Space d) : □ i ψ x = x i * ψ x
  simp [positionOperator_apply_fun]

/-!
### A.2. Radius powers (regularized)
-/
TODO "Incorporate normRegularizedPow into Space.Norm"

/-- Power of regularized norm, `(‖x‖² + ε²)^(s/2)`. -/
def normRegularizedPow (d : ℕ) (ε s : ℝ) : Space d → ℝ :=
  fun x ↦ (‖x‖ ^ 2 + ε ^ 2) ^ (s / 2)

lemma norm_sq_add_unit_sq_pos {d : ℕ} (ε : ℝˣ) (x : Space d) : 0 < ‖x‖ ^ 2 + ε ^ 2 :
    Left.add_pos_of_nonneg_of_pos (sq_nonneg ‖x‖) (sq_pos_iff.mpr <| Units.ne_zero ε

lemma normRegularizedPow_pos (d : ℕ) (ε : ℝˣ) (s : ℝ) (x : Space d) :
    0 < normRegularizedPow d ε s x :=
  Real.rpow_pos_of_pos (norm_sq_add_unit_sq_pos ε x) (s / 2)

lemma normRegularizedPow_hasTemperateGrowth (d : ℕ) (ε : ℝˣ) (s : ℝ) :
    HasTemperateGrowth (normRegularizedPow d ε s) := by
  -- Write `normRegularizedPow` as the composition of three simple functions
  -- to take advantage of `hasTemperateGrowth_one_add_norm_sq_rpow`.
  let f1 := fun (x : ℝ) ↦ (ε ^ 2) ^ (s / 2) * x
  let f2 := fun (x : Space d) ↦ (1 + ‖x‖ ^ 2) ^ (s / 2)
  let f3 := fun (x : Space d) ↦ ε.1⁻¹ • x
  have h123 : normRegularizedPow d ε s = f1 ∘ f2 ∘ f3 := by
    ext
    simp only [normRegularizedPow, f1, f2, f3, comp_apply, norm_smul, norm_inv, Real
    rw [← Real.mul_rpow (sq_nonneg ↑ε) (add_nonneg (zero_le_one' _) (sq_nonneg _))]
```

```
    simp [mul_add, mul_pow, add_comm]
  rw [h123]
  fun_prop

set_option backward.isDefEq.respectTransparency false in
/-- The radius operator to power `s`, regularized by `ε ≠ 0`, is the contin
  from `𝓢(Space d, ℂ)` to itself which maps `ψ` to `(‖x‖² + ε²)^(s/2) • ψ`.
/
def radiusRegPowOperator {d : ℕ} (ε : ℝˣ) (s : ℝ) : 𝓢(Space d, ℂ) →L[ℂ] 𝓢(S
  SchwartzMap.smulLeftCLM ℂ (Complex.ofReal ∘ normRegularizedPow d ε s)

@[inherit_doc radiusRegPowOperator]
notation "𝓡₀" => radiusRegPowOperator

@[inherit_doc radiusRegPowOperator]
notation "𝓡₀[" d' "]" => radiusRegPowOperator (d := d')

lemma radiusRegPowOperator_apply_fun {d : ℕ} (ε : ℝˣ) (s : ℝ) (ψ : 𝓢(Space
    𝓡₀ ε s ψ = fun x ↦ (‖x‖ ^ 2 + ε ^ 2) ^ (s / 2) • ψ x := by
  ext x
  dsimp [radiusRegPowOperator]
  refine smulLeftCLM_apply_apply ?_ ψ x
  exact HasTemperateGrowth.comp (by fun_prop) (normRegularizedPow_hasTemper

@[simp]
lemma radiusRegPowOperator_apply {d : ℕ} (ε : ℝˣ) (s : ℝ) (ψ : 𝓢(Space d, ℂ
    𝓡₀ ε s ψ x = (‖x‖ ^ 2 + ε ^ 2) ^ (s / 2) • ψ x := by
  rw [radiusRegPowOperator_apply_fun]

set_option backward.isDefEq.respectTransparency false in
@[simp]
lemma radiusRegPowOperator_comp_eq {d : ℕ} (ε : ℝˣ) (s t : ℝ) :
    𝓡₀[d] ε s ∘L 𝓡₀ ε t = 𝓡₀ ε (s+t) := by
  ext ψ x
  simp [add_div, Real.rpow_add (norm_sq_add_unit_sq_pos ε x), mul_assoc]

set_option backward.isDefEq.respectTransparency false in
@[simp]
lemma radiusRegPowOperator_zero {d : ℕ} (ε : ℝˣ) :
    𝓡₀ ε 0 = ContinuousLinearMap.id ℂ 𝓢(Space d, ℂ) := by
  ext
  simp

set_option backward.isDefEq.respectTransparency false in
lemma positionOperatorSqr_eq {d : ℕ} (ε : ℝˣ) :
    ∑ i, 𝓧 i ∘L 𝓧 i = 𝓡₀ ε 2 - ε.1 ^ 2 • ContinuousLinearMap.id ℂ 𝓢(Space d
```

```
  ext
  simp [Space.norm_sq_eq, add_mul, ← mul_assoc, ← pow_two, Finset.sum_mul]

/-!
### A.3. Radius powers
-/

open MeasureTheory
open SpaceDHilbertSpace

set_option backward.isDefEq.respectTransparency false in
/-- The radius operator to power `s` is the linear map from `𝓢(Space d, ℂ)` to `Spac
  maps `ψ` to `x ↦ ‖x‖ˢψ(x)` (which is 'nearly' Schwartz for general `s`). -
/
def radiusPowOperator {d : ℕ} (s : ℝ) : 𝓢(Space d, ℂ) →ₗ[ℂ] Space d → ℂ where
  toFun ψ := (fun x : Space d ↦ ‖x‖ ^ s) • ψ
  map_add' _ _ := by rw [← smul_add]; rfl
  map_smul' _ _ := by rw [smul_comm]; rfl

@[inherit_doc radiusPowOperator]
notation "𝓡" => radiusPowOperator

lemma radiusPowOperator_apply_fun {d : ℕ} (s : ℝ) (ψ : 𝓢(Space d, ℂ)) :
    𝓡 s ψ = fun x ↦ ‖x‖ ^ s • ψ x := rfl

@[simp]
lemma radiusPowOperator_apply {d : ℕ} (s : ℝ) (ψ : 𝓢(Space d, ℂ)) (x : Space d) :
    𝓡 s ψ x = ‖x‖ ^ s • ψ x := by
  rw [radiusPowOperator_apply_fun]

set_option backward.isDefEq.respectTransparency false in
/-- `x ↦ ‖x‖ˢψ(x)` is smooth away from `x = 0`. -/
@[fun_prop]
lemma radiusPowOperator_apply_contDiffAt {d : ℕ} (s : ℝ) (n : ℕ∞) (ψ : 𝓢(Space d, ℂ)
    (hx : x ≠ 0) : ContDiffAt ℝ n (𝓡 s ψ) x := by
  refine ContDiffAt.smul ?_ (ψ.contDiffAt n)
  have h (x : Space d) : ‖x‖ ^ s = (inner ℝ x x) ^ (s / 2) := by
    simp [← Real.rpow_natCast_mul, mul_div_cancel₀]
  simp only [h]
  exact ContDiffAt.rpow_const_of_ne (by fun_prop) (inner_self_ne_zero.mpr hx)

set_option backward.isDefEq.respectTransparency false in
/-- `x ↦ ‖x‖ˢψ(x)` is strongly measurable. -/
@[fun_prop]
lemma radiusPowOperator_apply_stronglyMeasurable {d : ℕ} (s : ℝ) (ψ : 𝓢(Space d, ℂ))
    StronglyMeasurable (𝓡 s ψ) := by
```

```
  rw [radiusPowOperator_apply_fun]
  exact StronglyMeasurable.smul (by measurability) ψ.continuous.stronglyMea

set_option backward.isDefEq.respectTransparency false in
/-- `x ↦ ‖x‖ˢψ(x)` is square-integrable provided `s` is not too negative. -
/
lemma radiusPowOperator_apply_memHS {d : ℕ} (s : ℝ) (h : 0 < d + 2 * s) (ψ
    MemHS (□ s ψ) := by
  rcases Nat.eq_zero_or_pos d with (rfl | hd)
  · simp only [MemHS, MemLp.of_discrete]
  · refine (MeasureTheory.memLp_two_iff_integrable_sq_norm (by fun_prop)).m
    suffices ∫⁻ (a : Space d), ‖‖ψ a‖ ^ 2 * ‖a‖ ^ (2 * s)‖ₑ < ⊤ by
      have hInt (x : Space d) : ‖□ s ψ x‖ ^ 2 = ‖ψ x‖ ^ 2 * ‖x‖ ^ (2 * s) :
        simp [radiusPowOperator, mul_pow, mul_comm, Real.rpow_mul]
      simpa only [HasFiniteIntegral, hInt]
    rw [← lintegral_add_compl _ (measurableSet_ball (x := 0) (ε := 1)), ENN
    constructor
    · -- `‖x‖ < 1`: bound `‖ψ x‖` by a constant
      obtain ⟨C, hC_pos, hC⟩ := ψ.decay 0 0
      simp only [pow_zero, norm_iteratedFDeriv_zero, one_mul] at hC
      suffices hBound : ∀ x, ‖‖ψ x‖ ^ 2 * ‖x‖ ^ (2 * s)‖ₑ ≤ ‖C ^ 2‖ₑ * ‖‖x‖
        calc
          _ ≤ ∫⁻ (x : Space d) in (Metric.ball 0 1), ‖C ^ 2‖ₑ * ‖‖x‖ ^ (2 *
            setLIntegral_mono' measurableSet_ball (fun x _ ↦ hBound x)
          _ = ‖C ^ 2‖ₑ * ∫⁻ (x : Space d) in (Metric.ball 0 1), ‖‖x‖ ^ (2 *
            lintegral_const_mul _ (by fun_prop)
        apply ENNReal.mul_lt_top enorm_lt_top
        exact ((integrableOn_norm_rpow_ball_iff hd Real.zero_lt_one _).mpr
      intro x
      simp_rw [← enorm_mul, enorm_le_iff_norm_le, norm_mul, norm_pow, Real.
      refine mul_le_mul_of_nonneg_right ?_ (abs_nonneg _)
      exact (sq_le_sq₀ (norm_nonneg _) hC_pos.le).mpr (hC x)
    · -- `1 ≤ ‖x‖`: bound `‖ψ x‖` by a suitable power of `‖x‖`
      obtain ⟨C, hC_pos, hC⟩ := ψ.decay (⌈s⌉.toNat + d) 0
      simp only [norm_iteratedFDeriv_zero, ← Real.rpow_natCast, Nat.cast_ad
      suffices hBound : ∀ x ∈ (Metric.ball 0 1)ᶜ,
          ‖‖ψ x‖ ^ 2 * ‖x‖ ^ (2 * s)‖ₑ ≤ ‖C ^ 2‖ₑ * ‖‖x‖ ^ (-
2 * d : ℝ)‖ₑ by
        calc
          _ ≤ ∫⁻ (x : Space d) in (Metric.ball 0 1)ᶜ, ‖C ^ 2‖ₑ * ‖‖x‖ ^ (-
2 * d : ℝ)‖ₑ :=
            setLIntegral_mono' (by measurability) hBound
          _ = ‖C ^ 2‖ₑ * ∫⁻ (x : Space d) in (Metric.ball 0 1)ᶜ, ‖‖x‖ ^ (-
2 * d : ℝ)‖ₑ :=
            lintegral_const_mul _ (by fun_prop)
        apply ENNReal.mul_lt_top enorm_lt_top
```

```
        have hd' : (d + -2 * d : ℝ) < 0 := by simp [hd]
        exact ((integrableOn_norm_rpow_ball_compl_iff hd zero_lt_one _).mpr hd').has
      intro x hx
      simp only [Set.mem_compl_iff, Metric.mem_ball, dist_zero_right, not_lt] at hx
      simp_rw [← enorm_mul, enorm_le_iff_norm_le, norm_mul, norm_pow, Real.norm_eq_a
        Real.abs_rpow_of_nonneg (norm_nonneg _), abs_norm]
      have hx' : 0 < ‖x‖ := by linarith
      have hψ : ‖ψ x‖ ≤ C * ‖x‖ ^ (-(⌈s⌉.toNat + d) : ℝ) := by
        rw [Real.rpow_neg hx'.le]
        exact (le_mul_inv_iff₀' <| Real.rpow_pos_of_pos hx' _).mpr (hC x)
      calc
        _ ≤ (C * ‖x‖ ^ (-(⌈s⌉.toNat + d) : ℝ)) ^ 2 * ‖x‖ ^ (2 * s) := by
          refine mul_le_mul_of_nonneg_right ?_ (Real.rpow_nonneg hx'.le _)
          exact pow_le_pow_left₀ (norm_nonneg _) hψ 2
        _ = C ^ 2 * ‖x‖ ^ (-2 * d : ℝ) * ‖x‖ ^ (2 * (s - ⌈s⌉.toNat) : ℝ) := by
          simp_rw [mul_pow, ← Real.rpow_mul_natCast hx'.le, mul_assoc, ← Real.rpow_a
          ring_nf
      suffices s ≤ ⌈s⌉.toNat by
        have h' : 0 < C ^ 2 * ‖x‖ ^ (-2 * d : ℝ) :=
          mul_pos (sq_pos_of_pos hC_pos) (Real.rpow_pos_of_pos hx' _)
        apply (mul_le_iff_le_one_right h').mpr
        exact Real.rpow_le_one_of_one_le_of_nonpos hx (by linarith)
      rcases lt_or_ge 0 s with (hs | hs)
      · have hs' : ⌈s⌉.toNat = (⌈s⌉ : ℝ) :=
          Int.cast_inj.mpr <| Int.toNat_of_nonneg <| Int.ceil_nonneg hs.le
        exact hs' ▸ Int.le_ceil s
      · have hs' : ⌈s⌉.toNat = (0 : ℝ) :=
          Nat.cast_eq_zero.mpr <| Int.toNat_of_nonpos <| Int.ceil_le.mpr (by rwa [In
        exact hs' ▸ hs

/-!
#### A.3.1. As limit of regularized operators
-/

open Filter

/-- Neighborhoods of "0" in the non-zero reals, i.e. those sets containing `(-
ε,0) ∪ (0,ε) ⊆ ℝˣ`
  for some `ε > 0`. -/
abbrev nhdsZeroUnits : Filter ℝˣ := comap (Units.coeHom ℝ) (nhds 0)

instance : NeBot nhdsZeroUnits := by
  refine comap_neBot fun t ht ↦ ?_
  obtain ⟨ε, hε_pos, hε⟩ := Metric.mem_nhds_iff.mp ht
  use Units.mk0 (ε / 2) (by linarith)
  apply hε
```

```
  simp [abs_of_pos, hε_pos]

/-- `□[ε,s] ψ` converges pointwise to `□[s] ψ` as `ε → 0` except perhaps at
/
lemma radiusRegPow_tendsto_radiusPow {d : ℕ} (s : ℝ) (ψ : □(Space d, ℂ)) {x
    (hx : x ≠ 0) : Tendsto (fun ε ↦ □₀ ε s ψ x) nhdsZeroUnits (nhds (□ s ψ
  have hpow : ‖x‖ ^ s = (‖x‖ ^ 2 + 0 ^ 2) ^ (s / 2) := by
    simp [← Real.rpow_natCast_mul, mul_div_cancel₀]
  simp only [radiusRegPowOperator_apply, radiusPowOperator_apply, Complex.r
  refine Tendsto.mul_const (ψ x) <| Tendsto.ofReal ?_
  refine Tendsto.rpow_const ?_ (Or.inl <| by simp [hx])
  exact Tendsto.const_add _ <| Tendsto.pow tendsto_comap 2

/-- `□[ε,s] ψ` converges pointwise to `□[s] ψ` as `ε → 0` provided `□[ε,s]
/
lemma radiusRegPow_tendsto_radiusPow' {d : ℕ} (s : ℝ) (ψ : □(Space d, ℂ)) (
    Tendsto (fun ε ↦ ⇑(□₀ ε s ψ)) nhdsZeroUnits (nhds (□ s ψ)) := by
  refine tendsto_pi_nhds.mpr fun x ↦ ?_
  rcases eq_zero_or_neZero x with (rfl | hx)
  · rcases h with (hs | hψ)
    · simp only [radiusRegPowOperator_apply, radiusPowOperator_apply, Compl
        ne_eq, OfNat.ofNat_ne_zero, not_false_eq_true, zero_pow, zero_add]
      have : (0 : ℝ) ^ s = (0 ^ 2) ^ (s / 2) := by
        rw [← Real.rpow_natCast_mul (le_refl 0), Nat.cast_ofNat, mul_div_ca
      rw [this]
      refine Tendsto.mul_const (ψ 0) <| Tendsto.ofReal ?_
      exact Tendsto.rpow_const (Tendsto.pow tendsto_comap 2) (Or.inr <| by
    · simp [hψ]
  · exact radiusRegPow_tendsto_radiusPow s ψ hx.ne

/-- a.e. version of `radiusRegPow_tendsto_radiusPow` -/
lemma radiusRegPow_ae_tendsto_radiusPow {d : ℕ} (hd : 0 < d) (s : ℝ) (ψ : □
    ∀ᵐ x, Tendsto (fun ε ↦ □₀ ε s ψ x) nhdsZeroUnits (nhds (□ s ψ x)) := by
  apply ae_iff.mpr
  suffices h : {x | ¬Tendsto (fun ε ↦ □₀ ε s ψ x) nhdsZeroUnits (nhds (□ s
    rcases Set.subset_singleton_iff_eq.mp h with (h' | h')
    · exact h' ▸ measure_empty
    · have : Nontrivial (Space d) := Nat.succ_pred_eq_of_pos hd ▸ Space.ins
      exact h' ▸ measure_singleton 0
  intro x hx
  by_contra hx'
  exact hx <| radiusRegPow_tendsto_radiusPow s ψ hx'

lemma radiusRegPow_ae_tendsto_iff {d : ℕ} (hd : 0 < d) {s : ℝ} {ψ : □(Space
    {φ : Space d → ℂ} : (∀ᵐ x, Tendsto (fun ε ↦ □₀ ε s ψ x) nhdsZeroUnits (
    ↔ φ =ᵐ[volume] □ s ψ := by
```

```
  let t₁ := {x | ¬Tendsto (fun ε ↦ □₀ ε s ψ x) nhdsZeroUnits (nhds (φ x))}
  let t₂ := {x | φ x ≠ □ s ψ x}
  show volume t₁ = 0 ↔ volume t₂ = 0
  suffices heq : t₁ ∪ {0} = t₂ ∪ {0} by
    have : Nontrivial (Space d) := Nat.succ_pred_eq_of_pos hd ▸ Space.instNontrivial
    have hUnion : ∀ t : Set (Space d), volume t = 0 ↔ volume (t ∪ {0}) = 0 :=
      fun _ ↦ by simp only [measure_union_null_iff, measure_singleton, and_true]
    rw [hUnion t₁, hUnion t₂, heq]
  ext x
  rcases eq_zero_or_neZero x with (rfl | hx)
  · simp
  · simp only [Set.union_singleton, Set.mem_insert_iff, hx.ne, false_or]
    have hLim := radiusRegPow_tendsto_radiusPow s ψ hx.ne
    exact not_congr ⟨fun h ↦ tendsto_nhds_unique h hLim, fun h ↦ h ▸ hLim⟩

end

/-!
## B. Unbounded operators
-/

noncomputable section

open SpaceDHilbertSpace

/-!
### B.1. Position vector
-/

set_option backward.isDefEq.respectTransparency false in
/-- The position operators defined on the Schwartz submodule. -
/
def positionOperatorSchwartz : schwartzSubmodule d →ₗ[ℂ] schwartzSubmodule d :=
  schwartzEquiv.toLinearMap ∘ₗ (□ i).toLinearMap ∘ₗ schwartzEquiv.symm.toLinearMap

set_option backward.isDefEq.respectTransparency false in
lemma positionOperatorSchwartz_isSymmetric : (positionOperatorSchwartz i).IsSymmetri
  intro ψ ψ'
  obtain ⟨_, rfl⟩ := schwartzEquiv.surjective ψ
  obtain ⟨_, rfl⟩ := schwartzEquiv.surjective ψ'
  unfold positionOperatorSchwartz
  simp only [LinearMap.coe_comp, LinearEquiv.coe_coe, Function.comp_apply, schwartzE
  congr with x
  simp only [LinearEquiv.symm_apply_apply, ContinuousLinearMap.coe_coe,
    positionOperator_apply, map_mul, Complex.conj_ofReal]
  ring
```

```
/-- The symmetric position unbounded operators with domain the Schwartz sub
  of the Hilbert space. -/
def positionUnboundedOperator : UnboundedOperator (SpaceDHilbertSpace d) (S
  UnboundedOperator.ofSymmetric (schwartzSubmodule_dense d) (positionOperat

/-!
### B.2. Radius powers (regularized)
-/

set_option backward.isDefEq.respectTransparency false in
/-- The (regularized) radius operators defined on the Schwartz submodule. -
/
def radiusRegPowOperatorSchwartz {d : ℕ} (ε : ℝˣ) (s : ℝ) :
    schwartzSubmodule d →ₗ[ℂ] schwartzSubmodule d :=
  schwartzEquiv.toLinearMap ∘ₗ (□₀ ε s).toLinearMap ∘ₗ schwartzEquiv.symm.t

set_option backward.isDefEq.respectTransparency false in
lemma radiusRegPowOperatorSchwartz_isSymmetric {d : ℕ} (ε : ℝˣ) (s : ℝ) :
    (radiusRegPowOperatorSchwartz (d := d) ε s).IsSymmetric := by
  intro ψ ψ'
  obtain ⟨_, rfl⟩ := schwartzEquiv.surjective ψ
  obtain ⟨_, rfl⟩ := schwartzEquiv.surjective ψ'
  simp only [radiusRegPowOperatorSchwartz, LinearMap.coe_comp, LinearEquiv.
    Function.comp_apply, schwartzEquiv_inner]
  congr with x -- match integrands
  simp only [LinearEquiv.symm_apply_apply, ContinuousLinearMap.coe_coe, rad
    Complex.real_smul, map_mul, Complex.conj_ofReal]
  ring

/-- The symmetric (regularized) radius unbounded operators with domain the
  of the Hilbert space. -/
def radiusRegPowUnboundedOperator {d : ℕ} (ε : ℝˣ) (s : ℝ) :
    UnboundedOperator (SpaceDHilbertSpace d) (SpaceDHilbertSpace d) :=
  UnboundedOperator.ofSymmetric (schwartzSubmodule_dense d)
    (radiusRegPowOperatorSchwartz_isSymmetric ε s)

end
end QuantumMechanics
```

1.236 Physlib/QuantumMechanics/DDimensions/Operators/Unbounded.

```
public import Physlib.Mathematics.InnerProductSpace.Submodule
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
```

1.237 Physlib/QuantumMechanics/DDimensions/SpaceDHilbertSpace/Basic.lea

```
public import Physlib.SpaceAndTime.Space.Module
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.238 Physlib/QuantumMechanics/DDimensions/SpaceDHilbertSpace/PolyBddS

```
/-
Copyright (c) 2026 Gregory J. Loges. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Gregory J. Loges
-/
module

public import Mathlib.Analysis.Calculus.BumpFunction.InnerProduct
public import Mathlib.MeasureTheory.Measure.Lebesgue.VolumeOfBalls
public import Physlib.QuantumMechanics.DDimensions.SpaceDHilbertSpace.SchwartzSubmod
/-!

# Polynomially-bounded Schwartz submodules

## i. Overview

In this module we define polynomially-bounded Schwartz submodules of `SpaceDHilbertS

For each `a : ℕ∞`, `polyBddSchwartzSubmodule d a` is the submodule corresponding to
maps `f` satisfying the polynomial growth bounds `‖x‖ ^ (-
k) * ‖f x‖ ≤ Cₖ` for all `(k : ℕ) ≤ a`.
In particular, for `a = ⊤` such a bound holds for all natural numbers.

These serve as a natural domain for singular unbounded operators. For example, the `
potential operator maps `polyBddSchwartzSubmodule d ⊤` to itself. In the same way th
a Schwartz map by any polynomial in the coordinates results in a square-
integrable function,
polynomially-bounded Schwartz maps may be multiplied by Laurent polynomials and rema
square-integrable (the precise condition depends on `d`, `a` and the negative degree
the Laurent polynomial).

Note: the condition defining polynomially-bounded Schwartz maps is phrased as
`‖x‖ ^ (-k) * ‖f x‖ ≤ Cₖ` rather than as `‖f x‖ ≤ Cₖ * ‖x‖ ^ k` to mirror `SchwartzM
```

```
These two conditions only differ at `x = 0` and are therefore equivalent fo
then `f 0` may be determined by continuity. For `d = 0` the former does not
(since `x = 0` is the only point and `0⁻¹ = 0`) while the latter does (and
their being dense in `SpaceDHilbertSpace 0 ≅ ℂ`).

## ii. Key results

- `polyBddSchwartzSubmodule d (a : ℕ∞)`: Restriction of `schwartzSubmodule
  which are bounded by powers of `‖x‖`.
- `polyBddSchwartzSubmodule_dense d a`: These submodules are dense in `Spac

## iii. Table of contents

- A. Definitions
- B. (In)equalities
- C. Density

## iv. References

-/

@[expose] public section

namespace QuantumMechanics
namespace SpaceDHilbertSpace

open MeasureTheory
open InnerProductSpace
open SchwartzMap

noncomputable section

/-!
### A. Definitions
-/

/-- A function is a bounded Schwartz map if it is both Schwartz and bounded
/
def polyBddSchwartzMap (d : ℕ) (a : ℕ∞) : Submodule ℂ 𝓢(Space d, ℂ) where
  carrier := {f : 𝓢(Space d, ℂ) |
    ∀ k : ℕ, k ≤ a → ∃ C : ℝ, 0 < C ∧ ∀ x : Space d, ‖x‖ ^ (-
k : ℤ) * ‖f x‖ ≤ C}
  add_mem' := by
    intro f g hf hg k hk
    obtain ⟨C₁, hC₁_pos, hC₁⟩ := hf k hk
    obtain ⟨C₂, hC₂_pos, hC₂⟩ := hg k hk
```

```
    refine ⟨C₁ + C₂, by positivity, fun x ↦ ?_⟩
    refine le_trans ?_ (add_le_add (hC₁ x) (hC₂ x))
    rw [← mul_add]
    exact mul_le_mul_of_nonneg_left (norm_add_le (f x) (g x)) (by positivity)
  zero_mem' := fun _ _ ↦ ⟨1, by simp⟩
  smul_mem' := by
    intro c f hf k hk
    obtain ⟨C, hC_pos, hC⟩ := hf k hk
    refine ⟨(1 + ‖c‖) * C, by positivity, fun x ↦ ?_⟩
    rw [smul_apply, norm_smul, mul_rotate', mul_comm ‖f x‖]
    exact le_trans (mul_le_mul_of_nonneg_left (hC x) (norm_nonneg c)) (by linarith)

/-- The continuous linear map `schwartzIncl` with domain restricted to `polyBddSchwa
/
def polyBddSchwartzIncl {d : ℕ} {a : ℕ∞} : polyBddSchwartzMap d a →L[ℂ] SpaceDHilber
  ⟨schwartzIncl.domRestrict (polyBddSchwartzMap d a),
    schwartzIncl.continuous_domRestrict schwartzIncl.continuous _⟩

/-- The submodule of `SpaceDHilbertSpace d` corresponding to bounded Schwartz maps.
/
abbrev polyBddSchwartzSubmodule (d : ℕ) (a : ℕ∞) : Submodule ℂ (SpaceDHilbertSpace d
  (polyBddSchwartzIncl (a := a)).range

lemma polyBddSchwartzIncl_injective (d : ℕ) (a : ℕ∞) :
    Function.Injective (polyBddSchwartzIncl (d := d) (a := a)) :=
  LinearMap.injective_domRestrict_iff.mpr <| schwartzIncl_ker.symm ▸ inf_bot_eq _

/-- The linear equivalence between polynomially-bounded Schwartz maps and the corres
  submodule of the Hilbert space. -/
def polyBddSchwartzEquiv {d : ℕ} {a : ℕ∞} :
    polyBddSchwartzMap d a ≃ₗ[ℂ] polyBddSchwartzSubmodule d a :=
  LinearEquiv.ofInjective polyBddSchwartzIncl.toLinearMap (polyBddSchwartzIncl_injec

/-!
### B. (In)equalities
-/

lemma polyBddSchwartzMap_zero_eq_top (d : ℕ) : polyBddSchwartzMap d 0 = ⊤ := by
  ext f
  have := f.decay 0 0
  simp_all [polyBddSchwartzMap]

lemma polyBddSchwartzMap_le_of_ge (d : ℕ) {a b : ℕ∞} (h : a ≤ b) :
    polyBddSchwartzMap d b ≤ polyBddSchwartzMap d a := fun _ hx k hk ↦ hx k (hk.tran

lemma polyBddSchwartzSubmodule_zero_eq (d : ℕ) :
```

```
    polyBddSchwartzSubmodule d 0 = schwartzSubmodule d := by
  simp [polyBddSchwartzSubmodule, polyBddSchwartzIncl, polyBddSchwartzMap_z

lemma polyBddSchwartzSubmodule_le (d : ℕ) (a : ℕ∞) :
    polyBddSchwartzSubmodule d a ≤ schwartzSubmodule d := LinearMap.range_d

lemma polyBddSchwartzSubmodule_le_of_ge (d : ℕ) {a b : ℕ∞} (h : a ≤ b) :
    polyBddSchwartzSubmodule d b ≤ polyBddSchwartzSubmodule d a := by
  simp only [polyBddSchwartzSubmodule, polyBddSchwartzIncl, LinearMap.range
  exact Submodule.map_mono (polyBddSchwartzMap_le_of_ge d h)

/-!
### C. Density
-/

open Filter Complex

private lemma enorm_bump_mul_le_enorm {𝕜 E : Type*} [RCLike 𝕜] [NormedAddCo
    [NormedSpace ℝ E] [HasContDiffBump E] {c : E} (f : ContDiffBump c) (g :
    ‖f x * g x‖ₑ ≤ ‖g x‖ₑ := by
  nth_rw 2 [← one_mul (g x)]
  simp_rw [enorm_mul]
  refine mul_le_mul_left ?_ ‖g x‖ₑ
  apply enorm_le_iff_norm_le.mpr
  rw [norm_algebraMap', Real.norm_eq_abs, norm_one, ← abs_one]
  exact abs_le_abs_of_nonneg f.nonneg f.le_one

private lemma tendsto_zero_iff_tendsto_zero_lintegral_enorm_sq
    {d : ℕ} {α : Type*} {l : Filter α} {ψ : α → SpaceDHilbertSpace d} :
    Tendsto ψ l (nhds 0) ↔ Tendsto (fun a ↦ ∫⁻ x : Space d, ‖ψ a x‖ₑ ^ 2) l
  trans Tendsto (fun a ↦ (∫⁻ x, ‖ψ a x‖ₑ ^ 2) ^ (2⁻¹ : ℝ)) l (nhds 0)
  · simp [tendsto_iff_edist_tendsto_0, edist_zero_right, Lp.enorm_def, eLpN
  constructor <;> intro h
  · apply Tendsto.ennrpow_const 2 at h
    simp_all [← ENNReal.rpow_mul_natCast]
  · apply Tendsto.ennrpow_const 2⁻¹ at h
    simp_all

private lemma polyBddSchwartzSubmodule_zero_top_dense :
    Dense (polyBddSchwartzSubmodule 0 ⊤ : Set (SpaceDHilbertSpace 0)) := by
  suffices polyBddSchwartzMap 0 ⊤ = ⊤ by
    simp [polyBddSchwartzSubmodule, polyBddSchwartzIncl, this, schwartzSubm
  refine Submodule.eq_top_iff'.mpr (fun f k hk ↦ ?_)
  refine ⟨1 + ‖f 0‖, by positivity, fun x ↦ ?_⟩
  simp only [Space.point_dim_zero_eq, norm_zero, zpow_neg, zpow_natCast]
  rcases eq_or_ne k 0 with (rfl | hk')
```

```
  · simp
  · simp [hk', add_nonneg]

lemma polyBddSchwartzSubmodule_top_dense (d : ℕ) :
    Dense (polyBddSchwartzSubmodule d ⊤ : Set (SpaceDHilbertSpace d)) := by
  rcases eq_zero_or_pos d with (rfl | hd)
  · -- `d = 0`: Every function `Space 0 ≃ {0} → ℂ` is in `polyBddSchwartzSubmodule 0
    exact polyBddSchwartzSubmodule_zero_top_dense
  · -- `d > 0`: Construct a sequence in `polyBddSchwartzSubmodule d ⊤` which tends t
    intro ξ
    apply mem_closure_iff_seq_limit.mpr
    -- `ψₙ = [fₙ]` is a sequence in `schwartzSubmodule` which tends to `ξ`
    obtain ⟨ψ, hψ, hψξ⟩ := mem_closure_iff_seq_limit.mp (schwartzSubmodule_dense d ξ
    let f (n : ℕ) : 𝓢(Space d, ℂ) := schwartzEquiv.symm ⟨ψ n, hψ n⟩
    -- `bₙ` is a sequence of bump functions with shrinking domain
    let b (n : ℕ) : ContDiffBump (0 : Space d) :=
      ⟨(n + 1)⁻¹, 2 * (n + 1 : ℝ)⁻¹, by positivity, lt_two_mul_self Nat.inv_pos_of_n
    -- `φₙ = [bₙfₙ]` is a sequence in `schwartzSubmodule` which tends to `0`
    let g (n : ℕ) : 𝓢(Space d, ℂ) := smulLeftCLM ℂ (b n) (f n)
    let φ (n : ℕ) : SpaceDHilbertSpace d := schwartzIncl (g n)
    have hg (n : ℕ) (x : Space d) : g n x = b n x * f n x := by
      have := (b n).hasCompactSupport.hasTemperateGrowth (b n).contDiff
      rw [smulLeftCLM_apply_apply this, ← Complex.coe_smul, smul_eq_mul]
    use ψ - φ
    constructor
    · intro n
      rw [SetLike.mem_coe, LinearMap.mem_range, Subtype.exists]
      refine ⟨f n - g n, ?_, by simp [f, φ, polyBddSchwartzIncl, ← schwartzEquiv_app
      intro k _
      obtain ⟨C, hC_pos, hC⟩ := (f n).decay 0 0
      simp only [pow_zero, norm_iteratedFDeriv_zero, one_mul] at hC
      use (n + 1) ^ k * C
      refine ⟨by positivity, fun x ↦ ?_⟩
      rcases le_or_gt ‖x‖ (b n).rIn with (hx | hx)
      · have h_one : b n x = 1 := (b n).one_of_mem_closedBall (mem_closedBall_zero_i
        exact le_of_eq_of_le (b := 0) (by simp [hg, h_one]) (by positivity)
      · refine mul_le_mul_of_nonneg ?_ ?_ (by positivity) hC_pos.le
        · rw [← inv_zpow', zpow_natCast]
          refine pow_le_pow_left₀ (by positivity) ?_ k
          exact (inv_lt_of_inv_lt₀ (Nat.cast_add_one_pos n) hx).le
        · refine le_trans ?_ (hC x)
          rw [sub_apply, ← one_mul (f n x), hg, ← sub_mul]
          simp_rw [norm_mul, ← ofReal_one, ← ofReal_sub, norm_real, Real.norm_eq_abs
          refine mul_le_mul_of_nonneg_right ?_ (norm_nonneg _)
          exact abs_sub_le_of_nonneg_of_le zero_le_one le_rfl (b n).nonneg (b n).le_
    · refine tendsto_of_sub_tendsto_zero ξ hψξ ?_
```

```
    rw [sub_sub_cancel_left, Pi.neg_def, ← neg_zero, tendsto_neg_iff]
    -- Split `φₙ = σₙ + (φₙ - σₐ)` with `σₙ ≔ [bₙξ]` a sequence in `Space
    let s (n : ℕ) : Space d → ℂ := fun x ↦ b n x * ξ x
    let σ (n : ℕ) : SpaceDHilbertSpace d := by
      refine mk (f := s n) ⟨?_, ?_⟩
      · refine Continuous.comp_aestronglyMeasurable₂ (by fun_prop) ?_ ξ.v
        exact continuous_ofReal.comp_aestronglyMeasurable (b n).continuou
      · refine lt_of_le_of_lt ?_ (coe_hilbertSpace_memHS ξ).2
        exact eLpNorm_mono_enorm (enorm_bump_mul_le_enorm (b n) ξ)
    have hψ_ae (n : ℕ) : ψ n =ᵐ[volume] f n := (schwartzEquiv_symm_coe_ae
    have hφ_ae (n : ℕ) : φ n =ᵐ[volume] g n := schwartzEquiv_coe_ae (g n)
    have hσ_ae (n : ℕ) : σ n =ᵐ[volume] s n := coe_mk_ae _
    have hφσ_ae (n : ℕ) : (φ - σ) n =ᵐ[volume] g n - s n :=
      (AEEqFun.coeFn_sub (φ n).val (σ n).val).trans <| (hφ_ae n).sub (hσ_
    have hψξ_ae (n : ℕ) : ψ n - ξ =ᵐ[volume] f n - ξ :=
      (AEEqFun.coeFn_sub (ψ n).val ξ.val).trans <| (hψ_ae n).sub Eventual
    refine tendsto_of_sub_tendsto_zero (f := σ) 0 ?_ ?_
    · -- `σ = bₙξ → 0` since the norms are bounded by the integral of `‖ξ
      -- on a domain which tends to zero
      apply tendsto_zero_iff_tendsto_zero_lintegral_enorm_sq.mpr
      let B (n : ℕ) : Set (Space d) := Metric.ball 0 (b n).rOut
      have hξB : Tendsto (fun n ↦ ∫⁻ x in B n, ‖ξ x‖ₑ ^ 2) atTop (nhds 0)
        refine tendsto_setLIntegral_zero ?_ ?_
        · apply lt_top_iff_ne_top.mp
          have hξ := L2.eLpNorm_rpow_two_norm_lt_top ξ
          simp_rw [eLpNorm_one_eq_lintegral_enorm, Real.rpow_ofNat, enorm
          exact hξ
        · have : Nontrivial (Space d) := Nat.succ_pred_eq_of_pos hd ▸ Spa
          let C : ℝ := (ENNReal.ofReal (√Real.pi ^ d / Real.Gamma (d / 2
          have hvolB : ⇑volume ∘ B = fun n ↦ ENNReal.ofReal (C * (b n).rO
            ext n
            simp [B, InnerProductSpace.volume_ball, C, mul_comm,
              ENNReal.ofReal_pow (b n).rOut_pos.le]
          simp_rw [hvolB, ← ENNReal.ofReal_zero, b, ← one_div, mul_pow, ←
          rw [← mul_zero (C * 2 ^ d), ← zero_pow (M₀ := ℝ) hd.ne']
          refine ENNReal.tendsto_ofReal <| Tendsto.const_mul (C * 2 ^ d)
          exact tendsto_one_div_add_atTop_nhds_zero_nat.pow d
      refine Tendsto.squeeze tendsto_const_nhds hξB (zero_le _) (fun n ↦
      suffices ∫⁻ x, ‖σ n x‖ₑ ^ 2 = ∫⁻ x in B n, ‖σ n x‖ₑ ^ 2 by
        rw [this]
        refine setLIntegral_mono_ae' measurableSet_ball ?_
        filter_upwards [hσ_ae n] with x h _
        exact ENNReal.pow_le_pow_left <| h ▸ enorm_bump_mul_le_enorm (b n
      have h (A : Set (Space d)) : ∫⁻ x in A, ‖σ n x‖ₑ ^ 2 = ∫⁻ x in A, ‖
        lintegral_congr_ae ((hσ_ae n).fun_comp (fun z ↦ ‖z‖ₑ ^ 2)).restri
      rw [← setLIntegral_univ, h, h, setLIntegral_univ]
```

```
        refine (setLIntegral_eq_of_support_subset ?_).symm
        refine Function.support_subset_iff'.mpr (fun x hx ↦ ?_)
        simp [s, (b n).zero_of_le_dist (not_lt.mp hx)]
      · -- `φₙ - σₙ = bₙ(ψₙ - ξ) → 0` since `ψₙ → ξ` (by definition) and the `bₙ` ar
        apply tendsto_zero_iff_tendsto_zero_lintegral_enorm_sq.mpr
        have hψξ : Tendsto (fun n ↦ ∫⁻ x, ‖(ψ n - ξ) x‖ₑ ^ 2) atTop (nhds 0) :=
          tendsto_zero_iff_tendsto_zero_lintegral_enorm_sq.mp (sub_self ξ ▸ hψξ.sub_
        refine Tendsto.squeeze tendsto_const_nhds hψξ (zero_le _) (fun n ↦ ?_)
        refine lintegral_mono_ae ?_
        filter_upwards [hφσ_ae n, hψξ_ae n] with x h h'
        simp_rw [h, h', Pi.sub_apply, hg, s, ← mul_sub]
        exact ENNReal.pow_le_pow_left <| enorm_bump_mul_le_enorm (b n) (fun x ↦ f n

lemma polyBddSchwartzSubmodule_dense (d : ℕ) (a : ℕ∞) :
    Dense (polyBddSchwartzSubmodule d a : Set (SpaceDHilbertSpace d)) :=
  (polyBddSchwartzSubmodule_top_dense d).mono (polyBddSchwartzSubmodule_le_of_ge d l

end

end SpaceDHilbertSpace
end QuantumMechanics
```

1.239 Physlib/QuantumMechanics/DDimensions/SpaceDHilbertSpace/Schwartz

```
public import Physlib.QuantumMechanics.DDimensions.SpaceDHilbertSpace.Basic
# Schwartz submodule

## i. Overview

In this module we define the Schwartz submodule of `SpaceDHilbertSpace`.

## ii. Key results

- `schwartzSubmodule d`: Submodule of `SpaceDHilbertSpace d` consisting of the L² eq
  classes of Schwartz maps `𝓢(Space d, ℂ)`.

## iii. Table of contents

## iv. References
/-!
## A. Schwartz submodule
-/

noncomputable section
```

```
set_option backward.isDefEq.respectTransparency false in
/-- The continuous linear map including Schwartz maps into `SpaceDHilbertSp
/
set_option backward.isDefEq.respectTransparency false in
/-- The submodule of `SpaceDHilbertSpace d` corresponding to Schwartz maps.
/
set_option backward.isDefEq.respectTransparency false in
/-- The linear equivalence between the Schwartz maps `𝓢(Space d, ℂ)` and th
  of `SpaceDHilbertSpace d`. -/
variable (f g : 𝓢(Space d, ℂ)) (ψ : schwartzSubmodule d)
lemma schwartzEquiv_symm_coe_ae : schwartzEquiv.symm ψ =ᵐ[volume] ψ := by
  nth_rw 2 [← schwartzEquiv.apply_symm_apply ψ]
  exact (schwartzEquiv_coe_ae _).symm

lemma schwartzEquiv_apply_coe : ↑(schwartzEquiv f) = schwartzIncl f := by s

lemma schwartzIncl_ker : schwartzIncl.ker = (⊥ : Submodule ℂ 𝓢(Space d, ℂ))
  ext; simp [← schwartzEquiv_apply_coe]
```

1.240 Physlib/QuantumMechanics/FiniteTarget/Basic.lean

```
public import Physlib.Meta.TODO.Basic
public import Physlib.QuantumMechanics.PlanckConstant


TODO "Define a smooth structure on `FiniteTarget`."
```

1.241 Physlib/QuantumMechanics/OneDimension/GeneralPotential/Ba

```
public import Physlib.QuantumMechanics.OneDimension.Operators.Momentum
open HilbertSpace Constants
set_option backward.isDefEq.respectTransparency false in
```

1.242 Physlib/QuantumMechanics/OneDimension/HarmonicOscillator/

```
public import Physlib.QuantumMechanics.OneDimension.Operators.Parity
public import Physlib.QuantumMechanics.OneDimension.Operators.Momentum
public import Physlib.QuantumMechanics.OneDimension.Operators.Position
open HilbertSpace
```

```
set_option backward.isDefEq.respectTransparency false in
```

1.243 Physlib/QuantumMechanics/OneDimension/HarmonicOscillator/Complet

```
public import Physlib.QuantumMechanics.OneDimension.HarmonicOscillator.Eigenfunction
public import Physlib.QuantumMechanics.OneDimension.HilbertSpace.Gaussians
open Physlib
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.244 Physlib/QuantumMechanics/OneDimension/HarmonicOscillator/Eigenfu

```
public import Physlib.QuantumMechanics.OneDimension.HarmonicOscillator.Basic
public import Physlib.Mathematics.SpecialFunctions.PhysHermite
open Nat Physlib HilbertSpace MeasureTheory Constants
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.245 Physlib/QuantumMechanics/OneDimension/HarmonicOscillator/Example

```
public import Physlib.QuantumMechanics.OneDimension.HarmonicOscillator.TISE
lake env lean Physlib/QuantumMechanics/OneDimension/HarmonicOscillator/Examples.lean
```

1.246 Physlib/QuantumMechanics/OneDimension/HarmonicOscillator/TISE.le

```
public import Physlib.QuantumMechanics.OneDimension.HarmonicOscillator.Eigenfunction
open Nat Physlib HilbertSpace Constants
set_option backward.isDefEq.respectTransparency false in
    one_div, mul_inv_rev, one_mul, Polynomial.aeval, Polynomial.aevalEquiv, Equiv.co
    physHermite_zero, eq_intCast, Int.cast_one, Polynomial.eval₂AlgHom_apply,
    Algebra.toRingHom_ofId, algebraMap_int_eq, Polynomial.eval₂_one, Complex.ofReal_
    Complex.ofReal_ofNat, Pi.mul_apply, Pi.smul_apply, smul_eq_mul, pow_one, factori
    physHermite_one, Polynomial.eval₂_mul, Polynomial.eval₂_ofNat, Polynomial.eval₂_
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
        rw [show (Polynomial.aeval (x / Q.ξ)) 2 = 2 by
          simp [Polynomial.aevalEquiv, Polynomial.aeval]]
```

1.247 Physlib/QuantumMechanics/OneDimension/HilbertSpace/Basic.

```
public import Mathlib.MeasureTheory.Function.L2Space
public import Mathlib.MeasureTheory.Measure.Haar.OfBasis
set_option backward.isDefEq.respectTransparency false in
```

1.248 Physlib/QuantumMechanics/OneDimension/HilbertSpace/Gaussi

```
public import Physlib.QuantumMechanics.OneDimension.HilbertSpace.Basic
public import Mathlib.Analysis.SpecialFunctions.Gaussian.GaussianIntegral
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.249 Physlib/QuantumMechanics/OneDimension/HilbertSpace/PlaneW

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.250 Physlib/QuantumMechanics/OneDimension/HilbertSpace/Positi

```
set_option backward.isDefEq.respectTransparency false in
```

1.251 Physlib/QuantumMechanics/OneDimension/HilbertSpace/Schwar

```
public import Physlib.QuantumMechanics.OneDimension.HilbertSpace.Basic
public import Mathlib.Analysis.Distribution.SchwartzSpace.Basic
set_option backward.isDefEq.respectTransparency false in
```

1.252 Physlib/QuantumMechanics/OneDimension/Operators/Commutati

```
public import Physlib.QuantumMechanics.OneDimension.Operators.Position
```

```
public import Physlib.QuantumMechanics.OneDimension.Operators.Momentum
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.253 Physlib/QuantumMechanics/OneDimension/Operators/Momentum.lean

```
public import Mathlib.Analysis.Calculus.FDeriv.Star
public import Physlib.QuantumMechanics.OneDimension.Operators.Unbounded
public import Physlib.QuantumMechanics.OneDimension.HilbertSpace.SchwartzSubmodule
public import Physlib.QuantumMechanics.PlanckConstant
public import Physlib.QuantumMechanics.OneDimension.HilbertSpace.PlaneWaves
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  · intro _ _
    apply Differentiable.star
  · fun_prop
```

1.254 Physlib/QuantumMechanics/OneDimension/Operators/Parity.lean

```
public import Physlib.QuantumMechanics.OneDimension.HilbertSpace.SchwartzSubmodule
public import Physlib.QuantumMechanics.OneDimension.Operators.Unbounded
public import Mathlib.MeasureTheory.Measure.Haar.Unique
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.255 Physlib/QuantumMechanics/OneDimension/Operators/Position.lean

```
public import Physlib.QuantumMechanics.OneDimension.HilbertSpace.PositionStates
public import Physlib.QuantumMechanics.OneDimension.Operators.Unbounded
public import Physlib.Mathematics.Distribution.PowMul
public import Physlib.QuantumMechanics.OneDimension.HilbertSpace.SchwartzSubmodule
open Physlib

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.256 Physlib/QuantumMechanics/OneDimension/Operators/Unbounded

```
public import Physlib.QuantumMechanics.OneDimension.HilbertSpace.Basic
```

1.257 Physlib/QuantumMechanics/OneDimension/ReflectionlessPoten

```
public import Physlib.QuantumMechanics.OneDimension.Operators.Momentum
public import Physlib.Mathematics.Trigonometry.Tanh

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.258 Physlib/QuantumMechanics/PlanckConstant.lean

1.259 Physlib/Relativity/Bispinors/Basic.lean

```
public import Physlib.Relativity.PauliMatrices.ToTensor
```

1.260 Physlib/Relativity/CliffordAlgebra.lean

```
public import Physlib.Meta.TODO.Basic
TODO "Prove injectivity of ofCliffordAlgebra and construct the full isomorp
set_option backward.isDefEq.respectTransparency false in
    simp only [γ, LinearMap.coe_sum, LinearMap.coe_smulRight, LinearMap.coe
      SetLike.mk_smul_mk, Finset.sum_apply, AddSubmonoidClass.coe_finset_su
    rw [Finset.sum_eq_single i]
    · simp
    · intro b _ hb
      simp [Pi.single_eq_of_ne hb]
      module
    · simp
set_option backward.isDefEq.respectTransparency false in
```

1.261 Physlib/Relativity/LorentzAlgebra/Basic.lean

```
public import Physlib.Relativity.MinkowskiMatrix
```

1.262 Physlib/Relativity/LorentzAlgebra/Basis.lean

```
public import Physlib.Relativity.LorentzAlgebra.Basic
public import Physlib.Meta.TODO.Basic
TODO can be completed by proving linear independence and spanning of these
TODO "Prove that boost generators are symmetric: \
TODO "Prove that boost generators are traceless: \
TODO "Prove that rotation generators are antisymmetric: \
TODO "Prove that rotation generators are traceless: \
```

1.263 Physlib/Relativity/LorentzAlgebra/ExponentialMap.lean

```
public import Physlib.Mathematics.DataStructures.Matrix.LieTrace
public import Physlib.Relativity.LorentzAlgebra.Basic
public import Physlib.Relativity.LorentzGroup.Restricted.Basic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.264 Physlib/Relativity/LorentzGroup/Basic.lean

```
public import Physlib.Relativity.MinkowskiMatrix
public import Physlib.Meta.TODO.Basic
TODO "Define the Lie group structure on the Lorentz group."
  simp [dual, SubtractionMonoid.neg_neg]
set_option backward.isDefEq.respectTransparency false in
```

1.265 Physlib/Relativity/LorentzGroup/Boosts/Apply.lean

```
public import Physlib.Relativity.LorentzGroup.Boosts.Basic
public import Physlib.Relativity.Tensors.RealTensor.Vector.Basic
```

1.266 Physlib/Relativity/LorentzGroup/Boosts/Basic.lean

```
public import Physlib.Relativity.LorentzGroup.Basic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.267 Physlib/Relativity/LorentzGroup/Boosts/Generalized.lean

```
public import Physlib.Relativity.LorentzGroup.Restricted.Basic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    refine (continuous_const_mul (η i i)).comp' (?_)
set_option backward.isDefEq.respectTransparency false in
    refine (continuous_const_mul (η i i)).comp' (?_)
set_option backward.isDefEq.respectTransparency false in
https://leanprover.zulipchat.com/#narrow/channel/479953-Physlib/topic/Loren
            mul_pow (v.1 (Sum.inl 0)) (‖u.1.spatialPart‖) 2,
            Velocity.norm_spatialPart_sq_eq u, Velocity.norm_spatialPart_sq
            real_inner_comm (u.1.spatialPart) (v.1.spatialPart)]
```

1.268 Physlib/Relativity/LorentzGroup/Orthochronous/Basic.lean

```
public import Physlib.Relativity.LorentzGroup.Proper
public import Physlib.Relativity.LorentzGroup.ToVector
public import Physlib.Relativity.Tensors.RealTensor.Velocity.Basic
TODO "Prove topological properties of the Orthochronous Lorentz Group."
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.269 Physlib/Relativity/LorentzGroup/Proper.lean

```
public import Physlib.Relativity.LorentzGroup.Basic
```

1.270 Physlib/Relativity/LorentzGroup/Restricted/Basic.lean

```
public import Physlib.Meta.Informal.SemiFormal
public import Physlib.Relativity.LorentzGroup.Orthochronous.Basic
```

```
TODO "Prove that every member of the restricted Lorentz group is
```

1.271 Physlib/Relativity/LorentzGroup/Restricted/FromBoostRotation.lean

```
public import Physlib.Relativity.LorentzGroup.Rotations
public import Physlib.Relativity.LorentzGroup.Boosts.Generalized
      simp only [toVector_rotation, Fin.isValue]
```

1.272 Physlib/Relativity/LorentzGroup/Rotations.lean

```
public import Physlib.Relativity.LorentzGroup.Restricted.Basic
      Matrix.mul_neg, neg_zero, mul_neg, SubtractionMonoid.neg_neg]
        simp [Matrix.fromBlocks_transpose, Matrix.fromBlocks_multiply,
          SubtractionMonoid.neg_neg] at hΛ
```

1.273 Physlib/Relativity/LorentzGroup/ToVector.lean

```
public import Physlib.Relativity.Tensors.RealTensor.Vector.MinkowskiProduct
set_option backward.isDefEq.respectTransparency false in
```

1.274 Physlib/Relativity/MinkowskiMatrix.lean

```
  simp [as_block, fromBlocks_multiply, SubtractionMonoid.neg_neg]
```

1.275 Physlib/Relativity/PauliMatrices/AsTensor.lean

```
/-
Copyright (c) 2024 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
module

public import Physlib.Relativity.Tensors.ComplexTensor.Weyl.Two
public import Physlib.Relativity.Tensors.ComplexTensor.Vector.Pre.Basic
/-!

## Pauli matrices as a tensor
```

```
The results in this file are primarily used to show that
the pauli matrices in invariant under the `SL(2,ℂ)` action.

-/

@[expose] public section

namespace PauliMatrix

open Complex
open Lorentz
open Fermion
open TensorProduct
open CategoryTheory.MonoidalCategory

noncomputable section

open Matrix
open MatrixGroups
open Complex
open TensorProduct

/-- The tensor `σ^μ^a^{dot a}` based on the Pauli-matrices as an element of
  `complexContr ⊗ leftHanded ⊗ rightHanded`. -/
def asTensor : (complexContr ⊗ leftHanded ⊗ rightHanded).V :=
  ∑ i, complexContrBasis i ⊗ₜ leftRightToMatrix.symm (pauliBasis i)

/-- The expansion of `asTensor` into complexContrBasis basis vectors . -
/
lemma asTensor_expand_complexContrBasis : asTensor =
    complexContrBasis (Sum.inl 0) ⊗ₜ leftRightToMatrix.symm (pauliBasis (Su
    + complexContrBasis (Sum.inr 0) ⊗ₜ leftRightToMatrix.symm (pauliBasis (
    + complexContrBasis (Sum.inr 1) ⊗ₜ leftRightToMatrix.symm (pauliBasis (
    + complexContrBasis (Sum.inr 2) ⊗ₜ leftRightToMatrix.symm (pauliBasis (
  rfl

set_option backward.isDefEq.respectTransparency false in
/-- The expansion of the pauli matrix `σ₀` in terms of a basis of tensor pr
/
lemma leftRightToMatrix_σSA_inl_0_expand : leftRightToMatrix.symm (pauliBas
    leftBasis 0 ⊗ₜ rightBasis 0 + leftBasis 1 ⊗ₜ rightBasis 1 := by
  rw [leftRightToMatrix_symm_expand_tmul]
  simp [pauliBasis, pauliSelfAdjoint, pauliMatrix]

set_option backward.isDefEq.respectTransparency false in
/-- The expansion of the pauli matrix `σ₁` in terms of a basis of tensor pr
```

```
/
lemma leftRightToMatrix_σSA_inr_0_expand : leftRightToMatrix.symm (pauliBasis (Sum.i
    leftBasis 0 ⊗ₜ rightBasis 1 + leftBasis 1 ⊗ₜ rightBasis 0:= by
  rw [leftRightToMatrix_symm_expand_tmul]
  simp [pauliBasis, pauliSelfAdjoint, pauliMatrix]

set_option backward.isDefEq.respectTransparency false in
/-- The expansion of the pauli matrix `σ₂` in terms of a basis of tensor product vec
/
lemma leftRightToMatrix_σSA_inr_1_expand : leftRightToMatrix.symm (pauliBasis (Sum.i
    -(I • leftBasis 0 ⊗ₜ[ℂ] rightBasis 1) + I • leftBasis 1 ⊗ₜ[ℂ] rightBasis 0 := by
  rw [leftRightToMatrix_symm_expand_tmul]
  simp [pauliBasis, pauliSelfAdjoint, pauliMatrix]

set_option backward.isDefEq.respectTransparency false in
/-- The expansion of the pauli matrix `σ₃` in terms of a basis of tensor product vec
/
lemma leftRightToMatrix_σSA_inr_2_expand : leftRightToMatrix.symm (pauliBasis (Sum.i
    leftBasis 0 ⊗ₜ rightBasis 0 - leftBasis 1 ⊗ₜ rightBasis 1 := by
  rw [leftRightToMatrix_symm_expand_tmul]
  simp [pauliBasis, pauliSelfAdjoint, pauliMatrix]
  rfl

set_option backward.isDefEq.respectTransparency false in
/-- The expansion of `asTensor` into complexContrBasis basis of tensor product vecto
/
lemma asTensor_expand : asTensor =
    complexContrBasis (Sum.inl 0) ⊗ₜ (leftBasis 0 ⊗ₜ rightBasis 0)
    + complexContrBasis (Sum.inl 0) ⊗ₜ (leftBasis 1 ⊗ₜ rightBasis 1)
    + complexContrBasis (Sum.inr 0) ⊗ₜ (leftBasis 0 ⊗ₜ rightBasis 1)
    + complexContrBasis (Sum.inr 0) ⊗ₜ (leftBasis 1 ⊗ₜ rightBasis 0)
    - I • complexContrBasis (Sum.inr 1) ⊗ₜ (leftBasis 0 ⊗ₜ rightBasis 1)
    + I • complexContrBasis (Sum.inr 1) ⊗ₜ (leftBasis 1 ⊗ₜ rightBasis 0)
    + complexContrBasis (Sum.inr 2) ⊗ₜ (leftBasis 0 ⊗ₜ rightBasis 0)
    - complexContrBasis (Sum.inr 2) ⊗ₜ (leftBasis 1 ⊗ₜ rightBasis 1) := by
  rw [asTensor_expand_complexContrBasis]
  rw [leftRightToMatrix_σSA_inl_0_expand, leftRightToMatrix_σSA_inr_0_expand,
    leftRightToMatrix_σSA_inr_1_expand, leftRightToMatrix_σSA_inr_2_expand]
  simp only [Fin.isValue, tmul_add, tmul_neg, tmul_smul, tmul_sub]
  rfl

set_option backward.isDefEq.respectTransparency false in
/-- The tensor `σ^μ^a^{dot a}` based on the Pauli-matrices as a morphism,
  `□_ (Rep ℂ SL(2,ℂ)) □ complexContr ⊗ leftHanded ⊗ rightHanded` manifesting
  the invariance under the `SL(2,ℂ)` action. -/
def asConsTensor : □_ (Rep ℂ SL(2,ℂ)) □ complexContr ⊗ leftHanded ⊗ rightHanded := R
```

```
{
toFun := fun a =>
  let a' : ℂ := a
  a' • asTensor,
map_add' := fun x y => by
  simp only [add_smul],
map_smul' := fun m x => by
  simp only [smul_smul]
  rfl
isIntertwining' M := by
  refine LinearMap.ext fun x : ℂ => ?_
  change x • asTensor =
    (TensorProduct.map (complexContr.ρ M)
      (TensorProduct.map (leftHanded.ρ M) (rightHanded.ρ M))) (x • asTe
  simp only [map_smul]
  apply congrArg
  nth_rewrite 2 [asTensor]
  simp only [map_sum, map_tmul]
  symm
  calc _ = ∑ x, ((complexContr.ρ M) (complexContrBasis x) ⊗ₜ[ℂ]
    leftRightToMatrix.symm (SL2C.toSelfAdjointMap M (pauliBasis x))) :=
        refine Finset.sum_congr rfl (fun x _ => ?_)
        rw [← leftRightToMatrix_ρ_symm_selfAdjoint]
    _ = ∑ x, ((∑ i, (SL2C.toLorentzGroup M).1 i x • (complexContrBasis
        ∑ j, leftRightToMatrix.symm ((SL2C.toLorentzGroup M⁻¹).1 x j •
        refine Finset.sum_congr rfl (fun x _ => ?_)
        rw [SL2CRep_ρ_basis, SL2C.toSelfAdjointMap_pauliBasis]
        simp only [SL2C.toLorentzGroup_apply_coe, Fintype.sum_sum_type,
          Fin.default_eq_zero, Fin.isValue, Finset.sum_singleton, map_i
          LorentzGroup.inv_eq_dual, AddSubgroup.coe_add, selfAdjoint.va
          AddSubgroup.val_finset_sum, map_add, map_sum]
    _ = ∑ x, ∑ i, ∑ j, ((SL2C.toLorentzGroup M).1 i x • (complexContrBa
          leftRightToMatrix.symm.toLinearMap
            ((SL2C.toLorentzGroup M⁻¹).1 x j • (pauliBasis j)) := by
        refine Finset.sum_congr rfl (fun x _ => ?_)
        rw [sum_tmul]
        refine Finset.sum_congr rfl (fun i _ => ?_)
        rw [tmul_sum]
        rfl
    _ = ∑ x, ∑ i, ∑ j, ((SL2C.toLorentzGroup M).1 i x • (complexContrBa
          ((SL2C.toLorentzGroup M⁻¹).1 x j • leftRightToMatrix.symm ((p
        refine Finset.sum_congr rfl (fun x _ => (Finset.sum_congr rfl (
          (Finset.sum_congr rfl (fun j _ => ?_)))))
        simp only [SL2C.toLorentzGroup_apply_coe, map_inv, LorentzGroup
          LinearMap.map_smul_of_tower, LinearEquiv.coe_coe, tmul_smul]
    _ = ∑ x, ∑ i, ∑ j, ((SL2C.toLorentzGroup M).1 i x * (SL2C.toLorentz
```

```
            • ((complexContrBasis i)) ⊗ₜ[ℂ] leftRightToMatrix.symm ((pauliBasis j))
            refine Finset.sum_congr rfl (fun x _ => (Finset.sum_congr rfl (fun i _ =
              (Finset.sum_congr rfl (fun j _ => ?_)))))
            rw [smul_tmul, smul_smul, tmul_smul]
          _ = ∑ i, ∑ x, ∑ j, ((SL2C.toLorentzGroup M).1 i x * (SL2C.toLorentzGroup M⁻¹
            • ((complexContrBasis i)) ⊗ₜ[ℂ] leftRightToMatrix.symm ((pauliBasis j))
              Finset.sum_comm
          _ = ∑ i, ∑ j, ∑ x, ((SL2C.toLorentzGroup M).1 i x * (SL2C.toLorentzGroup M⁻¹
            • ((complexContrBasis i)) ⊗ₜ[ℂ] leftRightToMatrix.symm ((pauliBasis j))
              Finset.sum_congr rfl (fun x _ => Finset.sum_comm)
          _ = ∑ i, ∑ j, (∑ x, (SL2C.toLorentzGroup M).1 i x * (SL2C.toLorentzGroup M⁻¹
            • ((complexContrBasis i)) ⊗ₜ[ℂ] leftRightToMatrix.symm ((pauliBasis j))
            refine Finset.sum_congr rfl (fun i _ => (Finset.sum_congr rfl (fun j _ =
            rw [Finset.sum_smul]
          _ = ∑ i, ∑ j, ((1 : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℝ) i j)
            • ((complexContrBasis i)) ⊗ₜ[ℂ] leftRightToMatrix.symm ((pauliBasis j)) :=
            refine Finset.sum_congr rfl (fun i _ => (Finset.sum_congr rfl (fun j _ =
            congr
            change ((SL2C.toLorentzGroup M) * (SL2C.toLorentzGroup M⁻¹)).1 i j = _
            rw [← SL2C.toLorentzGroup.map_mul]
            simp only [mul_inv_cancel, _root_.map_one, lorentzGroupIsGroup_one_coe]
          _ = ∑ i, ((1 : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℝ) i i)
            • ((complexContrBasis i)) ⊗ₜ[ℂ] leftRightToMatrix.symm ((pauliBasis i)) :=
            refine Finset.sum_congr rfl (fun i _ => ?_)
            refine Finset.sum_eq_single i (fun b _ hb => ?_) (fun hb => ?_)
            · simp [one_apply_ne' hb]
            · simp only [Finset.mem_univ, not_true_eq_false] at hb
          _ = asTensor := by
            refine Finset.sum_congr rfl (fun i _ => ?_)
            simp only [one_apply_eq, one_smul]}

/-- The map `□_ (Rep ℂ SL(2,ℂ)) □ complexContr ⊗ leftHanded ⊗ rightHanded` correspon
  to Pauli matrices, when evaluated on `1` corresponds to the tensor `PauliMatrix.as
/
lemma asConsTensor_apply_one : asConsTensor.hom (1 : ℂ) = asTensor := by
  change (1 : ℂ) • asTensor = asTensor
  simp only [one_smul]

end
end PauliMatrix
```

1.276 Physlib/Relativity/PauliMatrices/Basic.lean

1.277 Physlib/Relativity/PauliMatrices/CliffordAlgebra.lean

```
public import Physlib.Relativity.PauliMatrices.Basic
set_option backward.isDefEq.respectTransparency false in
    simp only [Fin.isValue, Nat.succ_eq_add_one, Nat.reduceAdd, LinearMap.c
      LinearMap.coe_smulRight, LinearMap.coe_proj, Function.eval, SetLike.m
      Finset.sum_apply, AddSubmonoidClass.coe_finset_sum]
    rw [Finset.sum_eq_single i]
    · simp
    · intro b _ hb
      simp [Pi.single_eq_of_ne hb]
      module
    · simp
```

1.278 Physlib/Relativity/PauliMatrices/Relations.lean

```
public import Physlib.Relativity.PauliMatrices.ToTensor
public import Physlib.Relativity.Tensors.ComplexTensor.Units.Basic

set_option backward.isDefEq.respectTransparency false in
    rw [← Physlib.RatComplexNum.toComplexNum.map_mul]
    rw [Physlib.RatComplexNum.ofNat_mul_toComplexNum 2]
    rw [← Physlib.RatComplexNum.toComplexNum.map_mul]
    rw [← Physlib.RatComplexNum.toComplexNum.map_mul]
  rw [← map_sum Physlib.RatComplexNum.toComplexNum]
  apply (Function.Injective.eq_iff Physlib.RatComplexNum.toComplexNum_injec
  apply (Function.Injective.eq_iff Physlib.RatComplexNum.toComplexNum_injec
  apply (Function.Injective.eq_iff Physlib.RatComplexNum.toComplexNum_injec
  apply (Function.Injective.eq_iff Physlib.RatComplexNum.toComplexNum_injec
  apply (Function.Injective.eq_iff Physlib.RatComplexNum.toComplexNum_injec
```

1.279 Physlib/Relativity/PauliMatrices/SelfAdjoint.lean

```
public import Physlib.Relativity.PauliMatrices.Basic
public import Physlib.Relativity.MinkowskiMatrix
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  | Sum.inr 0 => ⟨-σ1, by rw [AddSubgroup.neg_mem_iff]; exact pauliMatrix_s
  | Sum.inr 1 => ⟨-σ2, by rw [AddSubgroup.neg_mem_iff]; exact pauliMatrix_s
  | Sum.inr 2 => ⟨-σ3, by rw [AddSubgroup.neg_mem_iff]; exact pauliMatrix_s
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.280 Physlib/Relativity/PauliMatrices/ToTensor.lean

```
public import Physlib.Relativity.Tensors.ComplexTensor.Metrics.Basic
public import Physlib.Relativity.PauliMatrices.AsTensor
public import Physlib.Relativity.Tensors.ComplexTensor.Metrics.Basic

set_option backward.isDefEq.respectTransparency false in
    Tensor.basis ![Color.up, Color.upL, Color.upR]
      (fun | 0 => (0 : Fin 4) | 1 => (0 : Fin 2) | 2 => (0 : Fin 2))
    + Tensor.basis ![Color.up, Color.upL, Color.upR]
      (fun | 0 => (0 : Fin 4) | 1 => (1 : Fin 2) | 2 => (1 : Fin 2))
    + Tensor.basis ![Color.up, Color.upL, Color.upR]
      (fun | 0 => (1 : Fin 4) | 1 => (0 : Fin 2) | 2 => (1 : Fin 2))
    + Tensor.basis ![Color.up, Color.upL, Color.upR]
      (fun | 0 => (1 : Fin 4) | 1 => (1 : Fin 2) | 2 => (0 : Fin 2))
    - I • Tensor.basis ![Color.up, Color.upL, Color.upR]
      (fun | 0 => (2 : Fin 4) | 1 => (0 : Fin 2) | 2 => (1 : Fin 2))
    + I • Tensor.basis ![Color.up, Color.upL, Color.upR]
      (fun | 0 => (2 : Fin 4) | 1 => (1 : Fin 2) | 2 => (0 : Fin 2))
    + Tensor.basis ![Color.up, Color.upL, Color.upR]
      (fun | 0 => (3 : Fin 4) | 1 => (0 : Fin 2) | 2 => (0 : Fin 2))
    - Tensor.basis ![Color.up, Color.upL, Color.upR]
      (fun | 0 => (3 : Fin 4) | 1 => (1 : Fin 2) | 2 => (1 : Fin 2)) := by
set_option backward.isDefEq.respectTransparency false in
    σ^^^ = fromConstTriple (S := complexLorentzTensor)
      (c1 := Color.up) (c2 := Color.upL) (c3 := Color.upR) PauliMatrix.asConsTensor
  simp only [Nat.succ_eq_add_one, Nat.reduceAdd, Fin.isValue, map_sub,
    if b 0 = Fin.cast (by rfl) (0 : Fin 4) ∧ b 1 = b 2 then ⟨1, 0⟩ else
    if b 0 = Fin.cast (by rfl) (1 : Fin 4) ∧ b 1 ≠ b 2 then ⟨1, 0⟩ else
    if b 0 = Fin.cast (by rfl) (2 : Fin 4) ∧ b 1 = Fin.cast (by rfl) (0 : Fin 2) ∧
      b 2 = Fin.cast (by rfl) (1 : Fin 2) then ⟨0, -1⟩ else
    if b 0 = Fin.cast (by rfl) (2 : Fin 4) ∧ b 1 = Fin.cast (by rfl) (1 : Fin 2) ∧
      b 2 = Fin.cast (by rfl) (0 : Fin 2) then ⟨0, 1⟩ else
    if b 0 = Fin.cast (by rfl) (3 : Fin 4) ∧ b 1 = Fin.cast (by rfl) (0 : Fin 2) ∧
      b 2 = Fin.cast (by rfl) (0 : Fin 2) then ⟨1, 0⟩ else
    if b 0 = Fin.cast (by rfl) (3 : Fin 4) ∧ b 1 = Fin.cast (by rfl) (1 : Fin 2) ∧
```

```
      b 2 = Fin.cast (by rfl) (1 : Fin 2) then ⟨-1, 0⟩ else 0) := by
    smul_eq_mul, Physlib.RatComplexNum.I_mul_toComplexNum, mul_ite, ne_eq,
  apply (Function.Injective.eq_iff Physlib.RatComplexNum.toComplexNum_injec
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    if b 0 = Fin.cast (by rfl) (0 : Fin 4) ∧ b 1 = b 2 then ⟨1, 0⟩ else
    if b 0 = Fin.cast (by rfl) (1 : Fin 4) ∧ b 1 ≠ b 2 then ⟨-
1, 0⟩ else
    if b 0 = Fin.cast (by rfl) (2 : Fin 4) ∧ b 1 = Fin.cast (by rfl) (0 : F
      b 2 = Fin.cast (by rfl) (1 : Fin 2) then ⟨0, 1⟩ else
    if b 0 = Fin.cast (by rfl) (2 : Fin 4) ∧ b 1 = Fin.cast (by rfl) (1 : F
      b 2 = Fin.cast (by rfl) (0 : Fin 2) then ⟨0, -1⟩ else
    if b 0 = Fin.cast (by rfl) (3 : Fin 4) ∧ b 1 = Fin.cast (by rfl) (0 : F
      b 2 = Fin.cast (by rfl) (0 : Fin 2) then ⟨-1, 0⟩ else
    if b 0 = Fin.cast (by rfl) (3 : Fin 4) ∧ b 1 = Fin.cast (by rfl) (1 : F
      b 2 = Fin.cast (by rfl) (1 : Fin 2) then ⟨1, 0⟩ else ⟨0, 0⟩) := by
    rw [← Physlib.RatComplexNum.toComplexNum.map_mul]
    rw [← Physlib.RatComplexNum.toComplexNum.map_mul]
  rw [← map_sum Physlib.RatComplexNum.toComplexNum]
  apply (Function.Injective.eq_iff Physlib.RatComplexNum.toComplexNum_injec
    if b 0 = Fin.cast (by rfl) (0 : Fin 4) ∧ b 1 = b 2 then ⟨1, 0⟩ else
    if b 0 = Fin.cast (by rfl) (1 : Fin 4) ∧ b 1 ≠ b 2 then ⟨1, 0⟩ else
    if b 0 = Fin.cast (by rfl) (2 : Fin 4) ∧ b 1 = Fin.cast (by rfl) (0 : F
      b 2 = Fin.cast (by rfl) (1 : Fin 2) then ⟨0, -1⟩ else
    if b 0 = Fin.cast (by rfl) (2 : Fin 4) ∧ b 1 = Fin.cast (by rfl) (1 : F
      b 2 = Fin.cast (by rfl) (0 : Fin 2) then ⟨0, 1⟩ else
    if b 0 = Fin.cast (by rfl) (3 : Fin 4) ∧ b 1 = Fin.cast (by rfl) (1 : F
      b 2 = Fin.cast (by rfl) (1 : Fin 2) then ⟨-1, 0⟩ else
    if b 0 = Fin.cast (by rfl) (3 : Fin 4) ∧ b 1 = Fin.cast (by rfl) (0 : F
      b 2 = Fin.cast (by rfl) (0 : Fin 2) then ⟨1, 0⟩ else ⟨0, 0⟩) := by
    rw [← Physlib.RatComplexNum.toComplexNum.map_mul]
    rw [← Physlib.RatComplexNum.toComplexNum.map_mul]
    rw [← map_sum Physlib.RatComplexNum.toComplexNum]
    rw [← Physlib.RatComplexNum.toComplexNum.map_mul]
    rw [← Physlib.RatComplexNum.toComplexNum.map_mul]
  rw [← map_sum Physlib.RatComplexNum.toComplexNum]
  apply (Function.Injective.eq_iff Physlib.RatComplexNum.toComplexNum_injec
    if b 0 = Fin.cast (by rfl) (0 : Fin 4) ∧ b 1 = b 2 then ⟨1, 0⟩ else
    if b 0 = Fin.cast (by rfl) (1 : Fin 4) ∧ b 1 ≠ b 2 then ⟨-
1, 0⟩ else
    if b 0 = Fin.cast (by rfl) (2 : Fin 4) ∧ b 1 = Fin.cast (by rfl) (0 : F
      b 2 = Fin.cast (by rfl) (1 : Fin 2) then ⟨0, 1⟩ else
    if b 0 = Fin.cast (by rfl) (2 : Fin 4) ∧ b 1 = Fin.cast (by rfl) (1 : F
      b 2 = Fin.cast (by rfl) (0 : Fin 2) then ⟨0, -1⟩ else
    if b 0 = Fin.cast (by rfl) (3 : Fin 4) ∧ b 1 = Fin.cast (by rfl) (1 : F
      b 2 = Fin.cast (by rfl) (1 : Fin 2) then ⟨1, 0⟩ else
```

```
    if b 0 = Fin.cast (by rfl) (3 : Fin 4) ∧ b 1 = Fin.cast (by rfl) (0 : Fin 2) ∧
      b 2 = Fin.cast (by rfl) (0 : Fin 2) then ⟨-1, 0⟩ else 0) := by
    rw [← Physlib.RatComplexNum.toComplexNum.map_mul]
    rw [← Physlib.RatComplexNum.toComplexNum.map_mul]
    rw [← map_sum Physlib.RatComplexNum.toComplexNum]
    rw [← Physlib.RatComplexNum.toComplexNum.map_mul]
    rw [← Physlib.RatComplexNum.toComplexNum.map_mul]
  rw [← map_sum Physlib.RatComplexNum.toComplexNum]
  apply (Function.Injective.eq_iff Physlib.RatComplexNum.toComplexNum_injective).mpr
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.281 Physlib/Relativity/SL2C/Basic.lean

```
public import Physlib.Relativity.SL2C.SelfAdjoint
public import Physlib.Relativity.LorentzGroup.Restricted.Basic
lemma toSelfAdjointMap_apply (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
    toSelfAdjointMap M A = ⟨M.1 * A.1 * Matrix.conjTranspose M, by
        noncomm_ring [selfAdjoint.mem_iff, star_eq_conjTranspose,
        conjTranspose_mul, conjTranspose_conjTranspose,
        (star_eq_conjTranspose A.1).symm.trans $ selfAdjoint.mem_iff.mp A.2]⟩ := rfl

set_option backward.isDefEq.respectTransparency false in
    simp only [Fin.isValue, I_sq, mul_neg, mul_one, neg_mul, one_mul, sub_neg_eq_add
    simp only [Fin.isValue, I_sq, mul_neg, mul_one, neg_mul, one_mul, sub_neg_eq_add
    erw [Module.End.mul_apply]
    simp only [toSelfAdjointMap_apply, SpecialLinearGroup.coe_mul, conjTranspose_mul
      Subtype.mk.injEq]
    ext1
  positivity
```

1.282 Physlib/Relativity/SL2C/SelfAdjoint.lean

```
public import Physlib.Mathematics.SchurTriangulation
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.283 Physlib/Relativity/Special/ProperTime.lean

```
public import Physlib.SpaceAndTime.SpaceTime.Basic
```

```
public import Physlib.Relativity.Tensors.RealTensor.Vector.Causality.LightL
public import Physlib.Relativity.Tensors.RealTensor.Vector.Causality.TimeLi
```

1.284 Physlib/Relativity/Special/TwinParadox/Basic.lean

```
public import Physlib.Relativity.Special.ProperTime
TODO "Find the conditions for which the age gap for the twin paradox is zer
set_option backward.isDefEq.respectTransparency false in
TODO "Do the twin paradox with a non-instantaneous acceleration. This shoul
```

1.285 Physlib/Relativity/SpeedOfLight.lean

```
public import Mathlib.Data.Real.Basic
```

1.286 Physlib/Relativity/Tensors/Basic.lean

```
public import Physlib.Relativity.Tensors.TensorSpecies.Basic
public import Physlib.Meta.TODO.Basic
TODO "In this file (Physlib/Relativity/Tensors/Basic.lean), write an overvi
  implementation of tensors in Physlib. It should cover the main points:
- the definition of a tensor species,
- the meaning of color,
- the tensorial instances,
- other key definitions."

variable {k : Type} [CommRing k] {C G : Type} [Group G]
  {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basi
  (S : TensorSpecies k C G basisIdx)
variable {S : TensorSpecies k C G basisIdx} {n n' n2 : ℕ} {c : Fin n → C} {
set_option linter.unusedVariables false in
@[nolint unusedArguments]
abbrev ComponentIdx {n : ℕ} {S : TensorSpecies k C G basisIdx} (c : Fin n →
  Π j, basisIdx (c j)
    (i j : Fin n) (h : i = j) : b i = basisIdxCongr (by simp [h]) (b j) :=
      basisIdxCongr (by simp [hc]) (b (Fin.cast h.symm j))
abbrev Pure (S : TensorSpecies k C G basisIdx) (c : Fin n → C) : Type :=
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  rw [Physlib.PiTensorProduct.tmulEquiv_tmul_tprod]
set_option backward.isDefEq.respectTransparency false in
    (Finsupp.linearEquivFunOnFinite k k ((j : Fin n) → basisIdx (c j))).sym
```

```
lemma basis_congr {c1 c2 : C} (h : c1 = c2) (x : basisIdx c1)
    (y : basisIdx c2) (hxy : y = basisIdxCongr (by simp [h]) x) :
    S.FD.map (eqToHom (by simp [h])) ((S.basis c1) x) =
    (S.basis c2) y := by
  subst h hxy
  simp
instance {k : Type} [Field k] {C G : Type} [Group G]
    {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basisIdx c)
    (S : TensorSpecies k C G basisIdx)
noncomputable instance {k : Type} [RCLike k] {C G : Type} [Group G]
    {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basisIdx c)
    (S : TensorSpecies k C G basisIdx)
instance {k : Type} [RCLike k] {C G : Type} [Group G]
    {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basisIdx c)
    (S : TensorSpecies k C G basisIdx)
set_option backward.isDefEq.respectTransparency false in
TODO "Prove that if `σ` satisfies `PermCond c c1 σ` then `PermCond.inv σ h`
    Pure.basisVector c1 (fun i => basisIdxCongr (by simp [h.preserve_color]) (b (σ i
  have h1 {c1 c2 : C} (h : c1 = c2) (x : basisIdx c1) :
      (S.basis c2) (basisIdxCongr (by simp [h]) x) := by
  toFun t := (S.F.map h.toHom).hom t
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    simp only [Pure.permP_basisVector, basisIdxCongr_apply_apply, Function.comp_appl
    rfl
      basisIdxCongr (by simp [PermCond.inv_perserve_color]) (b (h.inv σ i))) := by
      simpa [← h'] using ComponentIdx.congr_right _ _ _ (PermCond.inv_apply_apply σ
      simpa [h'] using (ComponentIdx.congr_right _ _ _ (PermCond.apply_inv_apply σ h
set_option backward.isDefEq.respectTransparency false in
```

1.287 Physlib/Relativity/Tensors/Color/Basic.lean

```
set_option backward.isDefEq.respectTransparency false in
```

1.288 Physlib/Relativity/Tensors/Color/Discrete.lean

```
public import Mathlib.RepresentationTheory.Rep.Basic
    (y : ((Discrete.functor (Discrete.mk ∘ τ) ⋙ F).obj ⟨c⟩)) (h : c = c1) :
```

1.289 Physlib/Relativity/Tensors/Color/Lift.lean

```
public import Physlib.Relativity.Tensors.Color.Basic
public import Mathlib.RepresentationTheory.Rep.Basic
public import Physlib.Mathematics.PiTensorProduct
public import Physlib.Meta.Informal.Basic
set_option backward.isDefEq.respectTransparency false in
    simp_all [LinearMap.map_add]
  simp only [Functor.id_obj, PiTensorProduct.tprodCoeff_eq_smul_tprod, Line
set_option backward.isDefEq.respectTransparency false in
    simp_all [LinearMap.map_add]
    LinearMap.map_smul, PiTensorProduct.map_tprod, LinearMap.id_coe, id_eq]
set_option backward.isDefEq.respectTransparency false in
  (F.mapIso (Discrete.eqToIso h)).hom.hom.toLinearMap
  (F.mapIso (Discrete.eqToIso h)).inv.hom.toLinearMap
  (by simp [← Representation.IntertwiningMap.comp_toLinearMap, ← Rep.hom_co
  (by simp [← Representation.IntertwiningMap.comp_toLinearMap, ← Rep.hom_co
    Iso.refl_inv, LinearEquiv.ofLinear_apply]
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
def homToRepHom {f g : OverColor C} (m : f ⟶ g) : toRep F f ⟶ toRep F g :=
  {
    toLinearMap := (linearIsoOfHom F m).toLinearMap
    isIntertwining' M := by
      apply LinearMap.ext
      intro x
      refine PiTensorProduct.induction_on' x ?_ (by
        intro x y hx hy
        simp only [map_add, hx, hy])
      intro r x
      simp only [PiTensorProduct.tprodCoeff_eq_smul_tprod, map_smul]
      apply congrArg
      change (linearIsoOfHom F m) (((toRep F f).ρ M) ((PiTensorProduct.tpro
        ((toRep F g).ρ M) ((linearIsoOfHom F m) ((PiTensorProduct.tprod k)
      rw [linearIsoOfHom_tprod, toRep_ρ_tprod]
      simp only [Functor.id_obj]
      rw [linearIsoOfHom_tprod, toRep_ρ_tprod]
      apply congrArg
      funext i
      rw [linearIsoOfEq_comm_ρ]
  }
set_option backward.isDefEq.respectTransparency false in
  simp only [PiTensorProduct.tprodCoeff_eq_smul_tprod, map_smul]
  simp only [homToRepHom, Rep.hom_ofHom, LinearEquiv.coe_coe, Rep.hom_id,
    Representation.IntertwiningMap.toLinearMap_id, LinearMap.id_coe, id_eq]
    Iso.refl_hom, Iso.refl_inv, LinearEquiv.ofLinear_apply]
```

```
set_option backward.isDefEq.respectTransparency false in
  simp only [PiTensorProduct.tprodCoeff_eq_smul_tprod, map_smul]
  simp only [linearIsoOfEq, Functor.mapIso_hom, eqToIso.hom, Functor.mapIso_inv, eqT
    LinearEquiv.ofLinear_apply, Representation.IntertwiningMap.coe_toLinearMap]
    simp_all only [eqToHom_refl, Discrete.functor_map_id, Rep.hom_id,
      Representation.IntertwiningMap.id_apply]
  Rep.mkIso <| Representation.Equiv.mk (PiTensorProduct.isEmptyEquiv Empty).symm <|
    simp only [LinearMap.coe_comp, LinearEquiv.coe_coe, Function.comp_apply]
      PiTensorProduct.isEmptyEquiv_symm_apply, LinearMap.id_coe, id_eq]
    rfl
  Physlib.PiTensorProduct.tmulEquiv □□ₗ PiTensorProduct.congr (discreteSumEquiv F)
set_option backward.isDefEq.respectTransparency false in
    (discreteSumEquiv F i) (Physlib.PiTensorProduct.elimPureTensor p q i) := by
  rw [Physlib.PiTensorProduct.tmulEquiv_tmul_tprod]
    (PiTensorProduct.tprod k (Physlib.PiTensorProduct.elimPureTensor p q)) = _
set_option backward.isDefEq.respectTransparency false in
  Rep.mkIso <| Representation.Equiv.mk (μModEquiv F X Y) <| fun M => by
    refine Physlib.PiTensorProduct.induction_tmul (fun p q => ?_)
      rfl
    discreteSumEquiv F i (Physlib.PiTensorProduct.elimPureTensor p q i) :=
    discreteSumEquiv F i (Physlib.PiTensorProduct.elimPureTensor p q i) :=
set_option backward.isDefEq.respectTransparency false in
  ext1
  refine Physlib.PiTensorProduct.induction_tmul (fun p q => ?_)
  simp only [toRep_V_carrier, tensorObj_of_left, tensorObj_of_hom]
    (Physlib.PiTensorProduct.elimPureTensor p q i))
  simp [LinearMap.rTensor]
  erw [TensorProduct.map_tmul]
  simp [homToRepHom_tprod]
  erw [μ_tmul_tprod]
      Functor.mapIso_refl, Iso.refl_hom, Iso.refl_inv]
set_option backward.isDefEq.respectTransparency false in
  ext1
  refine Physlib.PiTensorProduct.induction_tmul (fun p q => ?_)
  simp only [Rep.tensor_V, Rep.tensor_ρ, Rep.hom_comp, Rep.hom_whiskerLeft,
    Representation.IntertwiningMap.comp_toLinearMap,
    Representation.IntertwiningMap.toLinearMap_lTensor, LinearMap.coe_comp,
    Representation.IntertwiningMap.coe_toLinearMap, Function.comp_apply,]
    (discreteSumEquiv F i) (Physlib.PiTensorProduct.elimPureTensor p q i))
      Iso.refl_inv, Functor.id_obj]
set_option backward.isDefEq.respectTransparency false in
  ext1
  refine Physlib.PiTensorProduct.induction_assoc' (fun p q m => ?_)
  simp only [toRep_V_carrier, CategoryStruct.comp]
    (Physlib.PiTensorProduct.elimPureTensor p q i)) ⊗ₜ[k] (PiTensorProduct.tprod k)
    (discreteSumEquiv F i) (Physlib.PiTensorProduct.elimPureTensor q m i))
```

```
      linearIsoOfEq, eqToIso_refl, Functor.mapIso_refl, Iso.refl_hom,
      linearIsoOfEq, eqToIso_refl, Functor.mapIso_refl, Iso.refl_hom,
      linearIsoOfEq, eqToIso_refl, Functor.mapIso_refl, Iso.refl_hom,
set_option backward.isDefEq.respectTransparency false in
  ext1
  apply Physlib.PiTensorProduct.induction_mod_tmul (fun x q => ?_)
  simp only [toRep_V_carrier, CategoryStruct.comp, tensorObj_of_left, tenso
    tensorObj_of_hom]
  simp only [toRep_V_carrier, lid_tmul, PiTensorProduct.isEmptyEquiv, Linea
  simp only [toRep_V_carrier, linearIsoOfEq, eqToIso_refl, Functor.mapIso_r
    Iso.refl_inv, LinearEquiv.ofLinear_apply]
set_option backward.isDefEq.respectTransparency false in
  ext1
  apply Physlib.PiTensorProduct.induction_tmul_mod (fun p x => ?_)
  simp only [CategoryStruct.comp]
  simp only [rid_tmul, PiTensorProduct.isEmptyEquiv, LinearEquiv.coe_symm_m
  simp only [toRep_V_carrier, linearIsoOfEq, eqToIso_refl, Functor.mapIso_r
    Iso.refl_inv, LinearEquiv.ofLinear_apply]
set_option backward.isDefEq.respectTransparency false in
  ext1
  apply Physlib.PiTensorProduct.induction_tmul (fun p q => ?_)
  simp only [toRep_V_carrier, CategoryStruct.comp]
      Functor.mapIso_refl, Iso.refl_hom, Iso.refl_inv, LinearEquiv.ofLinear
      Functor.mapIso_refl, Iso.refl_hom, Iso.refl_inv, LinearEquiv.ofLinear
    inferInstanceAs (IsIso (toRepUnitIso F).hom)
    fun X Y => inferInstanceAs (IsIso (μ F X Y).hom)
set_option backward.isDefEq.respectTransparency false in
def repNatTransOfColorApp (X : OverColor C) : (toRepFunc F).obj X ⟶ (toRepF
  Rep.ofHom <|
  {
    toLinearMap := PiTensorProduct.map (fun x => (η.app (Discrete.mk (X.hom
    isIntertwining' M := by
      apply LinearMap.ext
      intro x
      refine PiTensorProduct.induction_on' x ?_ (by
        intro x y hx hy
        simp only [map_add]
        erw [hx, hy])
      intro r x
      simp only [PiTensorProduct.tprodCoeff_eq_smul_tprod, _root_.map_smul]
      apply congrArg
      change (PiTensorProduct.map fun x => (η.app { as := X.hom x }).hom.to
        ((((toRepFunc F).obj X).ρ M) ((PiTensorProduct.tprod k) x)) =
        (((toRepFunc F').obj X).ρ M) ((PiTensorProduct.map fun x =>
        (η.app { as := X.hom x }).hom.toLinearMap) ((PiTensorProduct.tprod
      rw [PiTensorProduct.map_tprod]
```

```
      simp only [Functor.id_obj, toRepFunc]
      change (PiTensorProduct.map fun x => (η.app { as := X.hom x }).hom.toLinearMap
        (((toRep F X).ρ M) ((PiTensorProduct.tprod k) x)) =
        ((toRep F' X).ρ M) ((PiTensorProduct.tprod k) fun i =>
        (η.app { as := X.hom i }).hom.toLinearMap (x i))
      rw [toRep_ρ_tprod, toRep_ρ_tprod]
      erw [PiTensorProduct.map_tprod]
      apply congrArg
      funext i
      have h := ((η.app (Discrete.mk (X.hom i))).hom.isIntertwining' M)
      simpa using LinearMap.congr_fun h (x i)
}
set_option backward.isDefEq.respectTransparency false in
  simp only [PiTensorProduct.tprodCoeff_eq_smul_tprod, map_smul]
  simp
set_option backward.isDefEq.respectTransparency false in
  simp only [toRepFunc, toRep_V_carrier, tensorUnit_of_left, tensorUnit_of_hom, Rep.
    Rep.tensorUnit_ρ, Rep.Hom.hom, ConcreteCategory.hom, CategoryStruct.comp,
    Representation.IntertwiningMap.comp_toLinearMap, LinearMap.coe_comp,
    Representation.IntertwiningMap.coe_toLinearMap, Function.comp_apply]
set_option backward.isDefEq.respectTransparency false in
  ext1
  refine Physlib.PiTensorProduct.induction_tmul (fun p q => ?_)
  simp only [toRepFunc, toRep_V_carrier, tensorObj_of_left, tensorObj_of_hom, Rep.te
    Rep.tensor_ρ, Rep.Hom.hom, ConcreteCategory.hom, CategoryStruct.comp,
    Representation.IntertwiningMap.comp_toLinearMap, LinearMap.coe_comp,
    Representation.IntertwiningMap.coe_toLinearMap, Function.comp_apply]
set_option backward.isDefEq.respectTransparency false in
    simp only [Representation.IntertwiningMap.coe_toLinearMap, Rep.hom_id,
      Representation.IntertwiningMap.toLinearMap_id, LinearMap.id_coe, id_eq]
    simp only [PiTensorProduct.tprodCoeff_eq_smul_tprod, map_smul]
    discreteSumEquiv F i (Physlib.PiTensorProduct.elimPureTensor p q i) := by
  change (Representation.equivOfIso (Functor.Monoidal.μIso (lift.obj F).toFunctor X
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    Rep.mkIso <| Representation.Equiv.mk (forgetLiftAppV F c) <| fun g => by
    simp
    simp only [Fin.isValue, PiTensorProduct.tprodCoeff_eq_smul_tprod, map_smul]
    simp only [Fin.isValue, PiTensorProduct.map_tprod]
    rfl
  simp_all only [Nat.succ_eq_add_one, Iso.trans_inv, Functor.mapIso_inv]
```

1.290 Physlib/Relativity/Tensors/ComplexTensor/Basic.lean

```
Authors: Joseph Tooby-Smith, Nikolai Kashcheev
public import Physlib.Relativity.Tensors.ComplexTensor.Metrics.Pre
public import Physlib.Relativity.Tensors.ComplexTensor.Weyl.Metric

/-- The dimensions of each of the different types of complex Lorentz vector
/
abbrev repDim (c : Color) : ℕ :=
  match c with
  | Color.upL => 2
  | Color.downL => 2
  | Color.upR => 2
  | Color.downR => 2
  | Color.up => 4
  | Color.down => 4

def complexLorentzTensor : TensorSpecies ℂ complexLorentzTensor.Color SL(2,
    (fun c => Fin (repDim c)) where
lemma basis_contr (c : complexLorentzTensor.Color) (i : Fin (repDim c))
    (j : Fin (repDim (complexLorentzTensor.τ c))) :
lemma basisIdxCongr_eq_cast {c1 c2 : complexLorentzTensor.Color}
    (h : c1 = c2) (i : Fin (repDim c1)) :
    TensorSpecies.basisIdxCongr (basisIdx := fun c => Fin (repDim c)) h i =
      Fin.cast (by simp [h]) i := by
  subst h
  rfl

lemma repDim_tau {c : complexLorentzTensor.Color} :
    repDim (complexLorentzTensor.τ c) = repDim c := by
  cases c <;> simp [repDim] <;> rfl

/-
@[simp]
lemma basis_eq_complexContrBasisFin4 :
    complexLorentzTensor.basis Color.up = Lorentz.complexContrBasisFin4 :=
  ext i
  exact basis_up_eq

@[simp]
lemma basis_eq_complexCoBasisFin4 :
    complexLorentzTensor.basis Color.down = Lorentz.complexCoBasisFin4 := b
  ext i
  exact basis_down_eq

@[simp]
```

```
lemma FD_obj_up (Λ : SL(2, ℂ)) :
    (complexLorentzTensor.FD.obj { as := Color.up }).ρ Λ = Lorentz.complexContr.ρ Λ
  rfl

@[simp]
lemma FD_obj_down (Λ : SL(2, ℂ)) :
    (complexLorentzTensor.FD.obj { as := Color.down }).ρ Λ = Lorentz.complexCo.ρ Λ :
  rfl

/-!

## Vector slot component formulas (`Color.up` / `Color.down`)

The colors `Color.up` and `Color.down` are the standard Lorentz vector colors. The l
`repr_ρ_basis_vector_up` and `repr_ρ_basis_vector_down` are stated for `Fin 4` indic
(definitionally `Fin (repDim Color.up)` and `Fin (repDim Color.down)`).

When a slot is only known up to `c₀ = Color.up` or `Color.down`, use
`repr_ρ_basis_vector_up_of_eq` / `repr_ρ_basis_vector_down_of_eq`.

-/

section vectorSlotRepr

open TensorSpecies Tensor Lorentz Lorentz.SL2C Module

set_option backward.isDefEq.respectTransparency false in
/--
Component formula for the standard contravariant vector slot `Color.up`.

For `b, i : Fin 4`, the `i`-component of `ρ Λ` on `basis Color.up b` equals the corr
of `LorentzGroup.toComplex (SL2C.toLorentzGroup Λ)` in `Fin 1 ⊕ Fin 3` coordinates.
-/
lemma repr_ρ_basis_vector_up (Λ : SL(2, ℂ)) (b i : Fin 4) :
    ((complexLorentzTensor.basis Color.up).repr
        (((complexLorentzTensor.FD.obj { as := Color.up }).ρ Λ)
          (complexLorentzTensor.basis Color.up b))) i =
      (LorentzGroup.toComplex (Lorentz.SL2C.toLorentzGroup Λ))
        (finSumFinEquiv.symm i) (finSumFinEquiv.symm b) := by
  erw [Lorentz.complexContrBasis_reindex_apply_eq_fin4]
  simp_rw [basis_eq_complexContrBasisFin4, Lorentz.complexContrBasisFin4_eq_reindex]
  rw [Basis.repr_reindex_apply, Basis.reindex_apply]
  simp_rw [FD_obj_up]
  conv_lhs => erw [← LinearMap.toMatrix_apply]
  exact Lorentz.complexContrBasis_ρ_apply (M := Λ) (i := finSumFinEquiv.symm i)
    (j := finSumFinEquiv.symm b)
```

```
/--
Transport version of `repr_ρ_basis_vector_up` for a color `c₀` that is prop
-/
lemma repr_ρ_basis_vector_up_of_eq (c₀ : Color) (h : c₀ = Color.up) (Λ : SL
    (b i : Fin (repDim c₀)) :
    ((complexLorentzTensor.basis c₀).repr
        (((complexLorentzTensor.FD.obj { as := c₀ }).ρ Λ)
          (complexLorentzTensor.basis c₀ b))) i =
      (LorentzGroup.toComplex (Lorentz.SL2C.toLorentzGroup Λ))
        (finSumFinEquiv.symm (Fin.cast (by rw [h];) i))
        (finSumFinEquiv.symm (Fin.cast (by rw [h];) b)) := by
  subst h
  simpa using repr_ρ_basis_vector_up Λ b i

set_option backward.isDefEq.respectTransparency false in
/--
Component formula for the standard covariant vector slot `Color.down`.

For `b, i : Fin 4`, the `i`-component of `ρ Λ` on `basis Color.down b` matc
Lorentz matrix as in `Lorentz.complexCoBasis_ρ_apply` (with transpose index
-/
lemma repr_ρ_basis_vector_down (Λ : SL(2, ℂ)) (b i : Fin 4) :
    ((complexLorentzTensor.basis Color.down).repr
        (((complexLorentzTensor.FD.obj { as := Color.down }).ρ Λ)
          (complexLorentzTensor.basis Color.down b))) i =
      (LorentzGroup.toComplex (Lorentz.SL2C.toLorentzGroup Λ))⁻¹
        (finSumFinEquiv.symm b) (finSumFinEquiv.symm i) := by
  erw [Lorentz.complexCoBasis_reindex_apply_eq_fin4]
  simp_rw [basis_eq_complexCoBasisFin4, Lorentz.complexCoBasisFin4_eq_reind
  rw [Basis.repr_reindex_apply, Basis.reindex_apply]
  simp_rw [FD_obj_down]
  conv_lhs => erw [← LinearMap.toMatrix_apply]
  erw [Lorentz.complexCoBasis_ρ_apply (M := Λ) (i := finSumFinEquiv.symm i)
    (j := finSumFinEquiv.symm b)]
  simp only [Matrix.transpose_apply]

/--
Transport version of `repr_ρ_basis_vector_down` for a color `c₀` that is pr
`Color.down`.
-/
lemma repr_ρ_basis_vector_down_of_eq (c₀ : Color) (h : c₀ = Color.down) (Λ
    (b i : Fin (repDim c₀)) :
    ((complexLorentzTensor.basis c₀).repr
        (((complexLorentzTensor.FD.obj { as := c₀ }).ρ Λ)
          (complexLorentzTensor.basis c₀ b))) i =
```

```
      (LorentzGroup.toComplex (Lorentz.SL2C.toLorentzGroup Λ))⁻¹
        (finSumFinEquiv.symm (Fin.cast (by rw [h];) b))
        (finSumFinEquiv.symm (Fin.cast (by rw [h];) i)) := by
  subst h
  simpa using repr_ρ_basis_vector_down Λ b i

end vectorSlotRepr
```

1.291 Physlib/Relativity/Tensors/ComplexTensor/Lemmas.lean

```
public import Physlib.Relativity.Tensors.ComplexTensor.Basic

set_option backward.isDefEq.respectTransparency false in
set_option maxHeartbeats 450000 in
  · simp only [Nat.reduceAdd, Nat.succ_eq_add_one, Fin.isValue, id_eq, LinearMap.map
```

1.292 Physlib/Relativity/Tensors/ComplexTensor/Matrix/Pre.lean

```
public import Physlib.Relativity.Tensors.ComplexTensor.Vector.Pre.Basic

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.293 Physlib/Relativity/Tensors/ComplexTensor/Metrics/Basic.lean

```
public import Physlib.Relativity.Tensors.ComplexTensor.OfRat
```

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    (Tensor.basis (S := complexLorentzTensor) ![Color.down, Color.down]
      (fun | 0 => (0 : Fin 4) | 1 => (0 : Fin 4)))
    - (Tensor.basis (S := complexLorentzTensor) ![Color.down, Color.down]
      (fun | 0 => (1 : Fin 4) | 1 => (1 : Fin 4)))
    - (Tensor.basis (S := complexLorentzTensor) ![Color.down, Color.down]
      (fun | 0 => (2 : Fin 4) | 1 => (2 : Fin 4)))
      (fun | 0 => (3 : Fin 4) | 1 => (3 : Fin 4))) := by
    (Tensor.basis (S := complexLorentzTensor) ![Color.up, Color.up]
      (fun | 0 => (0 : Fin 4) | 1 => (0 : Fin 4)))
    - (Tensor.basis (S := complexLorentzTensor) ![Color.up, Color.up]
      (fun | 0 => (1 : Fin 4) | 1 => (1 : Fin 4)))
    - (Tensor.basis (S := complexLorentzTensor) ![Color.up, Color.up]
      (fun | 0 => (2 : Fin 4) | 1 => (2 : Fin 4)))
      (fun | 0 => (3 : Fin 4) | 1 => (3 : Fin 4))) := by
    - (Tensor.basis (S := complexLorentzTensor) ![Color.upL, Color.upL]
      (fun | 0 => (0 : Fin 2) | 1 => (1 : Fin 2)))
      ![Color.upL, Color.upL] (fun | 0 => (1 : Fin 2) | 1 => (0 : Fin 2)))
    (Tensor.basis (S := complexLorentzTensor) ![Color.downL, Color.downL]
      (fun | 0 => (0 : Fin 2) | 1 => (1 : Fin 2)))
      ![Color.downL, Color.downL] (fun | 0 => (1 : Fin 2) | 1 => (0 : Fin 2
    - (Tensor.basis (S := complexLorentzTensor) ![Color.upR, Color.upR]
      (fun | 0 => (0 : Fin 2) | 1 => (1 : Fin 2)))
      ![Color.upR, Color.upR] (fun | 0 => (1 : Fin 2) | 1 => (0 : Fin 2)))
      ![Color.downR, Color.downR] (fun | 0 => (0 : Fin 2) | 1 => (1 : Fin 2
      ![Color.downR, Color.downR] (fun | 0 => (1 : Fin 2) | 1 => (0 : Fin 2
    if f 0 = Fin.cast (by rfl) (0 : Fin 4) ∧ f 1 = Fin.cast (by rfl) (0 : F
    if f 0 = Fin.cast (by rfl) (0 : Fin 4) ∧ f 1 = Fin.cast (by rfl) (0 : F
    if f 0 = Fin.cast (by rfl) (0 : Fin 2) ∧ f 1 = Fin.cast (by rfl) (1 : F
    if f 1 = Fin.cast (by rfl) (0 : Fin 2) ∧ f 0 = Fin.cast (by rfl) (1 : F
      1 else 0 := by
    if f 0 = Fin.cast (by rfl) (0 : Fin 2) ∧ f 1 = Fin.cast (by rfl) (1 : F
    if f 1 = Fin.cast (by rfl) (0 : Fin 2) ∧ f 0 = Fin.cast (by rfl) (1 : F
      - 1 else 0 := by
    if f 0 = Fin.cast (by rfl) (0 : Fin 2) ∧ f 1 = Fin.cast (by rfl) (1 : F
    if f 1 = Fin.cast (by rfl) (0 : Fin 2) ∧ f 0 = Fin.cast (by rfl) (1 : F
    if f 0 = Fin.cast (by rfl) (0 : Fin 2) ∧ f 1 = Fin.cast (by rfl) (1 : F
    if f 1 = Fin.cast (by rfl) (0 : Fin 2) ∧ f 0 = Fin.cast (by rfl) (1 : F
      - 1 else 0 := by
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.294 Physlib/Relativity/Tensors/ComplexTensor/Metrics/Lemmas.lean

```
public import Physlib.Relativity.Tensors.ComplexTensor.Metrics.Basic
public import Physlib.Relativity.Tensors.ComplexTensor.Units.Basic
```

1.295 Physlib/Relativity/Tensors/ComplexTensor/Metrics/Pre.lean

```
public import Physlib.Relativity.Tensors.ComplexTensor.Units.Pre

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
def contrMetric : □[_] (Rep ℂ SL(2,ℂ)) □ complexContr ⊗ complexContr := Rep.ofHom {
      rfl
    isIntertwining' M := by
      refine LinearMap.ext fun x : ℂ => ?_
      change x • contrMetricVal =
        (TensorProduct.map (complexContr.ρ M) (complexContr.ρ M)) (x • contrMetricVa
      simp only [map_smul]
      apply congrArg
      simp only [contrMetricVal]
      rw [contrContrToMatrix_ρ_symm]
      apply congrArg
      simp only [LorentzGroup.toComplex_mul_minkowskiMatrix_mul_transpose]
}
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
def coMetric : □[_] (Rep ℂ SL(2,ℂ)) □ complexCo ⊗ complexCo := Rep.ofHom {
      rfl
    isIntertwining' M := by
      refine LinearMap.ext fun x : ℂ => ?_
      change x • coMetricVal =
        (TensorProduct.map (complexCo.ρ M) (complexCo.ρ M)) (x • coMetricVal)
      simp only [map_smul]
      apply congrArg
      simp only [coMetricVal]
      rw [coCoToMatrix_ρ_symm]
      apply congrArg
```

```
      rw [LorentzGroup.toComplex_inv]
      simp only [LorentzGroup.inv_eq_dual, SL2C.toLorentzGroup_apply_coe,
        LorentzGroup.toComplex_transpose_mul_minkowskiMatrix_mul_self]
}
set_option backward.isDefEq.respectTransparency false in
    ((complexContr ◁ (λ_ complexCo).hom)
    ((complexContr ◁ contrCoContraction ▷ complexCo)
    ((complexContr ◁ (α_ complexContr complexCo complexCo).inv)
    ((α_ complexContr complexContr (complexCo ⊗ complexCo)).hom
  simp [← Representation.IntertwiningMap.toLinearMap_apply]
  repeat erw [contrCoContraction_basis']
  simp only [Fin.isValue, ↓reduceIte, one_smul, reduceCtorEq, zero_smul, su
    Sum.inr.injEq, one_ne_zero, Fin.reduceEq, sub_neg_eq_add, zero_ne_one,
  erw [coContrUnit_apply_one, coContrUnitVal_expand_tmul]
set_option backward.isDefEq.respectTransparency false in
    ((complexCo ◁ (λ_ complexContr).hom)
    ((complexCo ◁ coContrContraction ▷ complexContr)
    ((complexCo ◁ (α_ complexCo complexContr complexContr).inv)
    ((α_ complexCo complexCo (complexContr ⊗ complexContr)).hom
  simp [← Representation.IntertwiningMap.toLinearMap_apply]
  repeat erw [coContrContraction_basis']
  simp only [Fin.isValue, ↓reduceIte, one_smul, reduceCtorEq, Sum.inr.injEq
    Fin.reduceEq, zero_ne_one]
  erw [contrCoUnit_apply_one, contrCoUnitVal_expand_tmul]
  module
```

1.296 Physlib/Relativity/Tensors/ComplexTensor/OfRat.lean

```
public import Physlib.Relativity.Tensors.ComplexTensor.Basic
public import Physlib.Mathematics.RatComplexNum
public import Physlib.Relativity.Tensors.Dual
open Physlib.RatComplexNum
open Physlib
    ((ofRat (fun b => f (ComponentIdx.prod b).1 *
      f1 (ComponentIdx.prod b).2))) := by
set_option backward.isDefEq.respectTransparency false in
    rw [← Physlib.RatComplexNum.toComplexNum.map_mul]
  rw [← map_sum Physlib.RatComplexNum.toComplexNum]
  rw [ofRat_basis_repr_apply]
  simp [basisIdxCongr_eq_cast]
set_option backward.isDefEq.respectTransparency false in
      f (DropPairSection.ofFinEquiv h.1 b (x, Fin.cast (by
        simp [← h.2, complexLorentzTensor.repDim_tau]) x)))))) := by
  rw [Finset.sum_eq_single (Fin.cast (by simp [← h.2, repDim_tau]) x)]
```

```
  congr
  ext i
  simp [basisIdxCongr_eq_cast]
```

1.297 Physlib/Relativity/Tensors/ComplexTensor/Units/Basic.lean

```
public import Physlib.Relativity.Tensors.ComplexTensor.OfRat

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.298 Physlib/Relativity/Tensors/ComplexTensor/Units/Pre.lean

```
public import Physlib.Relativity.Tensors.ComplexTensor.Matrix.Pre
public import Physlib.Relativity.Tensors.ComplexTensor.Vector.Pre.Contraction

lemma contrCoUnitVal_eq_sum_tmul : contrCoUnitVal =
    ∑ i, complexContrBasis i ⊗ₜ[ℂ] complexCoBasis i := by
  simp [contrCoUnitVal_expand_tmul, Fin.isValue, Fin.sum_univ_three]
  module

set_option backward.isDefEq.respectTransparency false in
def contrCoUnit : □_ (Rep ℂ SL(2,ℂ)) □ complexContr ⊗ complexCo := Rep.ofHom
{
      rfl
    isIntertwining' M := by
      refine LinearMap.ext fun x : ℂ => ?_
      change x • contrCoUnitVal =
        (TensorProduct.map (complexContr.ρ M) (complexCo.ρ M)) (x • contrCoUnitVal)
      simp only [map_smul]
      apply congrArg
      simp only [contrCoUnitVal]
      rw [contrCoToMatrix_ρ_symm]
      apply congrArg
      simp
}
lemma coContrUnitVal_eq_sum_tmul : coContrUnitVal =
    ∑ i, complexCoBasis i ⊗ₜ[ℂ] complexContrBasis i := by
  simp [coContrUnitVal_expand_tmul, Fin.isValue, Fin.sum_univ_three]
```

```
  module

set_option backward.isDefEq.respectTransparency false in
def coContrUnit : □_ (Rep ℂ SL(2,ℂ)) □ complexCo ⊗ complexContr := Rep.ofHo
      rfl
    isIntertwining' M := by
      refine LinearMap.ext fun x : ℂ => ?_
      change x • coContrUnitVal =
        (TensorProduct.map (complexCo.ρ M) (complexContr.ρ M)) (x • coContr
      simp only [map_smul]
      apply congrArg
      simp only [coContrUnitVal]
      rw [coContrToMatrix_ρ_symm]
      apply congrArg
      symm
      refine transpose_eq_one.mp ?h.h.h.a
      simp
}
set_option backward.isDefEq.respectTransparency false in
  rw [contrCoUnit_apply_one, contrCoUnitVal_eq_sum_tmul]
  simp [- Fintype.sum_sum_type, tmul_sum, sum_tmul, map_sum, Representation
    ← Representation.IntertwiningMap.toLinearMap_apply, smul_tmul]
  conv_lhs =>
    enter [2,x, 2, y]
    erw [coContrContraction_basis']
  simp

set_option backward.isDefEq.respectTransparency false in
  rw [coContrUnit_apply_one, coContrUnitVal_eq_sum_tmul]
  simp [- Fintype.sum_sum_type, tmul_sum, sum_tmul, map_sum, Representation
    ← Representation.IntertwiningMap.toLinearMap_apply, smul_tmul]
  conv_lhs =>
    enter [2,x, 2, y]
    erw [contrCoContraction_basis']
  simp
```

1.299 Physlib/Relativity/Tensors/ComplexTensor/Units/Symm.lean

```
public import Physlib.Relativity.Tensors.ComplexTensor.Units.Basic
```

1.300 Physlib/Relativity/Tensors/ComplexTensor/Vector/Pre/Basic.lean

```
public import Physlib.Relativity.Tensors.ComplexTensor.Vector.Pre.Modules
lemma complexContrBasisFin4_eq_reindex :
    complexContrBasisFin4 = complexContrBasis.reindex finSumFinEquiv :=
  rfl

lemma complexContrBasis_reindex_apply_eq_fin4 (j : Fin 4) :
    (complexContrBasis.reindex finSumFinEquiv) j = complexContrBasisFin4 j :=
  rfl

lemma complexCoBasisFin4_eq_reindex :
    complexCoBasisFin4 = complexCoBasis.reindex finSumFinEquiv :=
  rfl

lemma complexCoBasis_reindex_apply_eq_fin4 (j : Fin 4) :
    (complexCoBasis.reindex finSumFinEquiv) j = complexCoBasisFin4 j :=
  rfl

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.301 Physlib/Relativity/Tensors/ComplexTensor/Vector/Pre/Contraction.

```
public import Physlib.Relativity.Tensors.ComplexTensor.Vector.Pre.Basic

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
def contrCoContraction : complexContr ⊗ complexCo □ □_ (Rep ℂ SL(2,ℂ)) := Rep.ofHom
  {
    toLinearMap := TensorProduct.lift contrCoContrBi,
    isIntertwining' M := TensorProduct.ext' fun ψ φ => by
      change ((LorentzGroup.toComplex (SL2C.toLorentzGroup M)) *ᵥ ψ.toFin13ℂ) □ᵥ
        ((LorentzGroup.toComplex (SL2C.toLorentzGroup M))⁻¹ᵀ *ᵥ φ.toFin13ℂ) =
          ψ.toFin13ℂ □ᵥ φ.toFin13ℂ
      rw [dotProduct_mulVec, vecMul_transpose, mulVec_mulVec]
      rw [inv_mul_of_invertible (LorentzGroup.toComplex (SL2C.toLorentzGroup M))]
      simp
  }
def coContrContraction : complexCo ⊗ complexContr □ □_ (Rep ℂ SL(2,ℂ)) := Rep.ofHom
  {
  toLinearMap := TensorProduct.lift contrContrCoBi,
  isIntertwining' M := TensorProduct.ext' fun φ ψ => by
  }
```

1.302 Physlib/Relativity/Tensors/ComplexTensor/Vector/Pre/Modul

```
public import Physlib.Relativity.SL2C.Basic
```

1.303 Physlib/Relativity/Tensors/ComplexTensor/Weyl/Basic.lean

```
public import Physlib.Relativity.Tensors.ComplexTensor.Weyl.Modules
public import Physlib.Relativity.SL2C.Basic
def leftHandedToAlt : leftHanded  altLeftHanded := Rep.ofHom {
  toFun := fun ψ => AltLeftHandedModule.toFin2ℂEquiv.symm (!![0, 1; -
1, 0] *ᵥ ψ.toFin2ℂ),
  map_add' := by
    intro ψ ψ'
    simp only [mulVec_add, LinearEquiv.map_add]
  map_smul' := by
    intro a ψ
    simp only [mulVec_smul, LinearEquiv.map_smul]
    rfl
  isIntertwining' := by
}
def leftHandedAltTo : altLeftHanded  leftHanded := Rep.ofHom {
  toFun := fun ψ =>
  map_add' := by
    intro ψ ψ'
    simp only [map_add]
    rw [mulVec_add, LinearEquiv.map_add]
  map_smul' := by
    intro a ψ
    simp only [LinearEquiv.map_smul]
    rw [mulVec_smul, LinearEquiv.map_smul]
    rfl
  isIntertwining' := by
}
    simp only [Rep.hom_comp, Representation.IntertwiningMap.comp_toLinearMa
      Representation.IntertwiningMap.coe_toLinearMap, Function.comp_apply,
      Representation.IntertwiningMap.toLinearMap_id, LinearMap.id_coe, id_e
    simp only [Rep.hom_comp, Representation.IntertwiningMap.comp_toLinearMa
      Representation.IntertwiningMap.coe_toLinearMap, Function.comp_apply,
      Representation.IntertwiningMap.toLinearMap_id, LinearMap.id_coe, id_e
```

1.304 Physlib/Relativity/Tensors/ComplexTensor/Weyl/Contraction.lean

```
public import Physlib.Relativity.Tensors.ComplexTensor.Weyl.Basic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
def leftAltContraction : leftHanded ⊗ altLeftHanded □ □_ (Rep ℂ SL(2,ℂ)) := Rep.ofHo
  {
    toLinearMap := TensorProduct.lift leftAltBi
    isIntertwining' M := TensorProduct.ext' fun ψ φ => by
      change (M.1 *ᵥ ψ.toFin2ℂ) □ᵥ (M.1⁻¹ᵀ *ᵥ φ.toFin2ℂ) = ψ.toFin2ℂ □ᵥ φ.toFin2ℂ
      rw [dotProduct_mulVec, vecMul_transpose, mulVec_mulVec]
      simp
  }
def altLeftContraction : altLeftHanded ⊗ leftHanded □ □_ (Rep ℂ SL(2,ℂ)) := Rep.ofHo
  toLinearMap := TensorProduct.lift altLeftBi
  isIntertwining' M := TensorProduct.ext' fun φ ψ => by
}
def rightAltContraction : rightHanded ⊗ altRightHanded □ □_ (Rep ℂ SL(2,ℂ)) := Rep.o
  toLinearMap := TensorProduct.lift rightAltBi
  isIntertwining' M := TensorProduct.ext' fun ψ φ => by
}
def altRightContraction : altRightHanded ⊗ rightHanded □ □_ (Rep ℂ SL(2,ℂ)) := Rep.o
  toLinearMap := TensorProduct.lift altRightBi
  isIntertwining' M := TensorProduct.ext' fun φ ψ => by
}
```

1.305 Physlib/Relativity/Tensors/ComplexTensor/Weyl/Metric.lean

```
public import Physlib.Relativity.Tensors.ComplexTensor.Weyl.Unit
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
def leftMetric : □_ (Rep ℂ SL(2,ℂ)) □ leftHanded ⊗ leftHanded := Rep.ofHom {
      rfl
    isIntertwining' M := by
      refine LinearMap.ext fun x : ℂ => ?_
      change x • leftMetricVal =
        (TensorProduct.map (leftHanded.ρ M) (leftHanded.ρ M)) (x • leftMetricVal)
      simp only [map_smul]
      apply congrArg
      simp only [leftMetricVal, map_neg, neg_inj]
      rw [leftLeftToMatrix_ρ_symm]
```

```
      apply congrArg
      rw [comm_metricRaw, mul_assoc, ← @transpose_mul]
      simp only [SpecialLinearGroup.det_coe, isUnit_iff_ne_zero, ne_eq, one
        not_false_eq_true, mul_nonsing_inv, transpose_one, mul_one]
}
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
def altLeftMetric : □_ (Rep ℂ SL(2,ℂ)) □ altLeftHanded ⊗ altLeftHanded := R
      rfl
    isIntertwining' M := by
      refine LinearMap.ext fun x : ℂ => ?_
      change x • altLeftMetricVal =
        (TensorProduct.map (altLeftHanded.ρ M) (altLeftHanded.ρ M)) (x • al
      simp only [map_smul]
      apply congrArg
      simp only [altLeftMetricVal]
      rw [altLeftaltLeftToMatrix_ρ_symm]
      apply congrArg
      rw [← metricRaw_comm, mul_assoc]
      simp only [SpecialLinearGroup.det_coe, isUnit_iff_ne_zero, ne_eq, one
        not_false_eq_true, mul_nonsing_inv, mul_one]
}
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
def rightMetric : □_ (Rep ℂ SL(2,ℂ)) □ rightHanded ⊗ rightHanded := Rep.ofH
      rfl
    isIntertwining' M := by
      refine LinearMap.ext fun x : ℂ => ?_
      change x • rightMetricVal =
        (TensorProduct.map (rightHanded.ρ M) (rightHanded.ρ M)) (x • rightM
      simp only [map_smul]
      apply congrArg
      simp only [rightMetricVal, map_neg, neg_inj]
      trans rightRightToMatrix.symm ((M.1).map star * metricRaw * ((M.1).ma
      · apply congrArg
        rw [star_comm_metricRaw, mul_assoc]
        have h1 : ((M.1)⁻¹ᴴ * ((M.1).map star)ᵀ) = 1 := by
          trans (M.1)⁻¹ᴴ * ((M.1))ᴴ
          · rfl
          rw [← @conjTranspose_mul]
          simp only [SpecialLinearGroup.det_coe, isUnit_iff_ne_zero, ne_eq,
            not_false_eq_true, mul_nonsing_inv, conjTranspose_one]
        rw [h1]
        simp
      · rw [← rightRightToMatrix_ρ_symm metricRaw M]
}
```

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
def altRightMetric : □_ (Rep ℂ SL(2,ℂ)) □ altRightHanded ⊗ altRightHanded := Rep.ofH
    isIntertwining' M := by
      refine LinearMap.ext fun x : ℂ => ?_
      change x • altRightMetricVal =
        (TensorProduct.map (altRightHanded.ρ M) (altRightHanded.ρ M)) (x • altRightM
      simp only [map_smul]
      apply congrArg
      trans altRightAltRightToMatrix.symm
        (((M.1)⁻¹).conjTranspose * metricRaw * (((M.1)⁻¹).conjTranspose)ᵀ)
      · rw [altRightMetricVal]
        apply congrArg
        rw [← metricRaw_comm_star, mul_assoc]
        have h1 : ((M.1).map star * (M.1)⁻¹ᴴᵀ) = 1 := by
          refine transpose_eq_one.mp ?_
          rw [@transpose_mul]
          simp only [transpose_transpose, RCLike.star_def]
          change (M.1)⁻¹ᴴ * (M.1)ᴴ = 1
          rw [← @conjTranspose_mul]
          simp
        rw [h1, mul_one]
      · rw [← altRightAltRightToMatrix_ρ_symm metricRaw M]
        rfl
}
set_option backward.isDefEq.respectTransparency false in
    ((leftHanded ◁ (λ_ altLeftHanded).hom)
    ((leftHanded ◁ leftAltContraction ▷ altLeftHanded)
    ((leftHanded ◁ (α_ leftHanded altLeftHanded altLeftHanded).inv)
    ((α_ leftHanded leftHanded (altLeftHanded ⊗ altLeftHanded)).hom
  simp [← Representation.IntertwiningMap.toLinearMap_apply]
  repeat erw [leftAltContraction_basis]
  simp only [Fin.isValue, Fin.val_one, Fin.val_zero, one_ne_zero, ↓reduceIte, one_sm
  erw [altLeftLeftUnit_apply_one, altLeftLeftUnitVal_expand_tmul]
  module
set_option backward.isDefEq.respectTransparency false in
    ((altLeftHanded ◁ (λ_ leftHanded).hom)
    ((altLeftHanded ◁ altLeftContraction ▷ leftHanded)
    ((altLeftHanded ◁ (α_ altLeftHanded leftHanded leftHanded).inv)
    ((α_ altLeftHanded altLeftHanded (leftHanded ⊗ leftHanded)).hom
  simp [← Representation.IntertwiningMap.toLinearMap_apply]
  repeat erw [altLeftContraction_basis]
  simp only [Fin.isValue, Fin.val_zero, ↓reduceIte, one_smul, Fin.val_one, one_ne_ze
  erw [leftAltLeftUnit_apply_one, leftAltLeftUnitVal_expand_tmul]
  module
set_option backward.isDefEq.respectTransparency false in
```

```
    ((rightHanded ◁ (λ_ altRightHanded).hom)
    ((rightHanded ◁ rightAltContraction ▷ altRightHanded)
    ((rightHanded ◁ (α_ rightHanded altRightHanded altRightHanded).inv)
    ((α_ rightHanded rightHanded (altRightHanded ⊗ altRightHanded)).hom
  simp [← Representation.IntertwiningMap.toLinearMap_apply]
  repeat erw [rightAltContraction_basis]
  simp only [Fin.isValue, Fin.coe_ofNat_eq_mod, Nat.mod_succ, Nat.zero_mod,
    one_smul, zero_ne_one]
  erw [altRightRightUnit_apply_one, altRightRightUnitVal_expand_tmul]
  module
set_option backward.isDefEq.respectTransparency false in
    ((altRightHanded ◁ (λ_ rightHanded).hom)
    ((altRightHanded ◁ altRightContraction ▷ rightHanded)
    ((altRightHanded ◁ (α_ altRightHanded rightHanded rightHanded).inv)
    ((α_ altRightHanded altRightHanded (rightHanded ⊗ rightHanded)).hom
  simp [← Representation.IntertwiningMap.toLinearMap_apply]
  repeat erw [altRightContraction_basis]
  simp only [Fin.isValue, Fin.coe_ofNat_eq_mod, Nat.zero_mod, ↓reduceIte, o
    one_ne_zero, zero_smul, sub_zero, zero_ne_one]
  erw [rightAltRightUnit_apply_one, rightAltRightUnitVal_expand_tmul]
```

1.306 Physlib/Relativity/Tensors/ComplexTensor/Weyl/Two.lean

```
public import Physlib.Relativity.Tensors.ComplexTensor.Weyl.Basic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.307 Physlib/Relativity/Tensors/ComplexTensor/Weyl/Unit.lean

```
public import Physlib.Relativity.Tensors.ComplexTensor.Weyl.Two
public import Physlib.Relativity.Tensors.ComplexTensor.Weyl.Contraction
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
def leftAltLeftUnit : □_ (Rep ℂ SL(2,ℂ)) □ leftHanded ⊗ altLeftHanded := Rep.ofHom
{
      rfl
    isIntertwining' M := by
      refine LinearMap.ext fun x : ℂ => ?_
      change x • leftAltLeftUnitVal =
        (TensorProduct.map (leftHanded.ρ M) (altLeftHanded.ρ M)) (x • leftAltLeftUni
      simp only [map_smul]
      apply congrArg
      simp only [leftAltLeftUnitVal]
      rw [leftAltLeftToMatrix_ρ_symm]
      apply congrArg
      simp
}
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
def altLeftLeftUnit : □_ (Rep ℂ SL(2,ℂ)) □ altLeftHanded ⊗ leftHanded := Rep.ofHom {
```

```
  toFun := fun a =>
  map_add' := fun x y => by
    simp only [add_smul]
  map_smul' := fun m x => by
    simp only [smul_smul]
    rfl
  isIntertwining' M := by
}
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
def rightAltRightUnit : □_ (Rep ℂ SL(2,ℂ)) □ rightHanded ⊗ altRightHanded :
{
      rfl
    isIntertwining' M := by
      refine LinearMap.ext fun x : ℂ => ?_
      change x • rightAltRightUnitVal =
        (TensorProduct.map (rightHanded.ρ M) (altRightHanded.ρ M)) (x • rig
      simp only [map_smul]
      apply congrArg
      simp only [rightAltRightUnitVal]
      rw [rightAltRightToMatrix_ρ_symm]
      apply congrArg
      simp only [RCLike.star_def, mul_one]
      symm
      refine transpose_eq_one.mp ?h.h.h.a
      simp only [transpose_mul, transpose_transpose]
      change (M.1)⁻¹ᴴ * (M.1)ᴴ = 1
      rw [@conjTranspose_nonsing_inv]
      simp
  }
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
def altRightRightUnit : □_ (Rep ℂ SL(2,ℂ)) □ altRightHanded ⊗ rightHanded :
      rfl
    isIntertwining' M := by
      refine LinearMap.ext fun x : ℂ => ?_
      change x • altRightRightUnitVal =
        (TensorProduct.map (altRightHanded.ρ M) (rightHanded.ρ M)) (x • alt
      simp only [map_smul]
      apply congrArg
      simp only [altRightRightUnitVal]
      rw [altRightRightToMatrix_ρ_symm]
      apply congrArg
      simp only [mul_one, RCLike.star_def]
      symm
      change (M.1)⁻¹ᴴ * (M.1)ᴴ = 1
```

```
      rw [@conjTranspose_nonsing_inv]
      simp
  }
set_option backward.isDefEq.respectTransparency false in
  simp only [Rep.tensor_V, Rep.tensorUnit_V, Rep.tensor_ρ, Rep.tensorUnit_ρ, Rep.hom
    Rep.hom_whiskerRight, Rep.hom_inv_associator, Fin.sum_univ_two, Fin.isValue, tmu
    smul_tmul, tmul_smul, ← Representation.IntertwiningMap.toLinearMap_apply,
    Representation.TensorProduct.assoc_symm_toLinearMap, LinearMap.map_add, map_smul
    LinearEquiv.coe_coe, assoc_symm_tmul, Representation.IntertwiningMap.toLinearMap
    LinearMap.rTensor_tmul, Representation.IntertwiningMap.coe_toLinearMap,
    Representation.TensorProduct.toLinearMap_lid, lid_tmul]
  repeat erw [leftAltContraction_basis]
  simp

set_option backward.isDefEq.respectTransparency false in
  simp only [Rep.tensor_V, Rep.tensorUnit_V, Rep.tensor_ρ, Rep.tensorUnit_ρ, Rep.hom
    Rep.hom_whiskerRight, Rep.hom_inv_associator, Fin.sum_univ_two, Fin.isValue, tmu
    smul_tmul, tmul_smul, ← Representation.IntertwiningMap.toLinearMap_apply,
    Representation.TensorProduct.assoc_symm_toLinearMap, LinearMap.map_add, map_smul
    LinearEquiv.coe_coe, assoc_symm_tmul, Representation.IntertwiningMap.toLinearMap
    LinearMap.rTensor_tmul, Representation.IntertwiningMap.coe_toLinearMap,
    Representation.TensorProduct.toLinearMap_lid, lid_tmul]
  repeat erw [altLeftContraction_basis]
  simp

set_option backward.isDefEq.respectTransparency false in
  simp only [Rep.tensor_V, Rep.tensorUnit_V, Rep.tensor_ρ, Rep.tensorUnit_ρ, Rep.hom
    Rep.hom_whiskerRight, Rep.hom_inv_associator, Fin.sum_univ_two, Fin.isValue, tmu
    smul_tmul, tmul_smul, ← Representation.IntertwiningMap.toLinearMap_apply,
    Representation.TensorProduct.assoc_symm_toLinearMap, LinearMap.map_add, map_smul
    LinearEquiv.coe_coe, assoc_symm_tmul, Representation.IntertwiningMap.toLinearMap
    LinearMap.rTensor_tmul, Representation.IntertwiningMap.coe_toLinearMap,
    Representation.TensorProduct.toLinearMap_lid, lid_tmul]
  repeat erw [rightAltContraction_basis]
  simp

set_option backward.isDefEq.respectTransparency false in
  simp only [Rep.tensor_V, Rep.tensorUnit_V, Rep.tensor_ρ, Rep.tensorUnit_ρ, Rep.hom
    Rep.hom_whiskerRight, Rep.hom_inv_associator, Fin.sum_univ_two, Fin.isValue, tmu
    smul_tmul, tmul_smul, ← Representation.IntertwiningMap.toLinearMap_apply,
    Representation.TensorProduct.assoc_symm_toLinearMap, LinearMap.map_add, map_smul
    LinearEquiv.coe_coe, assoc_symm_tmul, Representation.IntertwiningMap.toLinearMap
    LinearMap.rTensor_tmul, Representation.IntertwiningMap.coe_toLinearMap,
    Representation.TensorProduct.toLinearMap_lid, lid_tmul]
  repeat erw [altRightContraction_basis]
  simp
```

1.308 Physlib/Relativity/Tensors/Constructors.lean

```
public import Physlib.Relativity.Tensors.Contraction.Products
variable {k : Type} [CommRing k] {C G : Type} [Group G]
  {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basi
  {S : TensorSpecies k C G basisIdx}
    change ((forgetLiftAppCon S.FD c).inv).hom.toLinearMap (x + y) = _
    change ((forgetLiftAppCon S.FD c).inv).hom.toLinearMap (r • x) = _
    (((lift.obj S.FD).mapIso (mkIso (by aesop))).hom.hom.toLinearMap p.toTe
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    (S.FD.map (eqToHom (congrArg Discrete.mk h))).hom.toLinearMap x) =
    fromPairT (TensorProduct.map LinearMap.id (S.FD.map (eqToHom (by rw [h]
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
      Functor.comp_obj, Discrete.functor_obj_eq_as, Function.comp_apply, ma
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    (b0 : basisIdx c) (b1 : basisIdx c1) :
set_option backward.isDefEq.respectTransparency false in
  exact congrArg _ (LinearMap.congr_fun (v.hom.isIntertwining' g) 1).symm
  simp only [Nat.succ_eq_add_one, Nat.reduceAdd]
  change fromPairT (TensorProduct.map LinearMap.id
    (S.FD.map (eqToHom (by rw [h])))).hom.toLinearMap _) = _
  simp only [Nat.succ_eq_add_one, Nat.reduceAdd, Fin.isValue]
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    (b0 : basisIdx c) (b1 : basisIdx c1)
    (b2 : basisIdx c2) :
set_option backward.isDefEq.respectTransparency false in
  exact congrArg _ (LinearMap.congr_fun (v.hom.isIntertwining' g) 1).symm
lemma fromConst_eq {n : ℕ} {c : Fin n → C}
    (T : □_ (Rep k G) □ S.F.obj (OverColor.mk c)) :
    fromConst T = T.hom (1 : k) := rfl
```

```
set_option backward.isDefEq.respectTransparency false in
  simp only [actionT_eq, fromConst_eq]
  exact (LinearMap.congr_fun (T.hom.isIntertwining' g) 1).symm
```

1.309 Physlib/Relativity/Tensors/Contraction/Basic.lean

```
public import Physlib.Relativity.Tensors.Contraction.Pure
variable {k : Type} [CommRing k] {C G : Type} [Group G]
  {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basisIdx c)]
  {S : TensorSpecies k C G basisIdx}
```

1.310 Physlib/Relativity/Tensors/Contraction/Basis.lean

```
public import Physlib.Relativity.Tensors.Contraction.Basic
variable {k : Type} [CommRing k] {C G : Type} [Group G]
  {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basisIdx c)]
  {S : TensorSpecies k C G basisIdx}
    (x : basisIdx (c i) × basisIdx (c j)) :
  if hi : m = i then basisIdxCongr (by subst hi; rfl) x.1
  else if hj : m = j then basisIdxCongr (by subst hj; rfl) x.2
    basisIdxCongr (by simp)
    (x : basisIdx (c i) × basisIdx (c j)) :
    ofFin (S := S) hij b x i = x.1 := by
    (x : basisIdx (c i) × basisIdx (c j)) :
    ofFin (S := S) hij b x j = x.2 := by
    (x : basisIdx (c i) × basisIdx (c j)) :
    ofFin (S := S) hij b x ∈ DropPairSection b := by
  symm
  apply ComponentIdx.congr_right
  simp
  `basisIdx (c i) × basisIdx (c j)`. -/
    basisIdx (c i) × basisIdx (c j) ≃ DropPairSection (S := S) b where
      symm
      apply ComponentIdx.congr_right
      simp
    (x : basisIdx (c i) × basisIdx (c j)) :
    (ofFinEquiv (S := S) hij b x).1 i = x.1 := by
    (x : basisIdx (c i) × basisIdx (c j)) :
    (ofFinEquiv (S := S) hij b x).1 j = x.2 := by
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    (S.basis (c i) (b'.1 i) ⊗ₜ[k] S.basis (S.τ (c i)) (basisIdxCongr (by rw [h.2]) (
        simp only [Basis.repr_self, Monoidal.tensorUnit_obj, Functor.comp_obj,
```

```
set_option backward.isDefEq.respectTransparency false in
    ∑ (x1 : basisIdx (c i)), ∑ (x2 : basisIdx (c j)),
    (S.basis (c i) x1 ⊗ₜ[k] S.basis (S.τ (c i)) (basisIdxCongr (by rw [h.2]
```

1.311 Physlib/Relativity/Tensors/Contraction/Products.lean

```
public import Physlib.Relativity.Tensors.Contraction.Basic
public import Physlib.Relativity.Tensors.Product
variable {k : Type} [CommRing k] {C G : Type} [Group G]
  {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basi
  {S : TensorSpecies k C G basisIdx}
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  simp only [contrPCoeff, Monoidal.tensorUnit_obj, Functor.comp_obj, Discre
    Function.comp_apply, prodP_apply_natAdd]
set_option backward.isDefEq.respectTransparency false in
  simp only [contrPCoeff, Monoidal.tensorUnit_obj, Functor.comp_obj, Discre
    Function.comp_apply, prodP_apply_castAdd]
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.312 Physlib/Relativity/Tensors/Contraction/Pure.lean

```
public import Physlib.Relativity.Tensors.Basic
variable {k : Type} [CommRing k] {C G : Type} [Group G]
  {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basi
  {S : TensorSpecies k C G basisIdx}
set_option backward.isDefEq.respectTransparency false in
TODO "Prove lemmas relating to the commutation rules of `dropPair` and `pro
  simp only [contrPCoeff, Monoidal.tensorUnit_obj,
  simp only [Monoidal.tensorUnit_obj,
set_option backward.isDefEq.respectTransparency false in
  change ((S.contr.app { as := c i })).hom.toLinearMap _ =
    ((S.contr.app { as := c i })).hom.toLinearMap _
    + ((S.contr.app { as := c i })).hom.toLinearMap _
  simp [Function.update_of_ne (Ne.symm hij.1), update, TensorProduct.add_tm
set_option backward.isDefEq.respectTransparency false in
  simp only [contrPCoeff, Monoidal.tensorUnit_obj,
  change ((S.contr.app { as := c i })).hom.toLinearMap _ =
    ((S.contr.app { as := c i })).hom.toLinearMap _
```

```
    + ((S.contr.app { as := c i })).hom.toLinearMap _
    change ((S.FD.map (eqToHom _))).hom.toLinearMap (x + y)
  simp only [Monoidal.tensorUnit_obj, TensorProduct.tmul_add, LinearMap.map_add]
set_option backward.isDefEq.respectTransparency false in
  simp only [contrPCoeff, Monoidal.tensorUnit_obj,
  change ((S.contr.app { as := c i })).hom.toLinearMap _ = r * _
  simp only [Monoidal.tensorUnit_obj, LinearMap.map_smul, smul_eq_mul]
  change ((S.contr.app { as := c i })).hom.toLinearMap _ =
    ((S.contr.app { as := c i })).hom.toLinearMap _
set_option backward.isDefEq.respectTransparency false in
  simp only [contrPCoeff, Monoidal.tensorUnit_obj,
  change ((S.contr.app { as := c i })).hom.toLinearMap _ = r * _
    change ((S.FD.map (eqToHom _))).hom.toLinearMap (r • _)
  simp only [Monoidal.tensorUnit_obj, LinearMap.map_smul, TensorProduct.tmul_smul, s
  simp only [Monoidal.tensorUnit_obj,
  change _ = (S.contr.app { as := c j }).hom _
set_option backward.isDefEq.respectTransparency false in
        simp only [Functor.comp_obj, Discrete.functor_obj_eq_as, Function.comp_apply
          (Discrete.functor (Discrete.mk ∘ S.τ)).obj { as := c i }))).hom.isIntertwi
        exact LinearMap.congr_fun h1 (p j)
  have h1 := (S.contr.app (Discrete.mk (c i))).hom.isIntertwining' g
  exact LinearMap.congr_fun h1 ((p i) ⊗ₜ ((S.FD.map (eqToHom (by simp [hij]))) (p j)
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.313 Physlib/Relativity/Tensors/Dual.lean

```
public import Physlib.Relativity.Tensors.MetricTensor
variable {k : Type} [CommRing k] {C G : Type} [Group G]
  {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basisIdx c)]
  {S : TensorSpecies k C G basisIdx}
set_option backward.isDefEq.respectTransparency false in
```

1.314 Physlib/Relativity/Tensors/Elab.lean

```
public import Physlib.Relativity.Tensors.Contraction.Basic
public import Physlib.Relativity.Tensors.Evaluation
public import Physlib.Relativity.Tensors.Tensorial
```

1.315 Physlib/Relativity/Tensors/Evaluation.lean

```
public import Physlib.Relativity.Tensors.Basic
variable {k : Type} [CommRing k] {C G : Type} [Group G]
  {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basi
  {S : TensorSpecies k C G basisIdx}
/-- Given a `i : Fin (n + 1)`, a `b : basisIdx (c i)` and a pure tensor
noncomputable def evalPCoeff (i : Fin (n + 1)) (b : basisIdx (c i)) (p : Pu
    (b : basisIdx (c i)) (p : Pure S c)
    (b : basisIdx (c i)) (p : Pure S c)
/-- Given a `i : Fin (n + 1)`, a `b : basisIdx (c i)` and a pure tensor
noncomputable def evalP (i : Fin (n + 1)) (b : basisIdx (c i)) (p : Pure S
set_option backward.isDefEq.respectTransparency false in
    (b : basisIdx (c i)) (p : Pure S c)
set_option backward.isDefEq.respectTransparency false in
    (b : basisIdx (c i)) (p : Pure S c)
set_option backward.isDefEq.respectTransparency false in
    (i : Fin (n + 1)) (b : basisIdx (c i)) :
      (b : basisIdx (c i)) :
    (b : basisIdx (c i)) (p : Pure S c) :
TODO "Add lemmas related to the interaction of evalT and permT, prodT and c
```

1.316 Physlib/Relativity/Tensors/MetricTensor.lean

```
public import Physlib.Relativity.Tensors.UnitTensor
variable {k : Type} [CommRing k] {C G : Type} [Group G]
  {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basi
  {S : TensorSpecies k C G basisIdx}
set_option backward.isDefEq.respectTransparency false in
```

1.317 Physlib/Relativity/Tensors/OfInt.lean

```
public import Physlib.Relativity.Tensors.Basic
variable {k : Type} [CommRing k] {C G : Type} [Group G]
  {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basi
  (S : TensorSpecies k C G basisIdx)
variable {S : TensorSpecies k C G basisIdx}
  (Finsupp.linearEquivFunOnFinite k k ((j : Fin n) → basisIdx (c j))).symm
```

1.318 Physlib/Relativity/Tensors/Product.lean

```
public import Physlib.Relativity.Tensors.Basic
```

```
  - A.1. The product of component indices as an equivalence
  {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basisIdx c)]
  {S : TensorSpecies k C G basisIdx} {n n' n2 : ℕ} {c : Fin n → C} {c' : Fin n' → C}
### A.1 The product of component indices as an equivalence
def ComponentIdx.prod {n1 n2 : ℕ} {c : Fin n1 → C} {c1 : Fin n2 → C} :
  toFun p := (fun i => basisIdxCongr (by simp) (p (Fin.castAdd n2 i)),
    fun i => basisIdxCongr (by simp) (p (Fin.natAdd n1 i)))
  invFun p := Fin.addCases (fun i => basisIdxCongr (by simp) (p.1 i))
    (fun i => basisIdxCongr (by simp) (p.2 i))
    ext1 i
    revert i
    simp [Fin.forall_fin_add]
  right_inv p := by simp
  (fun x => (Representation.equivOfIso
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    Pure.basisVector _ (ComponentIdx.prod.symm (b, b1)) := by
  cases i
  all_goals
    · simp [ComponentIdx.prod]
      rfl
    (p.prodP p1).component b = p.component (ComponentIdx.prod b).1 *
    p1.component (ComponentIdx.prod b).2 := by
  all_goals
    congr
    simp [ComponentIdx.prod]
    exact LinearMap.congr_fun ((S.FD.map (eqToHom h)).hom.isIntertwining' g) (p i)
    exact LinearMap.congr_fun ((S.FD.map (eqToHom h)).hom.isIntertwining' g) (p1 i)
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  (Representation.equivOfIso (S.F.mapIso
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    (Pure.basisVector _ (ComponentIdx.prod.symm (b, b1))).toTensor := by
set_option backward.isDefEq.respectTransparency false in
    (Tensor.basis c) (ComponentIdx.prod b).1 ⊗ₜ[k]
    (Tensor.basis c1) (ComponentIdx.prod b).2)
        (Tensor.basis c) (ComponentIdx.prod b).1 ⊗ₜ[k]
        (Tensor.basis c1) (ComponentIdx.prod b).2)
        (Tensor.basis c) (ComponentIdx.prod b).1 ⊗ₜ[k]
        (Tensor.basis c1) (ComponentIdx.prod b).2)
      change (ComponentIdx.prod.symm (ComponentIdx.prod b)) = _
set_option backward.isDefEq.respectTransparency false in
    (ComponentIdx.prod.symm)).map tensorEquivProd := by
```

```
  simp [ComponentIdx.prod, tensorEquivProd]
  rw [ComponentIdx.prod]
  | Sum.inl i => simp
  | Sum.inr i => simp
    (basis c).repr t (ComponentIdx.prod b).1 *
    (basis c1).repr t1 (ComponentIdx.prod b).2 := by
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.319 Physlib/Relativity/Tensors/RealTensor/Basic.lean

```
public import Physlib.Relativity.Tensors.RealTensor.Metrics.Pre
public import Physlib.Relativity.Tensors.Contraction.Basis

def realLorentzTensor (d : ℕ := 3) : TensorSpecies ℝ realLorentzTensor.Colo
    (LorentzGroup d) (fun _ => Fin 1 ⊕ Fin d) where
    | Color.up => Lorentz.contrBasis d
    | Color.down => Lorentz.coBasis d
    | Color.up => Lorentz.contrCoContract_apply_metric
    | Color.down => Lorentz.coContrContract_apply_metric
## Basis and discrete functor objects

These re-express fields of `realLorentzTensor d` in terms of `Lorentz` data
@[simp]
lemma basisIdxCongr_apply {d : ℕ} {c1 c2 : realLorentzTensor.Color} (h : c1
    (i : Fin 1 ⊕ Fin d) :
    TensorSpecies.basisIdxCongr (basisIdx := fun _ => Fin 1 ⊕ Fin d) h i =
  rfl
lemma basis_eq_contrBasis {d : ℕ} :
    (realLorentzTensor d).basis Color.up = Lorentz.contrBasis (d := d) := r
lemma basis_eq_coBasis {d : ℕ} :
    (realLorentzTensor d).basis Color.down = Lorentz.coBasis (d := d) := rf

lemma FD_obj_up {d : ℕ} :
    (realLorentzTensor d).FD.obj { as := Color.up } = Lorentz.Contr d := rf

lemma FD_obj_down {d : ℕ} :
    (realLorentzTensor d).FD.obj { as := Color.down } = Lorentz.Co d := rfl
set_option backward.isDefEq.respectTransparency false in
    (i : Fin 1 ⊕ Fin d)
```

```
    (j : Fin 1 ⊕ Fin d) :
      = (if i = j then 1 else 0) := by
      (Lorentz.contrBasis d i ⊗ₜ Lorentz.coBasis d j) = _
    simp only [Rep.tensorUnit_V, Lorentz.contrBasis_toFin1dℝ, Lorentz.coBasis_toFin1
      dotProduct_single, mul_one]
    simp only [eq_comm]
      (Lorentz.coBasis d i ⊗ₜ Lorentz.contrBasis d j) = _
    simp only [Rep.tensorUnit_V, Lorentz.coBasis_toFin1dℝ, Lorentz.contrBasis_toFin1
      dotProduct_single, mul_one]
    simp only [eq_comm]
    if (x.1 i) = (x.1 j) then 1 else 0 := by
    ∑ (x : Fin 1 ⊕ Fin d),
    (DropPairSection.ofFinEquiv h.1 b ⟨x, x⟩)) := by
  rw [Finset.sum_eq_single (x)]
  · rw [contr_basis]
    simp
    rw [contr_basis, if_neg]
    · simp_all
    · simp only [basisIdxCongr_apply]
      exact Ne.symm hy
```

1.320 Physlib/Relativity/Tensors/RealTensor/CoVector/Basic.lean

```
public import Physlib.Relativity.Tensors.Tensorial
public import Physlib.Relativity.Tensors.RealTensor.Basic
public import Mathlib.Geometry.Manifold.ChartedSpace

TODO "Split this module on Lorentz co-vectors into two: `Basic.lean` and `Tensorial.
  The former should contain the basic definitions and vector space structure of
  Lorentz co-vectors (e.g. the basis etc.). This should only depend on imports from
  The latter should contain the tensorial instance and everything which should
  depend on the `Tensorial` imports. A similar TODO item exists
  for Lorentz vectors."

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    simpa [norm_eq_equivEuclid, ← dist_eq_norm_neg_add] using
      dist_comm (equivEuclid d x) (equivEuclid d y)
    simpa [norm_eq_equivEuclid, ← dist_eq_norm_neg_add] using dist_triangle
    simp only [norm_eq_equivEuclid, map_add]
    simp at h
    grind
set_option backward.isDefEq.respectTransparency false in
```

```
    ComponentIdx (S := (realLorentzTensor d)) ![Color.down] ≃ Fin 1 ⊕ Fin d
  toFun f := f 0
  invFun i := fun _ => i
  left_inv f := by
    ext m
    fin_cases m
    simp
  right_inv i := by rfl
    ((Lorentz.coBasis d).repr (p 0)) (indexEquiv.symm i 0) := by
set_option backward.isDefEq.respectTransparency false in
  simp [Pure.basisVector]
    change (coBasis d) (indexEquiv.symm μ 0)
  simp [coBasis, indexEquiv, Pi.single_apply]
  congr 1
  exact Eq.propIntro (fun a => id (Eq.symm a)) fun a => id (Eq.symm a)
set_option backward.isDefEq.respectTransparency false in
    rw [coBasis_repr_apply]
    rw [toTensor_symm_pure, coBasis_repr_apply]
    rfl
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
        simp only [RingHom.id_apply]
set_option backward.isDefEq.respectTransparency false in
```

1.321 Physlib/Relativity/Tensors/RealTensor/Matrix/Pre.lean

```
public import Physlib.Relativity.Tensors.RealTensor.Vector.Pre.Basic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
```

1.322 Physlib/Relativity/Tensors/RealTensor/Metrics/Basic.lean

```
public import Physlib.Relativity.Tensors.RealTensor.Basic
public import Physlib.Relativity.Tensors.MetricTensor

set_option backward.isDefEq.respectTransparency false in
    minkowskiMatrix (b 0) (b 1) := by
    change (((Lorentz.coBasis d).tensorProduct (Lorentz.coBasis d)).repr
  rw [Finset.sum_eq_single (b 0)]
  · rw [Finset.sum_eq_single (b 1)]
      simp only [Fin.isValue, ↓reduceIte, mul_one, mul_ite, mul_zero,
      Finset.univ_unique, Fin.default_eq_zero, Finset.sum_singleton, Finset.sum_cons
      ite_eq_right_iff]
set_option backward.isDefEq.respectTransparency false in
    minkowskiMatrix (b 0) (b 1) := by
    change (((Lorentz.contrBasis d).tensorProduct (Lorentz.contrBasis d)).repr
  rw [Finset.sum_eq_single (b 0)]
  · rw [Finset.sum_eq_single (b 1)]
      simp only [Fin.isValue, ↓reduceIte, mul_one, mul_ite, mul_zero,
      Finset.univ_unique, Fin.default_eq_zero, Finset.sum_singleton,
      Finset.sum_const_zero, ite_eq_right_iff]
```

1.323 Physlib/Relativity/Tensors/RealTensor/Metrics/Pre.lean

```
/-
Copyright (c) 2024 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
module

public import Physlib.Relativity.Tensors.RealTensor.Units.Pre
/-!

# Metric for real Lorentz vectors

-/

@[expose] public section
noncomputable section
```

```
open Module Matrix MatrixGroups Complex TensorProduct CategoryTheory.Monoid

namespace Lorentz

/-- The metric `ηᵃᵃ` as an element of `(Contr d ⊗ Contr d).V`. -
/
def preContrMetricVal (d : ℕ := 3) : (Contr d ⊗ Contr d).V :=
  contrContrToMatrixRe.symm ((@minkowskiMatrix d))

set_option backward.isDefEq.respectTransparency false in
/-- Expansion of `preContrMetricVal` into basis. -/
lemma preContrMetricVal_expand_tmul {d : ℕ} : preContrMetricVal d =
    contrBasis d (Sum.inl 0) ⊗ₜ[ℝ] contrBasis d (Sum.inl 0) -
    ∑ i, contrBasis d (Sum.inr i) ⊗ₜ[ℝ] contrBasis d (Sum.inr i) := by
  simp only [preContrMetricVal, Fin.isValue]
  rw [contrContrToMatrixRe_symm_expand_tmul]
  simp only [Fintype.sum_sum_type, Finset.univ_unique, Fin.default_eq_zero,
    Fin.isValue, Finset.sum_singleton, ne_eq, reduceCtorEq, not_false_eq_tr
    minkowskiMatrix.off_diag_zero, zero_smul, Finset.sum_const_zero, add_ze
    minkowskiMatrix.inl_0_inl_0, one_smul, zero_add]
  rw [sub_eq_add_neg, ← Finset.sum_neg_distrib]
  congr
  funext x
  rw [Finset.sum_eq_single x]
  · simp [minkowskiMatrix.inr_i_inr_i]
  · simp only [Finset.mem_univ, ne_eq, smul_eq_zero, forall_const]
    intro b hb
    left
    refine minkowskiMatrix.off_diag_zero ?_
    simp only [ne_eq, Sum.inr.injEq]
    exact fun a => hb (id (Eq.symm a))
  · simp

set_option backward.isDefEq.respectTransparency false in
lemma preContrMetricVal_expand_tmul_minkowskiMatrix {d : ℕ} : preContrMetri
    ∑ i, (minkowskiMatrix i i) • (contrBasis d i ⊗ₜ[ℝ] contrBasis d i) := b
  rw [preContrMetricVal_expand_tmul]
  simp only [Fin.isValue, Fintype.sum_sum_type, Finset.univ_unique,
    Fin.default_eq_zero, Finset.sum_singleton, minkowskiMatrix.inl_0_inl_0,
    minkowskiMatrix.inr_i_inr_i, neg_smul, Finset.sum_neg_distrib]
  abel

set_option backward.isDefEq.respectTransparency false in
/-- The metric `ηᵃᵃ` as a morphism `𝟙_ (Rep ℝ (LorentzGroup d)) ⟶ Contr d ⊗
  making its invariance under the action of `LorentzGroup d`. -
/
```

```
def preContrMetric (d : ℕ := 3) : □_ (Rep ℝ (LorentzGroup d)) □ Contr d ⊗ Contr d :=
  {
    toFun := fun a => a • (preContrMetricVal d),
    map_add' := fun x y => by
      simp only [add_smul],
    map_smul' := fun m x => by
      simp only [smul_smul]
      rfl
    isIntertwining' M := by
      refine LinearMap.ext fun x : ℝ => ?_
      simp only [LinearMap.coe_comp, Function.comp_apply]
      change x • (preContrMetricVal d) =
        (TensorProduct.map ((Contr d).ρ M) ((Contr d).ρ M)) (x • (preContrMetricVal
      simp only [map_smul]
      apply congrArg
      simp only [preContrMetricVal]
      conv_rhs =>
        rw [contrContrToMatrixRe_ρ_symm]
      apply congrArg
      simp
    }

lemma preContrMetric_apply_one {d : ℕ} : (preContrMetric d).hom (1 : ℝ) = preContrMe
  change (1 : ℝ) • preContrMetricVal d = preContrMetricVal d
  rw [one_smul]

/-- The metric `ηᵢᵢ` as an element of `(Co d ⊗ Co d).V`. -
/
def preCoMetricVal (d : ℕ := 3) : (Co d ⊗ Co d).V :=
  coCoToMatrixRe.symm ((@minkowskiMatrix d))

set_option backward.isDefEq.respectTransparency false in
/-- Expansion of `preContrMetricVal` into basis. -/
lemma preCoMetricVal_expand_tmul {d : ℕ} : preCoMetricVal d =
    coBasis d (Sum.inl 0) ⊗ₜ[ℝ] coBasis d (Sum.inl 0) -
    ∑ i, coBasis d (Sum.inr i) ⊗ₜ[ℝ] coBasis d (Sum.inr i) := by
  simp only [preCoMetricVal, Fin.isValue]
  rw [coCoToMatrixRe_symm_expand_tmul]
  simp [minkowskiMatrix.inl_0_inl_0]
  rw [sub_eq_add_neg, ← Finset.sum_neg_distrib]
  congr
  funext x
  rw [Finset.sum_eq_single x]
  · simp [minkowskiMatrix.inr_i_inr_i]
  · simp only [Finset.mem_univ, ne_eq, smul_eq_zero, forall_const]
    intro b hb
```

```
    left
    refine minkowskiMatrix.off_diag_zero ?_
    simp only [ne_eq, Sum.inr.injEq]
    exact fun a => hb (id (Eq.symm a))
  · simp

set_option backward.isDefEq.respectTransparency false in
lemma preCoMetricVal_expand_tmul_minkowskiMatrix {d : ℕ} : preCoMetricVal d
    ∑ i, (minkowskiMatrix i i) • (coBasis d i ⊗ₜ[ℝ] coBasis d i) := by
  rw [preCoMetricVal_expand_tmul]
  simp only [Fin.isValue, Fintype.sum_sum_type, Finset.univ_unique,
    Fin.default_eq_zero, Finset.sum_singleton, minkowskiMatrix.inl_0_inl_0,
    minkowskiMatrix.inr_i_inr_i, neg_smul, Finset.sum_neg_distrib]
  abel

set_option backward.isDefEq.respectTransparency false in
/-- The metric `ηᵢᵢ` as a morphism `𝟙_ (Rep ℂ (LorentzGroup d))) ⟶ Co d ⊗ C
  making its invariance under the action of `LorentzGroup d`. -
/
def preCoMetric (d : ℕ := 3) : 𝟙_ (Rep ℝ (LorentzGroup d)) ⟶ Co d ⊗ Co d :=
  {
    toFun := fun a => a • preCoMetricVal d,
    map_add' := fun x y => by
      simp only [add_smul],
    map_smul' := fun m x => by
      simp only [smul_smul]
      rfl
    isIntertwining' M := by
      refine LinearMap.ext fun x : ℝ => ?_
      simp only [LinearMap.coe_comp, Function.comp_apply]
      change x • preCoMetricVal d =
        (TensorProduct.map ((Co d).ρ M) ((Co d).ρ M)) (x • preCoMetricVal d
      simp only [_root_.map_smul]
      apply congrArg
      simp only [preCoMetricVal]
      rw [coCoToMatrixRe_ρ_symm]
      apply congrArg
      rw [← LorentzGroup.coe_inv, LorentzGroup.transpose_mul_minkowskiMatri

    }

lemma preCoMetric_apply_one {d : ℕ} : (preCoMetric d).hom (1 : ℝ) = preCoMe
  change (1 : ℝ) • preCoMetricVal d = preCoMetricVal d
  rw [one_smul]

/-!
```

```
## Contraction of metrics

-/

open minkowskiMatrix in
set_option backward.isDefEq.respectTransparency false in
lemma contrCoContract_apply_metric {d : ℕ} :
    (β_ (Contr d) (Co d)).hom.hom
    (((Contr d) ◁ (λ_ (Co d)).hom).hom
    ((((Contr d) ◁ contrCoContract ▷ (Co d))).hom
    (((Contr d) ◁ (α_ ((Contr d)) (Co d) (Co d)).inv).hom
    ((α_ ((Contr d)) ((Contr d)) ((Co d) ⊗ (Co d))).hom.hom
    ((preContrMetric d).hom (1 : ℝ) ⊗ₜ[ℝ] (preCoMetric d).hom (1 : ℝ))))))
    = (preCoContrUnit d).hom (1 : ℝ) := by
  calc _
    _ = ((β_ (Contr d) (Co d)).hom.hom <| ((Contr d) ◁ (λ_ (Co d)).hom).hom <|
      (((Contr d) ◁ contrCoContract ▷ (Co d))).hom <|
      ((Contr d) ◁ (α_ ((Contr d)) (Co d) (Co d)).inv).hom <|
      (α_ ((Contr d)) ((Contr d)) ((Co d) ⊗ (Co d))).hom.hom <|
      ∑ i, ∑ j, ((η i i * η j j) •
      ((contrBasis d i ⊗ₜ[ℝ] contrBasis d i) ⊗ₜ[ℝ] (coBasis d j ⊗ₜ[ℝ] coBasis d j)))
        congr
        rw [preContrMetric_apply_one, preCoMetric_apply_one,
          preContrMetricVal_expand_tmul_minkowskiMatrix, preCoMetricVal_expand_tmul_
        simp [tmul_sum, sum_tmul, - Fintype.sum_sum_type, Finset.smul_sum]
        rw [Finset.sum_comm]
        congr 1
        funext x
        congr 1
        funext y
        simp [smul_tmul, smul_smul]
        rw [mul_comm]
    _ = ((β_ (Contr d) (Co d)).hom.hom <| ((Contr d) ◁ (λ_ (Co d)).hom).hom <|
      ∑ i, ∑ j, (minkowskiMatrix i i * minkowskiMatrix j j) •
        (contrBasis d i ⊗ₜ[ℝ] (contrCoContract (contrBasis d i ⊗ₜ[ℝ] coBasis d j)
          ⊗ₜ[ℝ] coBasis d j))) := by
        congr
        simp only [map_sum, map_smul]
        rfl
    _ = ((β_ (Contr d) (Co d)).hom.hom <| ((Contr d) ◁ (λ_ (Co d)).hom).hom <|
          ∑ i, contrBasis d i ⊗ₜ[ℝ] ((1 : ℝ) ⊗ₜ[ℝ] coBasis d i)) := by
        congr
        funext x
        rw [Finset.sum_eq_single x]
        · simp only [minkowskiMatrix.η_apply_mul_η_apply_diag, one_smul]
```

```
          rw [contrCoContract_basis]
          simp
        · intro b _ hb
          rw [contrCoContract_basis]
          rw [if_neg]
          · simp
          · exact id (Ne.symm hb)
        · simp
    _ = ((β_ (Contr d) (Co d)).hom.hom <| ∑ i, contrBasis d i ⊗ₜ[ℝ] coBasis
        congr
        simp only [map_sum]
        simp
  rw [preCoContrUnit_apply_one, preCoContrUnitVal_expand_tmul]
  simp

open minkowskiMatrix in
set_option backward.isDefEq.respectTransparency false in
lemma coContrContract_apply_metric {d : ℕ} :
    (β_ (Co d) (Contr d)).hom.hom
    (((Co d) ◁ (λ_ (Contr d)).hom).hom
    ((((Co d) ◁ coContrContract ▷ (Contr d))).hom
    (((Co d) ◁ (α_ ((Co d)) (Contr d) (Contr d)).inv).hom
    ((α_ ((Co d)) ((Co d)) ((Contr d) ⊗ (Contr d))).hom.hom
    ((preCoMetric d).hom (1 : ℝ) ⊗ₜ[ℝ] (preContrMetric d).hom (1 : ℝ))))))
    = (preContrCoUnit d).hom (1 : ℝ) := by
  calc _
    _ = ((β_ (Co d) (Contr d)).hom.hom <| ((Co d) ◁ (λ_ (Contr d)).hom).hom
      (((Co d) ◁ coContrContract ▷ (Contr d))).hom <|
      ((Co d) ◁ (α_ ((Co d)) (Contr d) (Contr d)).inv).hom <|
      (α_ ((Co d)) ((Co d)) ((Contr d) ⊗ (Contr d))).hom.hom <|
      ∑ i, ∑ j, ((η i i * η j j) •
      ((coBasis d i ⊗ₜ[ℝ] coBasis d i) ⊗ₜ[ℝ] (contrBasis d j ⊗ₜ[ℝ] contrBas
        congr
        rw [preCoMetric_apply_one, preContrMetric_apply_one,
          preCoMetricVal_expand_tmul_minkowskiMatrix, preContrMetricVal_exp
        simp [tmul_sum, sum_tmul, - Fintype.sum_sum_type, Finset.smul_sum]
        rw [Finset.sum_comm]
        congr 1
        funext x
        congr 1
        funext y
        simp [smul_tmul, smul_smul]
        rw [mul_comm]
    _ = ((β_ (Co d) (Contr d)).hom.hom <| ((Co d) ◁ (λ_ (Contr d)).hom).hom
      ∑ i, ∑ j, (minkowskiMatrix i i * minkowskiMatrix j j) •
        (coBasis d i ⊗ₜ[ℝ] (coContrContract (coBasis d i ⊗ₜ[ℝ] contrBasis d
```

```
          ⊗ₜ[ℝ] contrBasis d j))) := by
        congr
        simp only [map_sum, map_smul]
        rfl
    _ = ((β_ (Co d) (Contr d)).hom.hom <| ((Co d) ◁ (λ_ (Contr d)).hom).hom <|
          ∑ i, coBasis d i ⊗ₜ[ℝ] ((1 : ℝ) ⊗ₜ[ℝ] contrBasis d i)) := by
        congr
        funext x
        rw [Finset.sum_eq_single x]
        · simp only [minkowskiMatrix.η_apply_mul_η_apply_diag, one_smul]
          rw [coContrContract_basis]
          simp
        · intro b _ hb
          rw [coContrContract_basis]
          rw [if_neg]
          · simp
          · exact id (Ne.symm hb)
        · simp
    _ = ((β_ (Co d) (Contr d)).hom.hom <| ∑ i, coBasis d i ⊗ₜ[ℝ] contrBasis d i) :=
        congr
        simp only [map_sum]
        simp
  rw [preContrCoUnit_apply_one, preContrCoUnitVal_expand_tmul]
  simp

end Lorentz
end
```

1.324 Physlib/Relativity/Tensors/RealTensor/ToComplex.lean

```
Authors: Joseph Tooby-Smith, Nikolai Kashcheev
public import Physlib.Relativity.Tensors.ComplexTensor.Basic
lemma repDim_colorToComplex {c : realLorentzTensor.Color} :
    complexLorentzTensor.repDim (colorToComplex c) = 4 := by
  cases c <;> simp [colorToComplex]

/-- `simp` helper: reduce `match c j` after a case split on `c j`
  (avoids dependent `rw` / `Pi.smul_apply`). -/
lemma colorToComplex_match_up {n} {c : Fin n → realLorentzTensor.Color} {j}
    (hc : c j = realLorentzTensor.Color.up) :
    (match c j with
      | .up => complexLorentzTensor.Color.up
      | .down => complexLorentzTensor.Color.down)
      = complexLorentzTensor.Color.up := by
```

```
  rw [hc]

lemma colorToComplex_match_down {n} {c : Fin n → realLorentzTensor.Color} {
    (hc : c j = realLorentzTensor.Color.down) :
    (match c j with
      | .up => complexLorentzTensor.Color.up
      | .down => complexLorentzTensor.Color.down)
      = complexLorentzTensor.Color.down := by
  rw [hc]

lemma colorToComplex_comp_eq_match {n} (c : Fin n → realLorentzTensor.Color
    (colorToComplex ∘ c) j =
      (match c j with
        | .up => complexLorentzTensor.Color.up
        | .down => complexLorentzTensor.Color.down) := by
  rcases hc : c j with _ | _ <;> simp [colorToComplex, hc, Function.comp_ap

noncomputable def _root_.TensorSpecies.Tensor.ComponentIdx.complexify {n}
    {c : Fin n → realLorentzTensor.Color} :
  toFun b := fun j => Fin.cast repDim_colorToComplex.symm (finSumFinEquiv (
  invFun i := fun j => finSumFinEquiv.symm <| Fin.cast repDim_colorToComple
  left_inv i := by simp
  right_inv i := by simp
    (ComponentIdx.complexify f) j = Fin.cast repDim_colorToComplex.symm (fi
  simp only [toComplex, LinearMap.coe_mk, AddHom.coe_mk]
  simp

/-- The representation of `toComplex v` in the complexified basis equals
  the real representation coerced to complex. -/
lemma toComplex_repr {n} {c : Fin n → realLorentzTensor.Color}
    (v : ℝT(3, c)) (i : ComponentIdx (S := realLorentzTensor) c) :
    (Tensor.basis (S := complexLorentzTensor) (colorToComplex ∘ c)).repr
      (toComplex v) i.complexify =
    ↑((Tensor.basis (S := realLorentzTensor) c).repr v i) := by
  -- Expand toComplex v in the complexified basis
  rw [toComplex_eq_sum_basis]
  -- `repr` commutes with finite sums of tensors; then push the Finsupp eva
  rw [map_sum]
  simp only [Finsupp.coe_finset_sum, Finset.sum_apply]
  -- The sum has only one non-zero term (when k = i.complexify)
  rw [Fintype.sum_eq_single i.complexify]
  · -- Case k = i.complexify: show the term equals ↑(repr v i)
    have hsmul :
        ((Tensor.basis (S := realLorentzTensor) c).repr v
            (ComponentIdx.complexify.symm (ComponentIdx.complexify i))) •
          Tensor.basis (S := complexLorentzTensor) (colorToComplex ∘ c)
```

```
            (ComponentIdx.complexify i) =
          (↑((Tensor.basis (S := realLorentzTensor) c).repr v
              (ComponentIdx.complexify.symm (ComponentIdx.complexify i))) : ℂ) •
            Tensor.basis (S := complexLorentzTensor) (colorToComplex ∘ c)
              (ComponentIdx.complexify i) :=
        (Complex.coe_smul _ _).symm
      rw [hsmul]
      have hm :=
        LinearEquiv.map_smul
          (Tensor.basis (S := complexLorentzTensor) (colorToComplex ∘ c)).repr
          (↑((Tensor.basis (S := realLorentzTensor) c).repr v
              (ComponentIdx.complexify.symm (ComponentIdx.complexify i))) : ℂ)
          (Tensor.basis (S := complexLorentzTensor) (colorToComplex ∘ c)
            (ComponentIdx.complexify i))
      simp_rw [hm]
      simp only [Finsupp.coe_smul, Pi.smul_apply, smul_eq_mul, Basis.repr_self, Finsup
      simp only [Equiv.symm_apply_apply ComponentIdx.complexify, ite_true, mul_one]
    · -- Case k ≠ i.complexify: show the term equals 0
      intro k hk
      have hsmul :
          ((Tensor.basis (S := realLorentzTensor) c).repr v (ComponentIdx.complexify.s
            Tensor.basis (S := complexLorentzTensor) (colorToComplex ∘ c) k =
          (↑((Tensor.basis (S := realLorentzTensor) c).repr v (ComponentIdx.complexify
            Tensor.basis (S := complexLorentzTensor) (colorToComplex ∘ c) k :=
        (Complex.coe_smul _ _).symm
      rw [hsmul]
      have hm :=
        LinearEquiv.map_smul (Tensor.basis (S := complexLorentzTensor) (colorToComplex
          (↑((Tensor.basis (S := realLorentzTensor) c).repr v (ComponentIdx.complexify
          (Tensor.basis (S := complexLorentzTensor) (colorToComplex ∘ c) k)
      simp_rw [hm]
      simp only [Finsupp.coe_smul, Pi.smul_apply, smul_eq_mul, Basis.repr_self, Finsup
      split_ifs with h
      · exfalso
        exact hk h
      · rw [mul_zero]
open CategoryTheory
open Lorentz.SL2C

set_option backward.isDefEq.respectTransparency false in
/-- Local lemma for `toComplex_equivariant`: isolates the heavy matrix step
  for a contravariant slot. -/
lemma toComplex_equivariant_slot_repr_up {n} {c : Fin n → realLorentzTensor.Color}
    (k : Fin n) (h : c k = realLorentzTensor.Color.up) (Λ : SL(2, ℂ))
    (b : ComponentIdx (S := realLorentzTensor) c)
    (i : ComponentIdx (S := complexLorentzTensor) (colorToComplex ∘ c)) :
```

```
    (↑(((realLorentzTensor.basis (c k)).repr
          (((realLorentzTensor.FD.obj { as := c k}).ρ (Lorentz.SL2C.toLoren
            ((realLorentzTensor.basis (c k)) (b k))))
        (ComponentIdx.complexify.symm i k))) =
      (LorentzGroup.toComplex (Lorentz.SL2C.toLorentzGroup Λ)
        (finSumFinEquiv.symm (Fin.cast repDim_colorToComplex (i k))) (b k))
  rw [TensorSpecies.repr_ρ_basis_FDTransport (S := realLorentzTensor) (c :=
    (c₁ := realLorentzTensor.Color.up) h (Lorentz.SL2C.toLorentzGroup Λ)
    (ComponentIdx.complexify.symm i k) (b k)]
  simp_rw [FD_obj_up, basis_eq_contrBasis]
  erw [← LinearMap.toMatrix_apply]
  erw [Lorentz.contrBasis_ρ_apply (d := 3) (M := Lorentz.SL2C.toLorentzGrou
        (i := ComponentIdx.complexify.symm i k) (j := b k)]
  simp only [_root_.LorentzGroup.toComplex, MonoidHom.coe_mk, OneHom.coe_mk
    Complex.ofRealHom_eq_coe]
  refine Complex.ofReal_inj.mpr ?_
  rfl

set_option backward.isDefEq.respectTransparency false in
/-- Local lemma for `toComplex_equivariant`: isolates the heavy matrix step
/
lemma toComplex_equivariant_slot_repr_down {n} {c : Fin n → realLorentzTens
    (k : Fin n) (h : c k = realLorentzTensor.Color.down) (Λ : SL(2, ℂ))
    (b : ComponentIdx (S := realLorentzTensor) c)
    (i : ComponentIdx (S := complexLorentzTensor) (colorToComplex ∘ c)) :
    (↑(((realLorentzTensor.basis (c k)).repr
          (((realLorentzTensor.FD.obj { as := c k}).ρ (Lorentz.SL2C.toLoren
            ((realLorentzTensor.basis (c k)) (b k))))
        (ComponentIdx.complexify.symm i k))) =
      (LorentzGroup.toComplex (Lorentz.SL2C.toLorentzGroup Λ))⁻¹
        (b k)
        (finSumFinEquiv.symm (Fin.cast repDim_colorToComplex (i k))) := by
  rw [TensorSpecies.repr_ρ_basis_FDTransport (S := realLorentzTensor) (c :=
    (c₁ := realLorentzTensor.Color.down) h (Lorentz.SL2C.toLorentzGroup Λ)
    (ComponentIdx.complexify.symm i k) (b k)]
  simp_rw [FD_obj_down, basis_eq_coBasis]
  erw [← LinearMap.toMatrix_apply]
  erw [Lorentz.coBasis_ρ_apply (d := 3) (M := Lorentz.SL2C.toLorentzGroup Λ
        (i := ComponentIdx.complexify.symm i k)
        (j := b k)]
  simp only [Matrix.transpose_apply]
  rw [LorentzGroup.toComplex_inv]
  simp only [_root_.LorentzGroup.toComplex, MonoidHom.coe_mk, OneHom.coe_mk
    Complex.ofRealHom_eq_coe]
  refine Complex.ofReal_inj.mpr ?_
  rw [LorentzGroup.coe_inv (Λ := Lorentz.SL2C.toLorentzGroup Λ)]
```

```
  rfl
set_option backward.isDefEq.respectTransparency false in
  classical
  let P : ℝT(3, c) → Prop := fun t =>
    Λ • (toComplex t) = toComplex (Lorentz.SL2C.toLorentzGroup Λ • t)
  apply induction_on_basis (c := c) (P := P) (t := v)
  · intro b
    apply (Tensor.basis (S := complexLorentzTensor) (colorToComplex ∘ c)).repr.injec
    ext i
    conv_lhs => arg 1; rw [toComplex_basis (c := c) b]
    rw [Tensor.basis_apply (S := complexLorentzTensor) (colorToComplex ∘ c)
      (ComponentIdx.complexify b)]
    rw [Tensor.actionT_pure (S := complexLorentzTensor), Tensor.basis_repr_pure]
    conv_rhs =>
      rw [← Equiv.apply_symm_apply ComponentIdx.complexify i,
        toComplex_repr
          (Lorentz.SL2C.toLorentzGroup Λ • Tensor.basis (S := realLorentzTensor) c b
          (ComponentIdx.complexify.symm i)]
    rw [Tensor.basis_apply (S := realLorentzTensor) c b]
    rw [Tensor.actionT_pure (S := realLorentzTensor), Tensor.basis_repr_pure]
    refine Eq.trans (Fintype.prod_congr (α := Fin n) (M := ℂ)
      (fun k => (complexLorentzTensor.basis (colorToComplex (c k))).repr
        ((Λ • Pure.basisVector (colorToComplex ∘ c) (ComponentIdx.complexify b)) k)
      (fun k => ((realLorentzTensor.basis (c k)).repr
        ((Lorentz.SL2C.toLorentzGroup Λ • Pure.basisVector c b) k)
          ((ComponentIdx.complexify.symm i) k) : ℂ))
      (fun k => ?_)) (Eq.symm (Complex.ofReal_prod Finset.univ (fun k =>
        (realLorentzTensor.basis (c k)).repr
          ((Lorentz.SL2C.toLorentzGroup Λ • Pure.basisVector c b) k)
            ((ComponentIdx.complexify.symm i) k))))
    cases h : c k
    · simp only [Pure.actionP_eq (S := complexLorentzTensor),
        Pure.actionP_eq (S := realLorentzTensor), Function.comp_apply,
        Pure.basisVector, colorToComplex]
      have hcΛ : (colorToComplex ∘ c) k = complexLorentzTensor.Color.up := by
        rw [colorToComplex_comp_eq_match c k, colorToComplex_match_up h]
      change ((complexLorentzTensor.basis ((colorToComplex ∘ c) k)).repr
          (((complexLorentzTensor.FD.obj { as := (colorToComplex ∘ c) k }).ρ Λ)
            ((complexLorentzTensor.basis ((colorToComplex ∘ c) k)) (ComponentIdx.com
          (i k) =
        ↑(((realLorentzTensor.basis (c k)).repr
            (((realLorentzTensor.FD.obj { as := c k}).ρ (Lorentz.SL2C.toLorentzGroup
              ((realLorentzTensor.basis (c k)) (b k))))
          (ComponentIdx.complexify.symm i k))
      rw [repr_ρ_basis_vector_up_of_eq (c₀ := (colorToComplex ∘ c) k) hcΛ Λ
        (ComponentIdx.complexify b k) (i k)]
```

```
      simpa using (toComplex_equivariant_slot_repr_up k h Λ b i).symm
    · simp only [Pure.actionP_eq (S := complexLorentzTensor),
        Pure.actionP_eq (S := realLorentzTensor), Function.comp_apply,
        Pure.basisVector, colorToComplex]
      have hcΛ : (colorToComplex ∘ c) k = complexLorentzTensor.Color.down :
        rw [colorToComplex_comp_eq_match c k, colorToComplex_match_down h]
      change ((complexLorentzTensor.basis ((colorToComplex ∘ c) k)).repr
          (((complexLorentzTensor.FD.obj { as := (colorToComplex ∘ c) k }).
            ((complexLorentzTensor.basis ((colorToComplex ∘ c) k)) (Compone
          (i k) =
        ↑(((realLorentzTensor.basis (c k)).repr
            (((realLorentzTensor.FD.obj { as := c k}).ρ (Lorentz.SL2C.toLor
              ((realLorentzTensor.basis (c k)) (b k))))
          (ComponentIdx.complexify.symm i k))
      rw [repr_ρ_basis_vector_down_of_eq (c₀ := (colorToComplex ∘ c) k) hcΛ
        (ComponentIdx.complexify b k) (i k)]
      simpa using (toComplex_equivariant_slot_repr_down k h Λ b i).symm
  · simp [P]
  · intro r t ht
    dsimp [P] at ht ⊢
    rw [toComplex_map_smul, Tensor.actionT_smul (S := complexLorentzTensor)
      Tensor.actionT_smul (S := realLorentzTensor), toComplex_map_smul]
  · intro t1 t2 h1 h2
    dsimp [P] at h1 h2 ⊢
    rw [map_add, Tensor.actionT_add (S := complexLorentzTensor), h1, h2,
      Tensor.actionT_add (S := realLorentzTensor), map_add]
    = (Tensor.basis (S := realLorentzTensor) c1)
      (fun j => b (σ j)) := by
        simpa using congrArg (fun col => complexLorentzTensor.repDim col)
  simp [Tensor.basis_apply, permT_pure, Pure.permP_basisVector, basisIdxCon
set_option backward.isDefEq.respectTransparency false in
    ComponentIdx.complexify (c := Fin.append c c1) (ComponentIdx.prod.symm
      (ComponentIdx.prod.symm (ComponentIdx.complexify (c := c) b,
        ComponentIdx.complexify (c := c1) b1)) := by
    rw [ComponentIdx.complexify_apply]
    simp [ComponentIdx.prod]
    erw [basisIdxCongr_eq_cast]
    simp
    rw [ComponentIdx.complexify_apply]
    simp [ComponentIdx.prod]
    erw [basisIdxCongr_eq_cast]
    simp
set_option backward.isDefEq.respectTransparency false in
            colorToComplex_append,
            basisIdxCongr_eq_cast]
    have h_real := realLorentzTensor.contr_basis (c := c i) (b i) (b j)
```

```
    erw [← realLorentzTensor.basis_congr (h.2) ((b j))]
            ((b j))))).symm
            ((b j))) =
            ((b j))))) := congrArg (↑·) heq
    rw [map_basis_eq _ (by simp [h.2])]

    change _ = complexLorentzTensor.castToField _
    erw [complexLorentzTensor.basis_contr]
    simp only [ComponentIdx.complexify_apply, Fin.val_cast, basisIdxCongr_eq_cast]
    split_ifs with h1 h2
    · rfl
    · simp [h1] at h2
    · rename_i h2
      simp only [Complex.ofReal_zero, zero_ne_one]
      apply h1
      apply finSumFinEquiv.injective
      ext
      exact h2
    · rfl
set_option backward.isDefEq.respectTransparency false in
noncomputable def evalIdxToComplex {n : ℕ}
    (b : Fin 1 ⊕ Fin 3) : Fin (complexLorentzTensor.repDim ((colorToComplex ∘ c) i))
  Fin.cast repDim_colorToComplex.symm (finSumFinEquiv b)
    (b : Fin 1 ⊕ Fin 3) :
set_option backward.isDefEq.respectTransparency false in
    (i : Fin (n + 1)) (b : Fin 1 ⊕ Fin 3)
set_option backward.isDefEq.respectTransparency false in
    (i : Fin (n + 1)) (b : Fin 1 ⊕ Fin 3) (t : ℝT(3, c)) :
```

1.325 Physlib/Relativity/Tensors/RealTensor/Units/Pre.lean

```
public import Physlib.Relativity.Tensors.RealTensor.Matrix.Pre
public import Physlib.Relativity.Tensors.RealTensor.Vector.Pre.Contraction

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
def preContrCoUnit (d : ℕ := 3) : □_ (Rep ℝ (LorentzGroup d)) □ Contr d ⊗ Co d := Re
  {
      rfl
    isIntertwining' M := by
      refine LinearMap.ext fun x : ℝ => ?_
      simp only [LinearMap.coe_comp, Function.comp_apply]
      change x • preContrCoUnitVal d =
        (TensorProduct.map ((Contr d).ρ M) ((Co d).ρ M)) (x • preContrCoUnitVal d)
```

```
      simp only [map_smul]
      apply congrArg
      simp only [preContrCoUnitVal]
      rw [contrCoToMatrixRe_ρ_symm]
      apply congrArg
      simp
  }
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
def preCoContrUnit (d : ℕ) : □_ (Rep ℝ (LorentzGroup d)) □ Co d ⊗ Contr d :
  {
      rfl
    isIntertwining' M := by
      refine LinearMap.ext fun x : ℝ => ?_
      simp only [LinearMap.coe_comp, Function.comp_apply]
      change x • preCoContrUnitVal d =
        (TensorProduct.map ((Co d).ρ M) ((Contr d).ρ M)) (x • preCoContrUni
      simp only [map_smul]
      apply congrArg
      simp only [preCoContrUnitVal]
      rw [coContrToMatrixRe_ρ_symm]
      apply congrArg
      symm
      refine transpose_eq_one.mp ?h.h.h.a
      simp}
set_option backward.isDefEq.respectTransparency false in
    simp only [tmul_sum]
    simp only [Rep.tensor_V, Rep.tensor_ρ, Rep.hom_inv_associator, Fintype.
      Finset.univ_unique, Fin.default_eq_zero, Fin.isValue, Finset.sum_sing
      Representation.Equiv.coe_toIntertwiningMap, map_add, map_sum]
    rfl
    rw [map_sum]
    rfl
  have h3 (i : Fin 1 ⊕ Fin d) : (coContrContract.hom)
set_option backward.isDefEq.respectTransparency false in
    simp only [tmul_sum]
    simp only [Rep.tensor_V, Rep.tensor_ρ, Rep.hom_inv_associator, Fintype.
      Finset.univ_unique, Fin.default_eq_zero, Fin.isValue, Finset.sum_sing
      Representation.Equiv.coe_toIntertwiningMap, map_add, map_sum]
    rfl
    rw [map_sum]
    rfl
  have h3 (i : Fin 1 ⊕ Fin d) : (contrCoContract.hom)
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.326 Physlib/Relativity/Tensors/RealTensor/Vector/Basic.lean

```
public import Physlib.Relativity.Tensors.RealTensor.Basic
public import Physlib.Relativity.Tensors.Tensorial

TODO "Refactor: Split this module on Lorentz vectors into two: `Basic.lean` and `Ten
  The former should contain the basic definitions and vector space structure of
  Lorentz vectors (e.g. the basis etc.). This should only depend on imports from Mat
  The latter should contain the tensorial instance and everything which should
  depend on the `Tensorial` imports. A similar TODO item exists
  for Lorentz co-vectors."

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    simpa [norm_eq_equivEuclid, dist_eq_norm_neg_add] using dist_comm ((equivEuclid
    simpa [norm_eq_equivEuclid, dist_eq_norm_neg_add] using dist_triangle
    simp only [norm_eq_equivEuclid, map_add]
    simp at h
    grind
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    ComponentIdx (S := (realLorentzTensor d)) ![Color.up] ≃ Fin 1 ⊕ Fin d where
  toFun f := f 0
  invFun i := fun _ => i
  left_inv f := by
    ext m
    fin_cases m
    simp
  right_inv i := by rfl
    ((Lorentz.contrBasis d).repr (p 0)) (indexEquiv.symm i 0) := by
set_option backward.isDefEq.respectTransparency false in
  simp [Pure.basisVector]
    change (contrBasis d) (indexEquiv.symm μ 0)
  simp [contrBasis, indexEquiv, Pi.single_apply]
  congr 1
  exact Eq.propIntro (fun a => id (Eq.symm a)) fun a => id (Eq.symm a)
set_option backward.isDefEq.respectTransparency false in
    rw [contrBasis_repr_apply]
    rw [toTensor_symm_pure, contrBasis_repr_apply]
    rfl
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
        simp only [RingHom.id_apply]
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.327 Physlib/Relativity/Tensors/RealTensor/Vector/Causality/Ba

```
public import Physlib.Relativity.Tensors.RealTensor.Vector.MinkowskiProduct
  simp only [causalCharacter]
```

1.328 Physlib/Relativity/Tensors/RealTensor/Vector/Causality/Li

```
public import Physlib.Relativity.Tensors.RealTensor.Vector.Causality.Basic

set_option backward.isDefEq.respectTransparency false in
        push Not at h_not_le
        push Not at h_not_ge
```

1.329 Physlib/Relativity/Tensors/RealTensor/Vector/Causality/Ti

```
public import Physlib.Relativity.Tensors.RealTensor.Vector.Causality.Basic

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  simp only [PiLp.inner_apply]
```

1.330 Physlib/Relativity/Tensors/RealTensor/Vector/MinkowskiPro

```
public import Physlib.Relativity.Tensors.RealTensor.Vector.Basic
public import Physlib.Relativity.Tensors.Elab
public import Physlib.Relativity.Tensors.RealTensor.Metrics.Basic

    change minkowskiMatrix y x
    change x
    change y
  simp only [Fintype.sum_sum_type, Finset.univ_unique, Fin.default_eq_zero,
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.331 Physlib/Relativity/Tensors/RealTensor/Vector/Pre/Basic.lean

```
public import Physlib.Relativity.Tensors.RealTensor.Vector.Pre.Modules
public import Mathlib.RepresentationTheory.Rep.Basic
lemma contrBasis_repr_apply {d : ℕ} (p : Contr d) (i : Fin 1 ⊕ Fin d) :
    (contrBasis d).repr p i = p.val i := by
  simp only [contrBasis, Basis.ofEquivFun_repr_apply]
  rfl

set_option backward.isDefEq.respectTransparency false in
lemma coBasis_repr_apply {d : ℕ} (p : Co d) (i : Fin 1 ⊕ Fin d) :
    (coBasis d).repr p i = p.val i := by
  simp only [coBasis, Basis.ofEquivFun_repr_apply]
  rfl

def Contr.toCo (d : ℕ) : Contr d →ₗ[ℝ] Co d := Rep.ofHom <|
  {
      simp only [_root_.map_smul, mulVec_smul, RingHom.id_apply]
    isIntertwining' g := by
      ext1 ψ
      conv_lhs =>
        change CoMod.toFin1dℝEquiv.symm (η *ᵥ (g.1 *ᵥ ψ.toFin1dℝ))
        rw [mulVec_mulVec, LorentzGroup.minkowskiMatrix_comm, ← mulVec_mulVec]
      rfl
  }
def Co.toContr (d : ℕ) : Co d →ₗ[ℝ] Contr d := Rep.ofHom <|
    {
      toFun := fun ψ => ContrMod.toFin1dℝEquiv.symm (η *ᵥ ψ.toFin1dℝ)
      map_add' := by
        intro ψ ψ'
        simp only [map_add, mulVec_add]
      map_smul' := by
        intro r ψ
        simp only [_root_.map_smul, mulVec_smul, RingHom.id_apply]
      isIntertwining' g := by
        ext1 ψ
        conv_lhs =>
          change ContrMod.toFin1dℝEquiv.symm (η *ᵥ ((LorentzGroup.transpose g⁻¹).1 *
          rw [mulVec_mulVec, ← LorentzGroup.comm_minkowskiMatrix, ← mulVec_mulVec]
        rfl
    }
    simp only [Rep.hom_comp, Representation.IntertwiningMap.comp_toLinearMap, Linear
      Representation.IntertwiningMap.coe_toLinearMap, Function.comp_apply, Rep.hom_i
      Representation.IntertwiningMap.toLinearMap_id, LinearMap.id_coe, id_eq]
    simp only [Rep.hom_comp, Representation.IntertwiningMap.comp_toLinearMap, Linear
      Representation.IntertwiningMap.coe_toLinearMap, Function.comp_apply, Rep.hom_i
```

```
      Representation.IntertwiningMap.toLinearMap_id, LinearMap.id_coe, id_e
```

1.332 Physlib/Relativity/Tensors/RealTensor/Vector/Pre/Contract

```
public import Physlib.Relativity.Tensors.RealTensor.Vector.Pre.Basic

def contrCoContract : Contr d ⊗ Co d □ □_ (Rep ℝ (LorentzGroup d)) := Rep.o
  {
    toLinearMap := TensorProduct.lift (contrModCoModBi d)
    isIntertwining' Λ := by
      ext ψ φ
      simp only [Representation.tprod_apply, AlgebraTensorModule.curry_appl
        LinearMap.restrictScalars_self, curry_apply, LinearMap.coe_comp, Fu
        map_tmul, lift.tmul, Representation.isTrivial_def, LinearMap.id_com
      change (Λ.1 *ᵥ ψ.toFin1dℝ) □ᵥ ((LorentzGroup.transpose Λ⁻¹).1 *ᵥ φ.to
      rw [dotProduct_mulVec, LorentzGroup.transpose_val,
        vecMul_transpose, mulVec_mulVec, LorentzGroup.coe_inv, inv_mul_of_i
      simp only [one_mulVec]
      rfl
  }
def coContrContract : Co d ⊗ Contr d □ □_ (Rep ℝ (LorentzGroup d)) := Rep.o
  {
    toLinearMap := TensorProduct.lift (coModContrModBi d)
    isIntertwining' Λ := by
      ext ψ φ
      change ((LorentzGroup.transpose Λ⁻¹).1 *ᵥ ψ.toFin1dℝ) □ᵥ (Λ.1 *ᵥ φ.to
      rw [dotProduct_mulVec, LorentzGroup.transpose_val, mulVec_transpose,
        LorentzGroup.coe_inv, inv_mul_of_invertible Λ.1]
      simp only [vecMul_one]
      rfl
  }
  contrContrContract.hom.toLinearMap
  simp only [Rep.tensorUnit_V, Rep.tensor_V, Rep.tensor_ρ, Rep.tensorUnit_ρ
  LinearMap.congr_fun (contrContrContract.hom.isIntertwining' g) (x ⊗ₜ[ℝ] y
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  simp only [map_add, map_sub] at hp' hn'
  simp
set_option backward.isDefEq.respectTransparency false in
  simp only [Fin.isValue, I_sq, mul_neg, mul_one, sub_left_inj]
  ring
  simp only [contrBasis_toFin1dℝ, coBasis_toFin1dℝ, dotProduct_single, mul_
  simp only [coBasis_toFin1dℝ, contrBasis_toFin1dℝ, dotProduct_single, mul_
```

1.333 Physlib/Relativity/Tensors/RealTensor/Vector/Pre/Modules.lean

```
public import Physlib.Relativity.PauliMatrices.SelfAdjoint
public import Physlib.Relativity.LorentzGroup.Basic
@[reducible]
  dist_eq x y := Pi.normedAddCommGroup.dist_eq x.val y.val
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
      Pi.single_eq_same, MulAction.one_smul, ne_eq, reduceCtorEq, not_false_eq_true,
      Pi.single_eq_of_ne, MulActionWithZero.zero_smul, sub_zero, PauliMatrix.pauliBa
      Basis.coe_mk, PauliMatrix.pauliSelfAdjoint']
```

1.334 Physlib/Relativity/Tensors/RealTensor/Velocity/Basic.lean

```
public import Physlib.Relativity.Tensors.RealTensor.Vector.MinkowskiProduct
noncomputable instance : TopologicalSpace (Velocity d) := instTopologicalSpaceSubtyp
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.335 Physlib/Relativity/Tensors/TensorSpecies/Basic.lean

```
public import Physlib.Relativity.Tensors.Color.Discrete
public import Physlib.Relativity.Tensors.Color.Lift
structure TensorSpecies (k : Type) [CommRing k] (C : Type) (G : Type) [Group G]
    (basisIdx : C → Type) [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basisIdx c)

  basis : (c : C) → Basis (basisIdx c) k (FD.obj (Discrete.mk c)).V
variable {k : Type} [CommRing k] {C : Type} [Group G]
  {basisIdx : C → Type}

/-- The casting between `basisIdx c` and `basisIdx c1`. -
/
def basisIdxCongr {c c1 : C} (h : c = c1) :
    basisIdx c ≃ basisIdx c1 := Equiv.cast (by simp [h])

@[simp]
lemma basisIdxCongr_rfl (c : C) (i : basisIdx c) :
    basisIdxCongr (Eq.refl c) i = i := rfl

@[simp]
lemma basisIdxCongr_apply_apply {c c1 c2 : C} (h1 : c = c1) (h2 : c1 = c2) (i : basi
    basisIdxCongr h2 (basisIdxCongr h1 i) = basisIdxCongr (by simp [h1, h2]) i := by
  simp [basisIdxCongr]
```

```
variable  [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basisIdx c)]
  (S : TensorSpecies k C G basisIdx)
lemma basis_congr {c c1 : C} (h : c = c1) (i : basisIdx c) :
    S.basis c i = S.FD.map (eqToHom (by simp [h])) (S.basis c1 (basisIdxCon
lemma map_basis_eq {c c1 : C} (h : c = c1) (i : basisIdx c) :
    (S.FD.map (Discrete.eqToHom h)).hom (S.basis c i) = S.basis c1 (basisId
  subst h
  simp

lemma basis_congr_repr {c c1 : C} (h : c = c1) (i : basisIdx c)
    (basisIdxCongr (by simp [h]) i) := by
lemma FD_map_basis {c c1 : C} (h : c = c1) (i : basisIdx c) :
    (S.FD.map (Discrete.eqToHom h)).hom (S.basis c i)
    = S.basis c1 (basisIdxCongr (by simp [h]) i) := by
/-- Categorical bookkeeping: relate `basis.repr` of `ρ g` on a basis vector
  identifying slot colors `c` and `c₁` via `Discrete.eqToHom` (`basis_congr
  `Rep.hom_comm_apply`, `FD_map_basis` chained). Not a physical statement;
  calculations. -/
lemma repr_ρ_basis_FDTransport {c c₁ : C} (h : c = c₁) (g : G) (i :basisIdx
    (b : basisIdx c) :
    (S.basis c).repr (((S.FD.obj { as := c }).ρ g) (S.basis c b)) i =
      (S.basis c₁).repr
        (((S.FD.obj { as := c₁ }).ρ g) (S.basis c₁ (basisIdxCongr (by simp
        (basisIdxCongr (by simp [h]) i) := by
  rw [S.basis_congr_repr h i (((S.FD.obj { as := c }).ρ g) (S.basis c b))]
  rw [Rep.hom_comm_apply (S.FD.map (Discrete.eqToHom h)) g (S.basis c b)]
  rw [S.FD_map_basis h b]

def castToField {S : TensorSpecies k C G basisIdx}
lemma castToField_eq_self {S : TensorSpecies k C G basisIdx} {c}
set_option backward.isDefEq.respectTransparency false in
  simp only [castFin0ToField, mk_hom]
set_option backward.isDefEq.respectTransparency false in
def numIndices {S : TensorSpecies k C G basisIdx} {n : ℕ} {c : Fin n → C}
```

1.336 Physlib/Relativity/Tensors/Tensorial.lean

```
public import Physlib.Relativity.Tensors.Product
variable {k : Type} [CommRing k] {C G : Type} [Group G]
  {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basi
  {S : TensorSpecies k C G basisIdx}
    {basisIdx : outParam (C → Type)} [∀ c, Fintype (basisIdx c)] [∀ c, Deci
    {n : outParam ℕ} (S : outParam (TensorSpecies k C G basisIdx))
    (c :outParam (Fin n → C)) (M : Type)
```

```
noncomputable instance self {n : ℕ} (S : TensorSpecies k C G basisIdx) (c : Fin n →
lemma self_toTensor_apply {n : ℕ} (S : TensorSpecies k C G basisIdx)
    (c : Fin n → C) (t : S.Tensor c) :
noncomputable def numIndices (t : M) [Tensorial S c M] : ℕ :=
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
open Tensor in
    (ComponentIdx.prod.symm) := by
  simp only [ComponentIdx.prod, Module.Basis.map_apply, Module.Basis.coe_reindex,
variable {k : Type} [RCLike k] {C G : Type} [Group G]
    {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basisIdx c)
    (S : TensorSpecies k C G basisIdx) {c : Fin n → C} {M : Type} [AddCommGroup M] [
```

1.337 Physlib/Relativity/Tensors/UnitTensor.lean

```
public import Physlib.Relativity.Tensors.Constructors
variable {k : Type} [CommRing k] {C G : Type} [Group G]
  {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basisIdx c)]
  {S : TensorSpecies k C G basisIdx}
  change ((S.unit.app { as := c }).hom) (1 : k) = _
  simp only [Monoidal.tensorUnit_obj]
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.338 Physlib/SpaceAndTime/Space/Basic.lean

```
public import Physlib.Meta.TODO.Basic
public import Mathlib.Analysis.InnerProductSpace.PiL2
public import Mathlib.Geometry.Manifold.ContMDiff.Defs
`Physlib.SpaceAndTime.Space.Module`.
Physlib sits downstream of Mathlib, and above we import the necessary Mathlib module
The exact implementation of `Space d` into Physlib is discussed in numerous places
- https://leanprover.zulipchat.com/#narrow/channel/479953-
Physlib/topic/Space.20vs.20EuclideanSpace/with/575332888
In `Physlib.SpaceAndTime.Space.Module` we give `Space d` the structure of a `Module`
`d` dimensional (flat) Euclidean space with a given (but arbitrary)
  eq_of_apply <| fun i => Fin.elim0 i
TODO "Fix the manifold structure on `Space d`. In particular, we should not need to
  define `manifoldStructure`. Instead, we should be able to give `Space d` an instan
  of `IsManifold` directly."
```

```
set_option backward.isDefEq.respectTransparency false in
    (manifoldStructure d) (□(ℝ, EuclideanSpace ℝ (Fin d))) ⊤ (manifoldStruc
```

1.339 Physlib/SpaceAndTime/Space/ConstantSliceDist.lean

```
public import Mathlib.Analysis.Calculus.ContDiff.FiniteDimension
public import Physlib.SpaceAndTime.Space.Derivatives.Basic
public import Physlib.SpaceAndTime.Space.Slice

open Physlib
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
      · fun_prop
```

1.340 Physlib/SpaceAndTime/Space/CrossProduct.lean

```
public import Physlib.SpaceAndTime.Time.Derivatives
  simp only [WithLp.equiv_apply, WithLp.equiv_symm_apply, PiLp.inner_apply]
```

1.341 Physlib/SpaceAndTime/Space/Derivatives/Basic.lean

```
public import Physlib.Mathematics.Distribution.Basic
public import Physlib.Relativity.Tensors.RealTensor.Vector.Basic
public import Physlib.SpaceAndTime.Space.Module
open Physlib

lemma deriv_eq_fderiv_fun [AddCommGroup M] [Module ℝ M] [TopologicalSpace M
    exact (modelDiffeo.symm.mdifferentiable (WithTop.top_ne_zero)).mdiffere
    exact (modelDiffeo.mdifferentiable (WithTop.top_ne_zero)).mdifferentiab
TODO "Make the version of the derivative described through
set_option backward.isDefEq.respectTransparency false in
  · simp [-EuclideanSpace.coe_proj]
set_option backward.isDefEq.respectTransparency false in
```

1.342 Physlib/SpaceAndTime/Space/Derivatives/Curl.lean

```
/-
```

```
Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Zhi Kai Pong, Joseph Tooby-Smith, Lode Vermeulen
-/
module

public import Physlib.SpaceAndTime.Space.Derivatives.Laplacian
public import Mathlib.MeasureTheory.Integral.CurveIntegral.Poincare
public import Physlib.SpaceAndTime.Space.CrossProduct
public import Mathlib.Analysis.Calculus.ParametricIntervalIntegral

/-!

# Curl on Space

## i. Overview

In this module we define the curl of functions and distributions on 3-
dimensional
space `Space 3`.

We also prove some basic vector-identities involving of the curl operator.

## ii. Key results

- `curl` : The curl operator on functions from `Space 3` to `EuclideanSpace ℝ (Fin 3
- `distCurl` : The curl operator on distributions from `Space 3` to `EuclideanSpace
- `div_of_curl_eq_zero` : The divergence of the curl of a function is zero.
- `distCurl_distGrad_eq_zero` : The curl of the gradient of a distribution is zero.

## iii. Table of contents

- A. The curl on functions
  - A.1. The curl on the zero function
  - A.2. The curl on a constant function
  - A.3. Basic operations on curl
  - A.4. The curl of a linear map is a linear map
  - A.5. Preliminary lemmas about second derivatives
  - A.6. The div of a curl is zero
  - A.7. The curl of a grad is zero
  - A.8. The curl of a curl
  - A.9. A divergence-free field is a curl
  - A.10. A curl-free field is a gradient
- B. The curl on distributions
  - B.1. The components of the curl
  - B.2. Basic equalities
```

```
  - B.3. The curl of a grad is zero

## iv. References

-/

@[expose] public section

open Physlib

namespace Space

/-!

## A. The curl on functions

-/

/-- The vector calculus operator `curl`. -/
noncomputable def curl (f : Space → EuclideanSpace ℝ (Fin 3)) :
    Space → EuclideanSpace ℝ (Fin 3) := fun x =>
  -- get i-th component of `f`
  let fi i x := (f x) i
  -- derivative of i-th component in j-th coordinate
  -- ∂fᵢ/∂xⱼ
  let df i j x := ∂[j] (fi i) x
  WithLp.toLp 2 fun i =>
    match i with
    | 0 => df 2 1 x - df 1 2 x
    | 1 => df 0 2 x - df 2 0 x
    | 2 => df 1 0 x - df 0 1 x

@[inherit_doc curl]
macro (name := curlNotation) "∇" "×" f:term:100 : term => `(curl $f)

/-!

### A.1. The curl on the zero function

-/

@[simp]
lemma curl_zero : ∇ × (0 : Space → EuclideanSpace ℝ (Fin 3)) = 0 := by
  unfold curl Space.deriv
  simp only [Fin.isValue, Pi.ofNat_apply, fderiv_fun_const, ContinuousLinea
    sub_self]
```

```
  ext x i
  fin_cases i <;>
  rfl

/-!

### A.2. The curl on a constant function

-/

@[simp]
lemma curl_const : ∇ × (fun _ : Space => v₃) = 0 := by
  unfold curl Space.deriv
  simp only [Fin.isValue, fderiv_fun_const, Pi.ofNat_apply, ContinuousLinearMap.zero
    sub_self]
  ext x i
  fin_cases i <;>
  rfl

/-!

### A.3. Basic operations on curl

-/

lemma curl_add (f1 f2 : Space → EuclideanSpace ℝ (Fin 3))
    (hf1 : Differentiable ℝ f1) (hf2 : Differentiable ℝ f2) :
    ∇ × (f1 + f2) = ∇ × f1 + ∇ × f2 := by
  unfold curl
  ext x i
  fin_cases i <;>
  · simp only [Fin.isValue, Pi.add_apply, PiLp.add_apply, Fin.zero_eta]
    repeat rw [deriv_coord_add]
    simp only [Fin.isValue, Pi.add_apply]
    ring
    repeat assumption

lemma curl_smul (f : Space → EuclideanSpace ℝ (Fin 3)) (k : ℝ)
    (hf : Differentiable ℝ f) :
    ∇ × (k • f) = k • ∇ × f := by
  unfold curl
  ext x i
  fin_cases i <;>
  · simp only [Fin.isValue, Pi.smul_apply, PiLp.smul_apply, smul_eq_mul, Fin.zero_et
    rw [deriv_coord_smul, deriv_coord_smul, mul_sub]
    repeat fun_prop
```

```
lemma curl_neg (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : Differentiable
    ∇ × (-f) = -∇ × f := by
  rw [← neg_one_smul ℝ, curl_smul, neg_one_smul]
  · exact hf

lemma curl_sub (f1 f2 : Space → EuclideanSpace ℝ (Fin 3))
    (hf1 : Differentiable ℝ f1) (hf2 : Differentiable ℝ f2) :
    ∇ × (f1 - f2) = ∇ × f1 - ∇ × f2 := by
  rw [sub_eq_add_neg, curl_add, curl_neg, sub_eq_add_neg]
  repeat fun_prop

/-!

### A.4. The curl of a linear map is a linear map

-/

variable {W} [NormedAddCommGroup W] [NormedSpace ℝ W]

lemma curl_linear_map (f : W → Space 3 → EuclideanSpace ℝ (Fin 3))
    (hf : ∀ w, Differentiable ℝ (f w))
    (hf' : IsLinearMap ℝ f) :
    IsLinearMap ℝ (fun w => ∇ × (f w)) := by
  constructor
  · intro w w'
    rw [hf'.map_add]
    rw [curl_add]
    repeat fun_prop
  · intros k w
    rw [hf'.map_smul]
    rw [curl_smul]
    fun_prop

/-!

### A.5. Preliminary lemmas about second derivatives

-/

/-- Second derivatives distribute coordinate-wise over addition (all three
/
lemma deriv_coord_2nd_add (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : Cont
    ∂[i] (fun x => ∂[u] (fun x => f x u) x + (∂[v] (fun x => f x v) x + ∂[w
    (∂[i] (∂[u] (fun x => f x u))) + (∂[i] (∂[v] (fun x => f x v))) +
    (∂[i] (∂[w] (fun x => f x w))) := by
```

```
  repeat rw [deriv_eq_fderiv_fun]
  ext x
  rw [fderiv_fun_add, fderiv_fun_add]
  simp only [ContinuousLinearMap.add_apply, Pi.add_apply]
  ring
  repeat fun_prop

/-- Second derivatives distribute coordinate-wise over subtraction (two components f
/
lemma deriv_coord_2nd_sub (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : ContDiff ℝ 2
    ∂[u] (fun x => ∂[v] (fun x => f x w) x - ∂[w] (fun x => f x v) x) =
    (∂[u] (∂[v] (fun x => f x w))) - (∂[u] (∂[w] (fun x => f x v))) := by
  repeat rw [deriv_eq_fderiv_fun]
  ext x
  simp only [Pi.sub_apply]
  rw [fderiv_fun_sub]
  simp only [ContinuousLinearMap.coe_sub', Pi.sub_apply]
  repeat fun_prop

/-!

### A.6. The div of a curl is zero

-/

lemma div_of_curl_eq_zero (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : ContDiff ℝ 2
    ∇ ⬝ (∇ × f) = 0 := by
  unfold div curl Finset.sum
  ext x
  simp only [Fin.isValue, Fin.univ_val_map, List.ofFn_succ, Fin.succ_zero_eq_one,
    Fin.succ_one_eq_two, List.ofFn_zero, Multiset.sum_coe, List.sum_cons, List.sum_n
    Pi.zero_apply]
  rw [deriv_coord_2nd_sub, deriv_coord_2nd_sub, deriv_coord_2nd_sub]
  simp only [Fin.isValue, Pi.sub_apply]
  rw [deriv_commute fun x => f x 0, deriv_commute fun x => f x 1,
    deriv_commute fun x => f x 2]
  simp only [Fin.isValue, sub_add_sub_cancel', sub_self]
  repeat
    try apply contDiff_euclidean.mp
    exact hf

/-!

### A.7. The curl of a grad is zero

-/
```

```
lemma curl_of_grad_eq_zero (f : Space → ℝ) (hf : ContDiff ℝ 2 f) :
    ∇ × (∇ f) = 0 := by
  unfold curl grad
  ext x i
  fin_cases i <;>
  simp only [Fin.isValue, Pi.ofNat_apply, Fin.zero_eta, PiLp.zero_apply] <;
  · rw [deriv_commute]
    simp only [Fin.isValue, sub_self]
    · exact hf

/-!

### A.8. The curl of a curl

-/

lemma curl_of_curl (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : ContDiff ℝ
    ∇ × (∇ × f) = ∇ (∇ ⬝ f) - Δ f := by
  unfold laplacianVec laplacian div grad curl Finset.sum
  simp only [Fin.isValue, Fin.univ_val_map, List.ofFn_succ, Fin.succ_zero_e
    Fin.succ_one_eq_two, List.ofFn_zero, Multiset.sum_coe, List.sum_cons, L
  ext x i
  fin_cases i <;>
  · simp only [Fin.isValue, Fin.reduceFinMk, Pi.sub_apply]
    rw [deriv_coord_2nd_sub, deriv_coord_2nd_sub]
    simp only [Fin.isValue, Pi.sub_apply, PiLp.sub_apply]
    rw [deriv_coord_2nd_add]
    rw [deriv_commute fun x => f x 0, deriv_commute fun x => f x 1,
      deriv_commute fun x => f x 2]
    simp only [Fin.isValue, Pi.add_apply]
    ring
    repeat
      try apply contDiff_euclidean.mp
      exact hf

/-!

### A.9. A divergence-free field is a curl

We now prove that if the divergence of a function is zero,
then it is equal to the curl of some function.

This proof is involved, thus we have split it up into private definitions a
The proof is by explicit construction. If `f` has divergence zero,
then we construct
```

```
∫ t in 0..1, (t • x) × f (t • x)
```
and show that the curl of this is equal to `f`.

We call the integrand of this function the `homotopyOperatorIntegrand`.
We build API around this to help in our proof.

-/
```
open Matrix MeasureTheory

/-- The homotopy operator is defined as `∫ t in 0..1, (t • x) × f (t • x)`.
  This is the integrand of that function. -/
private noncomputable def homotopyOperatorIntegrand (f : Space → EuclideanSpace ℝ (F
  Space → ℝ → EuclideanSpace ℝ (Fin 3) := fun x t => (t • basis.repr x) ×₃ f (t • x

private lemma homotopyOperatorIntegrand_eq (f : Space → EuclideanSpace ℝ (Fin 3)) :
    homotopyOperatorIntegrand f = fun x t => (t • basis.repr x) ×₃ f (t • x) := by
@[fun_prop]
private lemma differentiable_homotopyOperatorIntegrand_space {f : Space → EuclideanS
    (hf : Differentiable ℝ f) (t : ℝ) :
    Differentiable ℝ (homotopyOperatorIntegrand f · t) := by
  simp [homotopyOperatorIntegrand]
  refine differentiable_euclidean.mpr ?_
  intro i
  fin_cases i
  all_goals
  · simp [crossProduct]
    fun_prop

@[fun_prop]
private lemma homotopyOperatorIntegrand_continuous_param {f : Space → EuclideanSpace
    (hf : Differentiable ℝ f) (x : Space) : Continuous (homotopyOperatorIntegrand f
  simp [homotopyOperatorIntegrand]
  have hf (i : Fin 3) : Continuous (fun t => f ((t : ℝ) • x) i) := by
    change Continuous (EuclideanSpace.proj i ∘ f ∘ fun t => t • x)
    apply Continuous.comp (by fun_prop)
    apply Continuous.comp ?_ ?_
    · exact hf.continuous
    · fun_prop
  refine Continuous.comp' (by fun_prop) ?_
  refine continuous_pi ?_
  intro i
  fin_cases i
  all_goals
  · simp [crossProduct]
```

```
    refine Continuous.fun_mul (by fun_prop) ?_
    refine Continuous.sub ?_ ?_
    all_goals
    · apply Continuous.mul ?_ ?_
      · fun_prop
      · exact hf _

private lemma intervalIntegrable_homotopyOperatorIntegrand {f : Space → Euc
    (hf : Differentiable ℝ f) (x : Space) :
    IntervalIntegrable (homotopyOperatorIntegrand f x ·) volume (0 : ℝ) 1 :
  apply Continuous.intervalIntegrable
  fun_prop

private lemma fderiv_homotopyOperatorIntegrand_eq_fderiv_crossProduct
    {f : Space → EuclideanSpace ℝ (Fin 3)}
    (hf : Differentiable ℝ f) (x : Space) (t : ℝ) (y : Space) (i : Fin 3) :
    fderiv ℝ (homotopyOperatorIntegrand f · t) x y i =
    t • (fderiv ℝ (fun x => (WithLp.toLp 2 ((crossProduct (basis.repr x).of
      (f (t • x)).ofLp)).ofLp i) x) y := by
  have cross_diff (t : ℝ) : Differentiable ℝ (fun x => WithLp.toLp 2
        ((crossProduct (basis.repr x).ofLp) (f (t • x)).ofLp)) := by
    refine differentiable_euclidean.mpr ?_
    intro i
    fin_cases i
    all_goals
    · simp [crossProduct]
      fun_prop
  conv_lhs =>
    simp [homotopyOperatorIntegrand]
    rw [fderiv_fun_smul (by fun_prop) (Differentiable.differentiableAt (cro
    simp
  change t • (fderiv ℝ (fun x => WithLp.toLp 2
    ((crossProduct (basis.repr x).ofLp) (f (t • x)).ofLp)) x) y _ = _
  trans t • ((fderiv ℝ (fun x => WithLp.toLp 2
    ((crossProduct (basis.repr x).ofLp) (f (t • x)).ofLp) i) x) y)
  · change _ = t • (fderiv ℝ (EuclideanSpace.proj i ∘
      (fun x => (WithLp.toLp 2 ((crossProduct (basis.repr x).ofLp) (f (t •
    rw [fderiv_comp]
    simp only [ContinuousLinearMap.fderiv, ContinuousLinearMap.coe_comp', F
      PiLp.proj_apply]
    · fun_prop
    · exact Differentiable.differentiableAt (cross_diff t)
  rfl

private lemma fderiv_homotopyOperatorIntegrand_apply_eq {f : Space → Euclid
    (hf : Differentiable ℝ f) (x : Space) (t : ℝ) (y : Space) (i : Fin 3) :
```

```
    fderiv ℝ (homotopyOperatorIntegrand f · t) x y i =
      t * (x (i+1) * t * fderiv ℝ f (t • x) y (i-1) + f (t • x) (i-
1) * y (i+1) -
      (x (i-1) * t * fderiv ℝ f (t • x) y (i+1) + f (t • x) (i+1) * y (i-
1))) := by
  have fderiv_f (x : Space) (t : ℝ) (y : Space)
      (i : Fin 3) : (fderiv ℝ (fun x => (f (t • x)).ofLp i) x) y = t * fderiv ℝ f (t
    change (fderiv ℝ (EuclideanSpace.proj i ∘ f ∘ fun x => t • x) x) y = _
    rw [fderiv_comp _ (by fun_prop) (by fun_prop),
      fderiv_comp _ (by fun_prop) (by fun_prop), fderiv_fun_smul (by fun_prop) (by f
    simp only [Function.comp_apply, ContinuousLinearMap.fderiv, fderiv_id', fderiv_f
      Pi.zero_apply, ContinuousLinearMap.zero_smulRight, add_zero, ContinuousLinearM
      ContinuousLinearMap.coe_smul', ContinuousLinearMap.coe_id', Pi.smul_apply, id_
      PiLp.proj_apply, smul_eq_mul]
  fin_cases i
  all_goals
    have : (-1 : Fin 3) = 2 := by rfl
    try rw [this]
    rw [fderiv_homotopyOperatorIntegrand_eq_fderiv_crossProduct]
    simp [crossProduct]
    rw [fderiv_fun_sub (by fun_prop) (by fun_prop), fderiv_fun_mul (by fun_prop) (by
      fderiv_fun_mul (by fun_prop) (by fun_prop)]
    simp [fderiv_f, this]
    ring_nf
    simp only [true_or]
    exact hf

private lemma continuous_uncurry_fderiv_homotopyOperatorIntegrand
    {f : Space → EuclideanSpace ℝ (Fin 3)} (hf : ContDiff ℝ 1 f) :
    Continuous (Function.uncurry (fun x t => fderiv ℝ (homotopyOperatorIntegrand f ·
  refine continuous_clm_apply.mpr ?_
  intro y
  suffices h1 : Continuous ((PiLp.continuousLinearEquiv 2 ℝ _).symm ∘
    (PiLp.continuousLinearEquiv 2 ℝ _) ∘
    (fun p : Space × ℝ => fderiv ℝ (homotopyOperatorIntegrand f · p.2) p.1 y)) by
    simpa using h1
  apply Continuous.comp (by fun_prop) ?_
  apply continuous_pi
  intro i
  change Continuous (fun p : Space × ℝ => fderiv ℝ (homotopyOperatorIntegrand f · p.
  have hf := hf.differentiable (by simp)
  fin_cases i
  all_goals
  · simp [fderiv_homotopyOperatorIntegrand_apply_eq hf]
    fun_prop
```

```
private lemma hasFDerivAt_intervalIntegral_homotopyOperatorIntegrand
    {f : Space → EuclideanSpace ℝ (Fin 3)} (hf : ContDiff ℝ 1 f) (x₀ : Spac
    HasFDerivAt (fun (x : Space) => ∫ (t : ℝ) in 0..1, homotopyOperatorInte
      (∫ (t : ℝ) in 0..1, fderiv ℝ (homotopyOperatorIntegrand f · t) x₀ ∂(v
  let F : Space → ℝ → EuclideanSpace ℝ (Fin 3) := homotopyOperatorIntegrand
  let F' : Space → ℝ → Space →L[ℝ] EuclideanSpace ℝ (Fin 3) := fun x t => f
  let s (x₀) : Set (Space) := Metric.closedBall x₀ 1
  obtain ⟨a, ha⟩ := IsCompact.exists_isMaxOn (s := s x₀ ×ˢ Set.Icc (0 : ℝ)
      (by exact (isCompact_closedBall x₀ 1).prod <|
        ConditionallyCompleteLinearOrder.isCompact_Icc 0 1)
      (f := fun (a : Space × ℝ) => ‖F' a.1 a.2‖)
      (by simp [s])
      (by
        apply (Continuous.comp ?_
          (continuous_uncurry_fderiv_homotopyOperatorIntegrand hf)).continu
        change Continuous (@norm _ ContinuousLinearMap.toSeminormedAddCommG
        fun_prop)
  change HasFDerivAt (fun (x : Space) => ∫ (t : ℝ) in 0..1, F x t ∂(volume)
      (∫ (t : ℝ) in 0..1, F' x₀ t ∂(volume)) x₀
  have hx :=hf.differentiable (by simp)
  apply intervalIntegral.hasFDerivAt_integral_of_dominated_of_fderiv_le (s
    (bound := fun t => ‖F' a.1 a.2‖)
  · exact Metric.closedBall_mem_nhds x₀ (by simp)
  · filter_upwards with x
    simp [F, homotopyOperatorIntegrand_eq]
    fun_prop
  · exact intervalIntegrable_homotopyOperatorIntegrand (hf.differentiable (
  · refine IntervalIntegrable.aestronglyMeasurable' ?_
    simp only [zero_le_one, sup_of_le_right, inf_of_le_left]
    apply Continuous.intervalIntegrable
    exact Continuous.uncurry_left x₀ (continuous_uncurry_fderiv_homotopyOpe
  · filter_upwards with t
    intro h x hx
    have hx2 := ha.2
    rw [isMaxOn_iff] at hx2
    apply hx2 (x, t)
    simp at h
    grind
  · apply Continuous.intervalIntegrable
    fun_prop
  · filter_upwards with t
    intro h x hx
    dsimp [F', F]
    apply (differentiable_homotopyOperatorIntegrand_space _ _).differentiab
    exact hf.differentiable (by simp)
```

```
private lemma deriv_intervalIntegral_homotopyOperatorIntegrand_sub
    (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : ContDiff ℝ 1 f) (i j k m : Fin 3) (
    ∂[i] (fun x => (∫ (t : ℝ) in 0..1, homotopyOperatorIntegrand f x t ∂(volume)) k)
    - ∂[j] (fun x => (∫ (t : ℝ) in 0..1, homotopyOperatorIntegrand f x t ∂(volume))
    ∫ (t : ℝ) in 0..1, (fderiv ℝ (homotopyOperatorIntegrand f · t) x₀ (basis i) k -
    fderiv ℝ (homotopyOperatorIntegrand f · t) x₀ (basis j) m) ∂(volume) := by
  let F : Space → ℝ → EuclideanSpace ℝ (Fin 3) := homotopyOperatorIntegrand f
  let F' : Space → ℝ → Space →L[ℝ] EuclideanSpace ℝ (Fin 3) := fun x t => fderiv ℝ (
  have F'_continuous : Continuous (Function.uncurry F') := by
    exact continuous_uncurry_fderiv_homotopyOperatorIntegrand (hf)
  have hfderiv (x₀ : Space) : HasFDerivAt (fun (x : Space) => ∫ (t : ℝ) in 0..1, F x
      (∫ (t : ℝ) in 0..1, F' x₀ t ∂(volume)) x₀ := by
    exact hasFDerivAt_intervalIntegral_homotopyOperatorIntegrand (hf) x₀
  have F'_apply_apply (x₀ : Space) (y : Space) (i : Fin 3) :
      ((∫ (t : ℝ) in 0..1, F' x₀ t) y).ofLp i =
      (∫ (t : ℝ) in 0..1, F' x₀ t y i) := by
    trans ((∫ (t : ℝ) in 0..1, F' x₀ t y)).ofLp i
    · rw [ContinuousLinearMap.intervalIntegral_apply]
      apply Continuous.intervalIntegrable
      fun_prop
    · rw [intervalIntegral.integral_of_le (by simp), intervalIntegral.integral_of_le
      rw [MeasureTheory.eval_integral_piLp]
      intro i
      apply MeasureTheory.IntegrableOn.integrable
      rw [← intervalIntegrable_iff_integrableOn_Ioc_of_le]
      apply Continuous.intervalIntegrable
      fun_prop
      simp
  repeat rw [Space.deriv_euclid, Space.deriv, (hfderiv x₀).fderiv]
  rotate_left
  · exact fun x => (hfderiv x).differentiableAt
  · exact fun x => (hfderiv x).differentiableAt
  rw [F'_apply_apply, F'_apply_apply, ← intervalIntegral.integral_sub]
  · apply Continuous.intervalIntegrable
    fun_prop
  · apply Continuous.intervalIntegrable
    fun_prop

lemma exists_curl_of_div_zero (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : ContDiff
    (hdiv : ∇ ⬝ f = 0) :
    ∃ g: Space → EuclideanSpace ℝ (Fin 3), f = curl g ∧ Differentiable ℝ g := by
  suffices hneg : ∃ g: Space → EuclideanSpace ℝ (Fin 3), f = - curl g ∧ Differentiab
    obtain ⟨g, hcurl, hdifferentiable⟩ := hneg
    use -g
    subst f
    simp_all
```

```
    rw [curl_neg]
    fun_prop
  have f_differentiable : Differentiable ℝ f := hf.differentiable (by simp)
  have fderiv_f_t (x : Space) (t : ℝ)
      (i : Fin 3) : (fderiv ℝ (fun t => (f (t • x)).ofLp i) t) 1 = fderiv ℝ
    change (fderiv ℝ (EuclideanSpace.proj i ∘ f ∘ fun (t : ℝ) => t • x) t)
    rw [fderiv_comp _ (by fun_prop) (by fun_prop), fderiv_comp _ (by fun_pr
      fderiv_fun_smul (by fun_prop) (by fun_prop)]
    simp only [Function.comp_apply, ContinuousLinearMap.fderiv, fderiv_fun_
      fderiv_id', ContinuousLinearMap.coe_comp', ContinuousLinearMap.add_ap
      ContinuousLinearMap.coe_smul', Pi.smul_apply, ContinuousLinearMap.zer
      ContinuousLinearMap.smulRight_apply, ContinuousLinearMap.coe_id', id_
      PiLp.proj_apply]
  have hi (x : Space) (i : Fin 3) : ∫ (t : ℝ) in 0..1, (t * f (t • x) i * 2
        t * (- fderiv ℝ f (t • x) (t • x)) i ∂(volume) = f x i := by
    trans ∫ (t : ℝ) in 0..1, fderiv ℝ (fun t => t ^ 2 * f (t • x) i) t 1 ∂(
    · congr
      funext t
      rw [fderiv_fun_mul (by fun_prop) (by fun_prop)]
      simp [fderiv_f_t]
      ring
    simp only [fderiv_eq_smul_deriv, smul_eq_mul, one_mul]
    rw [intervalIntegral.integral_deriv_eq_sub (by fun_prop)]
    simp only [one_pow, one_smul, one_mul, ne_eq, OfNat.ofNat_ne_zero, not_
      zero_smul, zero_mul, sub_zero]
    · apply Continuous.intervalIntegrable
      fun_prop
  use fun x => ∫ (t : ℝ) in 0..1, homotopyOperatorIntegrand f x t ∂(volume)
  apply And.intro
  swap
  · intro x
    exact (hasFDerivAt_intervalIntegral_homotopyOperatorIntegrand (hf) _).d
  ext x i
  have : (-1 : Fin 3) = 2 := by rfl
  fin_cases i <;> symm
  all_goals
    simp [curl]
    rw [deriv_intervalIntegral_homotopyOperatorIntegrand_sub _ hf]
    simp [fderiv_homotopyOperatorIntegrand_apply_eq (hf.differentiable (by
    rw [← hi]
    congr
    funext t
    have hdiv : div f (t • x) = 0 := by simp [hdiv]
    rw [div_eq_sum_fderiv _ (by fun_prop)] at hdiv
    simp [Fin.sum_univ_three] at hdiv
    have hx : x = ∑ i, basis.repr x i • basis i :=
```

```
      Eq.symm (OrthonormalBasis.sum_repr basis x)
    conv_rhs =>
      enter [2, 2,1, 1, 2]
      rw [hx]
  · linear_combination (norm := {simp [Fin.sum_univ_three]; ring}) - t ^ 2 * x 0 * h
  · linear_combination (norm := {simp [Fin.sum_univ_three, this]; ring}) - t ^ 2 * x
  · linear_combination (norm := {simp [Fin.sum_univ_three, this]; ring}) - t ^ 2 * x

/-!

### A.10. A curl-free field is a gradient

We show that if the curl of a function is zero, then it is equal to the gradient of
The key lemma here is
`Convex.exists_forall_hasFDerivAt_of_fderiv_symmetric`, which is
a more general version of this result in Mathlib. We turn this here into something
more familiar, to the physicists via `deriv_comm_of_curl_zero`.

-/

lemma deriv_comm_of_curl_zero (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : Different
    (hcurl : curl f = 0) (x : Space) (i j : Fin 3) :
    ∂[i] f x j = ∂[j] f x i := by
  fin_cases i <;> fin_cases j
  any_goals rfl
  all_goals
    simp only [Fin.reduceFinMk, Fin.isValue, Fin.zero_eta]
    rw [← deriv_euclid (by fun_prop), ← deriv_euclid (by fun_prop)]
    have hcurl' (i : Fin 3) : curl f x i = 0 := by simp [hcurl]
  · specialize hcurl' 2
    simp [curl] at hcurl'
    linear_combination (norm := ring_nf) hcurl'
  · specialize hcurl' 1
    simp [curl] at hcurl'
    linear_combination (norm := ring_nf) -hcurl'
  · specialize hcurl' 2
    simp [curl] at hcurl'
    linear_combination (norm := ring_nf) -hcurl'
  · specialize hcurl' 0
    simp [curl] at hcurl'
    linear_combination (norm := ring_nf) hcurl'
  · specialize hcurl' 1
    simp [curl] at hcurl'
    linear_combination (norm := ring_nf) hcurl'
  · specialize hcurl' 0
    simp [curl] at hcurl'
```

```
    linear_combination (norm := ring_nf) -hcurl'

open InnerProductSpace
lemma exists_grad_of_curl_zero (f : Space → EuclideanSpace ℝ (Fin 3)) (hf :
    (hcurl : curl f = 0) : ∃ g : Space → ℝ, f = grad g ∧ Differentiable ℝ g
  let s : Set (Space) := Set.univ
  have hs : Convex ℝ s := convex_univ
  have hso : IsOpen s := isOpen_univ
  let ω : Space → Space →L[ℝ] ℝ:= fun a => InnerProductSpace.toDual ℝ _ (ba
  have hω : DifferentiableOn ℝ ω s := by
    simp [ω]
    refine differentiableOn_univ.mpr ?_
    apply (InnerProductSpace.toDual ℝ (Space)).differentiable.fun_comp
    fun_prop
  have hω_fderiv (a x y : Space) : fderiv ℝ ω a x y =
      □(basis.repr.symm (fderiv ℝ f a x)), y□_ℝ:= by
    calc _
      _ = (fderiv ℝ (InnerProductSpace.toDual ℝ _ ∘
        fun a => (basis.repr.symm (f a))) a x) y := by rfl
    rw [fderiv_comp _ (by simpa using
      (InnerProductSpace.toDual ℝ (Space)).differentiable.differentiableAt)
    erw [(InnerProductSpace.toDual ℝ (Space)).fderiv]
    simp only [ContinuousLinearMap.coe_comp', ContinuousLinearEquiv.coe_coe
      LinearIsometryEquiv.coe_toContinuousLinearEquiv, Function.comp_apply]
    erw [InnerProductSpace.toDual_apply_apply]
    rw [fderiv_comp' _ (by fun_prop) (by fun_prop)]
    simp
  have hdω: ∀ a ∈ s, ∀ (x y : Space), ((fderiv ℝ ω a) x) y = ((fderiv ℝ ω a
    intro a ha x y
    rw [hω_fderiv, hω_fderiv]
    induction' y using basis_induction_on with i p1 p2 h1 h2 c p1 h1
    rotate_left
    · simp
    · simp [inner_add_left, inner_add_right, h1, h2]
    · simp [inner_smul_left, inner_smul_right, h1]
    induction' x using basis_induction_on with j p1 p2 h1 h2 c p1 h1
    rotate_left
    · simp
    · simp [inner_add_right, ← h1,← h2]
    · simp [inner_smul_right, ← h1]
    simp only [inner_basis, basis_repr_symm_apply]
    rw [← deriv_eq, ← deriv_eq]
    exact deriv_comm_of_curl_zero f hf hcurl a j i
  obtain ⟨g, hg⟩ := Convex.exists_forall_hasFDerivAt_of_fderiv_symmetric hs
  use g
  simp [ω, ← hasGradientAt_iff_hasFDerivAt, s] at hg
```

```
  apply And.intro _ (fun x => (hg x).differentiableAt)
  ext1 x
  specialize hg x
  simpa using (HasGradientAt.unique (hg.differentiableAt.hasGradientAt_grad x) hg).s

/-!

## B. The curl on distributions

-/

open MeasureTheory SchwartzMap InnerProductSpace Distribution

/-- The curl of a distribution `Space →d[ℝ] (EuclideanSpace ℝ (Fin 3))` as a distrib
  `Space →d[ℝ] (EuclideanSpace ℝ (Fin 3))`. -/
noncomputable def distCurl : (Space →d[ℝ] (EuclideanSpace ℝ (Fin 3))) →ₗ[ℝ]
    (Space) →d[ℝ] (EuclideanSpace ℝ (Fin 3)) where
  toFun f :=
    let curl : (Space →L[ℝ] (EuclideanSpace ℝ (Fin 3))) →L[ℝ] (EuclideanSpace ℝ (Fin
      toFun dfdx:= WithLp.toLp 2 fun i =>
        match i with
        | 0 => dfdx (basis 2) 1 - dfdx (basis 1) 2
        | 1 => dfdx (basis 0) 2 - dfdx (basis 2) 0
        | 2 => dfdx (basis 1) 0 - dfdx (basis 0) 1
      map_add' v1 v2 := by
        ext i
        fin_cases i
        all_goals
          simp only [Fin.isValue, ContinuousLinearMap.add_apply, PiLp.add_apply, Fin
          ring
      map_smul' a v := by
        ext i
        fin_cases i
        all_goals
          simp only [Fin.isValue, ContinuousLinearMap.coe_smul', Pi.smul_apply, PiLp
            smul_eq_mul, RingHom.id_apply, Fin.reduceFinMk]
          ring
      cont := by
        apply Continuous.comp
        · fun_prop
        rw [continuous_pi_iff]
        intro i
        fin_cases i
        all_goals
          fun_prop
        }
```

```
    curl.comp (Distribution.fderivD ℝ f)
  map_add' f1 f2 := by
    ext x
    simp
  map_smul' a f := by
    ext x
    simp

/-!

### B.1. The components of the curl

-/

lemma distCurl_apply_zero (f : Space →d[ℝ] (EuclideanSpace ℝ (Fin 3))) (η :
    distCurl f η 0 = - f (SchwartzMap.evalCLM ℝ Space ℝ (basis 2) (fderivCL
    + f (SchwartzMap.evalCLM ℝ Space ℝ (basis 1) (fderivCLM ℝ Space ℝ η)) 2
  simp [distCurl]
  rw [fderivD_apply, fderivD_apply]
  simp

lemma distCurl_apply_one (f : Space →d[ℝ] (EuclideanSpace ℝ (Fin 3))) (η :
    distCurl f η 1 = - f (SchwartzMap.evalCLM ℝ Space ℝ (basis 0) (fderivCL
    + f (SchwartzMap.evalCLM ℝ Space ℝ (basis 2) (fderivCLM ℝ Space ℝ η)) 0
  simp [distCurl]
  rw [fderivD_apply, fderivD_apply]
  simp

lemma distCurl_apply_two (f : Space →d[ℝ] (EuclideanSpace ℝ (Fin 3))) (η :
    distCurl f η 2 = - f (SchwartzMap.evalCLM ℝ Space ℝ (basis 1) (fderivCL
    + f (SchwartzMap.evalCLM ℝ Space ℝ (basis 0) (fderivCLM ℝ Space ℝ η)) 1
  simp [distCurl]
  rw [fderivD_apply, fderivD_apply]
  simp

/-!

### B.2. Basic equalities

-/

lemma distCurl_apply (f : Space →d[ℝ] (EuclideanSpace ℝ (Fin 3))) (η : □(Sp
    distCurl f η = WithLp.toLp 2 fun
    | 0 => - f (SchwartzMap.evalCLM ℝ Space ℝ (basis 2) (fderivCLM ℝ Space
      + f (SchwartzMap.evalCLM ℝ Space ℝ (basis 1) (fderivCLM ℝ Space ℝ η))
    | 1 => - f (SchwartzMap.evalCLM ℝ Space ℝ (basis 0) (fderivCLM ℝ Space
```

```
      + f (SchwartzMap.evalCLM ℝ Space ℝ (basis 2) (fderivCLM ℝ Space ℝ η)) 0
    | 2 => - f (SchwartzMap.evalCLM ℝ Space ℝ (basis 1) (fderivCLM ℝ Space ℝ η)) 0
      + f (SchwartzMap.evalCLM ℝ Space ℝ (basis 0) (fderivCLM ℝ Space ℝ η)) 1 := by
  ext i
  fin_cases i
  · simp [distCurl_apply_zero]
  · simp [distCurl_apply_one]
  · simp [distCurl_apply_two]

/-!

### B.3. The curl of a grad is zero

-/

/-- The curl of a grad is equal to zero. -/
@[simp]
lemma distCurl_distGrad_eq_zero (f : (Space) →d[ℝ] ℝ) :
    distCurl (distGrad f) = 0 := by
  ext η i
  fin_cases i
  all_goals
  · dsimp
    try rw [distCurl_apply_zero]
    try rw [distCurl_apply_one]
    try rw [distCurl_apply_two]
    rw [distGrad_eq_sum_basis, distGrad_eq_sum_basis]
    simp [Pi.single_apply]
    rw [← map_neg, ← map_add, ← ContinuousLinearMap.map_zero f]
    congr
    ext x
    simp only [Fin.isValue, SchwartzMap.add_apply, SchwartzMap.neg_apply, SchwartzMa
    rw [schwartMap_fderiv_comm]
    change ((SchwartzMap.evalCLM ℝ Space ℝ _)
      ((fderivCLM ℝ Space ℝ) ((SchwartzMap.evalCLM ℝ Space ℝ _) ((fderivCLM ℝ Space
      - ((SchwartzMap.evalCLM ℝ Space ℝ _)
        ((fderivCLM ℝ Space ℝ) ((SchwartzMap.evalCLM ℝ Space ℝ _) ((fderivCLM ℝ Spac
    simp

end Space
```

1.343 Physlib/SpaceAndTime/Space/Derivatives/Div.lean

```
public import Physlib.SpaceAndTime.Space.Derivatives.Grad
```

```
public import Physlib.SpaceAndTime.Space.DistOfFunction
open Physlib

lemma div_eq_sum_fderiv {d} (f : Space d → EuclideanSpace ℝ (Fin d))
    (hf : Differentiable ℝ f) (x : Space d) :
    (∇ ⬝ f) x = ∑ i, fderiv ℝ f x (basis i) i := by
  simp [div, Space.deriv]
  congr
  funext i
  rw [← Space.deriv_eq]
  rw [deriv_euclid]
  rfl
  fun_prop

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.344 Physlib/SpaceAndTime/Space/Derivatives/Grad.lean

```
public import Physlib.SpaceAndTime.Space.Derivatives.Basic
public import Physlib.SpaceAndTime.Space.IsDistBounded
public import Mathlib.Analysis.Calculus.Gradient.Basic
open Physlib

set_option backward.isDefEq.respectTransparency false in
lemma gradient_apply_eq_grad {d} (f : Space d → ℝ) (x : Space d) :
    gradient f x = basis.repr.symm (∇ f x) := by
  rw [grad_eq_gradiant f]
  simp

set_option backward.isDefEq.respectTransparency false in
lemma _root_.DifferentiableAt.hasGradientAt_grad {d} {f : Space d → ℝ} (x :
    (hf : DifferentiableAt ℝ f x) :
    HasGradientAt f (basis.repr.symm (∇ f x)) x := by
  rw [← gradient_apply_eq_grad]
  exact DifferentiableAt.hasGradientAt hf

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    PiLp.ofLp_single, Finset.sum_apply, Pi.smul_apply, Pi.single_apply, smu
lemma gradSchwartz_apply_eq_grad {d} (η : 𝓢(Space d, ℝ)) (x : Space d) :
```

1.345 Physlib/SpaceAndTime/Space/Derivatives/Iterated.lean

```
/-
Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Juan Jose Fernandez Morales
-/
module

public import Physlib.SpaceAndTime.Space.Derivatives.Basic
public import Physlib.SpaceAndTime.Space.Derivatives.MultiIndex
/-!
# Iterated derivatives on `Space d`

## i. Overview

This module defines iterated coordinate derivatives on `Space d` indexed by multi-
indices.

The implementation is intentionally modest. A multi-index is first expanded into a c
of coordinate directions, and the iterated derivative is then defined by repeated ap
`Space.deriv` along that list.

## ii. Key results

- `Space.iteratedDeriv` : iterated coordinate derivatives on `Space d`.
- `∂^[I] f` : notation for the iterated derivative indexed by the multi-
index `I`.
- `Space.iteratedDeriv_add`, `Space.iteratedDeriv_const_smul` :
  algebraic compatibility for smooth scalar-valued functions.
- `Space.iteratedDeriv_contDiff` : smooth scalar-valued functions remain smooth afte
  iterated coordinate differentiation.
- `Space.tsupport_iteratedDeriv_subset` :
  the support of an iterated spatial derivative is contained in that of the original

## iii. Table of contents

- A. Iterated derivatives on `Space d`
- B. Algebraic and regularity lemmas
- C. Support lemmas

## iv. References

-/

@[expose] public section
```

```
namespace Space

open Physlib
open scoped ContDiff

variable {M : Type} {d : ℕ}

/-!
## A. Iterated derivatives on `Space d`

-/

/-- The iterated coordinate derivative on `Space d` indexed by a multi-
index. -/
noncomputable def iteratedDeriv [AddCommGroup M] [Module ℝ M] [TopologicalS
    (I : MultiIndex d) (f : Space d → M) : Space d → M :=
  I.toList.foldr (fun i g => deriv i g) f

@[inherit_doc iteratedDeriv]
macro "∂^[" I:term "]" : term => `(iteratedDeriv $I)

private lemma iteratedDerivList_contDiff (L : List (Fin d)) {f : Space d →
    (hf : ContDiff ℝ ∞ f) :
    ContDiff ℝ ∞ (L.foldr (fun i g => deriv i g) f) := by
  induction L generalizing f with
  | nil =>
      simpa using hf
  | cons i L ih =>
      have htail : ContDiff ℝ ∞ (L.foldr (fun j g => deriv j g) f) := ih hf
      have hfamily : ContDiff ℝ ∞
          (fun x : Space d => fun j : Fin d => deriv j (L.foldr (fun j g =>
        simpa using (Space.deriv_contDiff (n := ∞) htail)
      have hfixed : ContDiff ℝ ∞
          (fun x => deriv i (L.foldr (fun j g => deriv j g) f) x) := by
        simpa using (contDiff_apply ℝ ℝ i).comp hfamily
      simpa using hfixed

private lemma iteratedDerivList_add (L : List (Fin d)) {f g : Space d → ℝ}
    (hf : ContDiff ℝ ∞ f) (hg : ContDiff ℝ ∞ g) :
    L.foldr (fun i h => deriv i h) (f + g) =
      L.foldr (fun i h => deriv i h) f + L.foldr (fun i h => deriv i h) g :
  induction L generalizing f g with
  | nil =>
      rfl
  | cons i L ih =>
```

```
      simp only [List.foldr]
      rw [ih hf hg]
      apply Space.deriv_add
      · exact (iteratedDerivList_contDiff L hf).differentiable (by simp)
      · exact (iteratedDerivList_contDiff L hg).differentiable (by simp)

private lemma iteratedDerivList_const_smul (L : List (Fin d)) (c : ℝ) {f : Space d →
    (hf : ContDiff ℝ ∞ f) :
    L.foldr (fun i h => deriv i h) (c • f) =
      c • L.foldr (fun i h => deriv i h) f := by
  induction L generalizing f with
  | nil =>
      rfl
  | cons i L ih =>
      simp only [List.foldr]
      rw [ih hf]
      apply Space.deriv_const_smul
      exact (iteratedDerivList_contDiff L hf).differentiable (by simp)

@[simp]
lemma iteratedDeriv_zero [AddCommGroup M] [Module ℝ M] [TopologicalSpace M]
    (f : Space d → M) : ∂^[0] f = f := by
  simp [iteratedDeriv, Physlib.MultiIndex.toList_zero]

@[simp]
lemma iteratedDeriv_increment_zero [AddCommGroup M] [Module ℝ M] [TopologicalSpace M
    (I : MultiIndex d.succ) (f : Space d.succ → M) :
    ∂^[MultiIndex.increment I 0] f = ∂[0] (∂^[I] f) := by
  simp [iteratedDeriv, Physlib.MultiIndex.toList_increment_zero]

@[simp]
lemma iteratedDeriv_single [AddCommGroup M] [Module ℝ M] [TopologicalSpace M]
    (i : Fin d) (f : Space d → M) :
    ∂^[MultiIndex.increment 0 i] f = ∂[i] f := by
  simp [iteratedDeriv, Physlib.MultiIndex.toList_single]

lemma iteratedDeriv_add (I : MultiIndex d) {f g : Space d → ℝ}
    (hf : ContDiff ℝ ∞ f) (hg : ContDiff ℝ ∞ g) :
    ∂^[I] (f + g) = ∂^[I] f + ∂^[I] g := by
  simpa [iteratedDeriv] using iteratedDerivList_add I.toList hf hg

lemma iteratedDeriv_const_smul (I : MultiIndex d) (c : ℝ) {f : Space d → ℝ}
    (hf : ContDiff ℝ ∞ f) :
    ∂^[I] (c • f) = c • ∂^[I] f := by
  simpa [iteratedDeriv] using iteratedDerivList_const_smul I.toList c hf
```

```
/-- Iterated spatial derivatives preserve smoothness for scalar-
valued functions. -/
lemma iteratedDeriv_contDiff (I : MultiIndex d) {f : Space d → ℝ}
    (hf : ContDiff ℝ ∞ f) :
    ContDiff ℝ ∞ (∂^[I] f) := by
  simpa [iteratedDeriv] using iteratedDerivList_contDiff I.toList hf

/-- The topological support of a spatial derivative is contained in that of
function. -/
lemma tsupport_deriv_subset (i : Fin d) {f : Space d → ℝ} :
    tsupport (deriv i f) ⊆ tsupport f := by
  simpa [deriv_eq_fderiv_fun] using
    (tsupport_fderiv_apply_subset (□ := ℝ) (f := fun x => f x) (v := basis

private lemma iteratedDerivList_commute_deriv (L : List (Fin d)) (i : Fin d
    {f : Space d → ℝ} (hf : ContDiff ℝ ∞ f) :
    L.foldr (fun j g => deriv j g) (deriv i f) =
      deriv i (L.foldr (fun j g => deriv j g) f) := by
  induction L generalizing f with
  | nil =>
      rfl
  | cons j L ih =>
      simp only [List.foldr]
      rw [ih hf]
      rw [Space.deriv_commute (u := j) (v := i)]
      have h2 : ContDiff ℝ (2 : ℕ∞) (L.foldr (fun j g => deriv j g) f) := b
        exact (iteratedDerivList_contDiff L hf).of_le (by
          exact WithTop.coe_le_coe.mpr le_top)
      exact h2

/-- An extra spatial derivative commutes with iterated spatial derivatives
scalar-valued functions. -/
lemma deriv_iteratedDeriv_commute (i : Fin d) (I : MultiIndex d) {f : Space
    (hf : ContDiff ℝ ∞ f) :
    deriv i (∂^[I] f) = ∂^[I] (deriv i f) := by
  symm
  simpa [iteratedDeriv] using iteratedDerivList_commute_deriv I.toList i hf

private lemma tsupport_iteratedDerivList_subset (L : List (Fin d)) {f : Spa
    tsupport (L.foldr (fun i g => deriv i g) f) ⊆ tsupport f := by
  induction L generalizing f with
  | nil =>
      simp
  | cons i L ih =>
      simpa [List.foldr] using
        (tsupport_deriv_subset i (f := L.foldr (fun j g => deriv j g) f)).t
```

```
/-- The topological support of an iterated spatial derivative is contained in that o
original function. -/
lemma tsupport_iteratedDeriv_subset (I : MultiIndex d) {f : Space d → ℝ} :
    tsupport (∂^[I] f) ⊆ tsupport f := by
  simpa [iteratedDeriv] using tsupport_iteratedDerivList_subset I.toList (f := f)

/-- An iterated spatial derivative vanishes outside the topological support of the o
function. -/
lemma iteratedDeriv_eq_zero_of_notMem_tsupport (I : MultiIndex d) {f : Space d → ℝ}
    (hx : x ∉ tsupport f) :
    ∂^[I] f x = 0 := by
  apply Function.notMem_support.mp
  intro hxI
  exact hx ((tsupport_iteratedDeriv_subset I) (subset_tsupport _ hxI))

end Space
```

1.346 Physlib/SpaceAndTime/Space/Derivatives/Laplacian.lean

```
public import Physlib.SpaceAndTime.Space.Derivatives.Div
```

1.347 Physlib/SpaceAndTime/Space/Derivatives/MultiIndex.lean

```
/-
Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Juan Jose Fernandez Morales
-/

module

public import Mathlib.Algebra.BigOperators.Fin

/-!
# Multi-indices

## i. Overview

This module defines the basic type of multi-indices used to index iterated partial d

A multi-index on `d` source coordinates is represented as a structure with an underl
`Fin d → ℕ`, together with the first basic operations needed later in the local Clas
```

```
Theory development.

## ii. Key results

- `Physlib.MultiIndex` : multi-indices on `d` coordinates.
- `MultiIndex.order` : the order `|I|` of a multi-index.
- `MultiIndex.increment` : increment a single coordinate of a multi-
index.
- `MultiIndex.toList` : the canonical ordered list of directions encoded by
index.

## iii. Table of contents

- A. The basic type of multi-indices
  - A.1. Basic operations
  - A.2. Basic lemmas
  - A.3. Canonical ordered lists of directions

## iv. References

-/

@[expose] public section

open scoped BigOperators

namespace Physlib

/-!
## A. The basic type of multi-indices

-/

/-- A multi-index on `d` source coordinates. -/
structure MultiIndex (d : ℕ) where
  /-- The coordinates of the multi-index. -/
  toFun : Fin d → ℕ
deriving DecidableEq

namespace MultiIndex

variable {d : ℕ}

instance : CoeFun (MultiIndex d) (fun _ => Fin d → ℕ) := ⟨MultiIndex.toFun⟩

instance : Zero (MultiIndex d) := ⟨⟨0⟩⟩
```

```
instance : Add (MultiIndex d) := ⟨fun I J => ⟨I.toFun + J.toFun⟩⟩

/-!
### A.1. Basic operations

-/

/-- The order `|I|` of a multi-index `I`, defined as the sum of its components. -
/
def order (I : MultiIndex d) : Nat := ∑ i, I i

/-- Increment the `i`-th coordinate of a multi-index by one. -
/
def increment (I : MultiIndex d) (i : Fin d) : MultiIndex d := ⟨I.toFun + Pi.single

/-!
### A.2. Basic lemmas

-/

@[ext]
lemma ext {I J : MultiIndex d} (h : ∀ i, I i = J i) : I = J := by
  cases I
  cases J
  simp only at h
  congr
  funext i
  exact h i

@[simp]
lemma zero_apply (i : Fin d) : (0 : MultiIndex d) i = 0 := rfl

@[simp]
lemma add_apply (I J : MultiIndex d) (i : Fin d) : (I + J) i = I i + J i := rfl

@[simp]
lemma increment_apply_same (I : MultiIndex d) (i : Fin d) :
    increment I i i = I i + 1 := by
  simp [increment]

@[simp]
lemma increment_apply_ne (I : MultiIndex d) {i j : Fin d} (h : j ≠ i) :
    increment I i j = I j := by
  simp [increment, Pi.single_eq_of_ne h]
```

```
@[simp]
lemma order_zero : order (0 : MultiIndex d) = 0 := by
  simp [order]

lemma order_add (I J : MultiIndex d) : order (I + J) = order I + order J :=
  simp [order, Finset.sum_add_distrib]

@[simp]
lemma order_single (i : Fin d) : order (⟨Pi.single i 1⟩ : MultiIndex d) = 1
  classical
  unfold order
  rw [Finset.sum_eq_single i]
  · simp
  · intro j _ hj
    simp [Pi.single_eq_of_ne hj]
  · intro hi
    simp at hi

@[simp]
lemma order_increment (I : MultiIndex d) (i : Fin d) :
    order (increment I i) = order I + 1 := by
  simp [increment, order, Finset.sum_add_distrib]

/-!
### A.3. Canonical ordered lists of directions

-/

/-- The tail of a multi-index on `d + 1` coordinates, dropping the `0`-
th coordinate. -/
def tail (I : MultiIndex d.succ) : MultiIndex d := ⟨fun i => I i.succ⟩

/-- The canonical ordered list of coordinate directions encoded by a multi-
index. -/
def toList : {d : ℕ} → MultiIndex d → List (Fin d)
  | 0, _ => []
  | _ + 1, I => List.replicate (I 0) 0 ++ (toList (tail I)).map Fin.succ

@[simp]
lemma tail_zero : tail (0 : MultiIndex d.succ) = 0 := by
  ext i
  rfl

@[simp]
lemma tail_increment_zero (I : MultiIndex d.succ) : tail (increment I 0) =
  ext i
```

```
  simp [tail, increment]

@[simp]
lemma tail_increment_succ (I : MultiIndex d.succ) (i : Fin d) :
    tail (increment I i.succ) = increment (tail I) i := by
  ext j
  by_cases h : j = i
  · subst h
    simp [tail, increment]
  · simp [tail, increment, h]

@[simp]
lemma toList_zero : toList (0 : MultiIndex d) = [] := by
  induction d with
  | zero => rfl
  | succ d ih =>
      simp [toList, ih]

lemma length_toList (I : MultiIndex d) : I.toList.length = I.order := by
  induction d with
  | zero =>
      simp [toList, MultiIndex.order]
  | succ d ih =>
      simp [toList, tail, MultiIndex.order, Fin.sum_univ_succ, ih]

@[simp]
lemma toList_increment_zero (I : MultiIndex d.succ) :
    toList (increment I 0) = 0 :: toList I := by
  simp only [toList, increment_apply_same, tail_increment_zero]
  rw [show I 0 + 1 = 1 + I 0 by omega, List.replicate_add]
  simp

@[simp]
lemma toList_single (i : Fin d) : toList (increment 0 i : MultiIndex d) = [i] := by
  induction d with
  | zero =>
      exact Fin.elim0 i
  | succ d ih =>
      refine Fin.cases ?_ ?_ i
      · simp [toList_increment_zero]
      · intro j
        have htail :
            tail (increment (0 : MultiIndex d.succ) j.succ) = increment (0 : MultiIn
          rw [tail_increment_succ, tail_zero]
        have hzero : increment (0 : MultiIndex d.succ) j.succ 0 = 0 := by
          simp [increment]
```

```
        simp [toList, hzero, htail, ih j]

end MultiIndex

end Physlib
```

1.348 Physlib/SpaceAndTime/Space/DistConst.lean

```
public import Physlib.SpaceAndTime.Space.Derivatives.Curl
open Physlib
```

1.349 Physlib/SpaceAndTime/Space/DistOfFunction.lean

```
public import Physlib.SpaceAndTime.Space.IsDistBounded
public import Physlib.Mathematics.Distribution.Basic

TODO "Add a general lemma specifying the derivative of
```

1.350 Physlib/SpaceAndTime/Space/Integrals/Basic.lean

```
public import Physlib.SpaceAndTime.Space.Module
public import Mathlib.MeasureTheory.Measure.Lebesgue.VolumeOfBalls
```

1.351 Physlib/SpaceAndTime/Space/Integrals/NormPow.lean

```
public import Physlib.SpaceAndTime.Space.Module
public import Mathlib.MeasureTheory.Constructions.HaarToSphere
public import Mathlib.Analysis.SpecialFunctions.ImproperIntegrals
set_option backward.isDefEq.respectTransparency false in
      npow_indicator_rpow_eq (Set.left_notMem_Ioo),
set_option backward.isDefEq.respectTransparency false in
```

1.352 Physlib/SpaceAndTime/Space/Integrals/RadialAngularMeasure

```
public import Physlib.SpaceAndTime.Space.Integrals.Basic

open NNReal Real
set_option backward.isDefEq.respectTransparency false in
```

1.353 Physlib/SpaceAndTime/Space/IsDistBounded.lean

```
public import Physlib.SpaceAndTime.Space.Integrals.RadialAngularMeasure
public import Physlib.SpaceAndTime.Time.Basic
public import Physlib.Relativity.Tensors.RealTensor.Vector.Basic
public import Mathlib.Analysis.Distribution.SchwartzSpace.Deriv

set_option backward.isDefEq.respectTransparency false in
        simp only [Prod.norm_mk, norm_zero, norm_nonneg, sup_of_le_right]
        ring_nf
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.354 Physlib/SpaceAndTime/Space/LengthUnit.lean

```
public import Mathlib.Analysis.SpecialFunctions.Trigonometric.Basic
public import Physlib.Meta.TODO.Basic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
TODO "For each unit of charge give the reference the literature where it's definitio
```

1.355 Physlib/SpaceAndTime/Space/Module.lean

```
public import Physlib.SpaceAndTime.Space.Basic
public import Mathlib.MeasureTheory.Measure.Haar.InnerProductSpace
public import Mathlib.Tactic.Cases
noncomputable instance {d} : SeminormedAddCommGroup (Space d) where
  dist_eq x y := by
    simp [dist_eq_norm, norm_eq]
    congr
    funext i
    ring
noncomputable instance : NormedAddCommGroup (Space d) where
  dist_eq x y := by
    simp [dist_eq_norm, norm_eq]
    congr
    funext i
    ring
set_option backward.isDefEq.respectTransparency false in
noncomputable instance {d : ℕ} : MeasurableSpace (Space d) := borel (Space d)
TODO "In the above documentation describe what an instance is, and why
```

```
TODO "In the above documentation describe the notion of a basis
lemma basis_induction_on {d} {P : Space d → Prop}
    (hb : ∀ i, P (basis i)) (hzero : P 0)
    (hadd : ∀ p1 p2, P p1 → P p2 → P (p1 + p2))
    (hsmul : ∀ (c : ℝ) p, P p → P (c • p)) (p : Space d) : P p := by
  rw [← OrthonormalBasis.sum_repr basis p]
  have hp_sum (s : Finset (Fin d)) (f : (Fin d) → Space d)
    (hi : ∀ i ∈ s, P (f i)) : P (∑ x ∈ s, f x) := by
    induction' s using Finset.induction with i s hi ih
    · simpa using hzero
    · rw [Finset.sum_insert]
      apply hadd
      · apply hi
        simp
      · apply ih
        intro i h'
        apply hi
        simp_all
      simp_all
  apply hp_sum
  intro i _
  apply hsmul
  apply hb
noncomputable def equivPi (d : ℕ) :
lemma mk_contDiff {d : ℕ} {n : WithTop ℕ∞}:
@[simp]
lemma fderiv_eval_apply {d : ℕ} (p y : Space d) (i : Fin d) :
    fderiv ℝ (fun p => p.val i) p y = y i := by
  trans fderiv ℝ (Space.coordCLM i) p y
  · congr
    funext i
    simp [Space.coordCLM, Space.coord_apply]
  simp only [ContinuousLinearMap.fderiv]
  simp [Space.coordCLM, Space.coord_apply]

    (μ : Fin d) (f : M → Space d) (hf : Differentiable ℝ f) (m dm : M) :
    fderiv ℝ f m dm μ = fderiv ℝ (fun m' => f m' μ) m dm := by
set_option backward.isDefEq.respectTransparency false in
  toFun p := p
  invFun p := p
  simp only [basis_apply, ContinuousLinearMap.fderiv, PiLp.proj_apply, PiLp
```

1.356 Physlib/SpaceAndTime/Space/Norm.lean

```
public import Physlib.SpaceAndTime.Space.Derivatives.Div

open SchwartzMap NNReal Physlib
set_option backward.isDefEq.respectTransparency false in
      · intro _ _
        apply Differentiable.differentiableAt
        fun_prop
set_option backward.isDefEq.respectTransparency false in
      · intro _ _
        apply Differentiable.differentiableAt
        fun_prop
set_option backward.isDefEq.respectTransparency false in
```

1.357 Physlib/SpaceAndTime/Space/Slice.lean

```
public import Physlib.SpaceAndTime.Space.Module
public import Mathlib.MeasureTheory.Constructions.Polish.Basic
- https://leanprover.zulipchat.com/#narrow/channel/479953-
Physlib/topic/API.20around.20.60Space.20.28d1.20.2B.20d2.29.60.20to.20.60Space.20d1.

open NNReal
```

1.358 Physlib/SpaceAndTime/Space/Translations.lean

```
public import Physlib.SpaceAndTime.Space.Derivatives.Div
open Physlib
```

1.359 Physlib/SpaceAndTime/SpaceTime/Basic.lean

```
public import Physlib.Relativity.SpeedOfLight
public import Physlib.Relativity.Tensors.RealTensor.Vector.Basic
public import Physlib.SpaceAndTime.Space.Integrals.Basic
public import Physlib.SpaceAndTime.Time.Basic
TODO "SpaceTime should be refactored into a structure, or similar, to prevent castin
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.360 Physlib/SpaceAndTime/SpaceTime/Boosts.lean

```
public import Physlib.Relativity.LorentzGroup.Boosts.Basic
public import Physlib.SpaceAndTime.SpaceTime.Basic
```

1.361 Physlib/SpaceAndTime/SpaceTime/Derivatives.lean

```
public import Physlib.SpaceAndTime.SpaceTime.LorentzAction
public import Physlib.Relativity.Tensors.RealTensor.CoVector.Basic
public import Physlib.SpaceAndTime.Space.Derivatives.Basic
public import Physlib.SpaceAndTime.Time.Derivatives
open Physlib
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  simp only [Lorentz.Vector.actionCLM_apply, ContinuousLinearMap.fderiv,
    ContinuousLinearMap.coe_comp', Function.comp_apply]
  simp only [map_sum, map_smul]
set_option backward.isDefEq.respectTransparency false in

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  simp only [smul_eq_mul]
  simp only [inv_smul_smul, smul_eq_mul]
open TensorSpecies.Tensorial Lorentz Tensor

    ∂_ (Lorentz.CoVector.indexEquiv (ComponentIdx.prod b).1)
        (ComponentIdx.prod b).2) x := by
  rw [Finset.sum_eq_single (Lorentz.CoVector.indexEquiv (ComponentIdx.prod
    generalize (Lorentz.CoVector.indexEquiv (ComponentIdx.prod b).1) = μ at
    generalize (ComponentIdx.prod b).2 = ν at *
/-- The expansion of `tensorDeriv` in terms of the tensor basis vector. -
/
lemma tensorDeriv_eq_sum_tensor_basis
    {f : SpaceTime d → M} (hf : Differentiable ℝ f) (x : SpaceTime d) :
    tensorDeriv f x = ∑ b, ∂_ (CoVector.indexEquiv (ComponentIdx.prod b).1)
      (fun x => (Tensor.basis _).repr (toTensor (f x)) (ComponentIdx.prod b
```

```
    toTensor.symm (Tensor.basis _ b) := by
  apply Tensorial.toTensor.injective
  apply (Tensor.basis (Fin.append _ _)).repr.injective
  ext b
  simp [Finsupp.single_apply, tensorDeriv_toTensor_basis_repr hf]

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    (distDeriv (Lorentz.CoVector.indexEquiv (ComponentIdx.prod b).1) f ε))
    (ComponentIdx.prod b).2 := by
  rw [Finset.sum_eq_single (Lorentz.CoVector.indexEquiv (ComponentIdx.prod b).1)]
```

1.362 Physlib/SpaceAndTime/SpaceTime/LorentzAction.lean

```
public import Physlib.SpaceAndTime.SpaceTime.Basic
public import Physlib.Mathematics.Distribution.Basic
open SchwartzMap Physlib
set_option backward.isDefEq.respectTransparency false in
      Lorentz.Vector.actionCLM_apply]
    rfl
    rw [lorentzGroup_smul_dist_apply]
    simp only [ContinuousLinearMap.add_apply, smul_add, lorentzGroup_smul_dist_apply
```

1.363 Physlib/SpaceAndTime/SpaceTime/TimeSlice.lean

```
public import Physlib.SpaceAndTime.SpaceTime.Derivatives
public import Physlib.SpaceAndTime.TimeAndSpace.Basic
open Physlib

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.364 Physlib/SpaceAndTime/Time/Basic.lean

```
public import Mathlib.Analysis.Calculus.FDeriv.Linear
public import Mathlib.MeasureTheory.Measure.Haar.InnerProductSpace
lemma norm_eq_val (t : Time) :
    ‖t‖ = ‖t.val‖ := rfl

  dist_eq t1 t2 := by
    simp [dist_eq_val, norm_eq_val]
```

```
    rw [abs_eq_iff_mul_self_eq]
    ring
  dist_eq t1 t2 := by
    simp [dist_eq_val, norm_eq_val]
    rw [abs_eq_iff_mul_self_eq]
    ring
set_option backward.isDefEq.respectTransparency false in
```

1.365 Physlib/SpaceAndTime/Time/Derivatives.lean

```
public import Physlib.Relativity.Tensors.RealTensor.Vector.Basic
public import Physlib.SpaceAndTime.Space.Module
public import Physlib.SpaceAndTime.Time.Basic
/-- Quotient rule for `Time.deriv` on real-valued functions: if `c` and `g`
  differentiable at `t` and `g t ≠ 0`, then
  `∂ₜ (c / g) t = (∂ₜ c t * g t - c t * ∂ₜ g t) / (g t)^2`. -
/
lemma deriv_div {c g : Time → ℝ}
    (hc : DifferentiableAt ℝ c t) (hg : DifferentiableAt ℝ g t) (hgz : g t
    ∂ₜ (fun s => c s / g s) t =
      (∂ₜ c t * g t - c t * ∂ₜ g t) / (g t) ^ 2 := by
  repeat rw [Time.deriv_eq]
  ring_nf
  simp [fderiv_fun_mul hc (DifferentiableAt.fun_inv (by fun_prop) hgz),
    fderiv_comp' t (differentiableAt_inv hgz) hg]
  grind

  · simp only [ContinuousLinearMap.fderiv, ContinuousLinearMap.coe_comp', F
    PiLp.proj_apply]
  · simp [-EuclideanSpace.coe_proj]
set_option backward.isDefEq.respectTransparency false in
```

1.366 Physlib/SpaceAndTime/Time/TimeTransMan.lean

```
public import Mathlib.Geometry.Manifold.Diffeomorph
public import Physlib.SpaceAndTime.Time.Basic
public import Physlib.SpaceAndTime.Time.TimeUnit
```

1.367 Physlib/SpaceAndTime/Time/TimeUnit.lean

```
public import Mathlib.Analysis.RCLike.Basic
set_option backward.isDefEq.respectTransparency false in
```

1.368 Physlib/SpaceAndTime/TimeAndSpace/Basic.lean

```
public import Physlib.SpaceAndTime.Space.Derivatives.Curl
public import Physlib.SpaceAndTime.Time.Derivatives

open Physlib
```

1.369 Physlib/SpaceAndTime/TimeAndSpace/ConstantTimeDist.lean

```
public import Physlib.SpaceAndTime.TimeAndSpace.Basic

open Physlib

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in

set_option backward.isDefEq.respectTransparency false in
      · fun_prop
```

1.370 Physlib/StatisticalMechanics/BoltzmannConstant.lean

1.371 Physlib/StatisticalMechanics/CanonicalEnsemble/Basic.lean

```
public import Mathlib.Topology.MetricSpace.Polish
public import Physlib.Thermodynamics.Temperature.Basic
  refine MeasureTheory.Integrable.fun_add ?_ ?_
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.372 Physlib/StatisticalMechanics/CanonicalEnsemble/Finite.lea

```
public import Physlib.StatisticalMechanics.CanonicalEnsemble.Lemmas
```

1.373 Physlib/StatisticalMechanics/CanonicalEnsemble/Lemmas.lea

```
public import Physlib.StatisticalMechanics.CanonicalEnsemble.Basic
public import Mathlib.Analysis.SpecialFunctions.Log.Deriv
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.374 Physlib/StatisticalMechanics/CanonicalEnsemble/TwoState.l

```
public import Physlib.StatisticalMechanics.CanonicalEnsemble.Finite
public import Physlib.Meta.Informal.Basic
```

1.375 Physlib/StringTheory/FTheory/SU5/Basic.lean

```
- `Physlib.Particles.SuperSymmetry.SU5`
  `Physlib.Particles.SuperSymmetry.SU5.Potential`, as not specific to F-
theory.
  This can be found in `Physlib.Particles.SuperSymmetry.SU5.Potential`, as
```

1.376 Physlib/StringTheory/FTheory/SU5/Charges/AnomalyFree.lean

```
public import Physlib.StringTheory.FTheory.SU5.Quanta.Basic
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.Map
public import Physlib.StringTheory.FTheory.SU5.Charges.Viable
open Physlib
```

1.377 Physlib/StringTheory/FTheory/SU5/Charges/OfRationalSectio

```
public import Physlib.Meta.TODO.Basic
TODO "The results in this file are currently stated, but not proved.
```

1.378 Physlib/StringTheory/FTheory/SU5/Charges/Viable.lean

```
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.PhenoClosed
public import Physlib.StringTheory.FTheory.SU5.Charges.OfRationalSection
open Physlib
```

1.379 Physlib/StringTheory/FTheory/SU5/Fluxes/NoExotics/ChiralIndices.

```
public import Physlib.StringTheory.FTheory.SU5.Fluxes.Basic
```

1.380 Physlib/StringTheory/FTheory/SU5/Fluxes/NoExotics/Completeness.l

```
public import Physlib.StringTheory.FTheory.SU5.Fluxes.NoExotics.ChiralIndices
public import Physlib.StringTheory.FTheory.SU5.Fluxes.NoExotics.Elems
- `Physlib.StringTheory.FTheory.SU5.Fluxes.NoExotics.Elems`
```

1.381 Physlib/StringTheory/FTheory/SU5/Fluxes/NoExotics/Elems.lean

```
public import Physlib.StringTheory.FTheory.SU5.Fluxes.Basic
```

1.382 Physlib/StringTheory/FTheory/SU5/Quanta/Basic.lean

```
public import Physlib.StringTheory.FTheory.SU5.Quanta.FiveQuanta
public import Physlib.StringTheory.FTheory.SU5.Quanta.TenQuanta
```

1.383 Physlib/StringTheory/FTheory/SU5/Quanta/FiveQuanta.lean

```
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.MinimallyAllowsTerm
public import Physlib.StringTheory.FTheory.SU5.Fluxes.NoExotics.Completeness
set_option backward.isDefEq.respectTransparency false in
```

1.384 Physlib/StringTheory/FTheory/SU5/Quanta/IsViable.lean

```
public import Physlib.StringTheory.FTheory.SU5.Charges.AnomalyFree
public import Mathlib.Data.ZMod.Defs
  Physlib.StringTheory.FTheory.SU5.Fluxes.Basic,Physlib.Particles.SuperSymmetry.SU5.
```

1.385 Physlib/StringTheory/FTheory/SU5/Quanta/TenQuanta.lean

```
public import Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.MinimallyA
public import Physlib.StringTheory.FTheory.SU5.Fluxes.NoExotics.Completenes
set_option backward.isDefEq.respectTransparency false in
```

1.386 Physlib/Thermodynamics/Temperature/Basic.lean

```
public import Physlib.StatisticalMechanics.BoltzmannConstant

set_option backward.isDefEq.respectTransparency false in
```

1.387 Physlib/Thermodynamics/Temperature/TemperatureUnits.lean

```
public import Mathlib.Analysis.RCLike.Basic
set_option backward.isDefEq.respectTransparency false in
```

1.388 Physlib/Units/Basic.lean

```
public import Physlib.SpaceAndTime.Time.TimeUnit
public import Physlib.SpaceAndTime.Space.LengthUnit
public import Physlib.ClassicalMechanics.Mass.MassUnit
public import Physlib.Electromagnetism.Charge.ChargeUnit
public import Physlib.Thermodynamics.Temperature.TemperatureUnits
public import Physlib.Units.Dimension
public import Mathlib.Analysis.SpecialFunctions.Pow.NNReal
Units within Physlib are implemented with the following convention:
  - `Physlib/SpaceAndTime/Time/TimeUnit.lean`
  - `Physlib/SpaceAndTime/Space/LengthUnit.lean`
  - `Physlib/ClassicalMechanics/Mass/MassUnit.lean`
  - `Physlib/Electromagnetism/Charge/ChargeUnit.lean`
  - `Physlib/Thermodynamics/Temperature/TemperatureUnit.lean`
- https://leanprover.zulipchat.com/#narrow/channel/479953-
Physlib/topic/physical.20units
in Physlib which is based exclusively on Reals.
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
TODO "Make SI : UnitChoices computable, probably by
  https://leanprover.zulipchat.com/#narrow/channel/479953-
Physlib/topic/physical.20units/near/534914807"
```

1.389 Physlib/Units/Examples.lean

```
public import Physlib.Units.WithDim.Speed
# Examples of units in Physlib
In this module we give some examples of how to use the units system in Physlib.
set_option backward.isDefEq.respectTransparency false in
  simp only [scaleUnit_apply_fst, scaleUnit_apply_fun, scaleUnit_symm_apply,
    scaleUnit_apply_fun_left]
TODO "Add an example involving derivatives."
```

1.390 Physlib/Units/FDeriv.lean

```
public import Physlib.Units.WithDim.Basic
public import Mathlib.Analysis.Calculus.FDeriv.Equiv
set_option backward.isDefEq.respectTransparency false in
```

1.391 Physlib/Units/Integral.lean

```
public import Physlib.Units.UnitDependent
public import Mathlib.MeasureTheory.Integral.Bochner.Basic
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
      simp
```

1.392 Physlib/Units/UnitDependent.lean

```
public import Physlib.Units.Basic
public import Mathlib.Topology.Algebra.Module.StrongTopology
```

1.393 Physlib/Units/WithDim/Area.lean

```
public import Physlib.Units.WithDim.Basic
```

1.394 Physlib/Units/WithDim/Basic.lean

```
public import Physlib.Units.UnitDependent
TODO "Induce instances on `WithDim d M` from instances on `M`."
```

1.395 Physlib/Units/WithDim/Energy.lean

```
public import Physlib.Units.WithDim.Basic
```

1.396 Physlib/Units/WithDim/Momentum.lean

```
public import Physlib.Units.WithDim.Basic
```

1.397 Physlib/Units/WithDim/Pressure.lean

```
public import Physlib.Units.WithDim.Basic
```

1.398 Physlib/Units/WithDim/Speed.lean

```
public import Physlib.Units.WithDim.Basic

set_option backward.isDefEq.respectTransparency false in
```

1.399 QuantumInfo.lean

```
module

public import QuantumInfo.ForMathlib
public import QuantumInfo.Finite.Channel.DegradableOrder
public import QuantumInfo.Finite.CPTPMap
public import QuantumInfo.Finite.Distance
public import QuantumInfo.Finite.Qubit.Basic
public import QuantumInfo.Finite.ResourceTheory.FreeState
public import QuantumInfo.Finite.ResourceTheory.SteinsLemma
public import QuantumInfo.Finite.Braket
public import QuantumInfo.Finite.Capacity
public import QuantumInfo.Finite.Ensemble
public import QuantumInfo.Finite.Entanglement
public import QuantumInfo.Finite.Entropy
public import QuantumInfo.Finite.MState
public import QuantumInfo.Finite.Pinching
public import QuantumInfo.Finite.POVM
public import QuantumInfo.Finite.Unitary
```

```
public import QuantumInfo.Finite.Capacity_doc
public import QuantumInfo.ClassicalInfo.Distribution
public import QuantumInfo.ClassicalInfo.Entropy
public import QuantumInfo.ClassicalInfo.Prob
public import QuantumInfo.StatMech.Hamiltonian
public import QuantumInfo.StatMech.IdealGas
public import QuantumInfo.StatMech.ThermoQuantities
```

1.400 QuantumInfo/ClassicalInfo/Capacity.lean

```
module

public import QuantumInfo.ClassicalInfo.Entropy
public import Mathlib.Data.Finset.Fin
public import Mathlib.Data.Fintype.Fin

@[expose] public section
 change the symbol set), and `FixedLengthCode` where the output lengths only depend
 lengths. -/
 the channel merely "corrupts" the data then this `O = I`; the erasure channel might
 take `O = I ⊕ Unit` or `Option I`.
```

1.401 QuantumInfo/ClassicalInfo/Channel.lean

```
module

public import QuantumInfo.ClassicalInfo.Distribution
public import Mathlib.MeasureTheory.Measure.MeasureSpaceDef

@[expose] public section
  symb_dist : I → ProbDistribution O
    -- change ∑ os in Fintype.piFinset fun x => (Finset.univ : Finset O),
    --  ∏ k : Fin n, ((C.symb_dist (is k)) (os k) : ℝ) = 1
  C.on_fin _ _ (fun k => is.get k)
```

1.402 QuantumInfo/ClassicalInfo/Distribution.lean

```
module
public import QuantumInfo.ClassicalInfo.Prob

public import Mathlib.Analysis.Convex.Combination
We define the type `Distribution α` on a `Fintype α`. By restricting ourselves to di
```

```
finite types, a lot of notation and casts are greatly simplified.
This suffices for (most) finite-dimensional quantum theory.

@[expose] public section

theorem constant_of_exists_one {D : ProbDistribution α} {x : α} (h : D x =
    D = ProbDistribution.constant x := by
/-- Given a distribution on α, extend it to a distribution on `Sum α β` by
  giving it no support on `β`. -/
/-- Given a distribution on α, extend it to a distribution on `Sum β α` by
  giving it no support on `β`. -/
set_option backward.isDefEq.respectTransparency false in
-- The two properties below (and congrRandVar) follow from the fact that Di
-- contravariant functor.
theorem congr_symm_apply (σ : α ≃ β) :
    (ProbDistribution.congr σ).symm = ProbDistribution.congr σ.symm := by
/-- The distribution on Fin 2 corresponding to a coin with probability p.
  Chance p of 1, 1-p of 0. -/
it is the "convex combination over a finite family" on the type `T`, afford
the `Mixable` instance, with the probability distribution of `X` as weights
/
    have hz : ∀ i ∈ Finset.univ, inst.to_U (X.var i) ∈ Set.range inst.to_U
      simp [Finset.mem_univ]
theorem expect_val_eq_mixable_mix (d : ProbDistribution (Fin 2)) (x₁ x₂ : T
    expect_val ⟨![x₁, x₂], d⟩ = Mixable.mix (d 0) x₁ x₂ := by
    ∑ i : Fin (Nat.succ 0).succ, (d i : ℝ) • Mixable.to_U (![x₁, x₂] i) =
        ∑ i, (d i : ℝ) • Mixable.to_U (![x₁, x₂] i) := by
/-- The expectation value of a random variable with constant probability di
  `constant x` is its value at `x` -/
theorem zero_le_expect_val (d : ProbDistribution α) (f : α → ℝ) (hpos : 0 ≤
    0 ≤ expect_val ⟨f, d⟩ := by
/-- Given a `T`-valued random variable `X` over `α`, mapping over `T` commu
  with the equivalence over `α` -/
theorem expect_val_congr_eq_expect_val (σ : α ≃ β) (X : RandVar α T) :
    expect_val (congrRandVar σ X) = expect_val X := by
```

1.403 QuantumInfo/ClassicalInfo/Entropy.lean

```
module

public import QuantumInfo.ClassicalInfo.Distribution
public import Mathlib.Analysis.SpecialFunctions.Log.NegMulLog
public import Mathlib.Analysis.SpecialFunctions.BinaryEntropy
public import QuantumInfo.ClassicalInfo.ForMathlib.Analysis.SpecialFunction
```

```
@[expose] public section

set_option backward.isDefEq.respectTransparency false in
  -- Since the sum of the probabilities is 1, we can apply Jensen's inequality for t
    -- convex function -x log x.
```

1.404 QuantumInfo/ClassicalInfo/ForMathlib/Analysis/SpecialFunctions/L

```
module

public import Mathlib.Analysis.SpecialFunctions.Log.NegMulLog

@[expose] public section
```

1.405 QuantumInfo/ClassicalInfo/Prob.lean

```
module

public import Mathlib.Analysis.Convex.Mul
public import Mathlib.Analysis.SpecialFunctions.Log.Basic
public import Mathlib.Analysis.SpecialFunctions.Log.ENNRealLog
public import Mathlib.Data.NNReal.Basic
public import Mathlib.Data.EReal.Basic
public import Mathlib.Tactic.Finiteness
public import Mathlib.Topology.UnitInterval
This defines a type `Prob`, which is simply any real number in the interval 0 to 1.
This then comes with additional statements such as its application to convex sets, a
it makes useful type alias for functions that only make sense on probabilities.
A significant application is in the `Mixable` typeclass, also in this file, which is
general notion of convex combination that applies to types as opposed to sets;
elements are `Mixable.mix`ed using `Prob`s.
@[expose] public section

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
/-- A `Mixable T` typeclass instance gives a compact way of talking about the
  action of probabilities for forming linear combinations in convex spaces.
  The notation `p [ x₁ ↔ x₂ ]` means to take a convex
  Mixable is defined by an "underlying" data type `U` with addition and scalar
  multiplication, and a bijection between the `T` and a convex set of `U`.
  For instance, in `Mixable (Distribution (Fin n))`, `U` is `n`-
element vectors
```

```
  (which form the probability simplex, degenerate in one dimension). For
  `QuantumInfo.Finite.MState` density matrices in quantum mechanics, which
  PSD matrices of trace 1, `U` is the underlying matrix.

  Why not just stick with existing notions of `Convex`? `Convex` requires t
  the type already forms an `AddCommMonoid` and `Module ℝ`. But many types,
  are not: there is no good notion of "multiplying a probability distributi
  distribution. We can coerce the distribution nto, say, a vector or a func
  doing arithmetic with distributions. Accordingly, the expression `0.3 * d
  cannot represent a distribution on its own. -/
/-- `Mixable.mix` represents the notion of "convex combination" on the type
  `Mixable` instance. It takes a `Prob`, that is, a `Real` between 0 and 1.
  with a Real, use `mix_ab`. -/
theorem mkT_instUniv [AddCommMonoid T] [Module ℝ T] {t : T} (h : ∃ t', to_U
    instUniv.mkT h = ⟨t, rfl⟩ :=
theorem instPi.lem_1 {D : Type*} {T U : D → Type*} [∀i, AddCommMonoid (U i)
    [inst : ∀i, Mixable (U i) (T i)]
    {u : (i : D) → U i} (h : ∃ (t : (i : D) → T i), (fun d => to_U (t d)) =
    ∃ (t : T d), to_U t = u d := by
variable {D : Type*} {T U : D → Type*} [∀i, AddCommMonoid (U i)] [∀ i, Modu
  [inst : ∀i, Mixable (U i) (T i)] in
theorem val_mkT_instPi (D : Type*) [inst : Mixable U T] {u : D → U} (h : ∃
    (instPi.mkT h).val = fun d ↦ (inst.mkT (instPi.lem_1 h d)).val :=
theorem to_U_instPi (D : Type*) [inst : Mixable U T] {t : D → T} :
    (instPi).to_U t = fun d ↦ inst.to_U (t d) :=
@[reducible]
abbrev mix [AddCommMonoid U] [Module ℝ U] [inst : Mixable U T]
    (p : Prob) (x₁ x₂ : T) := inst.mix p x₁ x₂
/-- Map a probability [0,1] to [0,+∞] with -log p. Special case that 0 maps
  Real.log does). This makes it `Antitone`.
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.406 QuantumInfo/Finite/AxiomatizedEntropy/Defs.lean

```
module

public import QuantumInfo.ClassicalInfo.Entropy
public import QuantumInfo.Finite.MState
public import QuantumInfo.Finite.CPTPMap
@[expose] public section
```

1.407 QuantumInfo/Finite/AxiomatizedEntropy/Renyi.lean

```
module

public import QuantumInfo.Finite.Entropy.Defs
@[expose] public section
```

1.408 QuantumInfo/Finite/Braket.lean

```
module

public import QuantumInfo.ForMathlib
public import QuantumInfo.ClassicalInfo.Distribution
@[expose] public section

lemma _root_.Ket.coe_fun_eq (ψ : Ket d) : (ψ : d → ℂ) = ψ.vec := rfl

lemma _root_.Bra.coe_fun_eq (ψ : Bra d) : (ψ : d → ℂ) = ψ.vec := rfl

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  push Not
```

1.409 QuantumInfo/Finite/CPTPMap.lean

```
module

public import QuantumInfo.Finite.CPTPMap.Bundled
public import QuantumInfo.Finite.CPTPMap.CPTP
public import QuantumInfo.Finite.CPTPMap.Dual
public import QuantumInfo.Finite.CPTPMap.MatrixMap
public import QuantumInfo.Finite.CPTPMap.Unbundled
```

1.410 QuantumInfo/Finite/CPTPMap/Bundled.lean

```
module
public import QuantumInfo.Finite.CPTPMap.Unbundled
public import QuantumInfo.Finite.MState

public import Mathlib.Topology.Order.Hom.Basic
```

```
@[expose] public section

/-!

## Hermitian-preserving maps

-/

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
noncomputable instance instFunLike : FunLike (HPMap dIn dOut ℂ) (HermitianM
lemma apply_hermitianMat_eq (Λ : HPMap dIn dOut ℂ) (ρ : HermitianMat dIn ℂ)
    Λ ρ = ⟨Λ.map ρ.1, Λ.HP ρ.2⟩ := rfl

set_option backward.isDefEq.respectTransparency false in
  map_smulₛₗ f c x := HermitianMat.ext <| by simp [apply_hermitianMat_eq]
/-!

## Positive-preserving maps

-/

noncomputable instance instFunLike : FunLike (PMap dIn dOut ℂ) (HermitianMa
lemma apply_hermitianMat_eq (Λ : PMap dIn dOut ℂ) (ρ : HermitianMat dIn ℂ)
    Λ ρ = ⟨Λ.map ρ.1, Λ.HP ρ.2⟩ := rfl

set_option backward.isDefEq.respectTransparency false in
  map_smulₛₗ f c x := HermitianMat.ext <| by simp [apply_hermitianMat_eq]
/-!

## Positive trace-preserving maps

-/
noncomputable instance instFunLike : FunLike (PTPMap dIn dOut ℂ) (Hermitian
lemma apply_hermitianMat_eq_toPMap (Λ : PTPMap dIn dOut ℂ) (ρ : HermitianMa
    Λ ρ = Λ.toPMap ρ := rfl

set_option backward.isDefEq.respectTransparency false in
  map_add f x y := by simp [apply_hermitianMat_eq_toPMap]
  map_smulₛₗ f c x := by simp [apply_hermitianMat_eq_toPMap]
noncomputable instance instMFunLike [DecidableEq dIn] [DecidableEq dOut] :
lemma apply_mstate_eq [DecidableEq dIn] [DecidableEq dOut] (Λ : PTPMap dIn
    Λ ρ = MState.mk
      (Λ.toHPMap ρ.M) (HermitianMat.zero_le_iff.mpr (Λ.pos ρ.psd)) (by
        rw [HermitianMat.trace_eq_one_iff, ← ρ.tr']
        exact Λ.TP ρ) := rfl
```

```
/-!

## Completely positive trace-preserving linear maps

-/

noncomputable instance instMFunLike [DecidableEq dOut] : FunLike (CPTPMap dIn dOut)
lemma apply_mState_eq_toPTPMap [DecidableEq dOut] (Λ : CPTPMap dIn dOut) (ρ : MState
    Λ ρ = Λ.toPTPMap ρ := rfl

noncomputable instance instFunLike : FunLike (PUMap dIn dOut ℂ) (HermitianMat dIn ℂ)
lemma apply_hermitianMat_eq_toPMap (Λ : PUMap dIn dOut ℂ) (ρ : HermitianMat dIn ℂ) :
    Λ ρ = Λ.toPMap ρ := rfl

set_option backward.isDefEq.respectTransparency false in
  map_smulₛₗ f c x := HermitianMat.ext <| by simp [apply_hermitianMat_eq_toPMap]
noncomputable instance instFunLike : FunLike (CPUMap dIn dOut ℂ) (HermitianMat dIn ℂ
lemma apply_hermitianMat_eq_toPMap (Λ : CPUMap dIn dOut ℂ) (ρ : HermitianMat dIn ℂ)
    Λ ρ = Λ.toPMap ρ := rfl

set_option backward.isDefEq.respectTransparency false in
  map_smulₛₗ f c x := HermitianMat.ext <| by simp [apply_hermitianMat_eq_toPMap]
```

1.411 QuantumInfo/Finite/CPTPMap/CPTP.lean

```
module

public import QuantumInfo.Finite.CPTPMap.Bundled
public import QuantumInfo.Finite.Unitary
A `CPTPMap` is a `ℂ`-linear map between matrices (`MatrixMap` is an alias), bundled
that it `IsCompletelyPositive` and `IsTracePreserving`. CPTP maps are typically rega
"most general quantum operation", as they map density matrices (`MState`s) to densit
type `PTPMap`, for maps that are positive (but not necessarily completely positive)
declared.
    operations on states. These correspond directly to `MState.SWAP`, `MState.assoc`
    `MState.assoc'`, `MState.traceLeft`, and `MState.traceRight`.
@[expose] public section

set_option backward.isDefEq.respectTransparency false in
  simp only [replacement, compose_eq, traceLeft_eq_MState_traceLeft]
  simp only [apply_mState_eq_toPTPMap, PTPMap.apply_mstate_eq, HPMap.apply_hermitian
    HPMap.map, HermitianMat.val_eq_coe, MState.mat_M, LinearMap.coe_mk, AddHom.coe_m
PROBLEM If a MatrixMap of_kraus K K is trace-preserving, then Σ_k K_k† K_k = 1.
```

```
PROVIDED SOLUTION The TP condition says for all X, trace((of_kraus K K) X)
of_kraus: trace(Σ_k K_k X K_k†) = Σ_k trace(K_k† K_k X) (by cycle) = trace(
trace(A X) = trace(X) for all X where A = Σ_k K_k† K_k, which means A = 1.
`Matrix.eq_of_trace_mul_eq` or the fact that trace is a faithful pairing on
1. The TP condition `Λ.TP` gives us `∀ x, (Λ.map x).trace = x.trace`, and s
K`, we substitute and simplify.
    Ket.coe_fun_eq, Bra.coe_fun_eq]
```

1.412 QuantumInfo/Finite/CPTPMap/Dual.lean

```
module

public import QuantumInfo.Finite.CPTPMap.Bundled
public import Mathlib.LinearAlgebra.Matrix.FiniteDimensional
@[expose] public section

open MatrixOrder
set_option backward.isDefEq.respectTransparency false in
    classical
    apply CStarAlgebra.nonneg_iff_eq_star_mul_self.mp
    exact Matrix.nonneg_iff_posSemidef.mpr hB
  -- By definition of dual, we know that for any A and B, the trace of (M A
  -- A * (M.dual B).
  -- By definition of dual, we know that
  -- $(M.dual (single j₁ j₂ 1)) i₁ i₂ = (M (single i₂ i₁ 1)) j₂ j₁$.
If the Choi matrix of a map is positive semidefinite, then the Choi matrix
positive semidefinite.
set_option backward.isDefEq.respectTransparency false in
The composition of the dual of the inverse of the dual basis isomorphism wi
isomorphism is the evaluation map.
The composition of the inverse of the dual basis isomorphism with the dual
isomorphism is the inverse of the evaluation map.
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  simp only [HPMap.apply_hermitianMat_eq, HPMap.map, HermitianMat.mat_mk,
set_option backward.isDefEq.respectTransparency false in
--Can define one for HPMap's that has 'easier' definitional properties, use
--structure, doesn't go through Module.Basis the same way. Requires the equ
linear
--maps of HermitianMats and ℂ-linear maps of matrices.
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.413 QuantumInfo/Finite/CPTPMap/MatrixMap.lean

```
module

public import Mathlib.LinearAlgebra.TensorProduct.Matrix
public import Mathlib.LinearAlgebra.PiTensorProduct
public import Mathlib.Data.Set.Card
public import Mathlib.Algebra.Module.LinearMap.Basic
public import QuantumInfo.ForMathlib
public import QuantumInfo.Finite.Braket
public import QuantumInfo.Finite.MState
 * `toMatrix` is the rectangular "transfer matrix", where matrix multiplication comm
   composition.
We provide simp lemmas for relating these facts, prove basic facts e.g. composition
and some facts about `IsTracePreserving` maps.
@[expose] public section

  -- By definition of `MatrixMap.of_choi_matrix`, we know that applying it to the Ch
  -- reconstructs `M`.
/-- The Kronecker product of MatrixMaps. Defined here using `TensorProduct.map M₁ M₂
  appropriate reindexing operations and `LinearMap.toMatrix`/`Matrix.toLin`. Notatio
/
set_option backward.isDefEq.respectTransparency false in
/-- The extensional definition of the Kronecker product `MatrixMap.kron`, in terms o
  its image. -/
set_option backward.isDefEq.respectTransparency false in
--   def matrixMap_kron (M₁ : Matrix (A₁ × B₁) (C₁ × D₁) R) (M₂ : Matrix (A₂ × B₂) (
--   Matrix ((A₁ × A₂) × (B₁ × B₂)) ((C₁ × C₂) × (D₁ × D₂)) R := Matrix.of fun ((a₁,
--   ((c₁, c₂), (d₁, d₂)) ↦ (M₁ (a₁, b₁) (c₁, d₁)) * (M₂ (a₂, b₂) (c₂, d₂))
/-- The operational definition of the Kronecker product `MatrixMap.kron`, that it ma
  product of inputs to the Kronecker product of outputs. It is the unique bilinear m
/
/-- Finite Pi-type tensor product of MatrixMaps. Defined as `PiTensorProduct.tprod`
  underlying Linear maps. Notation `⨂ₜₘ[R] i, f i`, eventually. -
/
```

1.414 QuantumInfo/Finite/CPTPMap/Unbundled.lean

```
module

public import QuantumInfo.Finite.CPTPMap.MatrixMap
@[expose] public section
```

```
/-- Simp lemma: the trace of the image of a IsTracePreserving map is the sa
trace. -/
set_option backward.isDefEq.respectTransparency false in
open MatrixOrder

set_option backward.isDefEq.respectTransparency false in
/-- The map that takes M and returns M ⊗ₖ C, where C is positive semidefini
  positive map. -/
    -- Since $x$ and $C$ are positive semidefinite, there exist matrices $U
    -- $x = U^*U$ and $C = V^*V$.
    obtain ⟨U, hU⟩ : ∃ U : Matrix (A × Fin n) (A × Fin n) R, x = star U * U
      classical
      apply CStarAlgebra.nonneg_iff_eq_star_mul_self.mp
      exact Matrix.nonneg_iff_posSemidef.mpr hx
    obtain ⟨V, hV⟩ : ∃ V : Matrix d d R, C = star V * V := by
      classical
      apply CStarAlgebra.nonneg_iff_eq_star_mul_self.mp
      exact Matrix.nonneg_iff_posSemidef.mpr h
set_option backward.isDefEq.respectTransparency false in
  obtain ⟨m', rfl⟩ : ∃ B, m = B.conjTranspose * B := by
    classical
    apply CStarAlgebra.nonneg_iff_eq_star_mul_self.mp
    exact Matrix.nonneg_iff_posSemidef.mpr h
```

1.415 QuantumInfo/Finite/Capacity.lean

```
module
public import Mathlib.Analysis.SpecialFunctions.Log.Base

public import QuantumInfo.Finite.Entropy
public import QuantumInfo.Finite.CPTPMap
public import QuantumInfo.Finite.Distance
@[expose] public section
```

1.416 QuantumInfo/Finite/Capacity_doc.lean

```
module

-/@[expose] public section
```

1.417 QuantumInfo/Finite/Channel/DegradableOrder.lean

```
module

public import QuantumInfo.Finite.CPTPMap
We define the "degradable order" on channels, where M ≤ N if N can be degraded to M.
scoped instance, because there are other reasonable orders on channels.
The degradable order is actually only a preorder, because channels that are degradab
are not necessarily equal. (For instance, unitary channels are degradable to the ide
versa.) It can compare channels of different output dimensions, but they must have t
dimension. For this reason the `DegradablePreorder` is parameterized by the channel
are degradable to each other, but cannot be degraded to each other by unitary operat
instance, let A be the replacement channel that goes to a mixed state, and B be a re
channel that goes to a pure state.

Technical notes: to model "channels of different output types", the preorder is on t
channels parameterized by their output type. And since the output type needs to be a
DecidableEq, the argument is also a Sigma to bring this along.
@[expose] public section

@[reducible]
```

1.418 QuantumInfo/Finite/Distance.lean

```
module

public import QuantumInfo.Finite.Distance.Fidelity
public import QuantumInfo.Finite.Distance.TraceDistance

@[expose] public section
```

1.419 QuantumInfo/Finite/Distance/Fidelity.lean

```
module

public import QuantumInfo.Finite.CPTPMap

@[expose] public section
/-- The fidelity of two quantum states. This is the quantum version of the Bhattacha
  coefficient. -/
```

1.420 QuantumInfo/Finite/Distance/TraceDistance.lean

```
module
public import QuantumInfo.Finite.MState

public import QuantumInfo.ForMathlib

@[expose] public section
set_option backward.isDefEq.respectTransparency false in
```

1.421 QuantumInfo/Finite/Ensemble.lean

```
module

public import QuantumInfo.Finite.MState

@[expose] public section
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.422 QuantumInfo/Finite/Entanglement.lean

```
module

public import QuantumInfo.Finite.Braket
public import QuantumInfo.Finite.Ensemble
public import QuantumInfo.Finite.Entropy
public import QuantumInfo.ClassicalInfo.Entropy
@[expose] public section

set_option backward.isDefEq.respectTransparency false in
  push Not
  push Not
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.423 QuantumInfo/Finite/Entropy.lean

```
module
```

```
public import QuantumInfo.Finite.Entropy.VonNeumann
public import QuantumInfo.Finite.Entropy.SSA
public import QuantumInfo.Finite.Entropy.Relative
public import QuantumInfo.Finite.Entropy.DPI
```

1.424 QuantumInfo/Finite/Entropy/DPI.lean

```
module

public import QuantumInfo.Finite.Entropy.Relative
public import QuantumInfo.ForMathlib.HermitianMat.Sqrt

@[expose] public section
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    simp only [mat_smul] at this
set_option backward.isDefEq.respectTransparency false in
/-

/- Removed due to module system.
-/
```

1.425 QuantumInfo/Finite/Entropy/Relative.lean

```
module
public import QuantumInfo.Finite.Entropy.VonNeumann

@[expose] public section
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in

      rw [ show x ^ (1 + h) * h⁻¹ - x * h⁻¹ - x * Real.log x = x * ( -
h⁻¹ + ( h⁻¹ * x ^ h - Real.log x ) ) by rw [ Real.rpow_add ( by linarith [
      rw [ abs_mul, abs_of_nonneg ( by linarith [ hx.1 ] : 0 ≤ x ) ]
      ring_nf

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  · have : α < 1 := by push Not at hα₂; exact lt_of_le_of_ne hα₂ h1
          simp_all only [MState.mat_M, PiLp.ofLp_single, Matrix.mulVec_sing
        push Not at h_contra;
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  have := HermitianMat.ker_reindex_le_iff  (σ : HermitianMat d ℂ ) ↑ρ e.sym
  split_ifs <;> simp_all [HermitianMat.conj_submatrix]
set_option backward.isDefEq.respectTransparency false in
    simp only [SandwichedRelRentropy, hα, ↓reduceDIte, Std.le_refl, sub_sel
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
    push Not at h
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.426 QuantumInfo/Finite/Entropy/SSA.lean

```
module

public import QuantumInfo.Finite.Entropy.VonNeumann
public import QuantumInfo.ForMathlib.HermitianMat.Sqrt
@[expose] public section

set_option backward.isDefEq.respectTransparency false in
  exact Matrix.le_iff.mpr h_pos
  simp [mul_comm]
  rfl
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.427 QuantumInfo/Finite/Entropy/VonNeumann.lean

```
module

public import QuantumInfo.Finite.Braket
public import QuantumInfo.Finite.CPTPMap
public import QuantumInfo.ClassicalInfo.Entropy
@[expose] public section

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
def vecToMat {d₁ d₂ : Type*} [Fintype d₁] [Fintype d₂] (v : d₁ × d₂ → ℂ) : Matrix d₁
set_option backward.isDefEq.respectTransparency false in
```

1.428 QuantumInfo/Finite/MState.lean

```
module
public import QuantumInfo.ForMathlib
public import QuantumInfo.ClassicalInfo.Distribution
public import QuantumInfo.Finite.Braket

public import Mathlib.Logic.Equiv.Basic
@[expose] public section

meta def evalMStateM : PositivityExt where eval {_u _α} _zα _pα e := do
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
lemma default_eq [Nonempty d] : (default : MState d) = uniform := rfl

  simp [default_eq, uniform]
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
  · exact ContinuousFunctionalCalculus.spectrum_nonempty _ ρ.M.H
  · exact ContinuousFunctionalCalculus.spectrum_nonempty _ σ.M.H
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
          rw [ Finset.sum_eq_add_sum_diff_singleton i _ (by simp) ] at h_di
          exact ⟨i, hi, fun j hj => Classical.not_not.1 fun hj' =>
            absurd h_diag ( by rw [ if_neg hi ] ; exact ne_of_gt ( lt_add_o
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.429 QuantumInfo/Finite/POVM.lean

```
module

public import QuantumInfo.Finite.CPTPMap

@[expose] public section
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.430 QuantumInfo/Finite/Pinching.lean

```
module

public import QuantumInfo.Finite.CPTPMap
public import QuantumInfo.Finite.MState
public import QuantumInfo.Finite.Entropy
public import QuantumInfo.ForMathlib.HermitianMat.CFC
@[expose] public section

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
```

1.431 QuantumInfo/Finite/Qubit/Basic.lean

```
module

public import QuantumInfo.Finite.CPTPMap
@[expose] public section
```

1.432 QuantumInfo/Finite/ResourceTheory/FreeState.lean

```
module
public import Mathlib.Algebra.Module.Submodule.Lattice
public import Mathlib.Analysis.Subadditive
public import Mathlib.CategoryTheory.Functor.FullyFaithful
public import Mathlib.CategoryTheory.Monoidal.Braided.Basic
public import Mathlib.Data.EReal.Basic
public import Mathlib.Tactic

public import QuantumInfo.Finite.CPTPMap
public import QuantumInfo.Finite.Entropy

@[expose] public section
attribute [reducible] ResourcePretheory.FinH
attribute [reducible] ResourcePretheory.DecEqH
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.433 QuantumInfo/Finite/ResourceTheory/HypothesisTesting.lean

```
module
public import Mathlib.Algebra.Module.Submodule.Lattice
public import Mathlib.Analysis.Subadditive
public import Mathlib.CategoryTheory.Functor.FullyFaithful
public import Mathlib.CategoryTheory.Monoidal.Braided.Basic
public import Mathlib.Data.EReal.Basic

public import QuantumInfo.Finite.CPTPMap
```

```
public import QuantumInfo.Finite.Entropy
public import QuantumInfo.Finite.POVM
@[expose] public section

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.434 QuantumInfo/Finite/ResourceTheory/ResourceTheory.lean

```
module

public import QuantumInfo.Finite.ResourceTheory.FreeState

@[expose] public section
```

1.435 QuantumInfo/Finite/ResourceTheory/SteinsLemma.lean

```
module
public import QuantumInfo.Finite.ResourceTheory.FreeState
public import QuantumInfo.Finite.ResourceTheory.HypothesisTesting
public import QuantumInfo.Finite.Pinching
public import QuantumInfo.ForMathlib.Matrix
public import QuantumInfo.ForMathlib.LimSupInf
public import QuantumInfo.ForMathlib.HermitianMat
public import QuantumInfo.ForMathlib.HermitianMat.Jordan

public import Mathlib.Tactic.Bound

@[expose] public section
set_option backward.isDefEq.respectTransparency false in
  push Not at h
set_option backward.isDefEq.respectTransparency false in
  push Not at h
```

```
noncomputable def R1 (ρ : MState (H i)) (ε : Prob) : ℝ≥0∞ :=
noncomputable def R2 (ρ : MState (H i)) : ((n : ℕ) → IsFree (i := i ^ n)) → ℝ≥0∞ :=
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
def σ₁ : MState (H i) :=
def σ₁_mineig := □ k, (σ₁ i).M.H.eigenvalues k
def σ₁_c (n : ℕ) : ℝ :=
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  · trans  (R1 ρ ε).toReal - (R1 ρ ε).toReal * □↑((□ n) (ρ ⊗ᵣ^[n])), P1 ε2 n□_ℝ +
           □↑((□ n) (ρ ⊗ᵣ^[n])), P1 ε2 n□_ℝ * (R2 ρ σ).toReal +
         □↑((□ n) (ρ ⊗ᵣ^[n])), P1 ε2 n□_ℝ * ε₀ +
       (ε2 - ε2 * □↑((□ n) (ρ ⊗ᵣ^[n])), P2 ε2 n□_ℝ) +
     (-(□↑((□ n) (ρ ⊗ᵣ^[n])), P2 ε2 n□_ℝ * (R2 ρ σ).toReal) - □↑((□ n) (ρ ⊗ᵣ^[n])),
   □↑((□ n) (ρ ⊗ᵣ^[n])), P2 ε2 n□_ℝ * c' ε2 n; swap
    · ring_nf
      rfl
    apply add_nonneg
set_option maxHeartbeats 1000000 in
set_option backward.isDefEq.respectTransparency false in
        have h1 := EquationS88 ρ σ (ε := ε) (ε' := ε') (by assumption)
          (by assumption) (by assumption) (by assumption) m
          (by assumption) (by assumption) (by assumption)
          (by assumption) (by assumption) (by assumption) (by assumption)
          (by assumption) (by assumption) (by assumption) (by assumption)
        convert h1 using 1
        simp [P1, P2]
        grind
set_option backward.isDefEq.respectTransparency false in
theorem Lemma7 (ρ : MState (H i)) {ε : Prob} (hε : 0 < ε ∧ ε < 1) (σ : (n : ℕ) → IsF
    rw [spectrum.smul_eq_smul _ _ (ContinuousFunctionalCalculus.spectrum_nonempty _
    apply ContinuousFunctionalCalculus.spectrum_nonempty
noncomputable def Lemma7_improver (ρ : MState (H i)) {ε : Prob} (hε : 0 < ε ∧ ε < 1)
  · push Not at h
set_option backward.isDefEq.respectTransparency false in
/-
/-
Commented out because of module system.
-/
```

1.436 QuantumInfo/Finite/Unitary.lean

```
module

public import QuantumInfo.Finite.MState
@[expose] public section

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.437 QuantumInfo/ForMathlib.lean

```
module

public import QuantumInfo.ForMathlib.ContinuousLinearMap
public import QuantumInfo.ForMathlib.ContinuousSup
public import QuantumInfo.ForMathlib.Filter
public import QuantumInfo.ForMathlib.HermitianMat
public import QuantumInfo.ForMathlib.Isometry
public import QuantumInfo.ForMathlib.LinearEquiv
public import QuantumInfo.ForMathlib.MatrixNorm.TraceNorm
public import QuantumInfo.ForMathlib.Matrix
public import QuantumInfo.ForMathlib.Minimax
public import QuantumInfo.ForMathlib.Misc
public import QuantumInfo.ForMathlib.Unitary

@[expose] public section
```

1.438 QuantumInfo/ForMathlib/ContinuousLinearMap.lean

```
module

public import Mathlib.Topology.Algebra.Module.LinearMap
public import Mathlib.Analysis.CStarAlgebra.Classes
public import Mathlib.Analysis.InnerProductSpace.Spectrum
public import Mathlib.Order.CompletePartialOrder

@[expose] public section
```

1.439 QuantumInfo/ForMathlib/ContinuousSup.lean

```
module

public import Mathlib.Analysis.InnerProductSpace.Basic
public import Mathlib.Analysis.Normed.Module.FiniteDimension
public import Mathlib.Data.Real.StarOrdered
public import Mathlib.Analysis.Normed.Operator.BoundedLinearMaps

@[expose] public section
```

1.440 QuantumInfo/ForMathlib/Filter.lean

```
module
public import Mathlib

public import Mathlib.Tactic.Bound

@[expose] public section
```

1.441 QuantumInfo/ForMathlib/HermitianMat.lean

```
module

public import QuantumInfo.ForMathlib.HermitianMat.Basic
public import QuantumInfo.ForMathlib.HermitianMat.CFC
public import QuantumInfo.ForMathlib.HermitianMat.Inner
public import QuantumInfo.ForMathlib.HermitianMat.LogExp
public import QuantumInfo.ForMathlib.HermitianMat.Order
public import QuantumInfo.ForMathlib.HermitianMat.Proj
public import QuantumInfo.ForMathlib.HermitianMat.Rpow
public import QuantumInfo.ForMathlib.HermitianMat.Reindex
public import QuantumInfo.ForMathlib.HermitianMat.Schatten
public import QuantumInfo.ForMathlib.HermitianMat.Sqrt
public import QuantumInfo.ForMathlib.HermitianMat.Trace
public import QuantumInfo.ForMathlib.HermitianMat.Unitary
```

1.442 QuantumInfo/ForMathlib/HermitianMat/Basic.lean

```
module

public import QuantumInfo.ForMathlib.Matrix
```

```
public import QuantumInfo.ForMathlib.IsMaximalSelfAdjoint
public import QuantumInfo.ForMathlib.ContinuousLinearMap
public import QuantumInfo.ForMathlib.Tactic.Commutes

public import Mathlib.Analysis.Matrix.Normed
public import Mathlib.Analysis.SpecialFunctions.ContinuousFunctionalCalculu
public import Mathlib.Analysis.SpecialFunctions.ContinuousFunctionalCalculu
public import Mathlib.Analysis.SpecialFunctions.Bernstein
public import Mathlib.Analysis.SpecialFunctions.Pow.NNReal
public import Mathlib.Tactic.NormNum.GCD

@[expose] public section
set_option backward.isDefEq.respectTransparency false in

set_option backward.isDefEq.respectTransparency false in

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
noncomputable def lin : EuclideanSpace □ n →L[□] EuclideanSpace □ n where
set_option backward.isDefEq.respectTransparency false in
noncomputable def ker : Submodule □ (EuclideanSpace □ n) :=
noncomputable def support : Submodule □ (EuclideanSpace □ n) :=
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.443 QuantumInfo/ForMathlib/HermitianMat/CFC.lean

```
module

public import QuantumInfo.ForMathlib.HermitianMat.Inner
public import QuantumInfo.ForMathlib.HermitianMat.NonSingular
public import QuantumInfo.ForMathlib.Isometry
```

```
public import QuantumInfo.ForMathlib.Unitary

public import Mathlib.Analysis.CStarAlgebra.ContinuousFunctionalCalculus.Continuity
public import Mathlib.Analysis.CStarAlgebra.ContinuousFunctionalCalculus.Basic
public import Mathlib.Analysis.CStarAlgebra.ContinuousFunctionalCalculus.Instances
public import Mathlib.Analysis.CStarAlgebra.ContinuousFunctionalCalculus.Commute
public import Mathlib.Analysis.CStarAlgebra.CStarMatrix
public import Mathlib.Algebra.Order.Group.Pointwise.CompleteLattice
public import Mathlib.Topology.TietzeExtension
public import Mathlib.MeasureTheory.Integral.IntervalIntegral.Basic

@[expose] public section
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option maxHeartbeats 400000 in
    obtain ⟨ ε, ε_pos, hε ⟩ := this; exact ⟨ ε, ε_pos, ε, ε_pos, fun B hB s hs => hε
      simp only [Metric.mem_ball, Prod.dist_eq,]
      refine max_lt ?_ ?_
      · simpa [dist_eq_norm] using hB
```

```
      · simpa [dist_eq_norm] using hs) ⟩ ;)))))
  filter_upwards [ Metric.ball_mem_nhds A₀ ( show 0 < Min.min δ 1 by positi
      Classical.not_not.1 fun h => h_not_in_K B
      ( lt_of_lt_of_le (by simpa [dist_eq_norm] using hB) ( min_le_left _ _
      ⟨ hM B ( lt_of_lt_of_le (by simpa [dist_eq_norm] using hB) ( min_le_r
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.444 QuantumInfo/ForMathlib/HermitianMat/Inner.lean

```
module

public import QuantumInfo.ForMathlib.HermitianMat.Order
public import Mathlib.Analysis.Convex.Contractible
public import Mathlib.Topology.Instances.Real.Lemmas
@[expose] public section

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
      simp only [WithLp.ofLp_zero, EuclideanSpace.single] at h2
  simp only [inner_def, mat_reindex, Matrix.reindex_apply, Equiv.symm_symm]
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
@[reducible]
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
@[reducible]
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.445 QuantumInfo/ForMathlib/HermitianMat/Jordan.lean

```
module
public import Mathlib.Algebra.Jordan.Basic

public import QuantumInfo.ForMathlib.HermitianMat.CFC
public import QuantumInfo.ForMathlib.HermitianMat.Order
@[expose] public section

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.446 QuantumInfo/ForMathlib/HermitianMat/LogExp.lean

```
module

public import QuantumInfo.ForMathlib.HermitianMat.Proj
public import Mathlib.MeasureTheory.Integral.IntervalIntegral.FundThmCalculus
public import Mathlib.Analysis.SpecialFunctions.Log.Deriv
@[expose] public section

set_option backward.isDefEq.respectTransparency false in
meta def evalHermitianMatExp : PositivityExt where eval {_u _α} _zα _pα e := do
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
open ComplexOrder MatrixOrder in
  obtain ⟨C, hC⟩ : ∃ C : Matrix d d □, B.mat - A.mat = C.conjTranspose * C := by
    classical
    apply CStarAlgebra.nonneg_iff_eq_star_mul_self.mp
```

```
    simpa using h
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.447 QuantumInfo/ForMathlib/HermitianMat/NonSingular.lean

```
module
public import QuantumInfo.ForMathlib.HermitianMat.Order
public import QuantumInfo.ForMathlib.Isometry

@[expose] public section
noncomputable section
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.448 QuantumInfo/ForMathlib/HermitianMat/Order.lean

```
module

public import QuantumInfo.ForMathlib.HermitianMat.Trace
public import Mathlib.Analysis.RCLike.Basic

@[expose] public section
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
meta def evalHermitianMatTrace : PositivityExt where eval {_u _α} _zα _pα e
set_option backward.isDefEq.respectTransparency false in

set_option backward.isDefEq.respectTransparency false in
meta partial def findMatrixPSDInExpr (e : Expr) (p : Expr) (ty : Expr) :
meta def evalMatrixPSD : PositivityExt where eval {_u _α} _zα _pα e := do
meta partial def findHermitianMatPSDInExpr (e : Expr) (p : Expr) (ty : Expr
meta def evalHermitianMatPSD : PositivityExt where eval {_u _α} _zα _pα e :
meta def evalHermitianMatKronecker : PositivityExt where eval {_u _α} _zα _
meta def evalHermitianMatConj : PositivityExt where eval {_u _α} _zα _pα e
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
meta def evalHermitianMatMat : PositivityExt where eval {_u _α} _zα _pα e :
```

```
meta def evalHermitianMatVal : PositivityExt where eval {_u _α} _zα _pα e := do
meta def evalMatrixSelfMulConjTranspose : PositivityExt where eval {_u _α} _zα _pα e
meta def evalMatrixConjTransposeMulSelf : PositivityExt where eval {_u _α} _zα _pα e
meta def evalHermitianMatMk : PositivityExt where eval {_u _α} _zα _pα e := do
meta def evalMatrixEigenvalues : PositivityExt where eval {_u _α} _zα _pα e := do
```

1.449 QuantumInfo/ForMathlib/HermitianMat/Proj.lean

```
module
public import QuantumInfo.ForMathlib.HermitianMat.CFC

public import Mathlib.Analysis.CStarAlgebra.Classes
public import Mathlib.Analysis.SpecialFunctions.ContinuousFunctionalCalculus.PosPart
@[expose] public section

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.450 QuantumInfo/ForMathlib/HermitianMat/Reindex.lean

```
module

public import QuantumInfo.ForMathlib.HermitianMat.Basic
public import QuantumInfo.ForMathlib.ContinuousLinearMap
public import QuantumInfo.ForMathlib.LinearEquiv

@[expose] public section
set_option backward.isDefEq.respectTransparency false in
```

1.451 QuantumInfo/ForMathlib/HermitianMat/Rpow.lean

```
module

public import QuantumInfo.ForMathlib.HermitianMat.LogExp
public import QuantumInfo.ForMathlib.HermitianMat.Sqrt
public import QuantumInfo.ForMathlib.HermitianMat.Unitary
public import Mathlib.Analysis.SpecialFunctions.Integrability.Basic
public import Mathlib.Analysis.SpecialFunctions.ImproperIntegrals
public import Mathlib.MeasureTheory.Integral.IntegralEqImproper

@[expose] public section
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.452 QuantumInfo/ForMathlib/HermitianMat/Schatten.lean

```
module

public import QuantumInfo.ForMathlib.HermitianMat.Rpow
public import QuantumInfo.ForMathlib.Majorization

@[expose] public section
set_option backward.isDefEq.respectTransparency false in
```

1.453 QuantumInfo/ForMathlib/HermitianMat/Sqrt.lean

```
module

public import QuantumInfo.ForMathlib.HermitianMat.Proj

@[expose] public section
meta def evalHermitianMatSqrt : PositivityExt where eval {_u _α} _zα _pα e
```

1.454 QuantumInfo/ForMathlib/HermitianMat/Trace.lean

```
module

public import QuantumInfo.ForMathlib.HermitianMat.Reindex
@[expose] public section

set_option backward.isDefEq.respectTransparency false in
```

1.455 QuantumInfo/ForMathlib/HermitianMat/Unitary.lean

```
module

public import QuantumInfo.ForMathlib.HermitianMat.Inner
public import QuantumInfo.ForMathlib.HermitianMat.NonSingular
public import QuantumInfo.ForMathlib.Isometry

@[expose] public section
set_option backward.isDefEq.respectTransparency false in
```

1.456 QuantumInfo/ForMathlib/IsMaximalSelfAdjoint.lean

```
module

public import Mathlib.Analysis.Matrix.Normed

@[expose] public section
```

1.457 QuantumInfo/ForMathlib/Isometry.lean

```
module
public import Mathlib.Analysis.InnerProductSpace.JointEigenspace
public import Mathlib.Analysis.SpecialFunctions.Bernstein
public import Mathlib.Analysis.SpecialFunctions.Pow.NNReal
public import Mathlib.LinearAlgebra.Matrix.Permutation
public import Mathlib.LinearAlgebra.Matrix.IsDiag
public import Mathlib.Tactic.NormNum.GCD

public import QuantumInfo.ForMathlib.Matrix

@[expose] public section
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
@[reducible]
    rw [inner_sum, Finset.sum_eq_add_sum_diff_singleton _ _ (by simp)]
    PiLp.ofLp_single, ← mulVec_mulVec, sharedEigenvectorUnitary_mulVec, ← m
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.458 QuantumInfo/ForMathlib/Lieb.lean

```
module

public import QuantumInfo.ForMathlib.HermitianMat
@[expose] public section
```

1.459 QuantumInfo/ForMathlib/LimSupInf.lean

```
module

public import Mathlib.Algebra.Order.Ring.Star
public import Mathlib.Analysis.Normed.Ring.Lemmas
public import Mathlib.Data.Finset.Attr
public import Mathlib.Data.Int.Star
public import Mathlib.Data.Real.StarOrdered
public import Mathlib.Tactic.Bound
public import Mathlib.Tactic.Peel
public import Mathlib.Tactic.Common
public import Mathlib.Tactic.Continuity
public import Mathlib.Tactic.Finiteness.Attr
public import Mathlib.Tactic.SetLike
public import Mathlib.Util.CompileInductive
public import Mathlib.Topology.Instances.ENNReal.Lemmas
public import Mathlib.Topology.Instances.Nat

@[expose] public section
  push Not at h_contra;
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.460 QuantumInfo/ForMathlib/LinearEquiv.lean

```
module

public import Mathlib.Analysis.InnerProductSpace.PiL2

@[expose] public section
```

1.461 QuantumInfo/ForMathlib/Majorization.lean

```
module

public import Mathlib
public import QuantumInfo.ForMathlib.Matrix
@[expose] public section

set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.462 QuantumInfo/ForMathlib/Matrix.lean

```
module

public import Mathlib.Algebra.Algebra.Spectrum.Quasispectrum
public import Mathlib.Algebra.Order.Group.Pointwise.CompleteLattice
public import Mathlib.Analysis.CStarAlgebra.Matrix
public import Mathlib.Analysis.Matrix.Order
public import Mathlib.Analysis.SpecialFunctions.Bernstein
public import Mathlib.Analysis.SpecialFunctions.Pow.NNReal
public import Mathlib.Data.Multiset.Functor --Can't believe I'm having to import thi
public import Mathlib.LinearAlgebra.Matrix.Kronecker
public import Mathlib.LinearAlgebra.Matrix.PosDef
public import Mathlib.LinearAlgebra.Matrix.IsDiag
public import Mathlib.Tactic.Bound
public import Mathlib.Tactic.NormNum.GCD

public import QuantumInfo.ForMathlib.Misc

@[expose] public section
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

```
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
  simp only [not_and, not_forall]
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
          simp [Algebra.algebraMap_eq_smul_one]
set_option backward.isDefEq.respectTransparency false in
    exact ContinuousFunctionalCalculus.spectrum_nonempty _ hA
set_option backward.isDefEq.respectTransparency false in
open ComplexOrder MatrixOrder in
        classical
        apply CStarAlgebra.nonneg_iff_eq_star_mul_self.mp
        exact nonneg_iff_posSemidef.mpr (hA i)
    simp only [dotProduct_mulVec, y];
    simp only [dotProduct, vecMul, Pi.star_apply, RCLike.star_def, mul_comm
      star_prod];
    simp only [mul_apply, of_apply, star_apply, star_prod, RCLike.star_def,
      mul_left_comm, mulVec, dotProduct, mul_comm, map_sum, map_mul, map_pr
      RingHomCompTriple.comp_apply, RingHom.id_apply, mul_assoc];
```

1.463 QuantumInfo/ForMathlib/MatrixNorm/TraceNorm.lean

```
module

public import QuantumInfo.ForMathlib.Matrix
public import QuantumInfo.ForMathlib.Isometry

@[expose] public section
set_option backward.isDefEq.respectTransparency false in

open MatrixOrder Isometry
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.464 QuantumInfo/ForMathlib/Minimax.lean

```
module

public import QuantumInfo.ForMathlib.SionMinimax
public import Mathlib.Analysis.InnerProductSpace.Basic
public import Mathlib.GroupTheory.MonoidLocalization.Basic
public import Mathlib.LinearAlgebra.FiniteDimensional.Defs
public import Mathlib.Topology.Algebra.Module.StrongTopology
public import Mathlib.Topology.Algebra.Module.FiniteDimension

@[expose] public section
```

1.465 QuantumInfo/ForMathlib/Misc.lean

```
module

public import Mathlib.Analysis.SpecialFunctions.Log.Basic
public import Mathlib.Order.CompletePartialOrder

@[expose] public section
```

1.466 QuantumInfo/ForMathlib/SionMinimax.lean

```
module

public import Mathlib.Algebra.Order.Module.Field
public import Mathlib.Analysis.Convex.PathConnected
public import Mathlib.Analysis.Convex.Quasiconvex
public import Mathlib.Analysis.Convex.Topology
public import Mathlib.Analysis.Normed.Order.Lattice
public import Mathlib.Analysis.SpecificLimits.Basic
public import Mathlib.Data.EReal.Operations
public import Mathlib.Data.Fintype.Order
public import Mathlib.Topology.Algebra.InfiniteSum.Order
public import Mathlib.Topology.MetricSpace.Bounded

@[expose] public section
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.467 QuantumInfo/ForMathlib/Superadditive.lean

```
module

public import Mathlib.Analysis.Subadditive

@[expose] public section
```

1.468 QuantumInfo/ForMathlib/Tactic/Commutes.lean

```
module

public import QuantumInfo.ForMathlib.Tactic.Commutes.Attribute
public import Mathlib.Algebra.BigOperators.Group.List.Basic
public import Mathlib.Algebra.BigOperators.Ring.Finset
public import Mathlib.Algebra.BigOperators.Ring.List
public import Mathlib.Algebra.Group.Action.Defs
public import Mathlib.Algebra.Group.Center
public import Mathlib.Algebra.Group.Commute.Basic
public import Mathlib.Algebra.Group.Commute.Hom
public import Mathlib.Algebra.Group.Commute.Units
public import Mathlib.Algebra.Group.Invertible.Basic
public import Mathlib.Algebra.Group.Opposite
public import Mathlib.Algebra.Group.Pi.Lemmas
public import Mathlib.Algebra.Group.Prod
public import Mathlib.Algebra.GroupWithZero.Commute
public import Mathlib.Algebra.GroupWithZero.Semiconj
public import Mathlib.Algebra.Polynomial.Basic
public import Mathlib.Algebra.Ring.Commute
public import Mathlib.Algebra.Ring.Nat
public import Mathlib.Algebra.Star.SelfAdjoint
public import Mathlib.Data.Int.Cast.Lemmas
public import Mathlib.Data.Matrix.Basic
public import Mathlib.Data.Nat.Cast.Commute
public import Mathlib.Data.Rat.Cast.Defs
public import Mathlib.GroupTheory.GroupAction.Ring
public import Mathlib.LinearAlgebra.Matrix.ZPow
@[expose] public section
```

1.469 QuantumInfo/ForMathlib/Tactic/Commutes/Attribute.lean

```
module
```

```
public import Mathlib.Init
public import Aesop.Frontend.Command
@[expose] public section
```

1.470 QuantumInfo/ForMathlib/ULift.lean

```
module

public import Mathlib

@[expose] public section
```

1.471 QuantumInfo/ForMathlib/Unitary.lean

```
module

public import Mathlib.LinearAlgebra.Matrix.Kronecker
public import Mathlib.LinearAlgebra.Matrix.PosDef
public import QuantumInfo.ForMathlib.HermitianMat.Unitary

@[expose] public section
```

1.472 QuantumInfo/InfiniteDim/QState.lean

```
module
```

1.473 QuantumInfo/QI_DOC.md

```
QuantumInfo part of Physlib, explaining the exact conventions adopted, their motivat
```

1.474 QuantumInfo/Regularized.lean

```
module

public import QuantumInfo.ForMathlib.Superadditive
public import Mathlib.Order.LiminfLimsup
public import Mathlib.Topology.Order.MonotoneConvergence
```

```
@[expose] public section
```

1.475 QuantumInfo/StatMech/Hamiltonian.lean

```
module

public import Mathlib.Data.Fintype.Defs
public import Mathlib.Data.Real.Basic

@[expose] public section
```

1.476 QuantumInfo/StatMech/IdealGas.lean

```
module

public import QuantumInfo.StatMech.ThermoQuantities
public import Mathlib.Analysis.SpecialFunctions.Gaussian.FourierTransform

@[expose] public section
set_option backward.isDefEq.respectTransparency false in
    simp only [not_exists, not_lt, Prod.mk.eta]
      fun_prop
      fun_prop
      push Not
    simp only [not_exists, not_lt] at h_integrand_prod
      simp only [Real.norm_eq_abs, sq_abs]
```

1.477 QuantumInfo/StatMech/ThermoQuantities.lean

```
module

public import QuantumInfo.StatMech.Hamiltonian
public import Mathlib.Analysis.SpecialFunctions.Log.Deriv
public import Mathlib.Data.Real.StarOrdered
public import Mathlib.MeasureTheory.Constructions.Pi
public import Mathlib.MeasureTheory.Integral.Bochner.Basic
public import Mathlib.MeasureTheory.Integral.Bochner.L1
public import Mathlib.MeasureTheory.Integral.Bochner.VitaliCaratheodory
public import Mathlib.MeasureTheory.Measure.Haar.OfBasis
public import Mathlib.Order.CompletePartialOrder
```

```
@[expose] public section
set_option backward.isDefEq.respectTransparency false in
set_option backward.isDefEq.respectTransparency false in
```

1.478 README.md

```
<img src="./docs/Physlib-logo.jpeg" alt="Physlib logo" width="500">
[![](https://img.shields.io/badge/Getting-Started-darkgreen)](https://physlib.io/Get
[![](https://img.shields.io/badge/The-Website-darkgreen)](https://physlib.io)
[![](https://img.shields.io/badge/How_To-Get_Involved-darkgreen)](https://physlib.io
[![](https://img.shields.io/badge/Physlib_Zulip-Discussion-
darkgreen)](https://leanprover.zulipchat.com/#narrow/channel/479953-
Physlib/)
[![](https://img.shields.io/badge/TODO-List-darkgreen)](https://physlib.io/TODOList)
[![](https://img.shields.io/badge/View_The-Stats-blue)](https://physlib.io/Stats)
[![](https://img.shields.io/badge/Lean-v4.29.1-blue)](https://github.com/leanprover/
[![Gitpod Ready-to-Code](https://img.shields.io/badge/Gitpod-
ready--to--code-blue?logo=gitpod)](https://gitpod.io/#https://github.com/leanprover-
community/physlib)
[![Ask DeepWiki](https://deepwiki.com/badge.svg)](https://deepwiki.com/leanprover-
community/physlib)
[![api_docs](https://img.shields.io/badge/doc-API_docs-blue)](https://physlib.io/doc
## Contributing to Physlib
Physlib is open-source and community run, and we welcome contributions from anyone.
- our [todo list](https://physlib.io/TODOList)
- our [Get Involved page](https://physlib.io/GetInvolved.html)
> If stuck at any point there are lots of people happly to help on the [Physlib zuli
Physlib)
### Installing Physlib
At the moment Physlib is divided into two essentially disjoint halves, `Physlib` and
they offer two separate _build targets_: `Physlib` and `QuantumInfo`, as specified i
If you only want to build one, `lake build Physlib` or `lake build QuantumInfo` will
- Guide to [Physlib reviews](https://github.com/leanprover-
community/physlib/blob/master/docs/ReviewGuidelines.md).
> When making contributing to Physlib it is much better to do it with small steps. T
on the [Lean Zulip](https://leanprover.zulipchat.com/#narrow/channel/479953-
Physlib)
```bibtex
  author = {The Physlib community},
  title = {Physlib: The Lean Physics Library},
Physlib was formed by merging the general physics Lean library PhysLean (formerly ca
```bibtex
```bibtex
```
```

1.479 docs/MaintainerTeam.md

```
experience with either the Physlib or Mathlib review process.
```

1.480 docs/ReviewGuidelines.md

```
Below we give guidelines to what reviews of pull-requests (PRs) to Physlib
  post [here](https://leanprover.zulipchat.com/#narrow/channel/479953-
Physlib/topic/PR.20reviews/with/577663418).
```

1.481 lake-manifest.json


```
 [{"url": "https://github.com/leanprover-community/mathlib4.git",
   "rev": "5e932f97dd25535344f80f9dd8da3aab83df0fe6",
   "inputRev": "v4.29.1",
  {"url": "https://github.com/leanprover/doc-gen4",
   "rev": "aa4c3e4e14f5b31495b7c7238762eccedd9f52c",
   "name": "«doc-gen4»",
   "inputRev": "v4.29.0",
   "inherited": false,
   "rev": "83e90935a17ca19ebe4b7893c7f7066e266f50d3",
   "rev": "48d5698bc464786347c1b0d859b18f938420f060",
   "rev": "4dd0959c44d1af0462bd604d0f87c5781307d709",
   "inputRev": "v0.0.95+lean-v4.29.1",
   "rev": "7152850e7b216a0d409701617721b6e469d34bf6",
   "rev": "707efb56d0696634e9e965523a1bbe9ac6ce141d",
   "rev": "756e3321fd3b02a85ffda19fef789916223e578c",
   "configFile": "lakefile.toml"},
  {"url": "https://github.com/leanprover/lean4-cli",
   "type": "git",
   "subDir": null,
   "scope": "leanprover",
   "rev": "7802da01beb530bf051ab657443f9cd9bc3e1a29",
   "name": "Cli",
   "manifestFile": "lake-manifest.json",
   "inputRev": "v4.29.0",
   "inherited": true,
   "configFile": "lakefile.toml"},
  {"url": "https://github.com/kim-em/leansqlite",
   "type": "git",
   "subDir": null,
   "scope": "",
   "rev": "d14544c72b593af6a66131bc34cdab16bf7c0940",
```

```
   "name": "leansqlite",
   "manifestFile": "lake-manifest.json",
   "inputRev": "suppress-reducibility-warning",
   "inherited": true,
   "configFile": "lakefile.lean"},
  {"url": "https://github.com/fgdorais/lean4-unicode-basic",
   "type": "git",
   "subDir": null,
   "scope": "",
   "rev": "9539e34e5cb2d52a6454d9b6218f6b6835cad071",
   "name": "UnicodeBasic",
   "manifestFile": "lake-manifest.json",
   "inputRev": "main",
   "inherited": true,
   "configFile": "lakefile.lean"},
  {"url": "https://github.com/dupuisf/BibtexQuery",
   "type": "git",
   "subDir": null,
   "scope": "",
   "rev": "5d31b64fb703c5d77f6ef4d1fb958f9bdf1ea539",
   "name": "BibtexQuery",
   "manifestFile": "lake-manifest.json",
   "inputRev": "nightly-testing",
   "inherited": true,
   "configFile": "lakefile.toml"},
  {"url": "https://github.com/acmepjz/md4lean",
   "type": "git",
   "subDir": null,
   "scope": "",
   "rev": "6a3fb240133bcb7e1a066fdc784b3fdc304e3fc5",
   "name": "MD4Lean",
   "manifestFile": "lake-manifest.json",
   "inputRev": "main",
   "inherited": true,
   "configFile": "lakefile.lean"}],
 "name": "Physlib",
```


1.482 lakefile.lean

```
package «Physlib»
require «doc-gen4» from git "https://github.com/leanprover/doc-
gen4" @ "v4.29.0"
require "mathlib" from git "https://github.com/leanprover-
community/mathlib4.git" @ "v4.29.1"
```

```
lean_lib Physlib where
  moreLeanArgs := #[
    "-Dwarn.sorry=false",
    "-Dweak.says.verify=true"
  ]
lean_lib QuantumInfo where
  moreLeanArgs := #[
    "-Dwarn.sorry=false",
    "-Dweak.says.verify=true"
  ]
lean_exe runPhyslibLinters where
lean_exe mathlib_textLint_on_physlib where
-- lean_exe commands defined specifically for Physlib.
```

1.483 lean-toolchain

```
leanprover/lean4:v4.29.1
```

1.484 scripts/MetaPrograms/TODO_to_yml.lean

```
import Physlib.Meta.Basic
import Physlib.Meta.TODO.Basic
import Physlib.Meta.Informal.Post
import Physlib.Meta.Informal.SemiFormal
import Physlib.Meta.Linters.Sorry
open Lean System Meta Physlib
## Physlib categories.
inductive PhyslibCategory
def PhyslibCategory.string :  PhyslibCategory → String
def PhyslibCategory.emoji : PhyslibCategory → String
def PhyslibCategory.List :  List PhyslibCategory :=
  [ PhyslibCategory.ClassicalMechanics,
    PhyslibCategory.CondensedMatter,
    PhyslibCategory.Cosmology,
    PhyslibCategory.Elctromagnetism,
    PhyslibCategory.Mathematics,
    PhyslibCategory.Meta,
    PhyslibCategory.Optics,
    PhyslibCategory.Particles,
    PhyslibCategory.QFT,
    PhyslibCategory.QuantumMechanics,
    PhyslibCategory.Relativity,
    PhyslibCategory.StringTheory,
```

```
    PhyslibCategory.StatisticalMechanics,
    PhyslibCategory.Thermodynamics,
    PhyslibCategory.Other]

instance : ToString PhyslibCategory where
  toString := PhyslibCategory.string

def PhyslibCategory.ofFileName (n : Name) : PhyslibCategory :=
  if n.toString.startsWith "Physlib.ClassicalMechanics"  then
    PhyslibCategory.ClassicalMechanics
  else if n.toString.startsWith "Physlib.CondensedMatter" then
    PhyslibCategory.CondensedMatter
  else if n.toString.startsWith "Physlib.Cosmology" then
    PhyslibCategory.Cosmology
  else if n.toString.startsWith "Physlib.Electromagnetism" then
    PhyslibCategory.Elctromagnetism
  else if n.toString.startsWith "Physlib.Mathematics" then
    PhyslibCategory.Mathematics
  else if n.toString.startsWith "Physlib.Meta" then
    PhyslibCategory.Meta
  else if n.toString.startsWith "Physlib.Optics" then
    PhyslibCategory.Optics
  else if n.toString.startsWith "Physlib.Particles" then
    PhyslibCategory.Particles
  else if n.toString.startsWith "Physlib.QFT" then
    PhyslibCategory.QFT
  else if n.toString.startsWith "Physlib.QuantumMechanics" then
    PhyslibCategory.QuantumMechanics
  else if n.toString.startsWith "Physlib.Relativity" then
    PhyslibCategory.Relativity
  else if n.toString.startsWith "Physlib.StatisticalMechanics" then
    PhyslibCategory.StatisticalMechanics
  else if n.toString.startsWith "Physlib.Thermodynamics" then
    PhyslibCategory.Thermodynamics
    PhyslibCategory.Other
  category : PhyslibCategory
   isSemiFormalResult := false, category := PhyslibCategory.ofFileName t.fileName,
  let category := PhyslibCategory.ofFileName file
   isSemiFormalResult := true, category := PhyslibCategory.ofFileName t.fileName,
  category := PhyslibCategory.ofFileName s.fileName
  for c in PhyslibCategory.List do
  let env ← importModules (loadExts := true) #[`Physlib] {} 0
    -- Create directory if it doesn't exist
    let dir : System.FilePath := {toString := "./docs/_data"}
    if !(← System.FilePath.pathExists dir) then
      IO.FS.createDirAll dir
```

1.485 scripts/MetaPrograms/check_dup_tags.lean

```
import Physlib.Meta.Basic
import Physlib.Meta.TODO.Basic
import Physlib.Meta.Informal.Post
import Physlib.Meta.Informal.SemiFormal
open Lean System Meta Physlib Core
unsafe def main (_ : List String) : IO UInt32 := do
  let env ← importModules (loadExts := true) #[`Physlib] {} 0
```

1.486 scripts/MetaPrograms/check_rfl.lean

```
import Physlib.Meta.AllFilePaths
import Physlib.Meta.TransverseTactics
```

1.487 scripts/MetaPrograms/free_simps.lean

```
import Physlib.Meta.AllFilePaths
import Physlib.Meta.TransverseTactics
```

1.488 scripts/MetaPrograms/informal.lean

```
import Physlib.Meta.Informal.Post
appearing in the raw Lean code of Physlib.
 informal definitions and lemmas. You may also wish to contribute to Physli
    tooltip = \"Informal Physlib graph\";
    <title>Informal dependency graph for Physlib</title>
    <h1 style=\"text-align: center;\">Informal dependency graph for Physlib
    It does not show all results formalised into Physlib.</p>
  let rootModule := `Physlib
```

1.489 scripts/MetaPrograms/local_stats.lean

```
import Physlib.Meta.Informal.Post
import Physlib.Meta.AllFilePaths
# Local stats within Physlib
open Lean System Meta Physlib
  let imports ← Physlib.allImports
```

```
  let x ← imports.mapM Physlib.Imports.getUserConsts
  let imports ← Physlib.allImports
  let x ← imports.mapM Physlib.Imports.getUserConsts
  let statString ← CoreM.withImportModules #[`Physlib] (getStats dir).run'
```

1.490 scripts/MetaPrograms/module_doc_lint.lean

```
  let mods : Name := `Physlib
  let (physlibMod, _) ← readModuleData mFile
  let filePaths := physlibMod.imports.filterMap (fun imp ↦
```

1.491 scripts/MetaPrograms/module_doc_no_lint.txt

```
Physlib/ClassicalMechanics/Basic.lean
Physlib/ClassicalMechanics/EulerLagrange.lean
Physlib/ClassicalMechanics/HamiltonsEquations.lean
Physlib/ClassicalMechanics/Mass/MassUnit.lean
Physlib/ClassicalMechanics/RigidBody/Basic.lean
Physlib/ClassicalMechanics/RigidBody/SolidSphere.lean
Physlib/ClassicalMechanics/Scattering/RigidSphere.lean
Physlib/ClassicalMechanics/VectorFields.lean
Physlib/ClassicalMechanics/Vibrations/LinearTriatomic.lean
Physlib/ClassicalMechanics/WaveEquation/HarmonicWave.lean
Physlib/CondensedMatter/Basic.lean
Physlib/Cosmology/Basic.lean
Physlib/Cosmology/FLRW/Basic.lean
Physlib/Electromagnetism/Basic.lean
Physlib/Electromagnetism/Charge/ChargeUnit.lean
Physlib/Electromagnetism/Electrostatics/Basic.lean
Physlib/Electromagnetism/Electrostatics/OneDimension/Vacuum.lean
Physlib/Electromagnetism/Electrostatics/ThreeDimension/InfinitePlane.lean
Physlib/Electromagnetism/Electrostatics/ThreeDimension/PointParticle.lean
Physlib/Electromagnetism/FieldStrength/Basic.lean
Physlib/Electromagnetism/FieldStrength/Derivative.lean
Physlib/Electromagnetism/Homogeneous.lean
Physlib/Electromagnetism/MaxwellEquations.lean
Physlib/Mathematics/Calculus/AdjFDeriv.lean
Physlib/Mathematics/Calculus/Divergence.lean
Physlib/Mathematics/DataStructures/FourTree/Basic.lean
Physlib/Mathematics/DataStructures/FourTree/UniqueMap.lean
Physlib/Mathematics/DataStructures/Matrix/LieTrace.lean
Physlib/Mathematics/Distribution/Basic.lean
Physlib/Mathematics/Distribution/Function/InvPowMeasure.lean
```

Physlib/Mathematics/Distribution/Function/IsDistBounded.lean
Physlib/Mathematics/Distribution/Function/OfFunction.lean
Physlib/Mathematics/Distribution/PowMul.lean
Physlib/Mathematics/FDerivCurry.lean
Physlib/Mathematics/Fin.lean
Physlib/Mathematics/Fin/Involutions.lean
Physlib/Mathematics/Geometry/Metric/PseudoRiemannian/Defs.lean
Physlib/Mathematics/Geometry/Metric/Riemannian/Defs.lean
Physlib/Mathematics/InnerProductSpace/Adjoint.lean
Physlib/Mathematics/InnerProductSpace/Basic.lean
Physlib/Mathematics/InnerProductSpace/Calculus.lean
Physlib/Mathematics/LinearMaps.lean
Physlib/Mathematics/List.lean
Physlib/Mathematics/List/InsertIdx.lean
Physlib/Mathematics/List/InsertionSort.lean
Physlib/Mathematics/PiTensorProduct.lean
Physlib/Mathematics/RatComplexNum.lean
Physlib/Mathematics/SO3/Basic.lean
Physlib/Mathematics/SchurTriangulation.lean
Physlib/Mathematics/SpecialFunctions/PhysHermite.lean
Physlib/Mathematics/Trigonometry/Tanh.lean
Physlib/Mathematics/VariationalCalculus/Basic.lean
Physlib/Mathematics/VariationalCalculus/HasVarAdjDeriv.lean
Physlib/Mathematics/VariationalCalculus/HasVarAdjoint.lean
Physlib/Mathematics/VariationalCalculus/HasVarGradient.lean
Physlib/Mathematics/VariationalCalculus/IsLocalizedfunctionTransform.lean
Physlib/Mathematics/VariationalCalculus/IsTestFunction.lean
Physlib/Meta/AllFilePaths.lean
Physlib/Meta/Basic.lean
Physlib/Meta/Informal/Basic.lean
Physlib/Meta/Informal/Post.lean
Physlib/Meta/Informal/SemiFormal.lean
Physlib/Meta/Linters/Sorry.lean
Physlib/Meta/Notes/Basic.lean
Physlib/Meta/Notes/HTMLNote.lean
Physlib/Meta/Notes/NoteFile.lean
Physlib/Meta/Notes/ToHTML.lean
Physlib/Meta/Remark/Basic.lean
Physlib/Meta/Remark/Properties.lean
Physlib/Meta/TODO/Basic.lean
Physlib/Meta/TransverseTactics.lean
Physlib/Optics/Basic.lean
Physlib/Optics/Polarization/Basic.lean
Physlib/Particles/BeyondTheStandardModel/GeorgiGlashow/Basic.lean
Physlib/Particles/BeyondTheStandardModel/PatiSalam/Basic.lean
Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/Basic.lean

```
Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/FamilyMaps.lean
Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/NoGrav/Basic.lean
Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/Ordinary/Basic.lean
Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/Ordinary/DimSevenPl
Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/Ordinary/FamilyMaps
Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/Permutations.lean
Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/BMinusL.lean
Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/Basic.lean
Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/BoundPlaneDi
Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/FamilyMaps.l
Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/HyperCharge.
Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/PlaneNonSols
Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/QuadSol.lean
Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/QuadSolToSol
Physlib/Particles/BeyondTheStandardModel/Spin10/Basic.lean
Physlib/Particles/BeyondTheStandardModel/TwoHDM/Basic.lean
Physlib/Particles/BeyondTheStandardModel/TwoHDM/GaugeOrbits.lean
Physlib/Particles/FlavorPhysics/CKMMatrix/Basic.lean
Physlib/Particles/FlavorPhysics/CKMMatrix/Invariants.lean
Physlib/Particles/FlavorPhysics/CKMMatrix/PhaseFreedom.lean
Physlib/Particles/FlavorPhysics/CKMMatrix/Relations.lean
Physlib/Particles/FlavorPhysics/CKMMatrix/Rows.lean
Physlib/Particles/FlavorPhysics/CKMMatrix/StandardParameterization/Basic.lean
Physlib/Particles/FlavorPhysics/CKMMatrix/StandardParameterization/StandardParameter
Physlib/Particles/StandardModel/AnomalyCancellation/Basic.lean
Physlib/Particles/StandardModel/AnomalyCancellation/FamilyMaps.lean
Physlib/Particles/StandardModel/AnomalyCancellation/NoGrav/Basic.lean
Physlib/Particles/StandardModel/AnomalyCancellation/NoGrav/One/Lemmas.lean
Physlib/Particles/StandardModel/AnomalyCancellation/NoGrav/One/LinearParameterizatio
Physlib/Particles/StandardModel/AnomalyCancellation/Permutations.lean
Physlib/Particles/StandardModel/Basic.lean
Physlib/Particles/StandardModel/HiggsBoson/Potential.lean
Physlib/Particles/StandardModel/Representations.lean
Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/B3.lean
Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/Basic.lean
Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/HyperCharge.lean
Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/LineY3B3.lean
Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/OrthogY3B3/Basic.lean
Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/OrthogY3B3/PlaneWithY3B3.
Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/OrthogY3B3/ToSols.lean
Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/Permutations.lean
Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/Y3.lean
Physlib/QFT/AnomalyCancellation/GroupActions.lean
Physlib/QFT/PerturbationTheory/CreateAnnihilate.lean
Physlib/QFT/PerturbationTheory/FeynmanDiagrams/Basic.lean
Physlib/QFT/PerturbationTheory/FieldOpFreeAlgebra/Basic.lean
```

Physlib/QFT/PerturbationTheory/FieldOpFreeAlgebra/Grading.lean
Physlib/QFT/PerturbationTheory/FieldOpFreeAlgebra/NormTimeOrder.lean
Physlib/QFT/PerturbationTheory/FieldOpFreeAlgebra/NormalOrder.lean
Physlib/QFT/PerturbationTheory/FieldOpFreeAlgebra/SuperCommute.lean
Physlib/QFT/PerturbationTheory/FieldOpFreeAlgebra/TimeOrder.lean
Physlib/QFT/PerturbationTheory/FieldSpecification/Basic.lean
Physlib/QFT/PerturbationTheory/FieldSpecification/CrAnFieldOp.lean
Physlib/QFT/PerturbationTheory/FieldSpecification/CrAnSection.lean
Physlib/QFT/PerturbationTheory/FieldSpecification/Filters.lean
Physlib/QFT/PerturbationTheory/FieldSpecification/NormalOrder.lean
Physlib/QFT/PerturbationTheory/FieldSpecification/TimeOrder.lean
Physlib/QFT/PerturbationTheory/FieldStatistics/Basic.lean
Physlib/QFT/PerturbationTheory/FieldStatistics/ExchangeSign.lean
Physlib/QFT/PerturbationTheory/FieldStatistics/OfFinset.lean
Physlib/QFT/PerturbationTheory/Koszul/KoszulSign.lean
Physlib/QFT/PerturbationTheory/Koszul/KoszulSignInsert.lean
Physlib/QFT/PerturbationTheory/WickAlgebra/Basic.lean
Physlib/QFT/PerturbationTheory/WickAlgebra/Grading.lean
Physlib/QFT/PerturbationTheory/WickAlgebra/NormalOrder/Basic.lean
Physlib/QFT/PerturbationTheory/WickAlgebra/NormalOrder/Lemmas.lean
Physlib/QFT/PerturbationTheory/WickAlgebra/NormalOrder/WickContractions.lea
Physlib/QFT/PerturbationTheory/WickAlgebra/StaticWickTerm.lean
Physlib/QFT/PerturbationTheory/WickAlgebra/StaticWickTheorem.lean
Physlib/QFT/PerturbationTheory/WickAlgebra/SuperCommute.lean
Physlib/QFT/PerturbationTheory/WickAlgebra/TimeContraction.lean
Physlib/QFT/PerturbationTheory/WickAlgebra/TimeOrder.lean
Physlib/QFT/PerturbationTheory/WickAlgebra/Universality.lean
Physlib/QFT/PerturbationTheory/WickAlgebra/WickTerm.lean
Physlib/QFT/PerturbationTheory/WickAlgebra/WicksTheorem.lean
Physlib/QFT/PerturbationTheory/WickAlgebra/WicksTheoremNormal.lean
Physlib/QFT/PerturbationTheory/WickContraction/Basic.lean
Physlib/QFT/PerturbationTheory/WickContraction/Card.lean
Physlib/QFT/PerturbationTheory/WickContraction/Erase.lean
Physlib/QFT/PerturbationTheory/WickContraction/ExtractEquiv.lean
Physlib/QFT/PerturbationTheory/WickContraction/InsertAndContract.lean
Physlib/QFT/PerturbationTheory/WickContraction/InsertAndContractNat.lean
Physlib/QFT/PerturbationTheory/WickContraction/Involutions.lean
Physlib/QFT/PerturbationTheory/WickContraction/IsFull.lean
Physlib/QFT/PerturbationTheory/WickContraction/Join.lean
Physlib/QFT/PerturbationTheory/WickContraction/Perm.lean
Physlib/QFT/PerturbationTheory/WickContraction/Sign/Basic.lean
Physlib/QFT/PerturbationTheory/WickContraction/Sign/InsertNone.lean
Physlib/QFT/PerturbationTheory/WickContraction/Sign/InsertSome.lean
Physlib/QFT/PerturbationTheory/WickContraction/Sign/Join.lean
Physlib/QFT/PerturbationTheory/WickContraction/Singleton.lean
Physlib/QFT/PerturbationTheory/WickContraction/StaticContract.lean

```
Physlib/QFT/PerturbationTheory/WickContraction/SubContraction.lean
Physlib/QFT/PerturbationTheory/WickContraction/TimeCond.lean
Physlib/QFT/PerturbationTheory/WickContraction/TimeContract.lean
Physlib/QFT/PerturbationTheory/WickContraction/Uncontracted.lean
Physlib/QFT/PerturbationTheory/WickContraction/UncontractedList.lean
Physlib/QFT/QED/AnomalyCancellation/Basic.lean
Physlib/QFT/QED/AnomalyCancellation/BasisLinear.lean
Physlib/QFT/QED/AnomalyCancellation/ConstAbs.lean
Physlib/QFT/QED/AnomalyCancellation/Even/LineInCubic.lean
Physlib/QFT/QED/AnomalyCancellation/Even/Parameterization.lean
Physlib/QFT/QED/AnomalyCancellation/LineInPlaneCond.lean
Physlib/QFT/QED/AnomalyCancellation/LowDim/One.lean
Physlib/QFT/QED/AnomalyCancellation/LowDim/Three.lean
Physlib/QFT/QED/AnomalyCancellation/LowDim/Two.lean
Physlib/QFT/QED/AnomalyCancellation/Odd/LineInCubic.lean
Physlib/QFT/QED/AnomalyCancellation/Odd/Parameterization.lean
Physlib/QFT/QED/AnomalyCancellation/Permutations.lean
Physlib/QFT/QED/AnomalyCancellation/Sorts.lean
Physlib/QFT/QED/AnomalyCancellation/VectorLike.lean
Physlib/QuantumMechanics/FiniteTarget/Basic.lean
Physlib/QuantumMechanics/FiniteTarget/HilbertSpace.lean
Physlib/QuantumMechanics/OneDimension/GeneralPotential/Basic.lean
Physlib/QuantumMechanics/OneDimension/HarmonicOscillator/Basic.lean
Physlib/QuantumMechanics/OneDimension/HarmonicOscillator/Completeness.lean
Physlib/QuantumMechanics/OneDimension/HarmonicOscillator/Eigenfunction.lean
Physlib/QuantumMechanics/OneDimension/HarmonicOscillator/TISE.lean
Physlib/QuantumMechanics/OneDimension/HilbertSpace/Basic.lean
Physlib/QuantumMechanics/OneDimension/HilbertSpace/Gaussians.lean
Physlib/QuantumMechanics/OneDimension/HilbertSpace/PlaneWaves.lean
Physlib/QuantumMechanics/OneDimension/HilbertSpace/PositionStates.lean
Physlib/QuantumMechanics/OneDimension/HilbertSpace/SchwartzSubmodule.lean
Physlib/QuantumMechanics/OneDimension/Operators/Commutation.lean
Physlib/QuantumMechanics/OneDimension/Operators/Momentum.lean
Physlib/QuantumMechanics/OneDimension/Operators/Parity.lean
Physlib/QuantumMechanics/OneDimension/Operators/Position.lean
Physlib/QuantumMechanics/OneDimension/Operators/Unbounded.lean
Physlib/QuantumMechanics/OneDimension/ReflectionlessPotential/Basic.lean
Physlib/QuantumMechanics/PlanckConstant.lean
Physlib/Relativity/Bispinors/Basic.lean
Physlib/Relativity/CliffordAlgebra.lean
Physlib/Relativity/LorentzAlgebra/Basic.lean
Physlib/Relativity/LorentzAlgebra/Basis.lean
Physlib/Relativity/LorentzAlgebra/ExponentialMap.lean
Physlib/Relativity/LorentzGroup/Basic.lean
Physlib/Relativity/LorentzGroup/Boosts/Apply.lean
Physlib/Relativity/LorentzGroup/Boosts/Basic.lean
```

Physlib/Relativity/LorentzGroup/Boosts/Generalized.lean
Physlib/Relativity/LorentzGroup/Orthochronous/Basic.lean
Physlib/Relativity/LorentzGroup/Proper.lean
Physlib/Relativity/LorentzGroup/Restricted/Basic.lean
Physlib/Relativity/LorentzGroup/Restricted/FromBoostRotation.lean
Physlib/Relativity/LorentzGroup/Rotations.lean
Physlib/Relativity/LorentzGroup/ToVector.lean
Physlib/Relativity/PauliMatrices/AsTensor.lean
Physlib/Relativity/PauliMatrices/Basic.lean
Physlib/Relativity/PauliMatrices/CliffordAlgebra.lean
Physlib/Relativity/PauliMatrices/Relations.lean
Physlib/Relativity/PauliMatrices/SelfAdjoint.lean
Physlib/Relativity/PauliMatrices/ToTensor.lean
Physlib/Relativity/SL2C/Basic.lean
Physlib/Relativity/SL2C/SelfAdjoint.lean
Physlib/Relativity/Special/ProperTime.lean
Physlib/Relativity/Special/TwinParadox/Basic.lean
Physlib/Relativity/Tensors/Basic.lean
Physlib/Relativity/Tensors/Color/Basic.lean
Physlib/Relativity/Tensors/Color/Discrete.lean
Physlib/Relativity/Tensors/Color/Lift.lean
Physlib/Relativity/Tensors/ComplexTensor/Basic.lean
Physlib/Relativity/Tensors/ComplexTensor/Lemmas.lean
Physlib/Relativity/Tensors/ComplexTensor/Matrix/Pre.lean
Physlib/Relativity/Tensors/ComplexTensor/Metrics/Basic.lean
Physlib/Relativity/Tensors/ComplexTensor/Metrics/Lemmas.lean
Physlib/Relativity/Tensors/ComplexTensor/Metrics/Pre.lean
Physlib/Relativity/Tensors/ComplexTensor/OfRat.lean
Physlib/Relativity/Tensors/ComplexTensor/Units/Basic.lean
Physlib/Relativity/Tensors/ComplexTensor/Units/Pre.lean
Physlib/Relativity/Tensors/ComplexTensor/Units/Symm.lean
Physlib/Relativity/Tensors/ComplexTensor/Vector/Pre/Basic.lean
Physlib/Relativity/Tensors/ComplexTensor/Vector/Pre/Contraction.lean
Physlib/Relativity/Tensors/ComplexTensor/Vector/Pre/Modules.lean
Physlib/Relativity/Tensors/ComplexTensor/Weyl/Basic.lean
Physlib/Relativity/Tensors/ComplexTensor/Weyl/Contraction.lean
Physlib/Relativity/Tensors/ComplexTensor/Weyl/Metric.lean
Physlib/Relativity/Tensors/ComplexTensor/Weyl/Modules.lean
Physlib/Relativity/Tensors/ComplexTensor/Weyl/Two.lean
Physlib/Relativity/Tensors/ComplexTensor/Weyl/Unit.lean
Physlib/Relativity/Tensors/Constructors.lean
Physlib/Relativity/Tensors/Contraction/Basic.lean
Physlib/Relativity/Tensors/Contraction/Basis.lean
Physlib/Relativity/Tensors/Contraction/Products.lean
Physlib/Relativity/Tensors/Contraction/Pure.lean
Physlib/Relativity/Tensors/Dual.lean

```
Physlib/Relativity/Tensors/Elab.lean
Physlib/Relativity/Tensors/Evaluation.lean
Physlib/Relativity/Tensors/MetricTensor.lean
Physlib/Relativity/Tensors/OfInt.lean
Physlib/Relativity/Tensors/RealTensor/Basic.lean
Physlib/Relativity/Tensors/RealTensor/CoVector/Basic.lean
Physlib/Relativity/Tensors/RealTensor/Derivative.lean
Physlib/Relativity/Tensors/RealTensor/Matrix/Pre.lean
Physlib/Relativity/Tensors/RealTensor/Metrics/Basic.lean
Physlib/Relativity/Tensors/RealTensor/Metrics/Pre.lean
Physlib/Relativity/Tensors/RealTensor/ToComplex.lean
Physlib/Relativity/Tensors/RealTensor/Units/Pre.lean
Physlib/Relativity/Tensors/RealTensor/Vector/Basic.lean
Physlib/Relativity/Tensors/RealTensor/Vector/Causality/Basic.lean
Physlib/Relativity/Tensors/RealTensor/Vector/Causality/LightLike.lean
Physlib/Relativity/Tensors/RealTensor/Vector/Causality/TimeLike.lean
Physlib/Relativity/Tensors/RealTensor/Vector/MinkowskiProduct.lean
Physlib/Relativity/Tensors/RealTensor/Vector/Pre/Basic.lean
Physlib/Relativity/Tensors/RealTensor/Vector/Pre/Contraction.lean
Physlib/Relativity/Tensors/RealTensor/Vector/Pre/Modules.lean
Physlib/Relativity/Tensors/RealTensor/Velocity/Basic.lean
Physlib/Relativity/Tensors/TensorSpecies/Basic.lean
Physlib/Relativity/Tensors/UnitTensor.lean
Physlib/SpaceAndTime/Space/Basic.lean
Physlib/SpaceAndTime/Space/Distributions/Basic.lean
Physlib/SpaceAndTime/Space/LengthUnit.lean
Physlib/SpaceAndTime/Space/Translations.lean
Physlib/SpaceAndTime/SpaceTime/TimeSlice.lean
Physlib/SpaceAndTime/Time/TimeMan.lean
Physlib/SpaceAndTime/Time/TimeTransMan.lean
Physlib/SpaceAndTime/Time/TimeUnit.lean
Physlib/StatisticalMechanics/BoltzmannConstant.lean
Physlib/StatisticalMechanics/CanonicalEnsemble/Basic.lean
Physlib/StatisticalMechanics/CanonicalEnsemble/Finite.lean
Physlib/StatisticalMechanics/CanonicalEnsemble/Lemmas.lean
Physlib/StatisticalMechanics/CanonicalEnsemble/TwoState.lean
Physlib/StringTheory/Basic.lean
Physlib/StringTheory/FTheory/SU5/Basic.lean
Physlib/Thermodynamics/Basic.lean
Physlib/Thermodynamics/Temperature/Basic.lean
Physlib/Thermodynamics/Temperature/TemperatureUnits.lean
Physlib/Units/Basic.lean
Physlib/Units/Dimension.lean
Physlib/Units/Examples.lean
Physlib/Units/FDeriv.lean
Physlib/Units/Integral.lean
```

```
Physlib/Units/UnitDependent.lean
Physlib/Units/WithDim/Area.lean
Physlib/Units/WithDim/Basic.lean
Physlib/Units/WithDim/Energy.lean
Physlib/Units/WithDim/Mass.lean
Physlib/Units/WithDim/Momentum.lean
Physlib/Units/WithDim/Pressure.lean
Physlib/Units/WithDim/Speed.lean
Physlib/Units/WithDim/Velocity.lean
Physlib/Electromagnetism/Vacuum/Homogeneous.lean
Physlib/Electromagnetism/Vacuum/OneDimension.lean
Physlib/Particles/NeutrinoPhysics/Basic.lean
Physlib/QuantumMechanics/OneDimension/HarmonicOscillator/Examples.lean
```

1.492 scripts/MetaPrograms/notes.lean

```
import Physlib.Meta.Basic
import Physlib.Meta.Remark.Properties
import Physlib.Meta.Notes.ToHTML
import Physlib
open Lean System Meta Physlib
      as it appears in Physlib. Not every lemma or definition is covered he
     Lean 4, as part of the larger project Physlib.
     motivations for doing this which are discussed <a href = 'https://phys
    .name ``Physlib.physHermite .complete,
      implementation of tensors and index notation into Physlib, and
  let ymlString ← CoreM.withImportModules #[`Physlib] (n.toYML).run'
```

1.493 scripts/MetaPrograms/redundant_imports.lean

```
import Physlib.Meta.Basic
import ImportGraph
open Lean System Meta Physlib
  let _ ← CoreM.withImportModules #[`Physlib] (imports.mapM Imports.Redunda
```

1.494 scripts/MetaPrograms/regex_lint.lean

```
def PhyslibTextLinter : Type := Array String → Array (String × ℕ × ℕ)
def unneededParentheses : PhyslibTextLinter := fun lines ↦ Id.run do
def doubleEmptyLineLinter : PhyslibTextLinter := fun lines ↦ Id.run do
def doubleSpaceLinter : PhyslibTextLinter := fun lines ↦ Id.run do
def longLineLinter : PhyslibTextLinter := fun lines ↦ Id.run do
```

```
def substringLinter (s : String) : PhyslibTextLinter := fun lines ↦ Id.run do
def endLineLinter (s : String) : PhyslibTextLinter := fun lines ↦ Id.run do
def numInitialSpacesEven : PhyslibTextLinter := fun lines ↦ Id.run do
structure PhyslibErrorContext where
def printErrors (errors : Array PhyslibErrorContext) : IO Unit := do
def physlibLintFile (path : FilePath) : IO (Array PhyslibErrorContext) := do
    (Array.map (fun (e, n, c) ↦ PhyslibErrorContext.mk e n c path)) (lint lines)))
  let mods : Name :=  `Physlib
  let (physlibMod, _) ← readModuleData mFile
  let filePaths := physlibMod.imports.filterMap (fun imp ↦
  let errors := (← filePaths.mapM physlibLintFile).flatten
```

1.495 scripts/MetaPrograms/runPhyslibLinters.lean

```
A minimized version of the Batteries script runLinter dedicated to Physlib.
  let modulesToLint := #[`Physlib]
```

1.496 scripts/MetaPrograms/sorry_lint.lean

```
import Physlib.Meta.Informal.Post
import Physlib.Meta.Linters.Sorry
import Physlib.Meta.Sorry
  let env ← importModules (loadExts := true) #[`Physlib] {} 0
  let _ ← (Lean.Core.CoreM.toIO · ctx state) do (Physlib.sorryfulPseudoTest).run'
```

1.497 scripts/MetaPrograms/spelling.lean

```
import Physlib.Meta.Informal.Post
import Physlib.Meta.Linters.Sorry
import Physlib.Meta.Sorry
import Physlib.Meta.AllFilePaths
in Physlib, as well as module doc-strings, and compares them to a custom dictionary
/-- The strings appearing as documentation within Physlib. -
/
  let allModules ← allPhyslibModules
  let allConstants ← Physlib.allUserConsts
  let env ← importModules (loadExts := true) #[`Physlib] {} 0
```

1.498 scripts/MetaPrograms/style_lint.lean

```
# Physlib style linter
```

```
A number of linters on Physlib to enforce a consistent style.
def PhyslibTextLinter : Type := Array String → Array (String × ℕ × ℕ)
def doubleEmptyLineLinter : PhyslibTextLinter := fun lines ↦ Id.run do
def doubleSpaceLinter : PhyslibTextLinter := fun lines ↦ Id.run do
    if String.containsSubstr l.trimAsciiStart.copy "  " then
      let k := (Substring.Raw.findAllSubstr l "  ").toList.getLast?
        | some k => String.Pos.Raw.offsetOfPos l k.stopPos
def longLineLinter : PhyslibTextLinter := fun lines ↦ Id.run do
def substringLinter (s : String) : PhyslibTextLinter := fun lines ↦ Id.run
      let k := (Substring.Raw.findAllSubstr l s).toList.getLast?
        | some k => String.Pos.Raw.offsetOfPos l k.stopPos
def endLineLinter (s : String) : PhyslibTextLinter := fun lines ↦ Id.run do
def numInitialSpacesEven : PhyslibTextLinter := fun lines ↦ Id.run do
    let numSpaces := (l.takeWhile (· == ' ')).positions.length
structure PhyslibErrorContext where
def printErrors (errors : Array PhyslibErrorContext) : IO Unit := do
def physlibLintFile (path : FilePath) : IO (Array PhyslibErrorContext) := d
    (Array.map (fun (e, n, c) ↦ PhyslibErrorContext.mk e n c path)) (lint l
  let mods : Name :=  `Physlib
  let (physlibMod, _) ← readModuleData mFile
  let filePaths := physlibMod.imports.filterMap (fun imp ↦
  let errors := (← filePaths.mapM physlibLintFile).flatten
```

1.499 scripts/README.md

```
# Linting Physlib
Physlib has a number of linters which check for various different things.
are also automatically run on pull-requests to Physlib. These latter linter
Below we summarize the linters Physlib has, in each bullet point the initia
  - step 3: Checks all files are imported to `Physlib.lean` and that they a
  - step 5: Checks that all lemmas and definitions dependent on `sorry` or
- `lake exe runPhyslibLinters` : A linter which only does step 6 of `lake e
- `lake exe spelling` : Checks the spelling of words in Physlib against a g
```

1.500 scripts/Template.lean

```
          <a href="physlib.io" style="color: #fff; text-decoration: none; f
family: serif">
          Physlib Documentation</a>
```

1.501 scripts/check_file_imports.lean

```
This file checks that the imports in `Physlib.lean` are sorted and complete.
`addModulesIn`, `expectedPhyslibImports`, and `checkMissingImports`
/-- Compute imports expected by `Physlib.lean` by looking at file structure. -
/
def expectedPhyslibImports : IO (Array Name) := do
  for top in ← FilePath.readDir "Physlib" do
      let nm := `Physlib
    IO.print s!"Files are not imported add the following to the `Physlib` file: \n"
  let mods : Name := `Physlib
  let (physlibMod, _) ← readModuleData mFile
  let ePhyslibImports ← expectedPhyslibImports
  let sortedWarned ← arrayImportSorted physlibMod.imports
  let warned ← checkMissingImports physlibMod ePhyslibImports
    throw <| IO.userError s!"\x1b[31mThe Physlib.lean file is not sorted, or has mis
    IO.println s!"\x1b[32mAll files are imported correctly into Physlib.lean.\x1b[0m
```

1.502 scripts/find_TODOs.lean

```
This program finds all instances of `/<!> TODO: ...` (without the `<>`) in Physlib f
def PhyslibTODOItem : Type := Array String → Array (String × ℕ)
def TODOFinder : PhyslibTODOItem := fun lines ↦ Id.run do
  "https://github.com/leanprover-community/physlib/blob/master/"++
def physlibLintFile (path : FilePath) : IO String := do
This is an automatically generated list of TODOs appearing as `/-
! TODO:...` in Physlib.
  let mods : Name :=  `Physlib
  let (physlibMod, _) ← readModuleData mFile
  for imp in physlibMod.imports do
    let l ← physlibLintFile filePath
```

1.503 scripts/import-graph.py

```
# JSON files to be imported, generate this with `lake exe graph physlib.xdot_json`
graphFile = "physlib.xdot_json"
physlib = json.load(open(graphFile))
files = physlib['objects']
imports = physlib['edges']
physlib = pd.DataFrame([{'name':f['name'], 'imports':[], 'dependents':[]} for f in f
numFiles = len(physlib)
# Adds the subject of the file (the name of the folder under physlib) to the datafra
physlib['subject'] = physlib.name.apply(lambda str: str.split('.')[1] if len(str.spl
```

```
# The list of all current subjects in Physlib
subjectList = physlib.subject.unique()
    physlib.dependents.loc[i['tail']].append(i['head'])
    physlib.imports.loc[i['head']].append(i['tail'])
    return list(physlib.name.loc[index])
    return toName(physlib.imports.loc[index])
    return toName(physlib.dependents.loc[index])
    return physlib.index[physlib.name == name].tolist()
transImports = [None for i in range(len(physlib))]
    transImports[index] = physlib.imports.loc[index].copy()
    for i in physlib.imports.loc[index]:
for i in range(len(physlib)):
transDependents = [None for i in range(len(physlib))]
    transDependents[index] = physlib.dependents.loc[index].copy()
    for i in physlib.dependents[index]:
for i in range(len(physlib)):
physlib['transImports'] = transImports
physlib['transDependents'] = transDependents
physlib['numTransImps'] = physlib.transImports.apply(len)
physlib['numTransDeps'] = physlib.transDependents.apply(len)
physlib['importance'] = physlib.apply(lambda x: (x.numTransImps/numFiles)*(
plt.plot(physlib.numTransImps, physlib.numTransDeps, 'o')
plt.hist(physlib.importance)
```

1.504 scripts/lint-all.sh

```
echo "Building Physlib"
lake build Physlib
lake exe runLinter Physlib
lake exe shake Physlib
```

1.505 scripts/lint-style.sh

```
git ls-files 'Physlib/*.lean' | xargs ./scripts/lint-style.py "$@"
```

1.506 scripts/lint_all.lean

```
    let leanCheck ← IO.Process.output {cmd := "lake", args := #["exe", "run
```

1.507 scripts/mathlib_textLint_on_physlib.lean

```
to run text-based style linters from mathlib on Physlib.
  for s in ["Physlib.lean"] do
def physlib_lint_style : Cmd := `[Cli|
  "Run text-based style linters on every Lean file in Physlib (adapted from mathlib'
/-- The entry point to the `lake exe mathlib_textLint_on_physlib` command. -
/
def main (args : List String) : IO UInt32 := do physlib_lint_style.validate args
```

1.508 scripts/openAI_doc_check.lean

```
# Physlib OpenAI doc check
      let mods : Name :=  `Physlib
      let (physlibMod, _) ← readModuleData mFile
      let imports := physlibMod.imports
```

1.509 scripts/physlibNoShake.json

```
    "Physlib.Meta.Informal.Basic",
    "Physlib.Meta.Informal.SemiFormal",
    "Physlib.Meta.TODO.Basic",
    "Physlib.Relativity.Tensors.Elab",
    "Physlib.Relativity.Tensors.Elab"
```

1.510 scripts/stats.lean

```
import Physlib.Meta.Informal.Post
# Physlib Stats
This file concerns with statistics of Physlib.
open Lean System Meta Physlib
    <title>Stats for Physlib</title>
<h1>Stats for Physlib</h1>
<p>- Of which {noDefsVal - noInformalLemmasVal - noInformalDefsVal} are not <a href=
<p>- Of which {noLemmasVal} are not <a href=\"https://physlib.io/InformalGraph.html\
<h3>Number of <a href=\"https://physlib.io/TODOList\">TODOs</a>: {noTODOsVal}</h3>
  let statString ← CoreM.withImportModules #[`Physlib] (getStats).run'
    let html ← CoreM.withImportModules #[`Physlib] (Stats.toHtml).run'
```

1.511 scripts/treemap_gen.py

```
repo = Repo(".")
physlib_path = os.path.join(repo.working_tree_dir, "physlib")   # to analyz
# all folders in Physlib are named after a branch of physics, we assign a n
folders = [name for name in os.listdir(physlib_path) if os.path.isdir(os.pa
        if len(blob.path.split("/")) > 1 and blob.path.startswith("Physlib/
    )} for blob in repo.tree().traverse() if "Physlib" in blob.path.rsplit(
```

1.512 scripts/type_former_lint.lean

```
in Physlib.
/-- A rough definition of Upper Camel, checking that if a string starts wit
def IsUpperCamel (s : String) : Bool :=
  let mods : Name :=  `Physlib
  let (physlibMod, _) ← readModuleData mFile
  for imp in physlibMod.imports do
        ∧ ¬ IsUpperCamel c.name.toString then
    throw <| IO.userError s!"There are type former definitions not in upper
```